Programmazione Avanzata in Java e C

Dispensa del Corso – Anno Accademico 2025/2026

Raccolta dei Capitoli e Compendio Teorico-Pratico

Dalle lezioni del Prof. Massimiliano Redolfi

con la collaborazione del Prof. Ing. Giorgio Bontempi

Rielaborazione a cura degli studenti

Università degli Studi di Brescia – Dipartimento di Ingegneria dell'Informazione

Indice

1 Warm-up: Fondamenti di C e Java a Confronto **5**
- 1.1 Entry Point ed Esecuzione: Java vs C . . . 5
  - 1.1.1 Implementazione in C . . . 5
  - 1.1.2 Implementazione in Java . . . 5
  - 1.1.3 Aree di Memoria Runtime: Stack vs Heap e Passaggio dei Parametri per Valore . . . 6
- 1.2 Primo Tempo: Gestione dei Dati (Rettangoli) . . . 7
  - 1.2.1 L'Approccio Procedurale in C: Strutture e Typedef . . . 7
  - 1.2.2 L'Approccio a Oggetti in Java: Classi e Costruttori . . . 8
- 1.3 Secondo Tempo: Quadrati e Polimorfismo di Overloading . . . 9
  - 1.3.1 Il Limite Architetturale del C: Assenza di Overloading . . . 9
  - 1.3.2 La Soluzione in Java: Overloading dei Metodi . . . 9
- 1.4 Ereditarietà, Gerarchia e Catena di Costruzione . . . 9
  - 1.4.1 La Parola Chiave `extends` e la Construction Chain . . . 9
  - 1.4.2 La Classe Radice `java.lang.Object` . . . 10
  - 1.4.3 Uguaglianza e Tabelle Hash: `equals()` ed `hashCode()` . . . 10
- 1.5 Metodi di Istanza e Override . . . 11
  - 1.5.1 Spostamento dei Metodi all'Interno della Classe . . . 11
  - 1.5.2 Override dei Metodi (`@Override`) . . . 12
- 1.6 Terzo Tempo: Rombi, Forme Varie e Interfacce . . . 12
  - 1.6.1 La Crisi dell'Ereditarietà Singola . . . 12
  - 1.6.2 L'Interfaccia `Shape2d` . . . 13
  - 1.6.3 L'Arrivo delle Astronavi e la Segregazione dei Protocolli . . . 13
- 1.7 Determinazione dei Tipi: Compile-Time vs Run-Time . . . 14
  - 1.7.1 Enumerazioni e Switch Expression . . . 14
- 1.8 Il Java Collections Framework . . . 15
- 1.9 Java I/O: Accesso al File System e Flussi di Dati . . . 16
  - 1.9.1 La Classe `java.io.File` . . . 17
  - 1.9.2 Architettura degli Stream: Byte vs Caratteri . . . 17
  - 1.9.3 Pattern Robusto di Lettura e Scrittura Testuale . . . 17
  - 1.9.4 I/O Binario vs I/O Testuale: DataInputStream, DataOutputStream ed Endianness . . . 18
  - 1.9.5 Il Meccanismo Try-With-Resources: AutoCloseable, Ordine LIFO e Suppressed Exceptions 19
- 1.10 Tipi Enumerativi (enum) e Macchine a Stati Finiti . . . 19
  - 1.10.1 Struttura Avanzata di un `enum` . . . 19
  - 1.10.2 Implementazione di una Macchina a Stati Finiti (FSM) . . . 20
- 1.11 Studio di Caso d'Aula: Il Censimento del Bimondo in Stile Procedurale . . . 21

2 Dal Nome alla Funzione: Classi Anonime, Lambda, Method Reference e Nuove Interfacce **23**
- 2.1 Il Problema delle Classi Usa-e-Getta e del Boilerplate . . . 23
- 2.2 Classi Anonime: Sintassi, Uso e Limiti . . . 24
  - 2.2.1 Caratteristiche e Limitazioni delle Classi Anonime . . . 24
  - 2.2.2 Il Riferimento `this` nelle Classi Anonime . . . 25
- 2.3 Closure e il Modello di Memoria: La Regola di "Effectively Final" . . . 25
  - 2.3.1 Cosa Succede in Memoria: La Copia per Valore . . . 26
  - 2.3.2 Perché le Variabili Devono Essere "Effectively Final" . . . 26
- 2.4 Interfacce Funzionali e l'Annotazione `@FunctionalInterface` . . . 27
- 2.5 Lambda Expression: Sintassi, Target Typing ed Efficienza . . . 27
  - 2.5.1 Guida Rapida alla Sintassi delle Lambda . . . 28
  - 2.5.2 Come il Compilatore Capisce il Tipo: Il Target Typing . . . 28
  - 2.5.3 Differenze Sostanziali tra Lambda e Classi Anonime . . . 29

2.6 Method Reference: Le Quattro Categorie Canoniche . . . 29
2.6.1 Le Quattro Forme di Method Reference . . . 29
2.7 I Quattro Livelli di Astrazione a Confronto . . . 30
2.8 Iniezione di Comportamento e Inversione del Controllo . . . 31
2.9 Il Package `java.util.function` e la Composizione . . . 32
2.9.1 I Cinque Ruoli Fondamentali . . . 32
2.9.2 Tipi Primitivi e Zero-Boxing Overhead . . . 32
2.9.3 Comporre Funzioni e Predicati . . . 33
2.10 Il Motore della Calcolatrice Riscritto e Completo . . . 33
2.11 L'Evoluzione delle Interfacce: Metodi Default, Static e Private . . . 34
2.11.1 Perché Sono Nati i Metodi Default . . . 34
2.12 Risoluzione dei Conflitti tra Metodi Default . . . 34
2.12.1 Le Tre Regole della JVM per Risolvere i Conflitti . . . 35
2.12.2 Metodi Statici e Metodi Privati nelle Interfacce (Java 8 e Java 9) . . . 35
2.13 Contratti, Classi Astratte e i Due Volti dell'Ordinamento . . . 36
2.13.1 Interfaccia o Classe Astratta? Differenze e Scelte Architetturali . . . 36
2.13.2 Due Modi di Ordinare: Comparable vs Comparator . . . 37
2.13.3 Comparator Moderno: Metodi Statici, Combinatori e Null-Safety . . . 38
2.14 Gerarchie Chiuse: Tipi Sigillati (Sealed) e Switch sui Tipi . . . 39
2.14.1 Pattern Matching e Switch Esaustivo (Java 21) . . . 39
2.14.2 Confronto Sintetico: Gerarchie Aperte vs Gerarchie Chiuse . . . 40

3 Stream Computazionali: Pipeline, Valutazione Pigra e Collettori 42
3.1 Il Problema del Paradigma Procedurale e della Modifica Concorrente . . . 42
3.1.1 L'Anti-Pattern delle Liste Ausiliarie . . . 42
3.1.2 La Trappola della Modifica durante l'Iterazione . . . 43
3.2 Il Paradigma degli Stream come Flusso Computazionale . . . 43
3.2.1 L'Analogia delle Palline nel Tubo . . . 43
3.3 Anatomia di una Pipeline e Valutazione Pigra (Laziness) . . . 44
3.3.1 Dimostrazione Sperimentale della Laziness . . . 44
3.4 Ordine di Valutazione: Elaborazione Verticale vs Orizzontale . . . 45
3.4.1 Tracciare l'Ordine Reale di Esecuzione . . . 45
3.4.2 Operazioni Stateful e la Rottura della Verticalità . . . 46
3.5 Il Vincolo di Monouso e la Soluzione con `Supplier<Stream<T>` . . . 46
3.5.1 Perché lo Stream non è Riavvolgibile? . . . 46
3.5.2 La Soluzione Architetturale: `Supplier<Stream<T>` . . . 47
3.6 Creare Stream da Varie Sorgenti . . . 47
3.6.1 Da Collezioni e Array . . . 47
3.6.2 Stream Primitivi Specializzati: `IntStream`, `LongStream`, `DoubleStream` . . . 47
3.6.3 Stream Infiniti: `Stream.iterate` e `Stream.generate` . . . 48
3.7 Riduzioni Avanzate: Le Tre Forme di `reduce()` . . . 48
3.8 Raccogliere Risultati: `collect()` e la Classe `Collectors` . . . 49
3.8.1 Collettori Base e Concatenazione Testuale . . . 49
3.8.2 Raggruppamento e Partizionamento Avanzato . . . 50
3.8.3 Lo Studio del Caso d'Aula: Il Censimento del Bimondo in Una Riga . . . 50

4 Interfacce Grafiche Swing, Architettura ad Eventi ed Event Dispatch Thread 52
4.1 Dal Modello a Riga di Comando alla Programmazione ad Eventi . . . 52
4.1.1 Paradigmi a Confronto: Esecuzione Lineare vs Reattiva . . . 52
4.1.2 Il Dialogo Continuo tra Sistema Operativo e Applicazione . . . 52
4.1.3 Eventi Hardware a Basso Livello vs Eventi Semantici di Alto Livello . . . 53
4.2 Il Modello Architetturale: Observer Pattern e Delegation Event Model . . . 53
4.2.1 Multicast Events: Perché `add` e non `set`? . . . 53
4.3 Il Thread della Grafica: Event Dispatch Thread (EDT) . . . 54
4.3.1 Perché Swing non è Thread-Safe? . . . 54

4.3.2 La Chiamata Obbligatoria: `EventQueue.invokeLater` . . . 54
4.4 Gerarchia dei Componenti Swing e Contenitori . . . 55
4.4.1 I Livelli di Contenimento . . . 55
4.5 Gestione dello Spazio: I Layout Manager . . . 55
4.5.1 Perché il Posizionamento Assoluto (`setLayout(null)`) è Bandito . . . 55
4.5.2 Confronto Sinottico tra i Principali Layout Manager . . . 56
4.5.3 La Tecnica dei Pannelli Annidati (Nested Panels) . . . 56
4.6 L'Evoluzione dei Listener: Da Classi Esterne a Method Reference . . . 57
4.6.1 1. Classe Esterna Separata . . . 57
4.6.2 2. Classe Interna (Inner Class) . . . 57
4.6.3 3. Classe Anonima Inline (Pre-Java 8) . . . 57
4.6.4 4. Lambda Expression (Java 8+) . . . 57
4.6.5 5. Method Reference su Metodo Dedicato . . . 57
4.7 Progettazione di Componenti Custom: Il Caso del Tastierino Numerico (`PnlDigits`) . . . 58
4.7.1 Il Principio di Incapsulamento Rigoroso . . . 58
4.7.2 Il Componente come Proxy di Eventi . . . 58
4.7.3 Integrazione del Componente nel Frame Applicativo . . . 59
4.8 Concorrenza in Swing: L'Event Dispatch Thread (EDT) e il Ponte verso i Background Thread 60

5 Thread, Concorrenza, Executor e Virtual Threads 62
5.1 Introduzione: Una Finestra che Congela e l'Immagine Guida . . . 62
5.1.1 Il Problema: L'Applicazione PrimeFinderApp . . . 62
5.1.2 La Diagnosi: Perché la Finestra si Blocca? . . . 63
5.2 Processi, Thread e il Modello di Memoria della JVM . . . 64
5.2.1 Come Fanno i Sistemi Operativi Moderni (Linux vs Windows) . . . 64
5.2.2 Ispezione Forense: `jstack` e i Thread della JVM . . . 65
5.3 Il Primo Thread: Compito, Lavoratore e Ciclo di Vita . . . 65
5.3.1 Compito vs Lavoratore: `Runnable` e `Thread` . . . 65
5.3.2 Gli Stati del Ciclo di Vita di un Thread (`Thread.State`) . . . 66
5.3.3 Coordinamento: `sleep()` e `join()` . . . 66
5.3.4 Fermare un Thread: L'Interruzione Cooperativa (`interrupt()`) . . . 66
5.3.5 Lambda e Stato: Cattura ed *Effectively Final* (§3.5 D04) . . . 67
5.3.6 L'Esempio Guida: I Primi su Due Thread . . . 67
5.4 Il Framework Executor: Disaccoppiamento tra Compito ed Esecuzione . . . 68
5.4.1 Perché Non un Thread per Ogni Compito? . . . 68
5.4.2 Le Factory Standard di `Executors` . . . 68
5.4.3 Differenza Cruciale: `execute()` vs `submit()` . . . 69
5.4.4 Spegnere un Executor e il Supporto ad AutoCloseable . . . 69
5.4.5 Dimensionamento Ottimale del Pool . . . 69
5.5 Calcolo Asincrono con Valore: `Callable<V>` e `Future<V>` . . . 70
5.5.1 Da Runnable a Callable . . . 70
5.5.2 L'Esempio Guida: Contare i Primi fino a 10^8 su Tutti i Core . . . 70
5.5.3 Le Misure Empiriche e la Legge di Amdahl . . . 71
5.5.4 Perché 64 Compiti e Non 16? Lo Sbilanciamento del Carico . . . 71
5.6 Quando i Thread Condividono: Race Condition, Bytecode e Visibilità . . . 72
5.6.1 L'Anatomia di una Race Condition a Livello di Bytecode . . . 72
5.6.2 Visibilità della Memoria e la Parola Chiave `volatile` . . . 72
5.6.3 Sincronizzazione Intrinseca: Il Monitor Lock (`synchronized`) . . . 73
5.6.4 Coordinamento con `wait()`, `notify()` e la Regola del Loop While . . . 73
5.6.5 Deadlock e le Quattro Condizioni di Coffman . . . 74
5.6.6 Thread Safety per Costruzione . . . 74
5.7 Gli Strumenti Avanzati di `java.util.concurrent` . . . 74
5.7.1 Lock Espliciti: `ReentrantLock` . . . 74
5.7.2 Lettori e Scrittori: `ReadWriteLock` . . . 75
5.7.3 Risorse a Posti Limitati: `Semaphore` . . . 75

5.7.4 Variabili Atomiche Lock-Free (`java.util.concurrent.atomic`) . . . 75
5.7.5 La Soluzione alla Contesa Estrema: `LongAdder` . . . 75
5.7.6 Collezioni Concorrenti Thread-Safe . . . 76
5.7.7 Quale Strumento Scegliere: La Guida Decisionale (§6.6 D04) . . . 76
5.8 Il Tempo, l'Attesa e i Thread Virtuali (Java 21) . . . 76
5.8.1 Compiti Differiti e Periodici: `ScheduledExecutorService` . . . 76
5.8.2 Stream Paralleli Visti da Sotto: `parallelStream()` (§7.2 D04) . . . 76
5.8.3 La Rivoluzione dei Virtual Threads (Java 21 LTS, JEP 444) . . . 77
5.8.4 L'Esempio Guida in Tre Modi: Calcolo vs Attesa (§7.4 D04) . . . 78
5.9 Thread e Swing: Risolvere il Congelamento con SwingWorker . . . 79
5.9.1 La Regola del Single-Threaded Subsystem e `EventQueue.invokeLater()` . . . 79
5.9.2 L'Astrazione Dedicata: `javax.swing.SwingWorker<T, V>` . . . 79
5.9.3 La Soluzione Completa: PrimeFinderApp Reattiva con SwingWorker . . . 79
5.9.4 Rientro nell'EDT e la Trappola di `invokeAndWait` (§8.5 D04) . . . 81
5.9.5 Compiti Periodici sull'Interfaccia: `javax.swing.Timer` (§8.7 D04) . . . 81
5.10 Orizzonti Avanzati della Concorrenza Moderna . . . 82

A Eserciziario e Tracce di Laboratorio **83**
A.1 Capitolo 1: Warm-up e Fondamenti della Programmazione a Oggetti . . . 83
A.2 Capitolo 2: Classi Anonime, Lambda, Interfacce Moderne e Sealed . . . 88
A.3 Capitolo 3: Stream Computazionali, Pipeline e Collettori . . . 92
A.4 Capitolo 4: Interfacce Grafiche Swing ed Architettura ad Eventi . . . 96
A.5 Capitolo 5: Thread, Concorrenza, Executor e Virtual Threads . . . 102

B Compendio e Formulario Tecnico di Java **119**
B.1 PARTE I: SCHEMI CONCETTUALI E PATTERN ARCHITETTURALI . . . 119
B.1.1 Notazione UML e Relazioni tra Classi . . . 120
B.1.2 Principi Guida di Ingegneria del Software . . . 120
B.1.3 Ereditarietà, Polimorfismo e Classi Astratte . . . 121
B.1.4 Gestione delle Eccezioni . . . 121
B.1.5 Complessità Computazionale delle Strutture Dati . . . 122
B.1.6 Modelli Matematici della Scalabilità Concorrente: Amdahl e Goetz . . . 122
B.2 PARTE II: FORMULARIO OPERATIVO DELLE API DI JAVA . . . 123

Capitolo 1

Warm-up: Fondamenti di C e Java a Confronto

1.1 Entry Point ed Esecuzione: Java vs C

Ogni programma informatico richiede un punto di ingresso convenzionale (*entry point*) a partire dal quale il sistema operativo o il runtime avvia il thread di esecuzione primario. Il confronto tra la classica implementazione di "Hello World" nei due linguaggi mette in luce differenze architetturali profonde.

1.1.1 Implementazione in C

Nel linguaggio C, l'entry point è rappresentato dalla funzione libera globale `main`:

Hello World canonico in C (main.c)

```c
include <stdio.h>

/*
 * Entry point del programma in C.
 * 'void' specifica esplicitamente che la funzione non accetta argomenti.
 * Restituisce un intero (0 indica terminazione con successo al S.O.).
 */
int main(void) {
    printf("Hello World from C\n");
    return 0;
}
```

In C, le funzioni risiedono nello spazio globale dei simboli (o limitate al file tramite il qualificatore **`static`**). Non esiste l'obbligo di incapsulare il codice all'interno di classi.

1.1.2 Implementazione in Java

In Java, ogni frammento di codice deve risiedere tassativamente all'interno di una classe:

Hello World canonico in Java (HelloWorld.java)

```java
package it.unibs.eps;

public class HelloWorld {
    /*
     * Entry point universale per la Java Virtual Machine.
     */
    public static void main(String[] args) {
        System.out.printf("Hello World from JAVA!\n");
    }
}
```

Disamina Accademica della Firma di `main()` in Java

Durante la lezione, il docente ha analizzato punto per punto i vincoli della riga **`public static void main(String[] args)`**:

1. **`public`**: il metodo deve possedere visibilità pubblica globale per consentire al runtime della JVM (che risiede all'esterno del package dell'applicazione) di invocare il punto di ingresso.
2. **`static`**: è un metodo di classe e **non un metodo di istanza**. All'avvio del programma, nello heap *non esiste ancora alcun oggetto*. Se il metodo non fosse statico, la JVM si troverebbe in una situazione di stallo logico: non potrebbe invocare il metodo senza prima istanziare un oggetto, ma non saprebbe come costruirlo né quali parametri passare al costruttore. Il qualificatore **`static`**

svincola il metodo dall'esistenza di istanze nello heap.

3. `void`: il metodo non restituisce alcun valore nello stack di chiamata; eventuali codici di stato per il sistema operativo possono essere trasmessi esplicitamente tramite `System.exit(int)`.
4. `String[] args`: raccoglie i parametri passati all'applicazione tramite riga di comando sotto forma di array di stringhe.

1.1.3 Aree di Memoria Runtime: Stack vs Heap e Passaggio dei Parametri per Valore

Per comprendere a fondo il comportamento di qualsiasi programma Java, è indispensabile padroneggiare la suddivisione fondamentale della memoria gestita dalla JVM durante l'esecuzione:

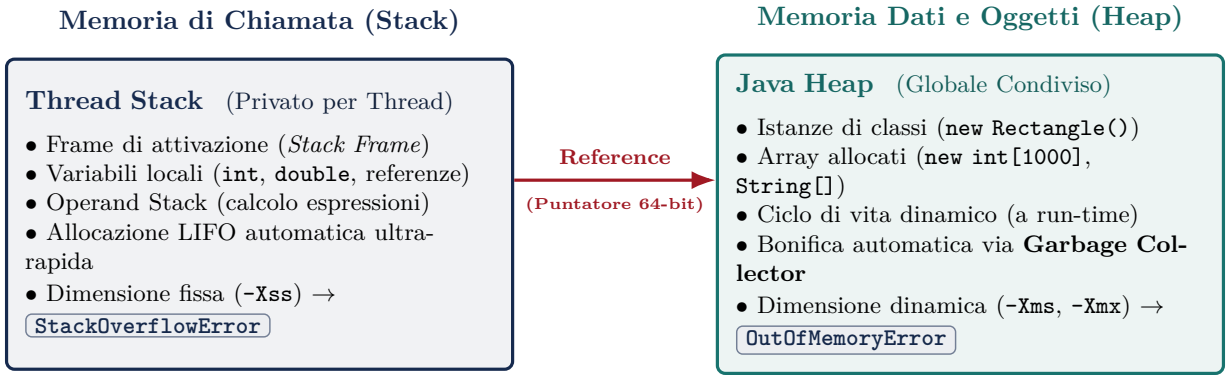


Figura 1.1: Disaccoppiamento strutturale della memoria Java: Stack privato dei thread vs Heap condiviso.

Principio Architetturale: Il Dogma Assoluto: Java Passa TASSATIVAMENTE Tutto per Valore

Nel linguaggio Java **non esiste il passaggio per riferimento del C++ o dei puntatori C**. Ogni passaggio di parametri nei metodi avviene **esclusivamente per valore** (*pass-by-value*):

- **Tipi Primitivi (`int`, `double`, `boolean`, ...)**: il parametro formale del metodo riceve una copia binaria pura del valore. Qualsiasi modifica apportata alla variabile all'interno del metodo agisce unicamente sullo stack frame del metodo stesso e non ha alcun effetto sulla variabile originale nel chiamante.
- **Tipi Riferimento (`Object`, Array)**: il parametro formale riceve una copia del **valore del riferimento** (il puntatore all'oggetto nello heap). Di conseguenza:
  1. Se il metodo invoca mutatori sullo stato dell'oggetto dereferenziato (es. `r.width = 20`), la modifica si riflette sullo stato fisico dell'oggetto condiviso nello heap ed è visibile a tutti i riferimenti;
  2. Se il metodo riassegna il parametro a una nuova istanza (es. `r = new Rectangle(1, 1)`), viene modificata unicamente la copia locale del puntatore residente nello stack frame corrente. Al ritorno dal metodo, il riferimento del chiamante continua a puntare all'oggetto originale, inalterato.

Dimostrazione Empirica del Passaggio dei Parametri per Valore

```
1  public class ParameterPassDemo {
2      public static void testSwap(Rectangle r, int num) {
3          num = 999;                // Modifica locale al frame: non altera il chiamante!
4          r.width = 42;             // Modifica lo stato dell'oggetto nello Heap:
       VISIBILE!
5          r = new Rectangle(0, 0); // Riassegna la copia locale del puntatore: NESSUN
       EFFETTO nel chiamante!
6      }
7
8      public static void main(String[] args) {
9          int x = 10;
10         Rectangle rett = new Rectangle(8, 5);
11         testSwap(rett, x);
12
```

```
        System.out.printf("x dopo il metodo: %d\n", x);            // Stampa: 10 (inalterato)
        System.out.printf("rett.width: %d\n", rett.width);         // Stampa: 42 (modificato nello heap)
        System.out.printf("rett.height: %d\n", rett.height);       // Stampa: 5 (non e' stato azzerato!)
    }
}
```

1.2 Primo Tempo: Gestione dei Dati (Rettangoli)

Immaginiamo di trovarci in un universo geometrico bidimensionale (il "Bimondo") e di dover modellare e catalogare gli oggetti che lo popolano, partendo dalle forme più semplici: i **rettangoli**.

1.2.1 L'Approccio Procedurale in C: Strutture e Typedef

In C, i dati compositi vengono raggruppati tramite il costrutto `struct`. Le funzioni che operano su tali dati sono entità separate ed esterne:

Modellazione procedurale di un rettangolo in C (main.c)

```c
#include <stdio.h>

/*
 * Definizione del tipo struttura per il rettangolo.
 * 'Rectangle_ST' e' il tag di struttura nel namespace struct.
 * 'Rectangle_T' e' il nuovo tipo di dato definito mediante typedef.
 */
typedef struct Rectangle_ST {
    int width;
    int height;
} Rectangle_T;

/* Funzione procedurale esterna per il perimetro */
int perimeter(Rectangle_T r) {
    return (r.width + r.height) * 2;
}

/* Funzione procedurale esterna per l'area */
int area(Rectangle_T r) {
    return r.width * r.height;
}

int main(void) {
    Rectangle_T r;
    r.width = 8;
    r.height = 5;

    printf("Perimetro rettangolo: %d\n", perimeter(r));
    printf("Area rettangolo: %d\n", area(r));
    return 0;
}
```

Dibattito d'Aula: `Rectangle_T` è un Tipo o una Variabile?

Il docente ha posto all'aula un quesito fondamentale di sintassi C: *"Nel codice sopra, `Rectangle_T` è una variabile o un tipo?"*.

La risposta corretta è che si tratta di un **tipo di dato**:

- In C standard, dichiarando `struct Nome { ... };`, ogni successiva istanziazione richiederebbe la sintassi verbosa `struct Nome r;`.
- L'istruzione `typedef struct Rectangle_ST { ... } Rectangle_T;` definisce un alias: `Rectangle_ST` è l'identificatore associato alla struct (*struct tag*), mentre `Rectangle_T` diventa a tutti gli effetti un nuovo identificatore di tipo, permettendo di dichiarare variabili con

`Rectangle_T r;`.

- Non viene allocata memoria fino a quando una variabile di tale tipo non viene formalmente dichiarata o allocata con `malloc`.

1.2.2 L'Approccio a Oggetti in Java: Classi e Costruttori

In Java, la separazione tra dati e funzioni comincia a dissolversi. In una prima versione elementare, definiamo la classe `Rectangle` come mero aggregato di campi:

Classe Rectangle elementare in Java (Rectangle.java)

```java
package it.unibs.eps;

public class Rectangle {
    public int width;
    public int height;

    /*
     * Costruttore esplicito con inizializzazione dello stato.
     * La parola chiave 'this' disambigua gli attributi di istanza dai parametri.
     */
    public Rectangle(int width, int height) {
        this.width = width;
        this.height = height;
    }

    /* Costruttore predefinito senza argomenti */
    public Rectangle() {
        this.width = 0;
        this.height = 0;
    }
}
```

Definizione: Operatore `new` e Costruttore

In Java, l'operatore **new** è responsabile dell'allocazione fisica di un nuovo blocco di memoria nello **Heap**. Esso restituisce il riferimento (*reference*) all'istanza creata. Subito dopo l'allocazione, viene invocato il **costruttore**, un metodo speciale (privo di tipo di ritorno e con nome identico alla classe) avente il compito di inizializzare lo stato iniziale dell'oggetto.

Main con istanziazione di Rectangle (Main.java)

```java
package it.unibs.eps;

public class Main {
    /* Metodi procedurali statici definiti nel Main */
    public static int perimeter(Rectangle r) {
        return (r.height + r.width) * 2;
    }

    public static int area(Rectangle r) {
        return r.height * r.width;
    }

    public static void main(String[] args) {
        // Creazione dell'istanza nello heap mediante costruttore esplicito
        Rectangle r = new Rectangle(8, 5);

        System.out.printf("Perimetro: %d\n", perimeter(r));
        System.out.printf("Area: %d\n", area(r));
    }
}
```

1.3 Secondo Tempo: Quadrati e Polimorfismo di Overloading

Il dominio applicativo si espande: dobbiamo introdurre i **quadrati** (`Square`).

1.3.1 Il Limite Architetturale del C: Assenza di Overloading

Nel linguaggio C, tutti i simboli di funzione condividono un unico spazio globale dei nomi. Non è ammessa la presenza di due funzioni con lo stesso nome, anche qualora accettassero parametri differenti. L'aggiunta del quadrato costringe a duplicare le funzioni:

Duplicazione forzata di funzioni in C per il tipo Square

```c
typedef struct Square_ST {
    int width;
    int height;
} Square_T;

/* In C dobbiamo inventare un nome differente! */
int perimeter_sq(Square_T s) {
    return (s.width + s.height) * 2;
}

int area_sq(Square_T s) {
    return s.width * s.height;
}
```

Questo vincolo degrada la scalabilità del software procedurale: all'aumentare delle forme, il numero di funzioni specifiche cresce linearmente (`perimeter_triangle`, `perimeter_circle`, ecc.).

1.3.2 La Soluzione in Java: Overloading dei Metodi

Java supporta nativamente il **polimorfismo di sovraccarico** (*method overloading*):

Overloading dei metodi statici in Java

```java
public class Main {
    // Versione per Rettangolo
    public static int perimeter(Rectangle r) {
        return (r.height + r.width) * 2;
    }

    // Versione sovraccaricata per Quadrato (stesso nome, firma diversa)
    public static int perimeter(Square s) {
        return (s.height + s.width) * 2;
    }
}
```

Definizione: Method Overloading (Polimorfismo a Tempo di Compilazione)

Il polimorfismo di sovraccarico consente a più metodi appartenenti alla stessa classe di condividere il medesimo nome, purché presentino una **firma differente** per numero o tipo dei parametri. La risoluzione della versione da invocare avviene staticamente a tempo di compilazione (*early binding*) in base al tipo statico della variabile passata.

1.4 Ereditarietà, Gerarchia e Catena di Costruzione

Dal punto di vista della teoria geometrica, un quadrato non è un'entità slegata dal rettangolo: un quadrato **è** un rettangolo speciale in cui i lati hanno la medesima lunghezza ($width = height$). In programmazione orientata agli oggetti, questa relazione concettuale è formalizzata dal principio di **ereditarietà** (*IS-A relationship*).

1.4.1 La Parola Chiave `extends` e la Construction Chain

Dichiarando `Square extends Rectangle`, la classe `Square` eredita automaticamente tutti gli attributi e i metodi accessibili della superclasse:

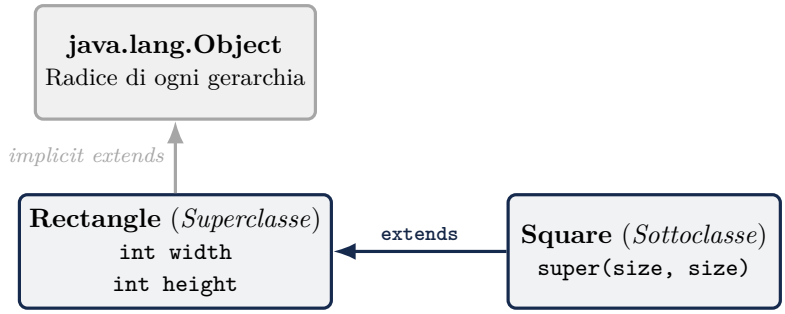


Figura 1.2: Gerarchia di ereditarietà tra `Square` e `Rectangle`, con derivazione da `Object`.

Definizione di Square tramite ereditarietà (Square.java)

```java
package it.unibs.eps;

public class Square extends Rectangle {
    /*
     * Costruttore di Square.
     * Invocando super(size, size) richiamiamo esplicitamente
     * il costruttore della superclasse Rectangle.
     */
    public Square(int size) {
        super(size, size);
    }
}
```

Principio Architetturale: Regola Aurea della Construction Chain

Ogni istanza di una sottoclasse racchiude al proprio interno una valida istanza della superclasse. Di conseguenza, il costruttore della classe derivata **deve tassativamente** invocare il costruttore della classe base (tramite `super(...)`) come primissima istruzione. Se non specificato, il compilatore inserisce automaticamente una chiamata a `super()` a vuoto; se la superclasse non possiede un costruttore senza parametri, si verifica un errore di compilazione.

1.4.2 La Classe Radice `java.lang.Object`

In Java, qualsiasi classe che non specifichi una clausola `extends` estende implicitamente la classe `Object`. Questa gerarchia unificata garantisce che ogni entità nello heap possegga un insieme garantito di comportamenti, tra cui:

- `toString()`: rappresentazione testuale dell'oggetto.
- `equals(Object)`: confronto semantico di uguaglianza tra contenuti.
- `hashCode()`: valore hash intero per l'indicizzazione nelle collezioni.
- `getClass()`: metadati di riflessione sul tipo a run-time.

1.4.3 Uguaglianza e Tabelle Hash: `equals()` ed `hashCode()`

Consideriamo una classe `Point` con due coordinate intere x e y, un costruttore e nient'altro, e proviamo a utilizzarla all'interno di un insieme:

Il problema dell'uguaglianza ereditata (PointDemo.java)

```java
var visited = new HashSet<Point>();
visited.add(new Point(1, 2));
System.out.println(visited.contains(new Point(1, 2))); // Stampa false!
```

Il punto è presente nell'insieme, ma `contains()` non lo trova. Il motivo è che `HashSet`, `HashMap` e tutte le collezioni basate su hash confrontano gli elementi con i metodi `equals()` e `hashCode()`. La versione predefinita ereditata da `Object` confronta l'**identità referenziale**: due riferimenti sono uguali solo se indicano la medesima istanza fisica nello heap (esattamente come l'operatore `==`). I due `new Point(1, 2)` sono oggetti distinti nello heap, quindi per `Object` risultano diversi.

Per confrontare gli oggetti **per contenuto** (uguaglianza semantica) è necessario sovrascrivere entrambi i metodi in modo coerente.

Principio Architetturale: Il Contratto tra `equals()` ed `hashCode()`

La regola che garantisce il funzionamento delle collezioni stabilisce:
Se due oggetti sono uguali secondo `equals()`, DEVONO produrre lo stesso valore intero da `hashCode()`:

$$x.\texttt{equals}(y) == \texttt{true} \implies x.\texttt{hashCode}() == y.\texttt{hashCode}()$$

Se due oggetti producono lo stesso codice hash non è richiesto che siano uguali (fenomeno delle collisioni), ma se manca `hashCode()`, due oggetti uguali finiranno in posizioni diverse della tabella hash e le collezioni non riusciranno a ritrovarli.

Inoltre, il contratto di `equals()` richiede che la relazione sia *riflessiva* ($x.\texttt{equals}(x)$ è vero), *simmetrica* ($x.\texttt{equals}(y) \iff y.\texttt{equals}(x)$), *transitiva*, *coerente* e che $x.\texttt{equals}(\texttt{null})$ restituisca `false`.

Implementazione Canonica di equals() e hashCode() (Rectangle.java)

```java
package it.unibs.pajc;

import java.util.Objects;

public class Rectangle {
    private int width;
    private int height;

    public Rectangle(int width, int height) {
        this.width = width;
        this.height = height;
    }

    @Override
    public boolean equals(Object o) {
        return o instanceof Rectangle r && width == r.width && height == r.height;
    }

    @Override
    public int hashCode() {
        return Objects.hash(width, height);
    }
}
```

Due dettagli fondamentali evidenziati a lezione:

- L'espressione `o instanceof Rectangle r` verifica il tipo e dichiara contestualmente la variabile `r` con pattern matching (Java 16+), gestendo automaticamente anche il caso in cui `o` sia `null` (restituendo `false`);
- L'espressione `r.width` accede direttamente a un campo privato di un'altra istanza: è perfettamente lecito perché il modificatore `private` in Java limita la visibilità alla **classe**, non alla singola istanza dell'oggetto.

1.5 Metodi di Istanza e Override

Fino a questo punto abbiamo trattato le classi come semplici contenitori di campi, delegando il calcolo a metodi statici esterni. Questo approccio è procedurale. Il vero paradigma ad oggetti richiede che i dati siano saldamente uniti ai metodi che li manipolano (**incapsulamento**).

1.5.1 Spostamento dei Metodi all'Interno della Classe

Trasformiamo `perimeter()` e `area()` in **metodi di istanza**:

Rectangle con metodi di istanza completi (Rectangle.java)

```java
package it.unibs.eps;

public class Rectangle {
    public int width;
    public int height;

    public Rectangle(int width, int height) {
        this.width = width;
        this.height = height;
    }

    /* Metodo di istanza: opera sullo stato dell'oggetto chiamante */
    public int perimeter() {
        return (this.width + this.height) * 2;
    }

    public int area() {
        return this.width * this.height;
    }
}
```

Quando eseguiamo `Square s = new Square(9); int p = s.perimeter();`, la classe `Square` eredita direttamente i metodi di `Rectangle`. Il perimetro restituito sarà corretto ($(9+9) \times 2 = 36$).

1.5.2 Override dei Metodi (`@Override`)

Tuttavia, per un quadrato, il calcolo del perimetro può essere ottimizzato o espresso in modo più specifico come lato moltiplicato per quattro ($4 \times l$). È possibile **sovrascrivere** l'implementazione ereditata:

Override del metodo perimeter in Square (Square.java)

```java
package it.unibs.eps;

public class Square extends Rectangle {
    public Square(int size) {
        super(size, size);
    }

    @Override
    public int perimeter() {
        // Calcolo specializzato per il quadrato
        return this.height * 4;
    }
}
```

Definizione: Method Overriding (Polimorfismo Dinamico)

L'**override** avviene quando una sottoclasse ridefinisce un metodo ereditato dalla superclasse mantenendo **identica la firma** (nome, parametri e tipo di ritorno compatibile). Durante l'esecuzione, la JVM adotta il meccanismo del *late binding* (o dynamic dispatch): viene sempre eseguita l'implementazione corrispondente al tipo dinamico effettivo dell'oggetto nello heap, indipendentemente dal tipo statico della variabile di riferimento.

1.6 Terzo Tempo: Rombi, Forme Varie e Interfacce

L'espansione del Bimondo richiede ora di introdurre i **Rombi** (`Diamond`), le **Ellissi** (`Ellipse`) e i **Cerchi** (`Circle`).

1.6.1 La Crisi dell'Ereditarietà Singola

Un rombo possiede una diagonale maggiore (`diagonaleMagg`) e una diagonale minore (`diagonaleMin`). Il calcolo dell'area è dato da (diagM $\times$ diagm)/2. Possiamo far derivare `Diamond` da `Rectangle`? **Assoluta-**

mente no! Un rombo non possiede base e altezza ortogonali, e tentare di forzare tale derivazione violerebbe il principio di coerenza e sostituibilità (Principio di Liskov).

Allo stesso modo, un cerchio è un'ellisse con i semiassi coincidenti ($xAxis = yAxis = radius$). Come possiamo gestire un catalogo eterogeneo di figure geometriche, calcolarne il perimetro e disegnarle senza creare gerarchie innaturali?

1.6.2 L'Interfaccia `Shape2d`

La soluzione consiste nel definire un'astrazione pura mediante un'**interfaccia**:

Interfaccia Shape2d come contratto geometrico puro (Shape2d.java)

```
package it.unibs.eps;

public interface Shape2d {
    int perimeter();
    int area();
    void draw();
}
```

Tutte le forme geometriche implementano l'interfaccia:

Snippet Java

```
public class Rectangle implements Shape2d { ... }
public class Diamond implements Shape2d { ... }
public class Ellipse implements Shape2d { ... }
```

1.6.3 L'Arrivo delle Astronavi e la Segregazione dei Protocolli

Supponiamo ora di dover inserire nel Bimondo un'astronave spaziale (`Spaceship`). Un'astronave possiede una velocità (`speed`), un livello di corazza (`armor`) e deve poter essere disegnata sullo schermo (`draw()`). Possiamo far implementare `Shape2d` all'astronave?

Perché un'Astronave NON è una `Shape2d`

Come rimarcato con enfasi dal docente durante la lezione: *"Le astronavi NON sono forme geometriche, sono astronavi!"*. Un'astronave non possiede un perimetro geometrico né un'area analitica. Se implementasse `Shape2d`, sarebbe costretta a definire metodi fittizi per `area()` e `perimeter()`, introducendo un anti-pattern distruttivo.

La soluzione architetturale corretta consiste nel definire un'interfaccia a grana più fine che descriva esclusivamente il protocollo di rendering:

Interfaccia comportamentale Drawable (Drawable.java)

```
package it.unibs.eps;

public interface Drawable {
    void draw();
}
```

L'interfaccia `Shape2d` estenderà a sua volta `Drawable`:

Snippet Java

```
public interface Shape2d extends Drawable {
    int perimeter();
    int area();
}
```

La classe `Spaceship` implementerà unicamente `Drawable`. Il motore grafico potrà così iterare su una collezione eterogenea di oggetti `Drawable` e invocarne il metodo `draw()` in modo uniforme e semanticamente ineccepibile.

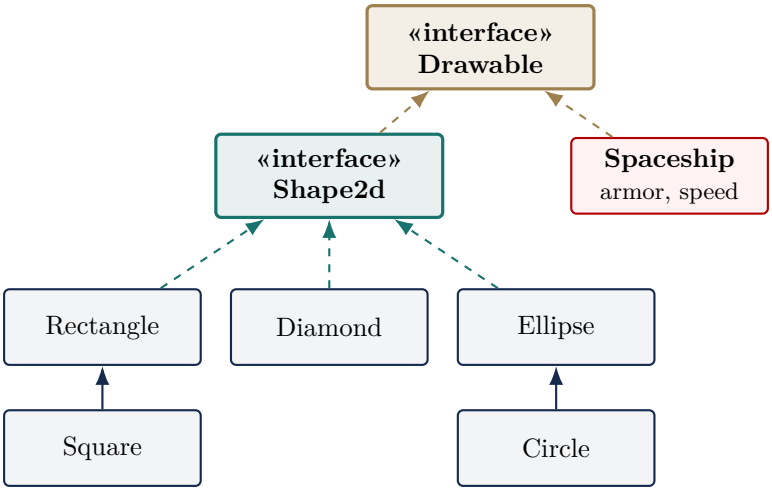


Figura 1.3: Diagramma architetturale completo: separazione semantica tra forme geometriche (`Shape2d`) e protocollo visivo (`Drawable`).

1.7 Determinazione dei Tipi: Compile-Time vs Run-Time

In un sistema a oggetti polimorfo, una variabile può possedere un tipo statico (dichiarato) più generale rispetto al tipo dinamico dell'istanza contenuta:

Controllo dei tipi con instanceof e Reflection

```
// Istanza reale: Square, Riferimento statico: Shape2d
Shape2d s = new Square(7);

/* 1. Controllo a tempo di compilazione tramite operatore instanceof */
if (s instanceof Shape2d) {
    int p = ((Shape2d) s).perimeter();
    System.out.printf("Perimetro calcolato: %d\n", p);
}

/* 2. Controllo dinamico a run-time tramite Reflection */
public boolean areObjectsAssignable(Object src, Object dest) {
    return dest.getClass().isInstance(src);
}
```

1.7.1 Enumerazioni e Switch Expression

Quando una variabile può assumere solo un insieme finito e prefissato di valori, la modellazione corretta in Java prevede l'uso delle enumerazioni (`enum`):

Dichiarazione di Enum (State.java)

```
public enum State { IDLE, RUNNING, PAUSED, DONE }
```

Un `enum` in Java è una classe a tutti gli effetti con un numero fisso di istanze immutabili, una per ciascuna costante: si confrontano in modo sicuro con `==`, si stampano con il proprio nome e mantengono un ordine naturale (`values()`).

Il caso d'uso tipico è una **macchina a stati**: per implementare la funzione di transizione tra stati, la moderna **Switch Expression** (introdotta stabilmente in Java 14) rappresenta lo strumento ideale.

Transizione di Stato con Switch Expression

```
state = switch (state) {
    case IDLE -> State.RUNNING;
    case RUNNING -> State.PAUSED;
    case PAUSED -> State.RUNNING;
    case DONE -> State.DONE;
};
```

La forma moderna offre tre vantaggi decisivi rispetto al vecchio `switch` ereditato dal C:

1. **Nessun fall-through accidentale**: la sintassi con la freccia (`->`) isola ogni ramo; non serve l'istruzione `break` e non si rischia di scivolare nel caso successivo;
2. **È un'espressione**: produce direttamente un valore che può essere assegnato a una variabile o restituito da un `return`, eliminando variabili temporanee mutabili;
3. **Esaustività verificata dal compilatore**: il compilatore verifica che tutti i valori dell'enum siano gestiti. Se in futuro viene aggiunto un nuovo stato all'enum e si dimentica di aggiornare lo switch, il programma rifiuta di compilare, segnalando immediatamente l'omissione.

Quando all'interno di un singolo caso sono necessarie più istruzioni, si racchiude il codice tra parentesi graffe e si produce il valore risultante tramite la parola chiave contestuale `yield`:

Blocco multi-istruzione con yield

```
String descrizione = switch (state) {
    case IDLE, DONE -> "fermo";
    case RUNNING -> "in corso";
    case PAUSED -> {
        System.out.println("[LOG] Macchina temporaneamente sospesa");
        yield "sospeso";
    }
};
```

Inoltre, poiché gli enum sono classi, ciascuna costante può avere campi propri, un costruttore (implicitamente privato) e metodi:

Enum con Stato e Comportamento (Operatore.java)

```
package it.unibs.pajc;

public enum Operatore {
    SOMMA("+"), SOTTRAZIONE("-"), PRODOTTO("*");

    private final String simbolo;

    Operatore(String simbolo) {
        this.simbolo = simbolo;
    }

    public String simbolo() {
        return simbolo;
    }
}
```

1.8 Il Java Collections Framework

Nei sistemi applicativi complessi, l'utilizzo degli array primitivi a dimensione fissa (`T[]`) risulta eccessivamente rigido. Java mette a disposizione il **Java Collections Framework** nel package `java.util`, una gerarchia unificata di interfacce e classi di supporto.

- `List`: elenchi ordinati di elementi che ammettono duplicati. L'accesso posizionale è garantito dall'indice.
  - `ArrayList`: basato su array dinamico ridimensionabile. Ottimo per accessi casuali $O(1)$, più lento negli inserimenti centrali $O(n)$. Non sincronizzato.
  - `Vector`: analogo ad `ArrayList`, ma con metodi sincronizzati (thread-safe). Presenta un sovraccarico prestazionale nei contesti single-thread.
- `Set`: collezioni matematiche di elementi distinti (duplicati non ammessi).
  - `HashSet`: tabella hash, tempo di inserimento e lookup medio $O(1)$, nessun ordinamento garantito.
  - `TreeSet`: albero rosso-nero, mantiene gli elementi rigidamente ordinati secondo l'ordinamento naturale (`Comparable`) o un `Comparator`. Complessità $O(\log n)$.
- `Map`: associazioni chiave-valore con chiavi univoche. Non estende `Collection`.
  - `HashMap`: tabella hash altamente efficiente.
  - `TreeMap`: albero ordinato per chiave.

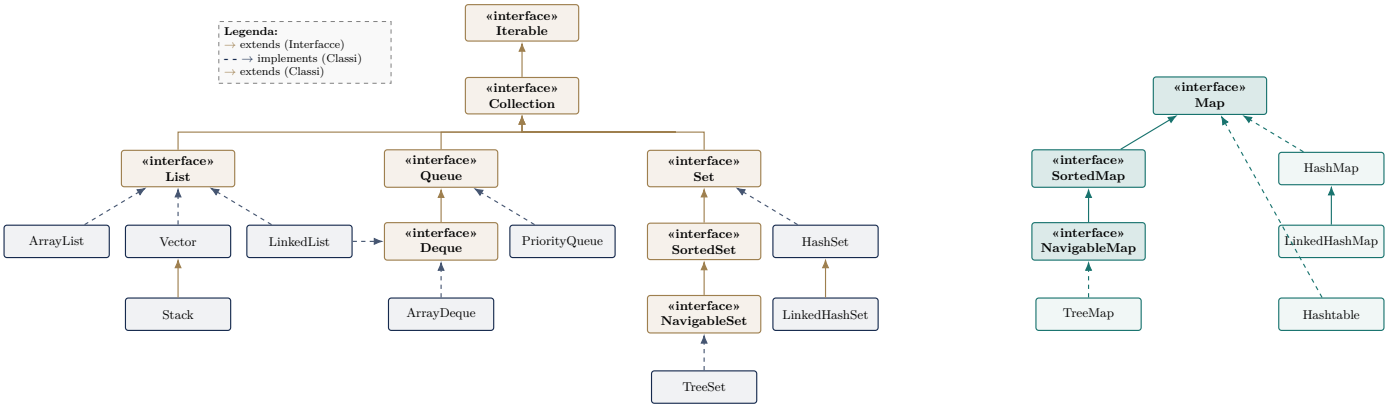


Figura 1.4: Tassonomia fondamentale del Java Collections Framework (le Mappe costituiscono una gerarchia indipendente da `Collection`).

Catalogazione con HashMap: Istogramma delle Forme

Il docente ha mostrato un pattern canonico per contare quante istanze di ciascuna classe siano presenti all'interno di una collezione eterogenea di `Shape2d`:

Conteggio e catalogazione tramite HashMap (MainCatalog.java)

```
package it.unibs.eps;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class MainCatalog {
    public static void main(String[] args) {
        List<Shape2d> listaOggetti = new ArrayList<>();
        listaOggetti.add(new Rectangle(5, 4));
        listaOggetti.add(new Square(6));
        listaOggetti.add(new Rectangle(2, 3));
        listaOggetti.add(new Square(9));
        listaOggetti.add(new Square(4));

        // Mappa per il conteggio: NomeClasse -> Frequenza
        Map<String, Integer> mappaOggetti = new HashMap<>();

        for (Shape2d s : listaOggetti) {
            String key = s.getClass().getSimpleName();
            if (mappaOggetti.containsKey(key)) {
                int counter = mappaOggetti.get(key);
                mappaOggetti.put(key, counter + 1);
            } else {
                mappaOggetti.put(key, 1);
            }
        }

        // Iterazione elegante sulle coppie chiave-valore
        for (Map.Entry<String, Integer> kv : mappaOggetti.entrySet()) {
            System.out.printf("%s: %d istanze registrate\n", kv.getKey(),
    kv.getValue());
        }
    }
}
```

1.9 Java I/O: Accesso al File System e Flussi di Dati

La gestione dell'Input/Output in Java è articolata su due livelli distinti: la gestione delle entità del file system (metadati) e il trasferimento effettivo dei byte/caratteri (stream).

1.9.1 La Classe `java.io.File`

La classe `File` rappresenta un'astrazione del percorso di file e cartelle. **Punto critico**: la classe `File` permette di interrogare gli attributi del file system (`exists()`, `isDirectory()`, `length()`, `mkdir()`, `delete()`), ma **non consente di leggere o scrivere il contenuto del file**.

1.9.2 Architettura degli Stream: Byte vs Caratteri

Per accedere al contenuto, Java adotta il pattern architetturale a flussi (*Stream*):

- **Stream Orientati ai Byte**: operano su sequenze grezze di dati a 8 bit.
  - `InputStream` / `OutputStream`: classi base (specializzate in `FileInputStream` e `FileOutputStream`).
  - `BufferedInputStream` / `BufferedOutputStream`: buffer di memoria intermedia per minimizzare le costose system call al disco.
  - `DataInputStream` / `DataOutputStream`: consentono la lettura/scrittura binaria diretta di tipi primitivi Java (`writeInt`, `readDouble`).
- **Stream Orientati ai Caratteri (Reader e Writer)**: operano su caratteri Unicode a 16 bit, gestendo automaticamente le tabelle di decodifica (es. UTF-8).
  - `FileReader` / `FileWriter`: aggancio ai file di testo.
  - `BufferedReader` / `BufferedWriter`: lettura ad alte prestazioni con buffer; `BufferedReader` introduce il metodo fondamentale `readLine()`.
  - `PrintWriter`: scrittura formattata ad alto livello (`printf`, `println`).

1.9.3 Pattern Robusto di Lettura e Scrittura Testuale

Metodi robusti per lettura e scrittura di file di testo (TextFileUtils.java)

```java
package it.unibs.eps;

import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;

public class TextFileUtils {

    /* Lettura completa riga per riga con numerazione */
    public static String read(String fname) {
        StringBuilder sb = new StringBuilder();
        // Idioma moderno try-with-resources per chiusura automatica
        try (BufferedReader in = new BufferedReader(new FileReader(fname))) {
            int lcount = 1;
            String s;
            while ((s = in.readLine()) != null) {
                sb.append(String.format("%3d| %s\n", lcount++, s));
            }
            return sb.toString();
        } catch (IOException ex) {
            throw new RuntimeException("Errore durante la lettura del file: " +
    fname, ex);
        }
    }

    /* Scrittura di una stringa su file di testo */
    public static void write(String fname, String body) {
        try (BufferedWriter out = new BufferedWriter(new FileWriter(fname))) {
            out.write(body);
        } catch (IOException ex) {
            throw new RuntimeException("Errore durante la scrittura del file: " +
    fname, ex);
        }
    }
}
```

1.9.4 I/O Binario vs I/O Testuale: DataInputStream, DataOutputStream ed Endianness

La distinzione tra file di testo e file binari riflette due filosofie di persistenza profondamente diverse:

- **File di Testo**: memorizzano informazioni destinate alla lettura umana mediante sequenze di caratteri codificati (tipicamente UTF-8). Un numero intero pari a `12345678` richiede 8 byte di testo (i caratteri ASCII `'1'`, `'2'`, ..., `'8'`). L'accesso richiede parsing tramite `Scanner` o `Integer.parseInt()`, con elevato overhead di CPU e consumo di spazio.
- **File Binari**: salvano direttamente la rappresentazione binaria compatta dei tipi di dato primitivi: un `int` a 32 bit occupa **sempre e solo 4 byte**, un `double` occupa esattamente 8 byte, e un valore booleano occupa 1 singolo byte.

Per la gestione dei dati binari strutturati, il package `java.io` fornisce le classi filtro `DataOutputStream` e `DataInputStream`:

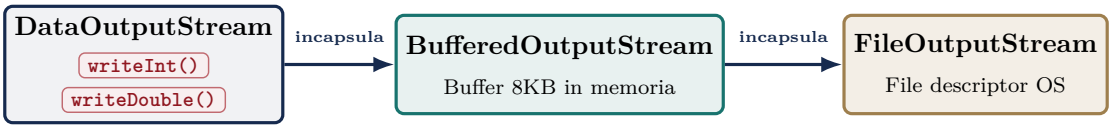


Figura 1.5: Composizione a strati dei filtri stream per I/O binario ad alte prestazioni.

Principio Architetturale: Portabilità del Byte Order: Lo Standard Big-Endian della JVM

Nei linguaggi a basso livello come C, la scrittura binaria di un intero a 32 bit con `fwrite(&val, sizeof(int), 1, fp)` riversa i byte nell'ordine imposto dall'architettura della CPU ospitante (generalmente *Little-Endian* sui processori x86/ARM, dove il byte meno significativo precede quello più significativo). Questo rende il file illeggibile o corrotto se trasferito su un'architettura differente.
In Java, la specifica della JVM impone che `DataOutputStream` e `DataInputStream` operino **tassativamente in formato Big-Endian** (*Network Byte Order*: il byte più significativo viene scritto per primo), indipendentemente dalla CPU fisica su cui gira il programma. Questo garantisce la totale portabilità binaria dei dati tra qualsiasi sistema operativo e architettura.

Scrittura e Lettura Binaria Deterministica (BinaryDataDemo.java)

```java
package it.unibs.pajc;

import java.io.*;

public class BinaryDataDemo {
    public static void salvaDati(String path, int id, double quota, String sigla)
    throws IOException {
        try (DataOutputStream out = new DataOutputStream(
                new BufferedOutputStream(new FileOutputStream(path)))) {
            out.writeInt(id);          // 4 byte esatti
            out.writeDouble(quota);    // 8 byte esatti
            out.writeUTF(sigla);       // Stringa binaria con prefisso di lunghezza a
    2 byte
        }
    }

    public static void caricaDati(String path) throws IOException {
        try (DataInputStream in = new DataInputStream(
                new BufferedInputStream(new FileInputStream(path)))) {
            int id = in.readInt();
            double quota = in.readDouble();
            String sigla = in.readUTF();
            System.out.printf("Letto record: ID=%d, Quota=%.2f, Sigla=%s\n", id,
    quota, sigla);
        }
    }
}
```

1.9.5 Il Meccanismo Try-With-Resources: AutoCloseable, Ordine LIFO e Suppressed Exceptions

Introdotto in Java 7, il costrutto **try-with-resources** costituisce il pilastro per la gestione sicura e deterministica delle risorse del sistema operativo (socket di rete, descrittori di file, lock fisici, connessioni database).

Definizione: Il Contratto dell'Interfaccia `java.lang.AutoCloseable`

Qualsiasi classe che implementi l'interfaccia `AutoCloseable` definisce un unico metodo formale:

`void close() throws Exception;`

La JVM garantisce che per qualsiasi risorsa istanziata nell'intestazione del **try**, il metodo `close()` verrà invocato automaticamente al termine del blocco protetto, sia che il blocco termini regolarmente, sia che si verifichi un'eccezione o un'istruzione di interruzione (**return**, **break**).

Principio Architetturale: Regola LIFO di Chiusura delle Risorse Annidate

Quando nell'intestazione del **try** vengono dichiarate più risorse separate da punto e virgola, la JVM garantisce che la loro chiusura avvenga in **ordine rigorosamente inverso rispetto alla dichiarazione** (*Last-In, First-Out*):

```
try (FileInputStream fis = new FileInputStream(file);        // 1. Aperto per primo
     BufferedInputStream bis = new BufferedInputStream(fis); // 2. Aperto per secondo
     DataInputStream dis = new DataInputStream(bis)) {       // 3. Aperto per terzo
    // Elaborazione dati...
} // Chiusura automatica garantita in ordine: dis.close() -> bis.close() -> fis.close()
```

Questo ordine previene errori di corruzione: lo stream più esterno esegue il flush del buffer prima che lo stream di basso livello sottostante venga chiuso.

Il Dramma Pre-Java 7 delle Eccezioni Soppresse (*Suppressed Exceptions*)

Nel codice legacy pre-Java 7, la chiusura delle risorse era affidata al blocco **finally**. Questo pattern nascondeva un vizio logico mortale:

1. Se il codice nel blocco **try** sollevava un'eccezione primaria (es. `DatiCorrottiException`);
2. E la successiva chiamata manuale a `in.close()` nel blocco **finally** sollevava a sua volta un'eccezione secondaria (es. `DiskFailureException`);
3. L'eccezione del **finally** **sovrascriveva e cancellava totalmente** l'eccezione primaria del blocco **try**, disperdendo per sempre la vera causa scatenante dell'errore.

Con il **try-with-resources**, questo problema è architetturalmente risolto: l'eccezione primaria del blocco **try** viene preservata e propagata, mentre l'eventuale eccezione sollevata durante la chiusura automatica di `close()` viene allegata come **eccezione soppressa**. Essa rimane accessibile tramite il metodo `Throwable.getSuppressed()`, consentendo diagnosi forensi ineccepibili.

1.10 Tipi Enumerativi (enum) e Macchine a Stati Finiti

Nel linguaggio C tradizionale, la keyword **enum** definisce semplici costanti intere associate a etichette mnemoniche, prive di sicurezza di tipo (un enum in C può essere assegnato a un qualunque intero).

In Java, gli **enum** sono veri e propri tipi ad oggetti di prima classe (*type-safe enum pattern*). Estendono implicitamente la classe `java.lang.Enum`, possono possedere costruttori, campi privati, metodi di istanza e interfacce.

1.10.1 Struttura Avanzata di un enum

Enum Java con stato interno e metodi (Colore.java)

```
package it.unibs.eps;

public enum Colore {
    RED("rosso brillante"),
    GREEN("verde foresta"),
```

```
    BLUE("blu notte");

    private final String descrizione;

    // Il costruttore di un enum e' sempre implicitamente privato
    Colore(String descrizione) {
        this.descrizione = descrizione;
    }

    public String getDescrizione() {
        return this.descrizione;
    }
}
```

1.10.2 Implementazione di una Macchina a Stati Finiti (FSM)

L'apice dell'espressività degli enumerativi in Java risiede nella possibilità di dichiarare un **metodo astratto** all'interno dell'enum e far sì che ogni singola costante ne fornisca un'implementazione specifica mediante una classe interna anonima. Questo consente di modellare **Macchine a Stati Finiti** in modo deterministico e compatto:

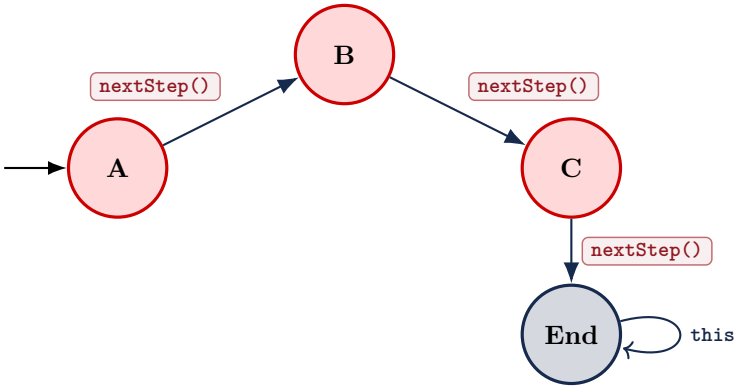


Figura 1.6: Diagramma di transizione degli stati della Macchina a Stati Finiti modellata con enum Java.

Macchina a stati finiti con pattern enum polimorfo (MacchinaStati.java)

```
package it.unibs.eps;

public enum MacchinaStati {
    A("Stato Iniziale - Alfa") {
        @Override
        public MacchinaStati nextStep() {
            return B;
        }
    },
    B("Stato di Elaborazione - Beta") {
        @Override
        public MacchinaStati nextStep() {
            return C;
        }
    },
    C("Stato di Verifica - Gamma") {
        @Override
        public MacchinaStati nextStep() {
            return End;
        }
    },
    End("Stato Finale - Terminato") {
        @Override
        public MacchinaStati nextStep() {
            return this; // Rimane nello stato terminale
        }
    };

```

```
    private final String desc;

    MacchinaStati(String desc) {
        this.desc = desc;
    }

    public String getDesc() {
        return this.desc;
    }

    /* Metodo astratto obbligatoriamente implementato da ogni stato */
    public abstract MacchinaStati nextStep();
}
```

Esecuzione del ciclo della macchina a stati (MainFSM.java)

```
package it.unibs.eps;

public class MainFSM {
    public static void main(String[] args) {
        MacchinaStati statoCorrente = MacchinaStati.A;

        System.out.println("=== Avvio Macchina a Stati ===");
        while (statoCorrente != MacchinaStati.End) {
            System.out.printf("Attivo: %s\n", statoCorrente.getDesc());
            statoCorrente = statoCorrente.nextStep();
        }
        System.out.printf("Raggiunto: %s\n", statoCorrente.getDesc());
        System.out.println("=== Esecuzione Conclusa ===");
    }
}
```

Principio Architetturale: Vantaggi Architetturali del Pattern FSM con Enum

Modellare una macchina a stati tramite enum garantisce:

- **Type Safety assoluta**: è impossibile transitare verso uno stato non contemplato dal sistema a tipi.
- **Incapsulamento della logica di transizione**: ogni stato è responsabile esclusivo della determinazione del proprio successore.
- **Assenza di switch/case complessi**: si elimina la proliferazione di costrutti `switch-case` procedurali, delegando il comportamento al polimorfismo interno delle costanti enumerative.

1.11 Studio di Caso d'Aula: Il Censimento del Bimondo in Stile Procedurale

A chiusura del modulo di Warm-up, il docente ha proposto una sintesi applicativa fondamentale: la catalogazione statistica di un universo popolato da forme geometriche eterogenee (il *Bimondo*).

Immaginiamo di disporre di una collezione di figure bidimensionali che implementano il contratto `Shape2d` e di voler determinare la frequenza di ciascuna classe concreta presente nel mondo:

Censimento del Bimondo: Approccio Procedurale con HashMap (BimondoProcedurale.java)

```
package it.unibs.eps;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class BimondoProcedurale {
    public static void main(String[] args) {
```

```
        // Popolamento del mondo geometrico eterogeneo
        List<Shape2d> mondo = new ArrayList<>();
        mondo.add(new Rectangle(4, 5));
        mondo.add(new Square(6));
        mondo.add(new Rectangle(2, 8));
        mondo.add(new Diamond(10, 6));
        mondo.add(new Square(3));
        mondo.add(new Square(7));

        // Mappa per il censimento delle frequenze: NomeClasse -> Conteggio
        Map<String, Integer> frequenze = new HashMap<>();
        double areaTotale = 0.0;

        for (Shape2d s : mondo) {
            String nomeClasse = s.getClass().getSimpleName();

            // Aggiornamento procedurale con getOrDefault
            int conteggioAttuale = frequenze.getOrDefault(nomeClasse, 0);
            frequenze.put(nomeClasse, conteggioAttuale + 1);

            // Accumulo procedurale dell'area complessiva
            areaTotale += s.area();
        }

        System.out.println("=== CENSIMENTO STATISTICO DEL BIMONDO ===");
        for (Map.Entry<String, Integer> entry : frequenze.entrySet()) {
            System.out.printf("Entita': %-12s | Rilevati: %d\n", entry.getKey(),
    entry.getValue());
        }
        System.out.printf("Area Totale del Mondo: %.2f\n", areaTotale);
    }
}
```

Il Punto di Partenza per la Rivoluzione Funzionale

Questo algoritmo procedurale, sebbene corretto, richiede la dichiarazione di variabili di stato mutevoli (`areaTotale`), una mappa esplicita e controlli manuali sulle chiavi. Nel Capitolo 3 vedremo come questa intera elaborazione statistica verrà compressa in un'unica ed elegante pipeline dichiarativa tramite le Stream API e i collettori `Collectors.groupingBy`.

Consultazione Esercizi di Laboratorio sul Modulo 1

Per consolidare l'architettura delle classi, la gestione della memoria Stack vs Heap, il contratto di `equals()` ed `hashCode()`, la manipolazione dei file binari e le macchine a stati finiti, si raccomanda di consultare e svolgere gli esercizi dettagliati nell'**Appendice A**:

- **Esercizio A.1 (StringMagic)**: incapsulamento, manipolazione stringhe e metodologia TDD;
- **Esercizio A.1 (FileUtil)**: I/O a caratteri e hex dump a byte con `try-with-resources`;
- **Esercizio A.1 (CollectionStats)**: statistiche descrittive su collezioni generiche;
- **Esercizio A.1 (Gerarchia Geometrica)**: separazione tra `Shape2d` e protocollo visivo `Drawable`;
- **Esercizio A.1 (Macchina a Stati Finiti)**: implementazione pattern State mediante enum polimorfi;
- **Esercizio A.1 (Polinomio Incapsulato)**: algebra simbolica e rispetto del contratto `equals`/`hashCode`;
- **Esercizio A.1 (MiniFileSystemSimulator)**: gerarchia composita multi-file (Composite Pattern).

Capitolo 2

Dal Nome alla Funzione: Classi Anonime, Lambda, Method Reference e Nuove Interfacce

In questo capitolo affrontiamo un problema classico della programmazione a oggetti: quando un programma cresce, spesso ci troviamo a creare decine di piccole classi che servono una sola volta (*classi usa-e-getta*). Questo riempie il progetto di file minuscoli e codice ripetitivo (*boilerplate code*).

L'obiettivo didattico è imparare a trattare i **comportamenti come entità di prima classe**: non vogliamo creare per forza un oggetto intero solo per eseguire un'azione; vogliamo poter passare direttamente una funzione a un metodo, memorizzarla in una mappa o restituirla come risultato di un calcolo.

2.1 Il Problema delle Classi Usa-e-Getta e del Boilerplate

Immaginiamo di progettare il motore (*engine*) di una calcolatrice per numeri interi. La calcolatrice riceve due numeri interi e un simbolo di operazione (ad esempio `"+"`, `"-"`, `"*"`, `"^"`), e deve calcolare il risultato corrispondente.

Nell'approccio classico a oggetti, separiamo nettamente la definizione dell'interfaccia dalle implementazioni concrete:

1. Definiamo un'interfaccia con il contratto per gli operatori binari;
2. Creiamo una classe separata per ogni singola operazione aritmetica.

Interfaccia di Contratto: BinaryOperator.java

```
public interface BinaryOperator {
    int eval(int a, int b);
}
```

Implementazione Tradizionale: Somma.java

```
public class Somma implements BinaryOperator {
    @Override
    public int eval(int a, int b) {
        return a + b;
    }
}
```

Allo stesso modo dovremmo creare i file `Sottrazione.java`, `Moltiplicazione.java` e `Potenza.java`. Poi, nel motore della calcolatrice, colleghiamo i simboli alle rispettive operazioni usando una `Map`:

Configurazione del Motore con Mappa di Operatori

```
Map<String, BinaryOperator> operatori = new HashMap<>();
operatori.put("+", new Somma());
operatori.put("-", new Sottrazione());
operatori.put("*", new Moltiplicazione());
operatori.put("^", new Potenza());

BinaryOperator op = operatori.get("+");
System.out.println(op.eval(3, 4)); // Stampa: 7
```

Vantaggi e Difetti di Questo Approccio

Dal punto di vista dell'architettura software, questo schema è formalmente corretto: rispetta il principio *Open/Closed* perché per aggiungere un nuovo operatore (ad esempio il modulo `"%"`) non dobbiamo modificare il codice della calcolatrice, ma solo registrare una nuova coppia chiave-valore.
Tuttavia, il costo di scrittura e manutenzione è davvero esagerato:

- Dobbiamo creare un file sorgente separato per ogni singola operazione di una riga;
- Dobbiamo ripetere ogni volta la struttura `public class ... implements BinaryOperator`;
- Di ogni classe creiamo **un solo oggetto in tutto il ciclo di vita del programma**, che registriamo nella mappa e di cui non useremo mai più il nome specifico;
- Il nome della classe (`Somma`) non serve a nulla nel resto del codice, perché la calcolatrice interagisce solo tramite l'interfaccia `BinaryOperator` e il metodo `eval(int, int)`.

Questo è un chiaro esempio di **boilerplate code**: codice cerimoniale richiesto dalla sintassi del linguaggio, che però non aggiunge alcun reale valore logico.

2.2 Classi Anonime: Sintassi, Uso e Limiti

Per evitare di creare un file separato per ogni classe usata una sola volta, Java 1.1 ha introdotto le **classi anonime** (*anonymous inner classes*).

Definizione: Classe Anonima

Una classe anonima è una classe priva di nome che viene dichiarata e istanziata nello stesso punto del codice in cui serve. La sintassi fonde l'allocazione tramite `new` con la definizione del corpo della classe:

`new Tipo(argomenti) { corpo della classe }`

dove `Tipo` è l'interfaccia da implementare oppure la classe da estendere.

Applicando questo costrutto alla nostra calcolatrice, possiamo scrivere l'operazione di somma direttamente sul posto:

Dichiarazione e Istanziazione di una Classe Anonima

```
1 BinaryOperator somma = new BinaryOperator() {
2     @Override
3     public int eval(int a, int b) {
4         return a + b;
5     }
6 };
```

Come Leggere Questa Sintassi

Questa istruzione può sembrare insolita a prima vista per tre motivi:

1. `new BinaryOperator()`: sembra che stiamo istanziando un'interfaccia (cosa vietata in Java). In realtà stiamo creando un'istanza di una nuova classe anonima che *implementa* quell'interfaccia. Le parentesi tonde vuote indicano che stiamo chiamando il costruttore predefinito di `Object`.
2. Le parentesi graffe `{ ... }` contengono il corpo della classe, dove possiamo implementare il metodo `eval(int, int)`.
3. Il punto e virgola finale `;` chiude l'istruzione di assegnamento: l'intera espressione produce il riferimento all'oggetto appena creato nello heap.

2.2.1 Caratteristiche e Limitazioni delle Classi Anonime

Una classe anonima è una vera e propria classe Java, ma per il fatto di non avere un nome ha regole ben precise:

Cosa può fare	**Spiegazione**
Dichiarare campi e metodi ausiliari	Può avere campi d'istanza e metodi privati di supporto, oltre a quelli richiesti dall'interfaccia.
Inizializzare lo stato	Può usare blocchi di inizializzazione (`{ ... }`) al posto del costruttore.
Accedere al contesto esterno	Può accedere a tutti i membri della classe contenitore e alle variabili locali del metodo circostante.
Cosa NON può fare	**Motivazione**
Nessun costruttore esplicito	In Java il costruttore deve avere lo stesso nome della classe; senza nome, non si può dichiarare un costruttore.
Nessun riutilizzo	Non avendo un nome, non possiamo invocare di nuovo `new` per creare un'altra istanza altrove.
Una sola interfaccia o classe base	Non può implementare più interfacce contemporaneamente, né estendere una classe e implementare un'interfaccia insieme.

Tabella 2.1: Capacità e limitazioni pratiche delle classi anonime in Java.

Cosa Succede nel Bytecode

Anche se nel codice sorgente non hanno nome, a livello di file compilati (`.class`) il compilatore assegna alle classi anonime un nome numerato progressivo: `Calcolatrice$1.class`, `Calcolatrice$2.class`, ecc. Questo meccanismo genera file fisici su disco e carica le classi a runtime tramite classloader.

2.2.2 Il Riferimento `this` nelle Classi Anonime

Un dettaglio importante riguarda la parola chiave `this`:

- **Nelle classi anonime**: `this` si riferisce **all'istanza della classe anonima stessa**. Se vogliamo riferirci all'oggetto della classe esterna, dobbiamo scrivere in modo qualificato: `ClasseEsterna.this`.
- **Nelle lambda expression** (come vedremo a breve): la lambda non crea un nuovo ambito per `this`; all'interno di una lambda, `this` indica sempre l'oggetto della classe esterna in cui ci troviamo.

2.3 Closure e il Modello di Memoria: La Regola di "Effectively Final"

Uno dei punti di forza delle classi definite all'interno di un metodo è la possibilità di **catturare le variabili locali** del metodo stesso (*closure*).

Consideriamo questo metodo che crea un operatore moltiplicato per un fattore di scala k:

Metodo Factory che Cattura una Variabile Locale

```
public static BinaryOperator sommaScalata(int k) {
    return new BinaryOperator() {
        @Override
        public int eval(int a, int b) {
            return k * (a + b); // k e' il parametro del metodo esterno!
        }
    };
}
```

Se usiamo questo metodo memorizzando l'operatore ottenuto:

Uso Differito dell'Operatore Creato

```
BinaryOperator operatoreDoppio = sommaScalata(2);
System.out.println(operatoreDoppio.eval(3, 4)); // Calcola: 2 * (3 + 4) = 14
```

Definizione: Closure (Chiusura)

Una **closure** è una funzione (o blocco di codice) che porta con sé l'ambiente in cui è stata creata, conservando l'accesso alle variabili locali del metodo circostante anche dopo che quel metodo ha terminato l'esecuzione.

2.3.1 Cosa Succede in Memoria: La Copia per Valore

Se osserviamo come viene eseguito il codice, notiamo una situazione particolare:

1. La variabile k risiedeva sullo **stack** della chiamata `sommaScalata(2)`;
2. Quando il metodo esegue il `return`, il suo stack frame viene deallocato e distrutto;
3. Più tardi, quando chiamiamo `eval(3, 4)`, il codice riesce comunque a leggere $k = 2$ e calcolare 14. Come fa?

La JVM risolve questo dilemma con una **copia per valore**: durante la creazione dell'oggetto nello heap, il compilatore crea un campo privato nascosto (ad esempio `final int val$k`) all'interno dell'istanza e vi copia dentro il valore di k.

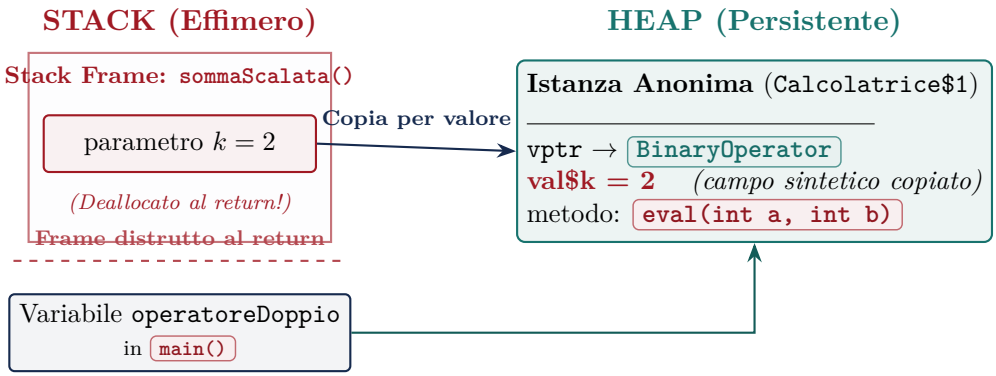


Figura 2.1: Modello di memoria della Closure: la variabile locale dello stack viene copiata in un campo privato dell'oggetto nello heap.

2.3.2 Perché le Variabili Devono Essere "Effectively Final"

Dato che la JVM lavora su una copia, capiamo subito il perché di una regola fondamentale di Java:

Definizione: Variabile Effectively Final

Una variabile locale (o parametro) si dice **effectively final** se dopo la sua inizializzazione non viene mai più modificata. In pratica, se aggiungessimo la parola chiave `final` prima del suo tipo, il programma continuerebbe a compilare senza errori.

Principio Architetturale: Regola sulle Variabili Catturate

Una classe anonima o una lambda expression può accedere alle variabili locali del metodo circostante solo se queste sono dichiarate `final` oppure sono **effectively final**.

Cosa succederebbe se Java permettesse di modificare la variabile? Supponiamo per assurdo che Java consentisse di modificare una variabile locale dopo averla catturata:

Esempio Errato: Violazione di Effectively Final

```
int contatore = 0;
Runnable r = new Runnable() {
    @Override
    public void run() {
        contatore++; // ERRORE DI COMPILAZIONE: contatore non e' effectively final!
    }
};
contatore = 10;
```

Se questo codice funzionasse, esisterebbero **due copie separate di `contatore`**:

- La variabile locale originale sullo stack del metodo;
- Il campo clonato `val$contatore` dentro l'oggetto nello heap.

Se il metodo scrivesse `contatore = 10`, la copia nello heap rimarrebbe ferma a 0. Se la lambda facesse `contatore++`, aggiornerebbe solo la copia nello heap lasciando lo stack invariato. I due valori si di-

sallineerebbero immediatamente. Per evitare queste incongruenze, Java impone che il dato catturato sia immutabile.

Come Condividere uno Stato Modificabile

Se abbiamo davvero bisogno di modificare un valore all'interno di una lambda o classe anonima, possiamo separare il contenitore dal contenuto tramite due pattern comuni:

1. **Array di un solo elemento**: scriviamo `final int[] contatore = {0};` e poi facciamo `contatore[0]++;`. Il riferimento all'array resta immutabile (`final`), mentre la cella al suo interno può cambiare valore liberamente.
2. **Classi atomiche**: usiamo classi dedicate della libreria concorrente come `AtomicInteger` (con il metodo `incrementAndGet()`), che garantiscono mutabilità sicura anche con più thread.

2.4 Interfacce Funzionali e l'Annotazione `@FunctionalInterface`

Prima di passare alla sintassi delle lambda, chiariamo il requisito necessario affinché Java capisca come interpretarle:

Definizione: Interfaccia Funzionale (SAM – Single Abstract Method)

Un'interfaccia si definisce **funzionale** se dichiara **esattamente un unico metodo astratto**. Nel calcolo di questo conteggio:

- I metodi con implementazione `default` **non contano**;
- I metodi `static` **non contano**;
- Eventuali dichiarazioni di metodi pubblici di `Object` (come `equals`) **non contano**, perché ogni classe li eredita già.

Perché serve un solo metodo astratto? Se l'interfaccia ha un solo metodo astratto da implementare, scrivere il codice della classe equivale semplicemente a **definire il corpo di quell'unico metodo**. Il compilatore conosce già il nome del metodo, i parametri attesi e il tipo di ritorno: non serve riscriverli ogni volta.

Interfaccia Funzionale Annotata

```
@FunctionalInterface
public interface BinaryOperator {
    int eval(int a, int b);
}
```

L'Utilità di `@FunctionalInterface`

L'annotazione non cambia il comportamento del codice a runtime, ma svolge due ruoli protettivi importanti:

1. **Documentazione chiara**: comunica subito a chi legge che l'interfaccia è pensata per essere usata con le lambda.
2. **Controllo del compilatore**: se in futuro qualcuno prova ad aggiungere per sbaglio un secondo metodo astratto all'interfaccia, il compilatore segnala subito un errore, impedendo di rompere le lambda già scritte nel progetto.

2.5 Lambda Expression: Sintassi, Target Typing ed Efficienza

Riprendiamo la classe anonima vista in precedenza:

La Classe Anonima Tradizionale

```
BinaryOperator somma = new BinaryOperator() {
    public int eval(int a, int b) {
        return a + b;
    }
};
```

Se analizziamo questo codice togliendo tutto ciò che è scontato:

- Il tipo `BinaryOperator` è già scritto a sinistra dell'uguale;
- Il metodo da implementare è per forza `eval`, l'unico disponibile;
- I tipi dei parametri (`int`, `int`) sono già definiti nel contratto;
- Le uniche due cose davvero nuove scritte dallo sviluppatore sono i **nomi dei due argomenti** (`a, b`) e la **formula di calcolo** (`a + b`).

Definizione: Lambda Expression

Una **lambda expression** è una funzione anonima scritta in forma compatta:

`(parametri) -> corpo`

che a runtime produce un'istanza di un'interfaccia funzionale.

Possiamo semplificare la classe anonima in due passaggi:

I Due Formati delle Lambda

```
// 1. Forma con blocco (Block Lambda con graffe e return esplicito):
BinaryOperator sommaBlocco = (a, b) -> { return a + b; };

// 2. Forma a espressione singola (Expression Lambda, senza graffe ne' return):
BinaryOperator somma = (a, b) -> a + b;
```

2.5.1 Guida Rapida alla Sintassi delle Lambda

In base al numero di parametri e alle istruzioni da eseguire, la sintassi si adatta per essere il più leggibile possibile:

- **Nessun parametro**: usiamo parentesi tonde vuote obbligatorie: `() -> 42`;
- **Un solo parametro**: possiamo omettere le parentesi tonde: `x -> x * 2`;
- **Due o più parametri**: le parentesi tonde sono obbligatorie: `(x, y) -> x + y`;
- **Tipi espliciti**: se vogliamo indicare i tipi per chiarezza, dobbiamo specificarli per tutti i parametri: `(int x, int y) -> x + y`;
- **Più istruzioni**: se il corpo contiene più righe, usiamo graffe e `return` esplicito: `x -> { log(x); return x * 2; }`.

2.5.2 Come il Compilatore Capisce il Tipo: Il Target Typing

Se scriviamo semplicemente l'espressione isolata `(a, b) -> a + b`, che tipo ha in Java?

Principio Architetturale: Principio di Target Typing

In Java, una lambda expression **non ha un tipo proprio autonomo**. Il suo tipo viene dedotto dal contesto in cui si trova, cioè dal tipo dell'interfaccia funzionale attesa in quel punto (**Target Typing**).

La stessa identica espressione `(a, b) -> a + b` può assumere tipi diversi a seconda dell'assegnamento:

Stessa Lambda, Tipi Bersaglio Differenti

```
// Assume il tipo della nostra interfaccia custom (metodo astratto: eval)
BinaryOperator op1 = (a, b) -> a + b;

// Assume il tipo standard di java.util.function (metodo astratto: applyAsInt)
IntBinaryOperator op2 = (a, b) -> a + b;
```

Da questo principio derivano due conseguenze pratiche:

1. **Non possiamo usare `var` da solo con una lambda**: Scrivere `var f = (a, b) -> a + b;` genera un **errore di compilazione**. Con `var` chiediamo al compilatore di dedurre il tipo dall'espressione a destra; ma l'espressione a destra non ha un tipo proprio e aspetta di riceverlo da sinistra. Senza un punto di riferimento, la compilazione fallisce.

2. **Serve un metodo astratto univoco**: Se un'interfaccia avesse due metodi astratti con la stessa firma, il compilatore non saprebbe quale dei due stiamo implementando con la lambda.

2.5.3 Differenze Sostanziali tra Lambda e Classi Anonime

Una lambda non è solo una classe anonima con una sintassi più corta:

1. **Comportamento di `this`**: Dentro una lambda, `this` fa riferimento all'oggetto della classe esterna in cui ci troviamo. Non crea un nuovo livello di scoping.
2. **Nessun mascheramento di variabili**: I parametri della lambda non possono avere lo stesso nome di variabili locali già presenti nel metodo (altrimenti il compilatore segnala un errore di nome duplicato). Una classe anonima, invece, crea un vero scope separato e può mascherare i nomi esterni.
3. **Prestazioni ed efficienza (Bytecode)**: Le classi anonime creano un file `.class` su disco per ogni classe e allocano sempre nuovi oggetti nello heap. Le lambda expression, invece, usano l'istruzione speciale di bytecode `invokedynamic` (introdotta da Java 7/8): la classe di supporto viene generata a runtime solo al primo utilizzo in modo molto più leggero ed efficiente.

2.6 Method Reference: Le Quattro Categorie Canoniche

Molto spesso il corpo di una lambda si limita a chiamare un metodo già esistente, passando semplicemente gli argomenti ricevuti:

Lambda di Semplice Inoltro

```
Consumer<String> stampa = s -> System.out.println(s);
```

In questi casi la lambda non fa nessun calcolo: fa solo da intermediario. Con i **Method Reference** possiamo fare a meno di riscrivere i parametri, indicando direttamente il nome del metodo preesistente.

Definizione: Method Reference

Un **method reference** è una sintassi compatta nella forma `qualificatore::nomeMetodo` che punta a un metodo o costruttore esistente. Quando la funzione viene eseguita, i parametri attesi dall'interfaccia funzionale vengono inoltrati automaticamente al metodo indicato.

La riga precedente diventa quindi:

Sostituzione con Method Reference

```
Consumer<String> stampa = System.out::println;
```

2.6.1 Le Quattro Forme di Method Reference

I method reference si dividono in quattro categorie a seconda di cosa scriviamo a sinistra dei doppi due punti (`::`):

- **1. Metodo d'istanza su un oggetto specifico (`oggetto::metodo`)**
  L'oggetto su cui chiamare il metodo è già noto e fissato in anticipo. Tutti i parametri della lambda diventano gli argomenti del metodo.
  Esempio: `System.out::println` ≡ `x -> System.out.println(x)`
- **2. Metodo statico di una classe (`Classe::metodoStatico`)**
  Non c'è nessun oggetto su cui chiamare il metodo; i parametri della lambda vengono passati direttamente come argomenti al metodo statico.
  Esempi: `Math::max` ≡ `(a, b) -> Math.max(a, b)`;
  `Integer::sum` ≡ `(a, b) -> Integer.sum(a, b)`
- **3. Metodo d'istanza su un oggetto arbitrario (`Classe::metodoDiIstanza`)**
  A sinistra scriviamo il nome della classe, ma il metodo referenziato è un metodo d'istanza.
  Come funziona: il **primo parametro** della lambda funge da bersaglio (*receiver*) su cui viene chiamato il metodo, mentre gli altri parametri diventano gli argomenti della chiamata.
  Esempio con un parametro: `String::trim` ≡ `s -> s.trim()`
  Esempio con due parametri: `String::compareTo` ≡ `(s, t) -> s.compareTo(t)`

- **4. Riferimento a costruttore (`Classe::new`)**
  Punta al costruttore della classe per creare una nuova istanza al momento opportuno.
  Esempio: `ArrayList::new` ≡ `() -> new ArrayList<>()`

Consultazione Rapida delle Sintassi

Per un promemoria rapido di tutte le quattro categorie con le relative equivalenze lambda prima dell'esame o del laboratorio, consulta la **Scheda dei Method Reference in Appendice B.2**.

Capire a Fondo la Categoria 3: Il Primo Parametro come Ricevente

La terza categoria è utilissima ogni volta che dobbiamo elaborare oggetti di una collezione. Ad esempio, quando dobbiamo confrontare due stringhe in un `Comparator<String>`, il comparatore riceve due stringhe (s, t). Scrivendo semplicemente:

`String::compareTo`

il compilatore capisce che la prima stringa s è l'oggetto su cui chiamare il metodo, e la seconda stringa t è il parametro da passargli: `(s, t) -> s.compareTo(t)`. Il codice diventa pulitissimo e si legge in modo naturale.

2.7 I Quattro Livelli di Astrazione a Confronto

Mettiamo ora a confronto i quattro modi che Java ci offre per realizzare la stessa identica operazione di somma all'interno del motore della calcolatrice:

La Scala Evolutiva: Quattro Modi per Scrivere la Stessa Azione

```java
// 1. Classe concreta separata (Java 1.0)
operatori.put("+", new Somma());

// 2. Classe anonima scritta sul posto (Java 1.1)
operatori.put("+", new BinaryOperator() {
    @Override
    public int eval(int a, int b) { return a + b; }
});

// 3. Lambda expression (Java 8)
operatori.put("+", (a, b) -> a + b);

// 4. Method reference (Java 8)
operatori.put("+", Integer::sum);
```

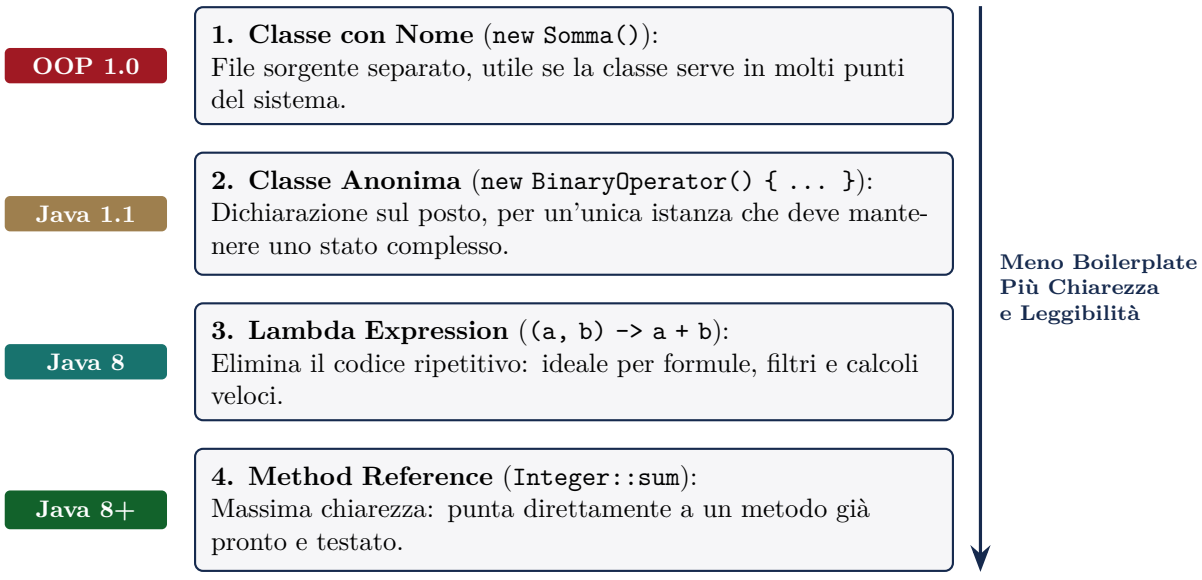


Figura 2.2: I quattro livelli di astrazione per implementare un comportamento in Java.

Principio Architetturale: Quale Forma Scegliere nella Pratica?

Per orientarsi nella scrittura del codice quotidiano:

1. **Scegli il Method Reference** quando il codice deve solo chiamare un metodo già esistente senza logica in più;
2. **Scegli la Lambda Expression** (meglio se su una sola riga senza graffe) quando devi fare un calcolo, combinare valori o applicare un piccolo filtro;
3. **Scegli la Classe Anonima** quando l'interfaccia ha più di un metodo astratto (non è SAM), oppure se devi gestire campi interni mutabili o ridefinire metodi di `Object`;
4. **Scegli una Classe Ordinaria con Nome** quando la classe deve essere riutilizzata in più parti del programma o fa parte dell'architettura pubblica dell'applicazione.

2.8 Iniezione di Comportamento e Inversione del Controllo

Oltre a scrivere meno codice, il vero salto di qualità delle lambda è l'**Iniezione di Comportamento** (*Behavior Parameterization*): la possibilità di passare un algoritmo o un'azione come parametro a un altro metodo.

Guardiamo questi due metodi:

Due Metodi con la Stessa Struttura di Ciclo

```
static void stampaInteri(int n) {
    for (int i = 0; i < n; i++)
        System.out.println(i);
}

static void stampaPari(int n) {
    for (int i = 0; i < n; i++)
        if (i % 2 == 0) System.out.println("pari: " + i);
}
```

La struttura del ciclo è identica: contiamo da 0 a $n-1$. L'unica cosa che cambia è *cosa facciamo dentro al ciclo ad ogni passaggio*. Nel codice tradizionale non potevamo unificare questi due metodi, perché potevamo passare come argomenti solo **dati**, non **azioni**.

Con un'interfaccia funzionale possiamo invece passare l'azione come argomento:

Unico Metodo che Riceve l'Azione da Eseguire

```
static void ripeti(int n, IntConsumer azione) {
    for (int i = 0; i < n; i++)
        azione.accept(i);
}
```

Ora i due metodi precedenti si scrivono in una riga:

Iniezione Diretta dell'Azione

```
ripeti(5, i -> System.out.println(i));
ripeti(5, i -> { if (i % 2 == 0) System.out.println("pari: " + i); });
```

I Cinque Scenari Tipici dell'Esecuzione Differita

Passare una funzione a un metodo non significa eseguirla all'istante: significa consegnare un blocco di codice a qualcun altro perché lo esegua **al momento giusto**, **in futuro** e **potenzialmente più volte**. Questo meccanismo è alla base di tutto il Java moderno:

1. **Esecuzione ripetuta**: metodi come `ripeti(n, azione)`;
2. **Collezioni e Stream**: passare un criterio di ordinamento a `list.sort()`, o un filtro a una pipeline di Stream;
3. **Eventi grafici o di rete**: collegare un'azione a un bottone (`bottone.addActionListener(e -> ...)`) che scatterà solo quando l'utente fa click;

4. **Valutazione ritardata (*Lazy Evaluation*)**: calcolare un messaggio di log pesante solo se il livello di log (es. DEBUG) è effettivamente attivo;
5. **Programmazione concorrente**: avviare un task asincrono in background su un nuovo thread con `new Thread(() -> calcolo()).start()`.

2.9 Il Package `java.util.function` e la Composizione

Per evitare che ogni sviluppatore debba creare da zero una propria interfaccia per ogni combinazione di parametri, la libreria standard di Java offre nel package `java.util.function` un insieme di interfacce pronte all'uso.

2.9.1 I Cinque Ruoli Fondamentali

Le interfacce standard si organizzano attorno a cinque concetti base:

- `Supplier<T>` (**Fornitore / Produttore**)
  Non riceve nessun parametro e produce un valore di tipo T tramite il metodo `get()`.
  Uso tipico: fabbriche di oggetti, generatori casuali: `() -> new Random()`.
- `Consumer<T>` (**Consumatore**)
  Riceve un elemento di tipo T, compie un'azione (effetto collaterale) e non restituisce nulla (`void`) tramite il metodo `accept(t)`.
  Uso tipico: operazioni di stampa o salvataggio su database: `s -> System.out.println(s)`.
- `Function<T, R>` (**Trasformatore / Mappatore**)
  Riceve un valore di tipo T e lo converte in un valore di tipo R tramite il metodo `apply(t)`.
  Uso tipico: estrazione di proprietà, conversioni di tipo: `s -> s.length()`.
- `Predicate<T>` (**Verificatore / Filtro**)
  Riceve un elemento di tipo T e restituisce un booleano tramite il metodo `test(t)`.
  Uso tipico: condizioni di ricerca o filtri su liste: `s -> s.isEmpty()`.
- `UnaryOperator<T>` e `BinaryOperator<T>` (**Operatori Omogenei**)
  Casi particolari di funzioni in cui i parametri e il valore di ritorno appartengono tutti allo stesso tipo T:
  - `UnaryOperator<T>`: prende un T e restituisce un T (`s -> s.toUpperCase()`);
  - `BinaryOperator<T>`: prende due elementi (T, T) e restituisce un T (`(a, b) -> a + b`).

Esistono inoltre le versioni a due argomenti: `BiFunction<T, U, R>`, `BiConsumer<T, U>` e `BiPredicate<T, U>`.

Scheda di Consultazione Completa del Formulario

La lista completa di tutte le interfacce del package, comprese le specializzazioni per tipi primitivi e tutti i metodi default di composizione, è disponibile in **Appendice B.2 (Catalogo Completo di java.util.function)**.

2.9.2 Tipi Primitivi e Zero-Boxing Overhead

In Java i generici funzionano solo con le classi (`Object`). Se usassimo `Function<Integer, Integer>`, la JVM dovrebbe continuamente trasformare i tipi da `int` a `Integer` (*autoboxing*) e viceversa (*unboxing*), creando decine di piccoli oggetti temporanei nello heap e rallentando il programma.

Per questo motivo, la libreria fornisce versioni dedicate ai tipi primitivi (`int`, `long`, `double`):

- `IntConsumer`, `LongConsumer`, `DoubleConsumer`;
- `IntPredicate`, `LongPredicate`, `DoublePredicate`;
- `IntFunction<R>`, `ToIntFunction<T>`;
- `IntUnaryOperator`, `IntBinaryOperator`.

La nostra interfaccia didattica con firma `int eval(int a, int b)` corrisponde alla classe standard di libreria `IntBinaryOperator`.

Attenzione ai Nomi Duplicati

Se nel codice importiamo tutto il package con `import java.util.function.*;`, il tipo standard `BinaryOperator` entra in conflitto con la nostra interfaccia didattica con lo stesso nome. Nei progetti reali basta usare il nome completo oppure usare direttamente `IntBinaryOperator`.

2.9.3 Comporre Funzioni e Predicati

Le interfacce funzionali standard non si limitano al loro metodo principale: mettono a disposizione comodi metodi per collegare più funzioni insieme in una catena logica:

Composizione di Predicati e Funzioni

```
// 1. Composizione di Predicati (AND, OR, NOT)
Predicate<String> nonVuota = ((Predicate<String>) String::isEmpty).negate();
Predicate<String> corta = s -> s.length() < 5;
Predicate<String> valida = nonVuota.and(corta); // Deve essere non vuota E corta

// 2. Concatenazione di Funzioni con andThen
Function<String, String> pulisciSpazi = String::trim;
Function<String, Integer> contaLettere = String::length;
Function<String, Integer> elabora = pulisciSpazi.andThen(contaLettere);

System.out.println(elabora.apply("   ciao mondo   ")); // Stampa: 10
```

Con `andThen` la seconda operazione viene eseguita dopo la prima, consentendo di costruire passaggi di trasformazione lineari e chiari.

2.10 Il Motore della Calcolatrice Riscritto e Completo

Tornando al problema iniziale della calcolatrice, vediamo come diventa compatto ed elegante il codice combinando lambda e method reference:

Motore della Calcolatrice: Motore.java

```
import java.util.HashMap;
import java.util.Map;

public class Motore {
    private final Map<String, BinaryOperator> operatori = new HashMap<>();

    public Motore() {
        // Registrazione semplice e immediata degli operatori
        operatori.put("+", (a, b) -> a + b);
        operatori.put("-", (a, b) -> a - b);
        operatori.put("*", (a, b) -> a * b);
        operatori.put("^", (a, b) -> (int) Math.pow(a, b));
        operatori.put("max", Math::max); // Method reference statico!
    }

    public int calcola(String simbolo, int a, int b) {
        BinaryOperator op = operatori.get(simbolo);
        if (op == null) {
            throw new IllegalArgumentException("Operatore sconosciuto: " + simbolo);
        }
        return op.eval(a, b);
    }
}
```

Abbiamo mantenuto tutti i vantaggi architetturali della mappa di operatori, ma abbiamo azzerato il codice inutile: non ci sono più file sparsi per ogni operazione, e aggiungere una nuova funzione richiede una sola riga nel costruttore.

2.11 L'Evoluzione delle Interfacce: Metodi Default, Static e Private

Fino a Java 7, un'interfaccia poteva contenere solo firme di metodi astratti (senza corpo) e costanti. A partire da Java 8 e 9, le interfacce si sono arricchite con tre nuovi strumenti:

Elemento	Introdotto	Scopo Principale	Regola
Metodi Astratti	Da sempre	Definiscono il contratto che le classi devono rispettare.	Obbligatori da implementare.
Metodi `default`	Java 8	Forniscono un'implementazione già pronta ereditabile dalle classi.	Ereditati automaticamente, sovrascrivibili.
Metodi `static`	Java 8	Metodi di supporto o factory legati al tipo dell'interfaccia.	Chiamati tramite `Interfaccia.metodo()`.
Metodi `private`	Java 9	Condividono codice comune tra più metodi default interni.	Invisibili all'esterno dell'interfaccia.

Tabella 2.2: Gli elementi di un'interfaccia in Java moderno.

2.11.1 Perché Sono Nati i Metodi Default

I metodi `default` sono stati introdotti principalmente per garantire la **retrocompatibilità** delle librerie esistenti.

Quando con Java 8 è nata la libreria degli Stream, gli ingegneri volevano che tutte le collezioni potessero usare il metodo `forEach(Consumer)`. Tuttavia, `forEach` doveva essere aggiunto all'interfaccia base `Iterable`.

Se fosse stato aggiunto come normale metodo astratto, **milioni di classi già scritte in tutto il mondo che implementavano `Iterable` avrebbero smesso istantaneamente di compilare**. Grazie alla parola chiave `default`, l'interfaccia ha potuto fornire un'implementazione standard con un normale ciclo `for`; tutte le classi esistenti hanno così ereditato il nuovo metodo senza dover modificare una sola riga di codice.

Esempio Completo di Interfaccia con Default, Static e Private

```
public interface Shape2d {
    double area();
    double perimeter();

    // Metodo statico: factory per creare un quadrato (Java 8)
    static Shape2d square(double side) {
        return new Rectangle(side, side);
    }

    // Metodo default: ereditato automaticamente da tutte le forme (Java 8)
    default String describe() {
        return name() + " | area=" + area() + " | perimetro=" + perimeter();
    }

    // Metodo privato: supporto interno condiviso (Java 9)
    private String name() {
        return getClass().getSimpleName();
    }
}
```

Qualsiasi classe che implementa `Shape2d` (come `Rectangle` o `Circle`) riceve subito il metodo `describe()` senza sforzo, pur mantenendo la libertà di farne l'override se vuole personalizzarlo.

2.12 Risoluzione dei Conflitti tra Metodi Default

Dato che una classe Java può implementare più interfacce, e ciascuna interfaccia può contenere metodi `default`, sorge spontanea una domanda: *cosa succede se due interfacce hanno un metodo default con lo stesso nome e gli stessi parametri?*

Due Interfacce con lo Stesso Metodo Default

```
interface Drawable {
    default String describe() { return "Entita Disegnabile"; }
}
```

```java

interface Shape2d {
    default String describe() { return "Forma Geometrica 2D"; }
}
```

Se una classe le implementa entrambe:

Conflitto da Risolvere

```java
class Circle implements Shape2d, Drawable {
    // ERRORE DI COMPILAZIONE SE NON SCEGLIAMO QUALE USARE!
}
```

2.12.1 Le Tre Regole della JVM per Risolvere i Conflitti

Per evitare ogni ambiguità, il compilatore Java applica tre regole in ordine di priorità:

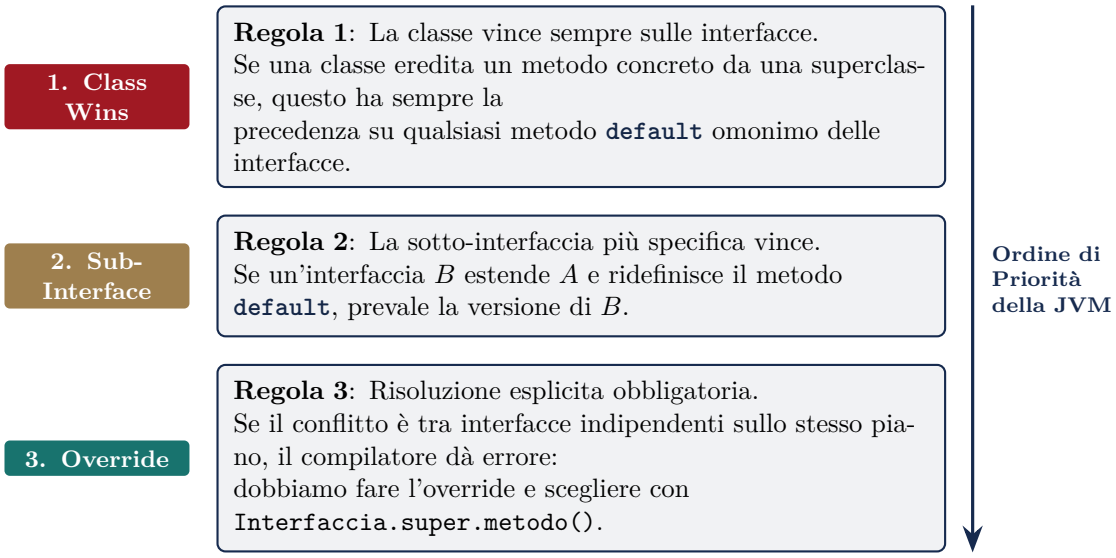


Figura 2.3: Le tre regole per risolvere i conflitti di metodi default in Java.

Nel nostro esempio con `Circle`, le due interfacce sono indipendenti (Regola 3). Dobbiamo quindi scrivere noi l'override:

Risoluzione Esplicita nel Cerchio

```java
class Circle implements Shape2d, Drawable {
    @Override
    public String describe() {
        // Possiamo scegliere una delle due, oppure combinarle entrambe
        return Shape2d.super.describe() + " (anche " + Drawable.super.describe() +
    ")";
    }
}
```

2.12.2 Metodi Statici e Metodi Privati nelle Interfacce (Java 8 e Java 9)

L'evoluzione delle interfacce verso componenti modulari sempre più ricchi è stata completata da due ulteriori introduzioni:

- **Metodi Statici (da Java 8)**: consentono di definire metodi di utilità e *factory method* direttamente all'interno dell'interfaccia correlata, senza dover istanziare classi helper esterne con costruttore privato (come la classica classe `Collections` affiancata a `Collection`). Un metodo statico in un'interfaccia:
  - Non fa parte dell'istanza e **non viene ereditato** dalle classi che implementano l'interfaccia;
  - Deve essere invocato usando esplicitamente il nome dell'interfaccia: `Shape2d.creaRettangolo(w, h)`.
- **Metodi Privati (da Java 9)**: quando più metodi `default` all'interno della stessa interfaccia necessitano di condividere frammenti complessi di logica algoritmica o controlli di validazione degli argomenti, si

creava in passato il dilemma di dover duplicare il codice oppure esporre un metodo ausiliario come pubblico nell'API contrattuale. Da Java 9, un'interfaccia può dichiarare metodi **`private`** (e **`private static`**), visibili unicamente all'interno dell'interfaccia stessa.

Metodi Default, Statici e Privati in un'Interfaccia (OperatoreNumerico.java)

```java
package it.unibs.eps;

public interface OperatoreNumerico {
    double calcola(double x);

    // Metodo default pubblico che sfrutta un metodo privato
    default double calcolaConLog(double x) {
        validaInput(x);
        double res = calcola(x);
        logOperazione("calcola", x, res);
        return res;
    }

    // Metodo statico factory pubblico
    static OperatoreNumerico identita() {
        return x -> x;
    }

    // Metodo privato di istanza (Java 9): fattorizza la validazione
    private void validaInput(double x) {
        if (Double.isNaN(x) || Double.isInfinite(x)) {
            throw new IllegalArgumentException("Input non valido: " + x);
        }
    }

    // Metodo privato statico (Java 9): ausiliario non esposto all'esterno
    private static void logOperazione(String op, double in, double out) {
        System.out.printf("[LOG %s] in=%.2f -> out=%.2f\n", op, in, out);
    }
}
```

2.13 Contratti, Classi Astratte e i Due Volti dell'Ordinamento

2.13.1 Interfaccia o Classe Astratta? Differenze e Scelte Architetturali

Nel capitolo precedente abbiamo definito l'interfaccia `Shape2d` per modellare figure geometriche aventi un'area e un perimetro. Un'interfaccia, nella sua forma fondamentale, rappresenta un **contratto puro**: un elenco formale di metodi che una classe promette di implementare. Chi utilizza un riferimento a `Shape2d` ignora volutamente quale classe concreta abbia davanti: è proprio questa *ignoranza voluta* che permette di aggiungere nuove forme al sistema senza modificare una singola riga del codice che le manipola (principio *Open/Closed*).

Tuttavia, sorge spontanea una domanda classica della progettazione ad oggetti: *quando conviene usare un'interfaccia e quando invece una classe astratta?*

Classe Astratta con Stato Condiviso: AbstractShape.java

```java
public abstract class AbstractShape implements Shape2d {
    private final String name; // Stato condiviso (campo privato)

    protected AbstractShape(String name) {
        this.name = name;
    }

    public String name() {
        return name; // Implementazione concreta condivisa
    }

    // area() e perimeter() rimangono astratti: ogni forma li calcola a modo suo
    @Override public abstract double area();
    @Override public abstract double perimeter();
```

```
}
```

Principio Architetturale: Criterio di Scelta: Contratto vs Stato Condiviso

I due costrutti rispondono a esigenze architetturali profondamente distinte:

- **La Classe Astratta** serve quando più classi derivate devono condividere **stato** (campi di istanza) e una radice comune di implementazione (costruttori, metodi di servizio protetti). Poiché in Java l'ereditarietà delle classi è rigorosamente **singola**, una classe può estendere una sola classe astratta.
- **L'Interfaccia** serve per definire una **capacità o protocollo comportamentale** che classi tassonomicamente lontanissime possono possedere (es. `Drawable`, `Comparable`, `AutoCloseable`). Una classe può implementare un numero arbitrario di interfacce.

Regola Aurea di Progettazione: Si definisca *sempre* il contratto fondamentale come interfaccia. Se utile, si affianchi a parte una classe astratta che ne fornisca un'implementazione parziale di supporto (come fa la Java Standard Library con `List` e `AbstractList`).

L'Anti-Pattern delle Costanti in Interfaccia

In Java le variabili dichiarate in un'interfaccia sono implicitamente `public static final`. In passato si usava creare interfacce prive di metodi al solo scopo di raggruppare costanti, per poi "implementarle" nelle classi applicative e avere i nomi a portata di mano. Questo schema (*Constant Interface Pattern*) è oggi considerato un grave anti-pattern architetturale: inquina l'API pubblica della classe ed espone dettagli implementativi interni. Le costanti devono risiedere nella classe che le utilizza, in una classe di utilità non istanziabile con costruttore privato, oppure all'interno di un `enum`.

2.13.2 Due Modi di Ordinare: Comparable vs Comparator

Ordinare una lista di forme geometriche richiede di stabilire quando un elemento "viene prima" di un altro. La libreria standard di Java offre due contratti distinti per formalizzare questo concetto:

I Due Contratti Fondamentali di Ordinamento

```
// 1. Ordinamento Naturale: implementato dalla classe stessa
public interface Comparable<T> {
    int compareTo(T other); // < 0 se this precede other, 0 se pari, > 0 se this segue
}

// 2. Ordinatore Estrinseco: oggetto esterno che confronta due istanze
public interface Comparator<T> {
    int compare(T a, T b);  // < 0 se a precede b, 0 se pari, > 0 se a segue b
}
```

- `Comparable<T>` definisce l'**ordinamento naturale** della classe, unico e intrinseco all'oggetto stesso. Una classe che implementa `Comparable<Rectangle>` dichiara qual è il suo criterio primario di confronto (ad esempio l'area):

Implementazione di Comparable

```
public class Rectangle implements Shape2d, Comparable<Rectangle> {
    private final double w, h;
    public Rectangle(double w, double h) { this.w = w; this.h = h; }
    public double width() { return w; }
    public double height() { return h; }
    @Override public double area() { return w * h; }
    @Override public double perimeter() { return 2 * (w + h); }

    @Override
    public int compareTo(Rectangle other) {
        return Double.compare(this.area(), other.area()); // Ordinamento naturale
    per area
    }
}
```

- `Comparator<T>` è invece un **oggetto esterno di confronto**: possono esisterne quanti se ne vuole (per area crescente, per perimetro decrescente, per larghezza, per nome alfabetico), e vengono scelti al momento dell'invocazione di `sort()`:

Uso di Comparable e Comparator

```
List<Rectangle> rs = new ArrayList<>(List.of(new Rectangle(3, 4), new Rectangle(1,
    10)));

Collections.sort(rs); // Sfrutta l'ordinamento naturale compareTo() (per area)

// Ordinamento ad-hoc per perimetro tramite lambda:
rs.sort((a, b) -> Double.compare(a.perimeter(), b.perimeter()));
```

Definizione: Il Contratto Matematico di un Ordinamento

Qualsiasi implementazione di `compareTo` o `compare` deve soddisfare tassativamente le proprietà matematiche di un ordine totale:

1. **Antisimmetria**: $\text{sgn}(compare(a,b)) == -\text{sgn}(compare(b,a))$.
2. **Transitività**: se $compare(a,b) > 0$ e $compare(b,c) > 0$, allora deve valere $compare(a,c) > 0$.
3. **Riflessività / Coerenza**: $compare(a,a) == 0$ per ogni a.

Coerenza con equals: È vivamente raccomandato che $(compare(a,b) == 0) \iff a.equals(b)$. Se questa coerenza viene violata, le collezioni ordinate (`TreeSet`, `TreeMap`), che basano l'unicità degli elementi esclusivamente sul comparatore e ignorano del tutto `equals()`, mostreranno comportamenti anomali rispetto alle collezioni standard (`HashSet`, `HashMap`). Ad esempio, in un `TreeSet` ordinato solo per area, due rettangoli distinti 2×6 e 3×4 verrebbero considerati duplicati e il secondo verrebbe scartato!

Principio Architetturale: L'Errore Fatale della Sottrazione nei Comparatori

Un errore comune tra i programmatori alle prime armi è implementare il confronto tramite sottrazione numerica:

```
return (int) (a.area() - b.area()); // ERRORE GRAVE!
```

Questo approccio fallisce in due modi disastrosi:

1. **Troncamento Floating-Point**: se $a.area() = 0.5$ e $b.area() = 0.2$, la differenza 0.3 convertita a `int` viene troncata a 0, dichiarando i due oggetti uguali pur avendo aree differenti.
2. **Overflow Numerico su Interi**: per i numeri interi, se $a = -2 \cdot 10^9$ e $b = 2 \cdot 10^9$, l'operazione $a - b$ va in underflow positivo, invertendo il segno del confronto e violando l'antisimmetria.

Soluzione Standard: Si utilizzi sempre e solo `Double.compare(a, b)` per numeri reali e `Integer.compare(a, b)` per numeri interi.

2.13.3 Comparator Moderno: Metodi Statici, Combinatori e Null-Safety

Grazie alle lambda e ai metodi default introdotti in Java 8, `Comparator<T>` è diventato il modello di riferimento per le API fluide e componibili della piattaforma Java:

- **Factory Statica** `Comparator.comparing(Function<T, U> keyExtractor)`: estrae la chiave di confronto e costruisce il comparatore senza scrivere lambda complesse;
- **Inversione** `reversed()`: inverte l'ordine corrente in modo dichiarativo;
- **Concatenazione** `thenComparing(Comparator)`: gestisce i criteri secondari a parità del primo;
- **Gestione dei Null**: `Comparator.nullsFirst(comp)` e `Comparator.nullsLast(comp)` per trattare elementi nulli senza sollevare `NullPointerException`.

Esempio Avanzato: Ordinamento Multi-Livello Fluido

```
List<Shape2d> forme = new ArrayList<>();

// Ordinamento su tre livelli:
// 1. Prima per area crescente;
```

```
// 2. A parita' di area, per perimetro decrescente;
// 3. A parita' di perimetro, per nome di classe alfabetico.
forme.sort(
    Comparator.comparing(Shape2d::area)
              .thenComparing(Comparator.comparing(Shape2d::perimeter).reversed())
              .thenComparing(s -> s.getClass().getSimpleName())
);
```

2.14 Gerarchie Chiuse: Tipi Sigillati (Sealed) e Switch sui Tipi

Nella programmazione a oggetti classica, le gerarchie sono aperte: chiunque può creare una nuova sottoclasse di una classe non `final`.

Tuttavia, in molti casi reali il numero di varianti possibili è limitato e noto fin dall'inizio (ad esempio i messaggi di un protocollo di rete, le figure geometriche gestite o gli stati di una macchina a stati). Prima di Java 17, l'unico modo per avere un insieme chiuso era usare una `enum`, che però vincola a semplici istanze costanti e non permette alle singole varianti di avere campi e dati differenti.

Con Java 17 arrivano le **classi e interfacce sigillate** (`sealed`):

Definizione: Interfaccia Sigillata (Sealed Interface)

Un'interfaccia sigillata dichiara esplicitamente, tramite la parola chiave `permits`, l'elenco tassativo delle sole classi che hanno il permesso di implementarla.

Gerarchia Sigillata con permits

```
public sealed interface Shape2d permits Rectangle, Circle, Diamond {
    double area();
    double perimeter();
}

// Ogni sottoclasse permessa DEVE dichiararsi in uno di questi tre modi:
// 1) final (non puo' essere ulteriormente estesa)
// 2) sealed (limita a sua volta le sue sottoclassi)
// 3) non-sealed (permette a chiunque di estenderla)

public final class Rectangle implements Shape2d {
    private final double w, h;
    public Rectangle(double w, double h) { this.w = w; this.h = h; }
    public double width() { return w; }
    public double height() { return h; }
    @Override public double area() { return w * h; }
    @Override public double perimeter() { return 2 * (w + h); }
}

public final class Circle implements Shape2d {
    private final double r;
    public Circle(double r) { this.r = r; }
    public double radius() { return r; }
    @Override public double area() { return Math.PI * r * r; }
    @Override public double perimeter() { return 2 * Math.PI * r; }
}

public final class Diamond implements Shape2d {
    private final double d1, d2;
    public Diamond(double d1, double d2) { this.d1 = d1; this.d2 = d2; }
    public double d1() { return d1; }
    public double d2() { return d2; }
    @Override public double area() { return (d1 * d2) / 2.0; }
    @Override public double perimeter() { return 2.0 * Math.hypot(d1, d2); }
}
```

2.14.1 Pattern Matching e Switch Esaustivo (Java 21)

Il grande vantaggio dei tipi sigillati si apprezza nello `switch` sui tipi introdotto in Java 21.

Poiché il compilatore conosce in anticipo tutte le implementazioni ammesse, può verificare che il nostro `switch` le gestisca tutte (**esaustività**):

Switch sui Tipi Senza Ramo Default

```
public static String formattaFigura(Shape2d s) {
    return switch (s) {
        case Rectangle r -> "Rettangolo " + r.width() + "x" + r.height();
        case Circle c    -> "Cerchio di raggio " + c.radius();
        case Diamond d   -> "Rombo con diagonali " + d.d1() + "x" + d.d2();
    }; // NESSUN DEFAULT NECESSARIO: copre tutti i casi possibili!
}
```

Pattern Matching con Guardie Condizionali (when) in Java 21

```
public static String formattaFiguraConGuardia(Shape2d s) {
    return switch (s) {
        // La clausola when raffina il caso senza rompere l'esaustivita':
        case Rectangle r when r.width() == r.height() ->
            "Quadrato perfetto speciale di lato " + r.width();
        case Rectangle r ->
            "Rettangolo ordinario " + r.width() + "x" + r.height();
        case Circle c when c.radius() > 10.0 ->
            "Grande cerchio di raggio " + c.radius();
        case Circle c ->
            "Cerchio ordinario di raggio " + c.radius();
        case Diamond d ->
            "Rombo con diagonali " + d.d1() + "x" + d.d2();
    };
}
```

Perché Non Mettere il Ramo Default è una Sicurezza

L'assenza della clausola `default` è una garanzia preziosa:

- Se mettessimo un ramo `default`, aggiungendo in futuro una nuova forma (es. `Triangle`), il programma continuerebbe a compilare e la nuova forma finirebbe zitta nel ramo generico, magari con bug invisibili;
- Senza ramo `default`, appena aggiungiamo `Triangle` alla clausola `permits`, **il compilatore ci segnala subito tutti i punti dell'applicazione dove dobbiamo gestire la nuova forma**, impedendo dimenticanze.

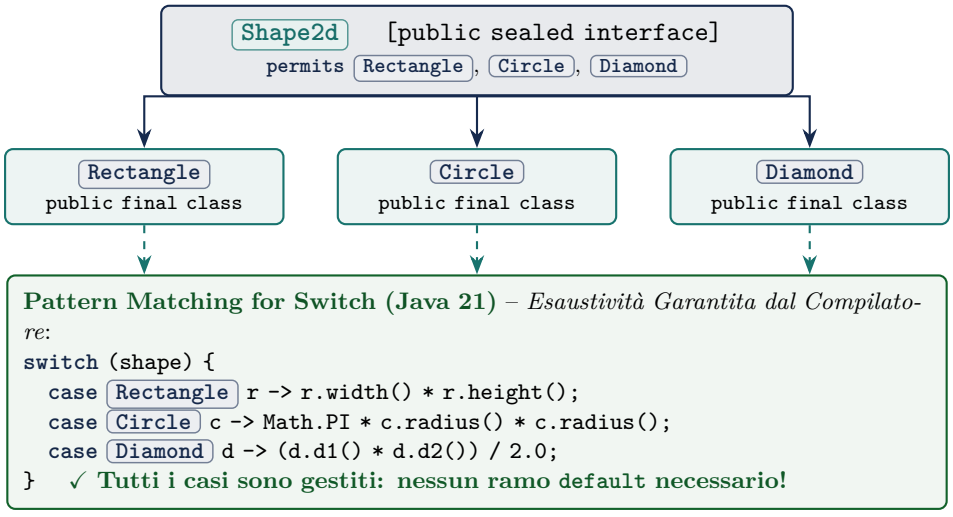


Figura 2.4: Gerarchia sigillata (`sealed`) e pattern matching: il compilatore verifica che tutti i casi siano gestiti senza clausola di fallback.

2.14.2 Confronto Sintetico: Gerarchie Aperte vs Gerarchie Chiuse

Possiamo riassumere le differenze tra i due approcci:

Caratteristica	**Gerarchia Aperta (Polimorfismo Tradizionale)**	**Gerarchia Chiusa (Sealed / Tipi Algebrici)**
Chi aggiunge tipi?	Chiunque, in qualsiasi punto o libreria esterna.	Solo chi controlla e dichiara l'interfaccia base con `permits`.
Aggiungere un'operazione	Richiede di aggiungere il metodo all'interfaccia e modificarne tutte le classi.	Molto facile: basta scrivere una funzione con uno `switch` sui tipi.
Aggiungere un nuovo tipo	Molto facile: basta implementare l'interfaccia senza toccare le classi esistenti.	Richiede di aggiornare la clausola `permits` e tutti gli `switch` dell'applicazione.
Quando usarla?	Per plugin, librerie estensibili e framework aperti.	Per insiemi chiusi: messaggi di rete, comandi, stati di un protocollo, token di un parser.

Tabella 2.3: Confronto pratico tra gerarchie aperte e gerarchie sigillate.

Consultazione Esercizi di Laboratorio sul Modulo 2

Per consolidare l'uso delle classi anonime, il refactoring funzionale verso lambda expression e method reference, la composizione fluida di comparatori null-safe e le gerarchie sigillate con pattern matching, si consultino gli esercizi dettagliati nell'**Appendice A**.

Capitolo 3

Stream Computazionali: Pipeline, Valutazione Pigra e Collettori

In questo capitolo esploriamo una delle innovazioni più radicali introdotte in Java a partire dalla versione 8: le **Stream API** e il paradigma di elaborazione dichiarativo e funzionale delle sequenze di dati.

Fino a Java 7, qualsiasi operazione di filtraggio, trasformazione o aggregazione su collezioni di dati richiedeva un approccio procedurale rigido, basato su cicli espliciti (`for`, `while`, `for-each`), variabili di stato provvisorie e liste ausiliarie. Questo stile non solo produce decine di righe di codice ripetitivo (*boilerplate code*), ma espone il programmatore a gravi trappole semantiche a tempo di esecuzione, prime fra tutte la modifica concorrente di una collezione durante la sua iterazione.

Con le Stream API, il paradigma si capovolge: il programmatore non descrive più *come* iterare fisicamente sui dati passo dopo passo (iterazione esterna), ma dichiara *cosa* vuole ottenere attraverso una sequenza fluida di trasformazioni (iterazione interna), demandando alla libreria sottostante l'ottimizzazione dell'accesso alla memoria, la valutazione differita (*laziness*) e l'eventuale parallelizzazione trasparente del calcolo.

3.1 Il Problema del Paradigma Procedurale e della Modifica Concorrente

Per comprendere il valore fondante degli stream, partiamo da un problema apparentemente banale: data una lista di stringhe che rappresentano nomi di città o parole del vocabolario, vogliamo selezionare solo quelle che iniziano con una determinata lettera (ad esempio la lettera `"C"`), convertirle in lettere maiuscole e scartare tutte le altre.

3.1.1 L'Anti-Pattern delle Liste Ausiliarie

Nell'approccio procedurale classico, il programmatore istanzia collezioni temporanee e accumula manualmente i risultati intermedi:

Approccio Procedurale con Liste di Appoggio

```
List<String> parole = Arrays.asList("como", "milano", "cremona", "brescia", "cantu",
    "bergamo");

// 1. Prima lista di accumulo: filtro per iniziale
List<String> filtrate = new ArrayList<>();
for (String p : parole) {
    if (p.startsWith("c")) {
        filtrate.add(p);
    }
}

// 2. Seconda lista di accumulo: trasformazione in maiuscolo
List<String> maiuscole = new ArrayList<>();
for (String p : filtrate) {
    maiuscole.add(p.toUpperCase());
}

System.out.println("Risultato: " + maiuscole);
```

Questo codice presenta diversi difetti strutturali:

1. **Prolissità e rumore visivo**: abbiamo impiegato oltre 15 righe per un'operazione concettualmente immediata. L'intento logico ("filtra per C e rendi maiuscolo") è sepolto sotto dettagli di allocazione memoria (`new ArrayList<>()`), chiamate manuali a `add()` e variabili temporanee di ciclo.
2. **Spreco di memoria e cache**: vengono allocate due strutture dati ridondanti (`filtrate` e `maiuscole`) che vivono nell'heap solo per trasportare elementi da un ciclo al successivo, sovraccaricando inutilmente il Garbage Collector.

3. **Mancanza di componibilità**: se volessimo ordinare il risultato o estrarre solo i primi due elementi, dovremmo annidare altri cicli o invocare metodi statici esterni (`Collections.sort()`), frammentando ulteriormente il flusso logico.

3.1.2 La Trappola della Modifica durante l'Iterazione

Una tentazione frequente per evitare la lista di appoggio è quella di rimuovere direttamente gli elementi indesiderati dalla lista originale all'interno di un ciclo `for-each`:

Errore Fatale: Rimozione in Ciclo for-each

```
List<String> citta = new ArrayList<>(Arrays.asList("como", "milano", "cremona",
    "brescia"));

// TENTATIVO ERRATO: lancia ConcurrentModificationException a runtime!
for (String c : citta) {
    if (!c.startsWith("c")) {
        citta.remove(c); // ERRORE CRITICO!
    }
}
```

L'Analogia dei Gradini della Scala

Come sottolineato con enfasi a lezione dal docente, il ciclo `for-each` di Java è un costrutto sintattico (*syntactic sugar*) che la JVM compila istanziando un oggetto `Iterator`. L'iteratore mantiene un cursore interno che punta all'indice corrente e un contatore di modifiche attese (`expectedModCount`).
Se invochiamo `citta.remove()` direttamente sulla collezione anziché sull'iteratore, modifichiamo la struttura sottostante all'insaputa dell'iteratore stesso. Alla successiva chiamata di `it.next()`, l'iteratore rileva una discrepanza tra `modCount` ed `expectedModCount` e lancia immediatamente una `ConcurrentModificationException`.
L'analogia didattica d'aula è memorabile: *"Iterare su una collezione mentre la si modifica strutturalmente è come salire una scala mentre qualcuno, gradino dopo gradino, sega i pioli sotto i vostri piedi. Prima o poi mettete il piede nel vuoto e precipitate."*
Per cancellare elementi in sicurezza nel mondo procedurale pre-Java 8, bisognava recuperare l'iteratore esplicito e chiamare esclusivamente `it.remove()`, raddoppiando ancora una volta il boilerplate.

3.2 Il Paradigma degli Stream come Flusso Computazionale

Definizione: Stream di Dati (`java.util.stream.Stream<T>`)

Uno **Stream** è una sequenza di elementi emessi da una sorgente dati che supporta operazioni aggregate sequenziali o parallele. A differenza di una collezione, uno stream **non memorizza elementi in memoria**, non possiede indici di accesso casuale e non altera la struttura originaria da cui ha origine (*non-interfering*). Uno stream è un canale computazionale effimero attraverso cui i dati fluiscono, vengono elaborati e infine consumati o raccolti.

Principio Architetturale: Differenza Ontologica: Collezione vs Stream

- Una **Collezione** è una struttura dati in memoria che **contiene** elementi. Tutti gli elementi sono già presenti contemporaneamente nell'heap prima che la collezione possa essere interrogata.
- Uno **Stream** è una struttura di computazione che **trasporta e trasforma** elementi su richiesta. I singoli elementi possono essere calcolati al volo (*on-demand*) o prelevati da sorgenti potenzialmente infinite.

3.2.1 L'Analogia delle Palline nel Tubo

Per visualizzare mentalmente il funzionamento di una pipeline di stream, il docente propone la metafora meccanica del tubo e delle palline:

- **La Sorgente (*Source*)**: è una tramoggia o un serbatoio iniziale (una `List`, un array, un file su disco, un generatore numerico) che lascia cadere le palline nel tubo una alla volta.

- **I Setacci e le Pompe (*Intermediate Operations*)**: all'interno del tubo sono collocate griglie di selezione (`filter()`) che fanno passare solo le palline con determinate caratteristiche scartando le altre, oppure meccanismi di verniciatura o deformazione (`map()`) che trasformano il colore o la forma della pallina mentre questa viaggia.
- **Il Bacino di Raccolta (*Terminal Operation*)**: all'estremità inferiore del tubo c'è un secchio o un misuratore finale. Finché non apriamo la valvola terminale o non posizioniamo il secchio, nessuna pallina cade dalla tramoggia iniziale.

Ecco come il problema iniziale delle città viene risolto con un'unica, elegante pipeline di stream:

Risoluzione Dichiarativa con le Stream API

```
List<String> parole = Arrays.asList("como", "milano", "cremona", "brescia", "cantu",
    "bergamo");

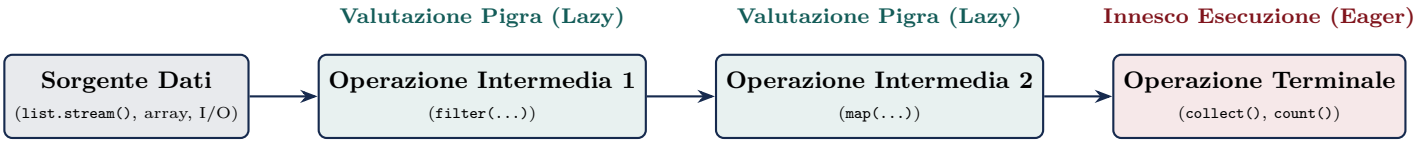
List<String> risultato = parole.stream()
    .filter(s -> s.startsWith("c"))
    .map(String::toUpperCase)
    .toList(); // oppure .collect(Collectors.toList())

System.out.println("Risultato: " + risultato); // [COMO, CREMONA, CANTU]
```

Non abbiamo creato alcuna lista ausiliaria esplicita, non ci siamo preoccupati di indici o iteratori e il codice si legge esattamente come le specifiche in linguaggio naturale: *"Dalla lista estrai uno stream, filtra gli elementi che iniziano con c, trasformali in maiuscolo e raccoglili in una nuova lista immutabile"*.

3.3 Anatomia di una Pipeline e Valutazione Pigra (Laziness)

Qualsiasi elaborazione basata su Stream è strutturata rigorosamente come una **pipeline** composta da tre fasi sequenziali:



1. **Sorgente (*Source*)**: fornisce gli elementi iniziali. Può essere finita (una collezione, un array) o teoricamente infinita (generatori algoritmici, stream da socket di rete).
2. **Operazioni Intermedie (*Intermediate Operations*)**: accettano uno stream in ingresso e restituiscono un nuovo stream trasformato. Possono essere concatenate a catena in numero arbitrario $(0, 1, 2, \dots)$. Sono sempre **lazy** (pigre): non eseguono alcun calcolo nel momento in cui vengono dichiarate, ma si limitano a costruire un grafo descrittivo della computazione.
3. **Operazione Terminale (*Terminal Operation*)**: chiude la pipeline, innesca la valutazione effettiva dei dati ed emette un risultato scalare (`long`, `boolean`, `Optional`) o una collezione (`List`, `Set`, `Map`), oppure consuma gli elementi per produrre un effetto collaterale (`forEach()`). Una volta invocata l'operazione terminale, lo stream è definitivamente **consumato** e non può più essere utilizzato.

3.3.1 Dimostrazione Sperimentale della Laziness

Per dimostrare l'inerzia assoluta delle operazioni intermedie in assenza di un'operazione terminale, consideriamo il seguente esperimento didattico:

L'Inerzia dello Stream senza Terminale

```
List<String> nomi = Arrays.asList("Anna", "Marco", "Giuseppe", "Lucia");

// Costruzione di una pipeline con filtri e stampe interne
nomi.stream()
    .filter(s -> {
        System.out.println("FILTRO eseguito su: " + s);
        return s.length() > 4;
    })
    .map(s -> {
```

```
        System.out.println("MAP eseguita su: " + s);
        return s.toUpperCase();
    });

System.out.println("--- Fine del blocco di codice ---");
```

Cosa stampa a console questo programma? Sorprendentemente, a console comparirà **unicamente**:

```
--- Fine del blocco di codice ---
```

Nessuna riga di "FILTRO eseguito" o "MAP eseguita" viene emessa. La JVM ha semplicemente registrato le due funzioni lambda all'interno dei metadati dello stream; poiché nessun consumatore terminale ha richiesto un risultato, l'intera computazione è rimasta inerte. Questo garantisce enormi risparmi prestazionali: se il risultato finale non serve, la CPU non spende un singolo ciclo di clock per elaborare i dati.

3.4 Ordine di Valutazione: Elaborazione Verticale vs Orizzontale

Uno dei fraintendimenti più diffusi tra i programmatori alle prime armi è immaginare che gli stream operino in maniera *orizzontale*: ovvero che prima il filtro venga eseguito su tutti i N elementi della lista creando una collezione filtrata invisibile, e solo dopo la mappatura venga applicata a tutti i superstiti.

In Java, l'elaborazione degli stream avviene di norma in modo rigorosamente VERTICALE: ogni singolo elemento prelevato dalla sorgente attraversa *l'intera pipeline in cascata* dall'inizio alla fine prima che il secondo elemento venga estratto dalla sorgente.

3.4.1 Tracciare l'Ordine Reale di Esecuzione

Aggiungiamo ora un'operazione terminale (`findFirst()`) alla nostra pipeline di test ed esaminiamo l'output dei log:

Tracciamento dell'Elaborazione Verticale e Short-Circuit

```
List<String> nomi = Arrays.asList("Anna", "Marco", "Giuseppe", "Lucia");

String trovato = nomi.stream()
    .filter(s -> {
        System.out.println("Passo 1: filter su " + s);
        return s.length() > 4;
    })
    .map(s -> {
        System.out.println("Passo 2: map su " + s);
        return s.toUpperCase();
    })
    .findFirst()
    .orElse("Nessuno");

System.out.println("Elemento estratto: " + trovato);
```

L'output generato a console rivela perfettamente il flusso verticale e l'effetto di **short-circuiting** (interruzione anticipata):

```
Passo 1: filter su Anna
Passo 1: filter su Marco
Passo 2: map su Marco
Elemento estratto: MARCO
```

Analizziamo cosa è successo:

1. "Anna" entra nel tubo: ha 4 caratteri, il filtro restituisce `false`. "Anna" viene scartata immediatamente e non raggiunge mai la `map()`.
2. "Marco" entra nel tubo: ha 5 caratteri, il filtro restituisce `true`. "Marco" prosegue immediatamente il tragitto verticale ed entra nella `map()`, che lo converte in `"MARCO"`.
3. `"MARCO"` raggiunge il terminale `findFirst()`: il terminale cercava il primo elemento valido e lo ha appena trovato.

4. **Chiusura anticipata**: la pipeline si arresta istantaneamente! "Giuseppe" e "Lucia" non vengono nemmeno prelevati dalla lista di partenza.

Principio Architetturale: L'Importanza Critica dell'Ordine dei Metodi Intermedi

Poiché l'elaborazione è verticale, **l'ordine di invocazione dei metodi intermedi impatta drasticamente le prestazioni**:

- Mettere `filter()` **prima** di `map()`: la trasformazione costosa viene eseguita *esclusivamente* sugli elementi che soddisfano il predicato.
- Mettere `map()` **prima** di `filter()`: costringe la CPU a trasformare tutti gli elementi della sorgente, inclusi quelli che verranno buttati via un istante dopo.

3.4.2 Operazioni Stateful e la Rottura della Verticalità

Esistono tuttavia alcune operazioni intermedie speciali che non possono operare in modo puramente verticale. Queste operazioni prendono il nome di **operazioni stateful** (con stato):

- `sorted()`: per ordinare una sequenza di elementi, non è possibile valutare un elemento alla volta isolatamente; è matematicamente necessario accumulare *tutti* gli elementi della sorgente in un buffer interno, confrontarli ed emetterli solo al termine.
- `distinct()`: deve conservare memoria di tutti gli elementi già transitati (solitamente tramite un `HashSet` interno) per scartare i duplicati.

Perché non si può ordinare uno stream infinito

L'operazione intermedia `sorted()` agisce come una diga idroelettrica lungo il tubo: blocca l'intero flusso verticale e attende che l'ultimo elemento sia transitato prima di iniziare a riversare i dati a valle.
Ne consegue una regola insormontabile: **è tassativamente impossibile ordinare uno stream infinito**. Se applichiamo `sorted()` a un generatore infinito (`Stream.iterate`), la pipeline cercherà di accumulare infiniti elementi nell'heap, esaurendo la memoria della JVM e collassando con un inesorabile `OutOfMemoryError`.

3.5 Il Vincolo di Monouso e la Soluzione con `Supplier<Stream<T>>`

Una delle caratteristiche più rigide del modello formale di Java è che **uno Stream può essere consumato una sola volta**:

Definizione: Vincolo di Consumo Unico (*Single-Use Stream*)

Non appena su un'istanza di `Stream` viene invocata un'operazione terminale (`collect()`, `count()`, `forEach()`, `anyMatch()`, ecc.), lo stream viene marcato internamente come chiuso e consumato. Qualsiasi successivo tentativo di invocare operazioni intermedie o terminali sulla medesima istanza scatena immediatamente l'eccezione a tempo di esecuzione:
`java.lang.IllegalStateException: stream has already been operated upon or closed`

Errore Fatale: Tentativo di Riuso di uno Stream Consumato

```
List<String> lista = Arrays.asList("a", "b", "c");
Stream<String> mioStream = lista.stream().filter(s -> !s.isEmpty());

// 1. Prima operazione terminale: conteggio
long conteggio = mioStream.count(); // OK: lo stream viene consumato qui!

// 2. SECONDA operazione terminale: ERRORE CRITICO A RUNTIME!
Optional<String> primo = mioStream.findFirst();
// Lancia IllegalStateException!
```

3.5.1 Perché lo Stream non è Riavvolgibile?

A differenza di un nastro magnetico o di una lista indicizzata, lo stream è modellato per consentire il processamento di sorgenti a flusso continuo (come letture da schede di rete, pipeline I/O di file da decine di gigabyte

o generatori stocastici). Mantenere in memoria la cronologia di tutti gli elementi transitati per consentire un eventuale "riavvolgimento" violerebbe il principio di efficienza e causerebbe memory leak massicci.

3.5.2 La Soluzione Architetturale: `Supplier<Stream<T>`

Quando la logica dell'applicazione richiede di rieseguire la medesima pipeline di trasformazione su richiesta (ad esempio per effettuare interrogazioni statistiche multiple o calcoli concorrenti), il pattern raccomandato consiste nell'incapsulare la catena di generazione all'interno di un'interfaccia funzionale `Supplier<Stream<T>`:

Pattern Factory Leggero con Supplier

```
List<String> dati = Arrays.asList("Alfa", "Beta", "Gamma", "Delta");

// Definiamo una factory pura di stream mediante lambda Supplier
Supplier<Stream<String>> streamFactory = () -> dati.stream().filter(s -> s.length() >
    4);

// 1. Primo utilizzo: calcoliamo il conteggio aprendo un nuovo stream
long quanti = streamFactory.get().count();
System.out.println("Elementi > 4 caratteri: " + quanti); // 2 (Gamma, Delta)

// 2. Secondo utilizzo: estraiamo il primo elemento aprendo un ALTRO stream fresco
Optional<String> primo = streamFactory.get().findFirst();
System.out.println("Primo elemento: " + primo.orElse("Nessuno")); // Gamma

// 3. Terzo utilizzo: verifichiamo se esiste una stringa con iniziale 'D'
boolean esisteD = streamFactory.get().anyMatch(s -> s.startsWith("D"));
System.out.println("Esiste iniziale D? " + esisteD); // true
```

Ogni invocazione di `streamFactory.get()` genera una pipeline completamente nuova e vergine a partire dalla sorgente originaria, garantendo piena thread-safety ed eliminando alla radice ogni rischio di `IllegalStateException`.

3.6 Creare Stream da Varie Sorgenti

Java offre una vastissima gamma di metodi factory per istanziare stream a partire da strutture dati preesistenti, tipi primitivi o formule matematiche generatrici.

3.6.1 Da Collezioni e Array

Il punto di partenza naturale per i progetti a oggetti:

- `collezione.stream()`: stream sequenziale da qualsiasi implementazione di `Collection` (`ArrayList`, `HashSet`, ecc.).
- `collezione.parallelStream()`: stream parallelo che partiziona automaticamente il carico sui core della CPU tramite il framework Fork/Join.
- `Arrays.stream(array)`: stream su array di oggetti o tipi primitivi.
- `Stream.of(val1, val2, val3)`: stream a partire da un elenco varargs esplicito di elementi.

3.6.2 Stream Primitivi Specializzati: `IntStream`, `LongStream`, `DoubleStream`

L'utilizzo di `Stream<Integer>` o `Stream<Double>` comporta un sovraccarico considerevole dovuto al fenomeno dell'**autoboxing/unboxing** continuo (conversione tra il tipo primitivo `int` e l'oggetto wrapper `Integer` nell'heap).

Per garantire prestazioni ad altissima efficienza e prevenire allocazioni dinamiche inutili nel percorso critico, Java mette a disposizione tre interfacce primitive specializzate: `IntStream`, `LongStream` e `DoubleStream`.

Utilizzo di IntStream e Statistiche Riassuntive

```
// Intervallo di interi da 1 a 100 inclusi
int somma = IntStream.rangeClosed(1, 100).sum(); // 5050

// Calcolo di statistiche complete in un singolo passaggio senza boxing
IntSummaryStatistics stats = IntStream.of(12, 45, 68, 2, 99, 34, 51)
    .summaryStatistics();

System.out.println("Minimo: " + stats.getMin());       // 2
System.out.println("Massimo: " + stats.getMax());     // 99
```

```java
System.out.println("Media: " + stats.getAverage());    // 44.428...
System.out.println("Totale elementi: " + stats.getCount()); // 7

// Conversione da primitivo a Stream di oggetti (boxing esplicito)
List<Integer> listaBoxed = IntStream.range(0, 5)
    .boxed()
    .toList();
```

3.6.3 Stream Infiniti: `Stream.iterate` e `Stream.generate`

Uno stream non deve necessariamente basarsi su dati memorizzati a priori. Possiamo generare sequenze matematiche infinite applicando una funzione unaria iterativa:

Definizione: Metodo `Stream.iterate(seed, UnaryOperator)`

Genera uno stream ordinato e infinito prodotto dall'applicazione iterativa della funzione a partire da un elemento iniziale (*seed*):

$$\text{Stream} = \{s_0,\ f(s_0),\ f(f(s_0)),\ f(f(f(s_0))),\ \ldots\}$$

Poiché la sequenza è infinita, è obbligatorio troncarla a valle mediante l'operazione intermedia di short-circuit `limit(n)` prima di richiamare un'operazione terminale.

Generazione della Serie di Fibonacci con Stream.iterate

```java
// Generiamo le coppie consecutive (F_n, F_{n+1}) partendo da (0, 1)
// Ogni coppia successiva e': (b, a + b)
List<Long> primiFibonacci = Stream.iterate(new long[]{0, 1}, coppia -> new
    long[]{coppia[1], coppia[0] + coppia[1]})
    .limit(10)                // Limitiamo ai primi 10 elementi
    .map(coppia -> coppia[0]) // Estraiamo il primo termine della coppia
    .toList();

System.out.println("Primi 10 numeri di Fibonacci: " + primiFibonacci);
// Stampa: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

3.7 Riduzioni Avanzate: Le Tre Forme di `reduce()`

Mentre i metodi speciali come `sum()`, `min()` o `count()` coprono aggregazioni standard, l'operazione terminale generica di aggregazione funzionale è `reduce()` (*fold*). Java fornisce tre versioni sovraccaricate di `reduce()`, ciascuna con specifiche garanzie matematiche:

1. Senza Identità

Forma 1: `Optional<T> reduce(BinaryOperator<T> accumulator)`
Nessun valore identità: se lo stream è vuoto, restituisce `Optional.empty()`.

2. Con Identità

Forma 2: `T reduce(T identity, BinaryOperator<T> accumulator)`
Con elemento neutro e: garantisce che per stream vuoto il risultato sia e.

3. Con Combiner

Forma 3: `<U> U reduce(U identity, BiFunction<U, ? super T, U> acc, BinaryOperator<U> combiner)`
Cambio di tipo ($T \to U$): l'accumulatore trasforma gli elementi, il **combiner** fonde i risultati parziali prodotti dai thread concorrenti negli stream paralleli.

Figura 3.1: Le tre forme sovraccaricate del metodo `reduce()` in Java.

Le Tre Forme di reduce() a Confronto (ReduceDemo.java)

```java
package it.unibs.eps;

import java.util.List;
import java.util.Optional;

```

```java
public class ReduceDemo {
    public static void main(String[] args) {
        List<Integer> numeri = List.of(2, 3, 4, 5);

        // 1. Forma 1: Prodotto senza identita' -> restituisce Optional
        Optional<Integer> prodOpt = numeri.stream().reduce((a, b) -> a * b);
        System.out.println("Prodotto: " + prodOpt.orElse(0)); // 120

        // 2. Forma 2: Somma con identita' (0 e' elemento neutro della somma) -> restituisce T
        int somma = numeri.stream().reduce(0, Integer::sum);
        System.out.println("Somma: " + somma); // 14

        // 3. Forma 3: Cambio di tipo da String (T) a Integer (U) - somma lunghezze
        List<String> parole = List.of("mela", "pera", "banana");
        int sommaLunghezze = parole.stream().reduce(
            0,                                  // Identita' di tipo U
            (acc, str) -> acc + str.length(),   // Accumulator: (U, T) -> U
            Integer::sum                        // Combiner: (U, U) -> U
        );
        System.out.println("Somma lunghezze: " + sommaLunghezze); // 4 + 4 + 6 = 14
    }
}
```

Principio Architetturale: Perché la Terza Forma Richiede Tassativamente il Combiner?

Negli stream sequenziali, il `combiner` non viene mai invocato perché un solo thread elabora gli elementi in serie. Tuttavia, la firma del metodo impone il combiner per garantire la correttezza formale della **parallelizzazione**:

- In uno stream parallelo (`parallelStream()`), il flusso di elementi di tipo T viene suddiviso tra molteplici thread del `ForkJoinPool`;
- Ciascun worker thread produce un accumulatore parziale indipendente di tipo U invocando l'`accumulator`;
- Al termine, i risultati parziali di tipo U devono essere aggregati a coppie risalendo l'albero di divisione binaria. Il `combiner` è la funzione puramente binaria $(U, U) \to U$ che unisce due sotto-risultati parziali.

Affinché la riduzione sia corretta, l'accumulatore e il combiner devono essere **associativi** e l'identità deve essere un autentico elemento neutro: combiner$(identity, u) == u$.

3.8 Raccogliere Risultati: `collect()` e la Classe `Collectors`

L'operazione terminale più versatile e potente del framework è `collect()`, che accetta un'istanza dell'interfaccia `Collector` per trasformare gli elementi dello stream in una struttura dati complessa (liste, insiemi, mappe gerarchiche, stringhe formattate). La classe di utilità `Collectors` fornisce decine di factory statiche pronte all'uso.

3.8.1 Collettori Base e Concatenazione Testuale

- `Collectors.toList()` / `toSet()`: accumula gli elementi in una lista o in un insieme mutabile (oppure `toUnmodifiableList()` per collezioni immutabili).
- `Collectors.toCollection(TreeSet::new)`: accumula gli elementi in una specifica implementazione decisa dal programmatore.
- `Collectors.joining(delimitatore, prefisso, suffisso)`: concatena elementi testuali con separatore in modo ultra-efficiente tramite un `StringBuilder` interno:

Esempio di Joining Testuale

```java
List<String> linguaggi = Arrays.asList("Java", "C", "Rust", "Python");

String csv = linguaggi.stream()
    .map(String::toUpperCase)
```

```
    .collect(Collectors.joining(", ", "[", "]"));

System.out.println(csv); // [JAVA, C, RUST, PYTHON]
```

3.8.2 Raggruppamento e Partizionamento Avanzato

Il vero potenziale analitico delle Stream API risiede nelle operazioni di aggregazione relazionale, analoghe a clausole `GROUP BY` del linguaggio SQL:

Definizione: Raggruppamento con `Collectors.groupingBy()`

Il metodo `groupingBy(Function<T, K> classifier)` suddivide gli elementi dello stream in partizioni in base al valore calcolato dalla funzione di classificazione, restituendo una `Map<K, List<T>>`. Inoltre, combinando un **downstream collector** (`groupingBy(classifier, downstream)`), è possibile applicare un'ulteriore riduzione sugli elementi di ciascun gruppo (conteggio, somma, mappatura, trasformazione in set).

Raggruppamenti Downstream e Partizionamento

```
List<String> parole = Arrays.asList("albero", "casa", "arco", "sole", "mela", "mare",
    "sasso");

// 1. Raggruppamento per lunghezza della stringa: Map<Integer, List<String>>
Map<Integer, List<String>> perLunghezza = parole.stream()
    .collect(Collectors.groupingBy(String::length));
// {4=[casa, sole, mela, mare], 5=[sasso], 6=[albero]}

// 2. Raggruppamento con downstream counting(): Map<Integer, Long>
Map<Integer, Long> conteggioPerLunghezza = parole.stream()
    .collect(Collectors.groupingBy(String::length, Collectors.counting()));
// {4=4, 5=1, 6=2}

// 3. Raggruppamento con downstream mapping(): Map<Integer, Set<String>>
Map<Integer, Set<String>> maiuscolePerLunghezza = parole.stream()
    .collect(Collectors.groupingBy(
        String::length,
        Collectors.mapping(String::toUpperCase, Collectors.toSet())
    ));

// 4. Raggruppamento con downstream filtering() (Java 9+):
Map<Integer, List<String>> soloConA = parole.stream()
    .collect(Collectors.groupingBy(
        String::length,
        Collectors.filtering(s -> s.contains("a"), Collectors.toList())
    ));

// 5. Partizionamento binario con predicato booleano: Map<Boolean, List<String>>
Map<Boolean, List<String>> iniziaConS = parole.stream()
    .collect(Collectors.partitioningBy(s -> s.startsWith("s")));
// {false=[albero, casa, arco, mela, mare], true=[sole, sasso]}
```

3.8.3 Lo Studio del Caso d'Aula: Il Censimento del Bimondo in Una Riga

Nel Capitolo 1 abbiamo affrontato l'esercizio del *Bimondo Geometrico*, in cui una lista eterogenea di figure bidimensionali doveva essere censita per contare quante istanze di ciascuna classe concreta fossero presenti nella simulazione.

Nell'approccio procedurale tradizionale, il censimento richiedeva la dichiarazione di una `Map<String, Integer>`, un ciclo di scansione, l'estrazione manuale del nome di classe con `s.getClass().getSimpleName()` e una serie di controlli `if (map.containsKey(...))` con relativi incrementi manuali.

Con le Stream API e i collettori avanzati, l'intera operazione si condensa in **un'unica, fulminea riga di codice dichiarativo**:

Il Censimento del Bimondo Risolto in Una Riga

```
List<Shape2d> mondo = Arrays.asList(
    new Cerchio(5.0),
    new Rettangolo(4.0, 6.0),
    new Cerchio(2.5),
    new Triangolo(3.0, 4.0),
    new Cerchio(1.0)
);

// Censimento istantaneo per tipo concreto con counting downstream:
Map<Class<?>, Long> censimento = mondo.stream()
    .collect(Collectors.groupingBy(Object::getClass, Collectors.counting()));

censimento.forEach((classe, quanti) ->
    System.out.println("Classe " + classe.getSimpleName() + " -> Presenti: " +
    quanti));
```

L'output prodotto a console:

```
Classe Cerchio -> Presenti: 3
Classe Rettangolo -> Presenti: 1
Classe Triangolo -> Presenti: 1
```

Consultazione Esercizi di Laboratorio sugli Stream

Per consolidare l'architettura delle Stream API, verificare empiricamente la laziness, implementare le tre forme di `reduce()`, progettare collettori avanzati downstream e profilare pipeline parallele, si rimanda agli esercizi dettagliati del **Capitolo A** (dall'Esercizio A.3 all'Esercizio A.3).

Capitolo 4

Interfacce Grafiche Swing, Architettura ad Eventi ed Event Dispatch Thread

In questo capitolo esploriamo la transizione dalla programmazione procedurale a riga di comando (CLI) allo sviluppo di applicazioni grafiche interattive (**GUI**) mediante il framework **Java Swing** e il toolkit fondamentale **Java AWT** (*Abstract Window Toolkit*).

Costruire un'interfaccia grafica non significa soltanto disegnare pulsanti o caselle di testo sullo schermo: significa abbracciare un radicale cambio di paradigma architetturale, noto come **Event-Driven Programming** (programmazione guidata dagli eventi), in cui il flusso di controllo del programma non è più scandito sequenzialmente dal codice dello sviluppatore, ma reagisce in modo asincrono alle azioni imprevedibili dell'utente e del sistema operativo.

4.1 Dal Modello a Riga di Comando alla Programmazione ad Eventi

4.1.1 Paradigmi a Confronto: Esecuzione Lineare vs Reattiva

Nelle applicazioni a riga di comando studiate finora, l'esecuzione del codice segue un flusso sequenziale deterministico governato dal programmatore:

Flusso Sequenziale Sincrono (CLI)

```
Scanner scanner = new Scanner(System.in);
System.out.print("Inserisci il primo numero: ");
int a = scanner.nextInt(); // IL THREAD MAIN SI BLOCCA QUI IN ATTESA
System.out.print("Inserisci il secondo numero: ");
int b = scanner.nextInt(); // IL THREAD MAIN SI BLOCCA DI NUOVO
System.out.println("Somma: " + (a + b));
```

In questo modello:

- Il programma decide *quando* e *cosa* l'utente può inserire;
- L'invocazione di metodi di I/O (come `scanner.nextInt()`) è **bloccante**: il thread della CPU si ferma in attesa che l'utente prema il tasto Invio;
- L'utente non ha alcuna libertà di interagire fuori dall'ordine imposto dallo sviluppatore.

Nelle applicazioni dotate di interfaccia grafica (**GUI**), questo modello è completamente insostenibile:

- Sullo schermo sono presenti contemporaneamente molteplici elementi interattivi (pulsanti, campi di testo, slider, menu a tendina);
- L'utente può compiere un'infinità di azioni imprevedibili in qualsiasi istante: cliccare un pulsante, ridimensionare la finestra, minimizzarla, spostare il cursore del mouse, digitare caratteri o chiudere l'applicazione;
- Il programma non può "bloccarsi in attesa": l'interfaccia deve rimanere fluida, ridisegnarsi costantemente a 60 frame al secondo e reagire istantaneamente alle sollecitazioni.

4.1.2 Il Dialogo Continuo tra Sistema Operativo e Applicazione

A livello architetturale di basso livello, il sistema operativo (Windows, Linux X11/Wayland, macOS) controlla costantemente le periferiche hardware di input (mouse, tastiera, touch screen). Quando l'utente muove il cursore del mouse o preme un tasto:

1. Il driver di periferica solleva un *hardware interrupt*;
2. Il kernel del sistema operativo confeziona un **messaggio di sistema** (ad esempio `WM_LBUTTONDOWN` o `WM_KEYDOWN` nell'architettura Win32) contenente coordinate (x, y), stato dei tasti e timestamp;
3. Il kernel deposita questo messaggio nella coda degli eventi della finestra su cui si trova il puntatore;

4. La Java Virtual Machine (JVM), attraverso i propri adapter nativi AWT, intercetta il messaggio del sistema operativo e lo trasforma in un'istanza fortemente tipizzata della gerarchia `java.awt.AWTEvent`.

4.1.3 Eventi Hardware a Basso Livello vs Eventi Semantici di Alto Livello

Nel toolkit Java AWT/Swing gli eventi si dividono in due grandi categorie:

Definizione: Tassonomia degli Eventi Java

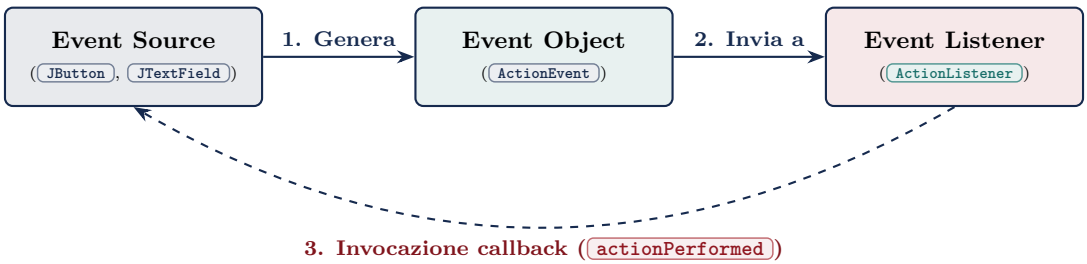
- **Eventi di Basso Livello (*Low-Level / Component Events*)**: rappresentano le singole interazioni fisiche grezze con l'hardware o con la finestra del sistema operativo. Appartengono a questa categoria: `MouseEvent` (movimento, ingresso, uscita, pressione del tasto del mouse), `KeyEvent` (pressione o rilascio di un singolo tasto fisico), `WindowEvent` (apertura, chiusura, iconificazione del frame) e `FocusEvent`.
- **Eventi Semantici (*Semantic / High-Level Events*)**: incapsulano l'**intento logico** dell'utente, astraendo completamente dal dispositivo fisico utilizzato per compierlo. Appartengono a questa categoria: `ActionEvent` (attivazione di un comando logico da un pulsante o voce di menu), `ItemEvent` (selezione/deselezione di una checkbox) e `ChangeEvent` (variazione di valore di uno slider).

Principio Architetturale: Astrazione mediante Eventi Semantici

Una buona architettura software per interfacce grafiche si ancora quasi sempre ad **eventi semantici**. Un `JButton` può essere attivato con un click sinistro del mouse, premendo la barra spaziatrice quando ha il focus, premendo il tasto Invio o tramite una combinazione di tasti di scelta rapida (*mnemonic*). Se il programmatore registrasse un ascoltatore grezzo di mouse (`MouseListener`), intercetterebbe solo il click fisico escludendo la tastiera e rompendo l'accessibilità. Registrando invece un `ActionListener` sull'evento semantico `ActionEvent`, l'applicazione risponde all'*intenzione* dell'utente a prescindere dal canale di input fisico.

4.2 Il Modello Architetturale: Observer Pattern e Delegation Event Model

Per gestire gli eventi in modo disaccoppiato, Java adotta il celebre **Delegation Event Model**, derivazione diretta del design pattern GoF **Observer**:



Il modello si fonda su tre attori architetturali distinti:

1. **La Sorgente dell'Evento (*Event Source*)**: è il componente grafico con cui l'utente interagisce (ad esempio un'istanza di `JButton`). La sorgente non sa *cosa* farà il programma quando verrà cliccata; sa soltanto gestire una lista interna di ascoltatori registrati.
2. **L'Oggetto Evento (*Event Object*)**: è una classe immutabile (derivata da `java.util.EventObject`) che trasporta tutte le informazioni contestuali dell'accaduto. In particolare, il metodo fondamentale `getSource()` restituisce il riferimento al componente che ha scatenato l'evento, mentre metodi specifici (es. `getActionCommand()`) forniscono dettagli aggiuntivi.
3. **L'Ascoltatore (*Event Listener*)**: è un oggetto che implementa un'interfaccia dedicata derivata da `java.util.EventListener`. Contiene uno o più metodi di callback (come il metodo `actionPerformed()`) che vengono invocati dalla sorgente quando l'evento si verifica.

4.2.1 Multicast Events: Perché `add` e non `set`?

Una finezza architetturale evidenziata a lezione riguarda la nomenclatura dei metodi di registrazione dei listener in Java:

La Distinzione tra `set` e `add`

Perché in Java scriviamo:
`pulsante.addActionListener(listener);` anziché `pulsante.setActionListener(listener);` ?
Il prefisso `set` indicherebbe una relazione $1:1$: una volta impostato un listener, impostarne un altro sovrascriverebbe e distruggerebbe il precedente.
Il prefisso `add` denota formalmente un'architettura **Multicast Event**: la sorgente mantiene una collezione interna di ascoltatori. Possono registrarsi $0, 1, 2, \ldots, N$ listener completamente indipendenti tra loro. Quando l'evento accade, la sorgente itera sulla collezione e notifica *tutti* gli ascoltatori registrati. Allo stesso modo, `removeActionListener()` consente di cancellare la sottoscrizione in qualunque momento.

4.3 Il Thread della Grafica: Event Dispatch Thread (EDT)

La comprensione dell'esecuzione concorrente è il pilastro fondamentale della sicurezza in ambiente Swing:

Principio Architetturale: Regola Aurea di Swing: Single-Threaded UI Rule

Il toolkit grafico Swing NON è thread-safe. I metodi di creazione, configurazione, manipolazione dello stato e visualizzazione dei componenti grafici devono essere invocati **esclusivamente dal thread dedicato alla grafica**, denominato **Event Dispatch Thread (EDT)**.

4.3.1 Perché Swing non è Thread-Safe?

Rendere thread-safe l'intera gerarchia di centinaia di classi di Swing (aggiungendo blocchi `synchronized` su ogni metodo di repaint, calcolo dei font, layout o gestione delle coordinate) avrebbe introdotto un costo computazionale mostruoso e creato un rischio perenne di **deadlock** tra i lock interni del toolkit e i lock applicativi dello sviluppatore.

I progettisti di Java (e di quasi tutti i moderni toolkit grafici, inclusi JavaFX, Android e iOS Cocoa) hanno quindi adottato il modello *Single-Threaded Subsystem*: tutti gli eventi grafici, le richieste di ridisegno (`repaint()`) e le chiamate ai listener vengono elaborate sequenzialmente da un unico thread di sistema, chiamato `AWT-EventQueue-0`.

4.3.2 La Chiamata Obbligatoria: `EventQueue.invokeLater`

Quando viene avviata un'applicazione Java, la JVM avvia il thread principale `main`. Se il thread `main` istanzia direttamente la finestra o ne modifica i componenti, si crea una grave **race condition** con l'EDT.

Per rispettare formalmente il contratto architetturale di Swing, l'avvio della GUI deve essere demandato all'EDT tramite il metodo statico `EventQueue.invokeLater(Runnable)`:

Punto di Ingresso Obbligatorio per Applicazioni Swing

```java
import java.awt.EventQueue;
import javax.swing.JFrame;

public class PrimaFinestraSwing {
    public static void main(String[] args) {
        // Demanda l'istanziazione e l'avvio della GUI all'Event Dispatch Thread (EDT)
        EventQueue.invokeLater(() -> {
            try {
                JFrame finestra = new JFrame("La Mia Prima GUI");
                finestra.setSize(400, 300);
                finestra.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
                finestra.setLocationRelativeTo(null); // Centra sullo schermo
                finestra.setVisible(true);
            } catch (Exception e) {
                e.printStackTrace();
            }
        });
    }
}
```

Il Pericolo del Congelamento della Finestra: Non Bloccare mai l'EDT!

Poiché l'EDT elabora tutti gli eventi sequenzialmente in un unico ciclo (*event loop*), se all'interno del metodo `actionPerformed()` di un listener eseguiamo un'operazione lenta o bloccante (come il download di un file da Internet, una query su database pesante o un ciclo da miliardi di iterazioni):

L'INTERA INTERFACCIA GRAFICA SI CONGELA ISTANTANEAMENTE!

La finestra smette di rispondere ai click, non si ridisegna più, assume un aspetto sbiadito e il sistema operativo mostra il messaggio *"L'applicazione non risponde"*. Le operazioni lunghe devono essere eseguite su un thread worker separato (ad esempio tramite la classe `javax.swing.SwingWorker`) e rientrare sull'EDT solo per aggiornare i componenti visivi.

4.4 Gerarchia dei Componenti Swing e Contenitori

Tutti i componenti visivi di Swing ereditano da una gerarchia di classi con radice in `java.awt.Component`:

- `Component` → classe astratta di AWT che incapsula la geometria bidimensionale (posizione x, y, dimensioni larghezza/altezza, colore di sfondo e font);
- `Container` → sottoclasse di `Component` capace di contenere al proprio interno altri componenti figli e governarne la disposizione tramite un `LayoutManager`;
- `JComponent` → classe base di quasi tutti i componenti Swing (prefisso 'J'). Introduce funzionalità avanzate: rendering a doppio buffer (*double buffering* per eliminare lo sfarfallio), bordi configurabili (`BorderFactory`), tooltip e supporto per il Look and Feel modificabile a runtime.

4.4.1 I Livelli di Contenimento

La progettazione di una finestra si articola su tre livelli gerarchici:

1. **Contenitori Top-Level (Primo Livello)**: sono le finestre fornite direttamente dal sistema operativo ospitante. Includono `JFrame` (finestra principale con decorazioni, barra del titolo e pulsanti riduci/chiudi) e `JDialog` (finestre di dialogo modali o non-modali). Una finestra `JFrame` non può essere contenuta in un'altra finestra.
2. **Il ContentPane**: è il pannello invisibile interno al `JFrame` su cui vengono fisicamente ancorati tutti gli elementi visivi. Si recupera tramite `frame.getContentPane()` (oppure chiamando direttamente `frame.add()` che in Java moderno delega in trasparenza al ContentPane).
3. **Contenitori Intermedi**: componenti come `JPanel` che non sono finestre autonome, ma fungono da "scatole" per aggregare e raggruppare sotto-componenti, isolandoli visivamente o applicando layout differenti.
4. **Componenti Atomici**: elementi terminali della gerarchia come `JButton`, `JLabel`, `JTextField`, `JCheckBox` o `JRadioButton`.

4.5 Gestione dello Spazio: I Layout Manager

4.5.1 Perché il Posizionamento Assoluto (`setLayout(null)`) è Bandito

Nei primi esperimenti con le GUI, molti programmatori sono tentati di posizionare i componenti specificando coordinate fisse in pixel (*Absolute Positioning*):

Anti-Pattern: Posizionamento Assoluto con setLayout(null)

```
// PRATICA PESSIMA: BANNATA NELLO SVILUPPO PROFESSIONALE!
pannello.setLayout(null);
JButton btn = new JButton("Premi Qui");
btn.setBounds(50, 100, 120, 30); // x=50, y=100, larghezza=120, altezza=30
pannello.add(btn);
```

Principio Architetturale: Il Fallimento del Posizionamento Assoluto

L'utilizzo di `setLayout(null)` e `setBounds()` produce interfacce **fragili e non portabili**:

- Se l'utente ridimensiona la finestra, i componenti rimangono bloccati nelle loro coordinate fisse in alto a sinistra, lasciando enormi spazi vuoti o finendo tagliati fuori dall'area visibile;

- Se l'applicazione viene eseguita su un display moderno con risoluzione 4K e fattore di scala DPI al 150% o 200%, le etichette con dimensioni fisse in pixel taglieranno il testo interno;
- Se la lingua del sistema operativo cambia o i font di sistema sono leggermente più larghi, il testo trabocchera dai pulsanti.

Per questa ragione, Java adotta i **Layout Manager**: oggetti che calcolano dinamicamente la posizione e le dimensioni di ogni componente in base allo spazio disponibile e alle dimensioni preferite (*preferred size*).

4.5.2 Confronto Sinottico tra i Principali Layout Manager

Layout Manager	Logica di Disposizione	Contesto d'Uso Tipico
`BorderLayout`	Suddivide lo spazio in 5 regioni: `NORTH`, `SOUTH`, `EAST`, `WEST`, `CENTER`. Le regioni laterali prendono la loro larghezza/altezza preferita; il `CENTER` assorbe tutto lo spazio residuo.	Layout predefinito del `ContentPane` di `JFrame`. Ideale per la struttura macroscopica della finestra.
`FlowLayout`	Dispone i componenti in una sequenza orizzontale da sinistra a destra con wrapping a capo automatico, rispettando le dimensioni preferite dei componenti.	Layout predefinito di `JPanel`. Ottimo per barre di pulsanti o raccolte di checkbox.
`GridLayout`	Suddivide il contenitore in una griglia regolare di R righe e C colonne di identiche dimensioni, forzando tutti i componenti ad avere la stessa area.	Calcolatrici, tastierini numerici, scacchiere.
`BoxLayout`	Dispone i componenti lungo un unico asse (orizzontale `X_AXIS` o verticale `Y_AXIS`), supportando molle (*glue*) e distanziatori (*strut*).	Barre laterali verticali o barre di stato complesse.

Tabella 4.1: Confronto tra i Layout Manager fondamentali in Java Swing.

4.5.3 La Tecnica dei Pannelli Annidati (Nested Panels)

L'approccio professionale per costruire interfacce ricche ed eleganti consiste nell'**annidamento gerarchico di pannelli**: si utilizza un `BorderLayout` per la struttura globale del frame e si inseriscono nei suoi quadranti (`NORTH`, `SOUTH`, ecc.) dei `JPanel` secondari dotati di `FlowLayout` o `GridLayout`:

Esempio Architetturale di Pannelli Annidati

```
JFrame frame = new JFrame("Layout Annidato Professionale");
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setSize(500, 350);

// Il ContentPane di default usa BorderLayout
Container cp = frame.getContentPane();
cp.setLayout(new BorderLayout(8, 8)); // gap orizzontale e verticale di 8px

// 1. ZONA CENTRALE: un'area di testo espandibile
cp.add(new JScrollPane(new JTextArea("Contenuto principale...")),
    BorderLayout.CENTER);

// 2. ZONA SUD: un pannello contenitore per i pulsanti di controllo
JPanel pnlSud = new JPanel(new FlowLayout(FlowLayout.RIGHT, 10, 5));
JButton btnAnnulla = new JButton("Annulla");
JButton btnConferma = new JButton("Salva Modifiche");
pnlSud.add(btnAnnulla);
pnlSud.add(btnConferma);

// Ancoriamo il pannello alla regione SOUTH del frame
```

```
20  cp.add(pnlSud, BorderLayout.SOUTH);
```

4.6 L'Evoluzione dei Listener: Da Classi Esterne a Method Reference

Nel corso della storia di Java, il modo in cui il programmatore collega un'azione a un pulsante ha subito una straordinaria evoluzione sintattica e concettuale, culminata con l'avvento delle lambda expression.

Esaminiamo i 5 approcci storici allo stesso problema: un pulsante `"Incrementa"` che aumenta un contatore intero visualizzato su una `JLabel`:

4.6.1 1. Classe Esterna Separata

Creazione di un file autonomo `GestoreClick.java`. Richiede di passare al costruttore i riferimenti al modello o ai componenti della GUI, frammentando il codice e aumentando l'accoppiamento.

4.6.2 2. Classe Interna (Inner Class)

La classe che implementa `ActionListener` viene definita come membro interno del frame. Ha accesso diretto ai campi privati della finestra (la label e il contatore), eliminando i costruttori complessi.

4.6.3 3. Classe Anonima Inline (Pre-Java 8)

L'approccio dominante fino a Java 7: la classe viene istanziata al volo nel punto esatto di registrazione del listener:

Ascoltatore con Classe Anonima (Java <= 7)

```
1  btnIncrementa.addActionListener(new ActionListener() {
2      @Override
3      public void actionPerformed(ActionEvent e) {
4          contatore++;
5          lblTesto.setText("Valore: " + contatore);
6      }
7  });
```

4.6.4 4. Lambda Expression (Java 8+)

Poiché `ActionListener` è un'interfaccia funzionale dotata dell'unico metodo astratto `actionPerformed()`, l'intera classe anonima collassa in una concisa espressione lambda:

Ascoltatore con Lambda Expression

```
1  btnIncrementa.addActionListener(e -> {
2      contatore++;
3      lblTesto.setText("Valore: " + contatore);
4  });
```

4.6.5 5. Method Reference su Metodo Dedicato

Se la logica di reazione supera le due o tre righe, l'inserimento di un blocco lambda corposo all'interno dell'albero di costruzione della GUI riduce la leggibilità. La best practice moderna consiste nel fattorizzare la logica in un metodo di istanza privato e associarlo tramite **Method Reference**:

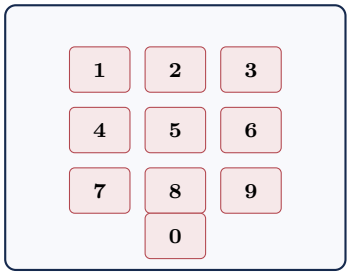
Associazione Tramite Method Reference

```
1  // Nel costruttore del Frame:
2  btnIncrementa.addActionListener(this::onIncrementaClick);
3
4  // Metodo di gestione privato e isolato:
5  private void onIncrementaClick(ActionEvent event) {
6      this.contatore++;
7      this.lblTesto.setText("Valore: " + this.contatore);
8  }
```

4.7 Progettazione di Componenti Custom: Il Caso del Tastierino Numerico (`PnlDigits`)

Uno dei contributi metodologici più rilevanti della lezione del 30/09 è la progettazione di un **componente grafico composito personalizzato**: il tastierino numerico telefonico o da calcolatrice (`PnlDigits`).

Componente Incapsulato: PnlDigits



- **Incapsulamento totale**
- **I bottoni interni sono privati**
- **Espone:** `addActionListener()`
- **Gestione con** `EventListenerList`

4.7.1 Il Principio di Incapsulamento Rigoroso

L'errore architetturale più grave compiuto dai programmatori inesperti è creare un pannello con bottoni e poi esporre i bottoni all'esterno tramite metodi getter pubblici:

```
// ERRORE ARCHITETTURALE: espone l'implementazione interna
public JButton getBtn1() { return this.btn1; }
```

Se facciamo ciò, distruggiamo l'incapsulamento del componente: la classe cliente (il frame) deve conoscere l'esistenza dei 10 bottoni singoli e deve registrare 10 listener diversi. Se un domani volessimo sostituire i pulsanti con una griglia disegnata su canvas o con un display touch virtuale, il codice cliente collasserebbe.

4.7.2 Il Componente come Proxy di Eventi

La soluzione architetturale corretta consiste nel trattare `PnlDigits` come una scatola nera:

1. I 10 pulsanti interni (0–9) sono **`private`**;
2. `PnlDigits` registra un listener interno su ciascun pulsante;
3. Quando l'utente preme un tasto numerico, `PnlDigits` intercetta il click e notifica i propri ascoltatori esterni emettendo un nuovo `ActionEvent` in cui la sorgente è `PnlDigits` stesso e il comando è la cifra digitata;
4. Il componente adotta `EventListenerList` di Swing per garantire sicurezza nei thread e flessibilità multicast.

Ecco l'implementazione completa e robusta del componente:

Implementazione del Componente: PnlDigits.java

```
import java.awt.GridLayout;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.JButton;
import javax.swing.JPanel;
import javax.swing.event.EventListenerList;

public class PnlDigits extends JPanel {
    // Lista sicura e thread-safe di ascoltatori registrati
    private final EventListenerList listenerList = new EventListenerList();

    public PnlDigits() {
        // Griglia 4 righe x 3 colonne
        super(new GridLayout(4, 3, 4, 4));

        // Listener interno comune a tutti i pulsanti della tastiera
        ActionListener internalListener = e -> {
            String cifra = e.getActionCommand();
            fireActionPerformed(new ActionEvent(this, ActionEvent.ACTION_PERFORMED,
    cifra));
        };

```

```
        // Generazione dei tasti da 1 a 9
        for (int i = 1; i <= 9; i++) {
            JButton btn = new JButton(String.valueOf(i));
            btn.addActionListener(internalListener);
            this.add(btn);
        }

        // Ultima riga: spazio vuoto, tasto 0, spazio vuoto
        this.add(new JPanel()); // segnaposto trasparente
        JButton btnZero = new JButton("0");
        btnZero.addActionListener(internalListener);
        this.add(btnZero);
        this.add(new JPanel());
    }

    // Registrazione di un ascoltatore esterno (Multicast Event)
    public void addActionListener(ActionListener listener) {
        listenerList.add(ActionListener.class, listener);
    }

    // Deregistrazione dell'ascoltatore esterno
    public void removeActionListener(ActionListener listener) {
        listenerList.remove(ActionListener.class, listener);
    }

    // Propagazione dell'evento a tutti gli ascoltatori registrati
    protected void fireActionPerformed(ActionEvent event) {
        Object[] listeners = listenerList.getListenerList();
        // L'array di EventListenerList e' strutturato a coppie: [TipoClasse,
    IstanzaListener]
        for (int i = listeners.length - 2; i >= 0; i -= 2) {
            if (listeners[i] == ActionListener.class) {
                ((ActionListener) listeners[i + 1]).actionPerformed(event);
            }
        }
    }
}
```

4.7.3 Integrazione del Componente nel Frame Applicativo

Grazie a questa architettura a scatola nera, la finestra che include il tastierino registra **un unico listener** su tutto il pannello:

Utilizzo Pulito del Componente PnlDigits

```
JFrame frame = new JFrame("Display Calcolatrice");
JTextField display = new JTextField(12);
display.setEditable(false);

PnlDigits tastierino = new PnlDigits();

// La finestra si iscrive agli eventi del pannello senza sapere come e' fatto dentro:
tastierino.addActionListener(e -> {
    String cifraPremuta = e.getActionCommand();
    display.setText(display.getText() + cifraPremuta);
});

frame.getContentPane().add(display, BorderLayout.NORTH);
frame.getContentPane().add(tastierino, BorderLayout.CENTER);
```

Avvertenza d'Esame: Limiti degli Editor Visuali (WindowBuilder) e Codifica Manuale

Durante le esercitazioni e lo studio individuale, gli studenti incontrano spesso strumenti di disegno visuale come **Eclipse WindowBuilder** o il GUI Designer di IntelliJ IDEA, che permettono di assemblare finestre tramite drag-and-drop WYSIWYG (*What You See Is What You Get*).

Il docente ha ribadito con estrema fermezza che **nella prova d'esame e nella pratica dell'ingegneria del software seria, il codice delle interfacce grafiche deve essere scritto rigorosamente a mano**:

1. **Proliferazione di Codice Monolitico e Incontrollabile**: gli editor visuali rigenerano continuamente l'albero di sintassi astratta (AST) accumulando centinaia di righe di codice spazzatura all'interno di un unico gigantesco metodo `initialize()`, violando il principio di singola responsabilità e rendendo il refactoring quasi impossibile.
2. **La Trappola del Layout Assoluto (`null layout`)**: per consentire il trascinamento libero dei componenti con il mouse, i visual builder impostano spesso posizionamenti rigidi a pixel fissi (`setBounds(x, y, w, h)`). Come dimostrato nella Sezione 4.5, questo codice collassa non appena l'applicazione viene eseguita su un display con diversa risoluzione DPI o su un sistema operativo differente.
3. **Incapacità di Progettare Componenti Incapsulati**: un editor visuale è incapace di concepire l'architettura a scatola nera di un componente personalizzato come `PnlDigits`. L'editor si limita ad appiattire tutti i bottoni nel frame principale, costringendo il programmatore a un design procedurale degradato.

Lo studente deve padroneggiare la composizione a strati dei pannelli (`JPanel`), la corretta gerarchia dei Layout Manager e la gestione rigorosa della concorrenza sull'Event Dispatch Thread.

4.8 Concorrenza in Swing: L'Event Dispatch Thread (EDT) e il Ponte verso i Background Thread

Il framework Swing adotta un'architettura fondamentale nota come **Single-Threaded Subsystem**: per massimizzare la reattività e azzerare i costosi lock di sincronizzazione nel rendering dei pixel, tutti i componenti grafici di Swing **non sono thread-safe**.

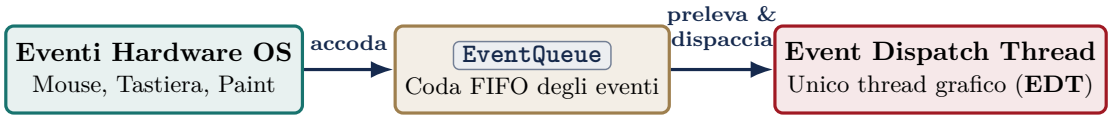


Figura 4.1: Architettura dell'Event Dispatch Thread: dalla coda degli eventi al ridisegno dello schermo.

Principio Architetturale: La Regola Aurea del Thread di Swing

Qualsiasi interazione con un componente grafico visibile (creazione di finestre, modifica di testi con `setText()`, aggiornamento di tabelle o repaint) **deve avvenire tassativamente all'interno dell'Event Dispatch Thread (EDT)**.
Per avviare un'applicazione grafica rispettando questa regola, l'entry point delega l'istanziazione della finestra tramite la coda di sistema con `SwingUtilities.invokeLater()`:

```
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        FinestraCalcolatrice gui = new FinestraCalcolatrice();
        gui.setVisible(true);
    });
}
```

Il Pericolo del Congelamento dell'Interfaccia (*GUI Freezing*)

Se un listener su un pulsante esegue un'operazione pesante (un loop intensivo di calcolo matematico, un trasferimento di rete HTTP o una lettura di un file da 1 GB), tale codice viene eseguito **direttamente sull'EDT**.
Poiché l'EDT è un unico thread, esso rimane intrappolato nell'elaborazione del task: non può prelevare nuovi eventi dalla coda, non può processare i click del mouse né inviare le istruzioni di ridisegno dello schermo al sistema operativo. L'utente percepisce la finestra come completamente bloccata o congelata (*"L'applicazione non risponde"*).
La regola ingegneristica per risolvere questo stallo è immediata:

"Lavora fuori dall'EDT, entra nell'EDT solo per aggiornare i pixel."
I calcoli intensivi devono essere delegati a thread operai di background. Questa esigenza introduce naturalmente la teoria e la pratica della programmazione concorrente e multi-thread, a partire dal caso di studio guida di `PrimeFinderApp` trattato capillarmente nel Capitolo 5 (§5.1).

Esercitazioni di Laboratorio su Swing ed Eventi

Per implementare finestre complete, analizzare sperimentalmente il comportamento dei diversi Layout Manager, realizzare componenti custom incapsulati e gestire l'interazione tra thread e interfaccia grafica, si rimanda agli esercizi del **Capitolo A** (dall'Esercizio A.4 all'Esercizio A.4).

Capitolo 5

Thread, Concorrenza, Executor e Virtual Threads

Nei sistemi software moderni, l'esecuzione sequenziale di un'unica istruzione alla volta rappresenta un limite insormontabile. I microprocessori integrano molteplici core fisici e logici capaci di eseguire flussi computazionali in autentico parallelismo hardware. Parallelamente, le applicazioni moderne trascorrono gran parte del loro tempo in attesa di operazioni di Input/Output (letture su disco, interrogazioni a database, chiamate di rete tramite socket HTTP o elaborazione di eventi grafici).

Questo capitolo, interamente basato sulle lezioni e sui testi d'aula del corso (*PAJC-26-D04* e *PAJC-26-L04a* del Prof. Massimiliano Redolfi), sviluppa in modo organico la concorrenza in Java attorno a due problemi cardinali che guidano l'intero percorso:

1. **Usare tutta la macchina**: distribuire un calcolo intensivo su tutti i core CPU disponibili, misurarne lo *speedup*, comprendere la legge di Amdahl e bilanciare il carico computazionale;
2. **Tenere viva la finestra**: impedire che un calcolo prolungato congeli l'interfaccia grafica desktop Swing, comprendendo la coda degli eventi, l'Event Dispatch Thread (EDT) e l'astrazione `SwingWorker`.

Dalle astrazioni di base del sistema operativo e della JVM fino al framework `java.util.concurrent`, ai monitor, ai lock espliciti, a `LongAdder` e alla rivoluzione architetturale dei **Virtual Threads** di Java 21 (Project Loom), ogni strumento viene introdotto per rispondere a un'esigenza ingegneristica precisa.

5.1 Introduzione: Una Finestra che Congela e l'Immagine Guida

5.1.1 Il Problema: L'Applicazione PrimeFinderApp

Il problema da cui parte l'intera trattazione è apparentemente elementare. Si consideri un'applicazione grafica Swing dotata di una casella di testo (`JTextField`), un pulsante "Conta" (`JButton`) e un'etichetta di stato (`JLabel`). L'utente digita un numero limite N, preme il pulsante, e l'etichetta deve mostrare quanti numeri primi esistono nell'intervallo $[1, N]$.

PrimeFinderApp: La Versione Ingenua Monothread (Capitolo 1 D04)

```
package it.unibs.pajc.thread;

import javax.swing.*;
import java.awt.*;

public class PrimeFinderApp extends JFrame {
    private final JTextField campoLimite = new JTextField("100000000", 12);
    private final JButton bottoneConta = new JButton("Conta");
    private final JLabel etichettaRisultato = new JLabel("Premi Conta per iniziare");

    public PrimeFinderApp() {
        super("Contatore Numeri Primi");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new FlowLayout());

        add(new JLabel("Fino a:"));
        add(campoLimite);
        add(bottoneConta);
        add(etichettaRisultato);

        // Listener ingenuo: calcola direttamente nel gestore dell'evento
        bottoneConta.addActionListener(e -> {
            long max = Long.parseLong(campoLimite.getText().trim());
            etichettaRisultato.setText("Calcolo in corso...");
            long totale = contaPrimi(1, max); // CHIAMATA BLOCCANTE!
```

```
            etichettaRisultato.setText(String.format("Trovati %d numeri primi.",
    totale));
        });

        pack();
        setLocationRelativeTo(null);
    }

    public static boolean isPrime(long n) {
        if (n <= 1) return false;
        if (n <= 3) return true;
        if (n % 2 == 0 || n % 3 == 0) return false;
        for (long i = 5; i * i <= n; i += 6) {
            if (n % i == 0 || n % (i + 2) == 0) return false;
        }
        return true;
    }

    public static long contaPrimi(long da, long a) {
        long c = 0;
        for (long i = da; i <= a; i++) {
            if (isPrime(i)) c++;
        }
        return c;
    }

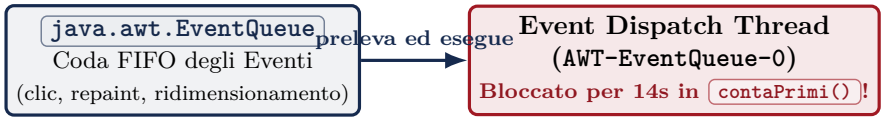
    public static void main(String[] args) {
        EventQueue.invokeLater(() -> new PrimeFinderApp().setVisible(true));
    }
}
```

5.1.2 La Diagnosi: Perché la Finestra si Blocca?

Se si avvia il programma e si richiede il calcolo per $N = 100.000.000$, accade un fenomeno catastrofico:

- Il pulsante "Conta" rimane visivamente premuto e non ritorna nella posizione rilassata;
- L'etichetta non mostra affatto la scritta intermedia `"Calcolo in corso..."`;
- La finestra non risponde al trascinamento, non si ridimensiona e non si minimizza;
- Se si tenta di chiuderla, il cursore si trasforma nella classica "girandola" (o clessidra) e il sistema operativo avverte che *"L'applicazione non risponde"*.
- Dopo circa 14 secondi (su una CPU moderna standard), all'improvviso la finestra si risveglia, il bottone si sblocca e appare il risultato finale (5.761.455 primi).

Questo accade perché in Java Swing l'intera interfaccia grafica è governata da un **unico thread**, denominato **Event Dispatch Thread (EDT)** (identificato nel runtime come `AWT-EventQueue-0`).



Cosa succede durante i 14 secondi di calcolo:
1. L'EDT è intrappolato nel metodo `contaPrimi()` invocato da `actionPerformed()`;
2. I nuovi eventi utente (spostamento mouse, click su altri bottoni, ridisegno) si accumulano nella coda;
3. Nessun evento di repaint viene processato: l'interfaccia congela completamente!

Figura 5.1: L'immagine guida della coda e dell'EDT: il blocco computazionale sull'Event Dispatch Thread.

Principio Architetturale: L'Immagine Guida: Una Coda e Chi la Svuota

Un'interfaccia grafica è un ciclo continuo: una coda di eventi (`EventQueue`) alimentata dal sistema operativo e dall'utente, e un lavoratore dedicato (`AWT-EventQueue-0`) che estrae un evento alla volta e lo esegue.
Se il lavoratore riceve un compito che richiede 14 secondi, per 14 secondi la coda non viene svuotata.

L'utente percepisce il blocco perché gli eventi di ridisegno (*paint*) rimangono intrappolati nella coda.
Per risolvere il problema, l'architettura impone una regola tassativa:
Il calcolo pesante deve essere affidato a un lavoratore separato (un Thread di background).
L'EDT deve rimanere sempre libero per svuotare la coda degli eventi.

5.2 Processi, Thread e il Modello di Memoria della JVM

Prima di invocare `new Thread(...)`, è indispensabile comprendere formalmente cosa si sta chiedendo al sistema operativo sottostante.

Definizione: Processo vs Thread

- Un **Processo** è un programma in esecuzione a cui il sistema operativo assegna uno **spazio di indirizzamento virtuale isolato**, una tabella dei descrittori di file aperti, credenziali di sicurezza e un identificatore univoco (PID). Due processi non possono leggere o scrivere l'uno nella memoria dell'altro senza ricorrere a costosi meccanismi IPC (*Inter-Process Communication*: socket, pipe, shared memory). Il cambio di contesto (*context switch*) tra processi è un'operazione pesante che richiede l'invalidazione della Translation Lookaside Buffer (TLB) della CPU e il cambio della tabella delle pagine di memoria.
- Un **Thread** (o *flusso di controllo leggero*) è l'unità fondamentale di schedulazione della CPU all'interno di un processo. Tutti i thread generati all'interno del medesimo processo **condividono lo stesso spazio di memoria (lo Heap)** e i medesimi file descriptor, ma ciascun thread possiede un proprio **Stack privato**, un Program Counter (PC) e un insieme di registri CPU.

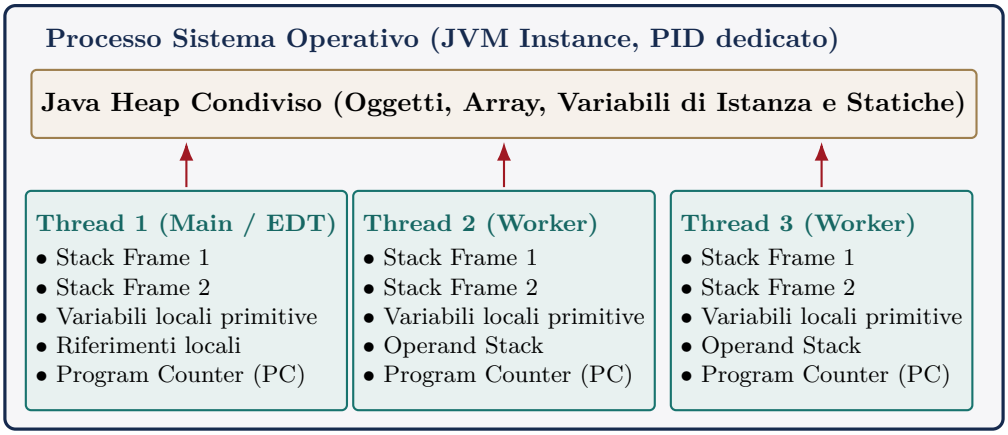


Figura 5.2: Modello di Memoria dei Thread in Java: Stack privati per-thread e Heap comune condiviso.

5.2.1 Come Fanno i Sistemi Operativi Moderni (Linux vs Windows)

Il thread di Java standard (*Platform Thread*) è un'astrazione sottile costruita direttamente sopra i thread del kernel del sistema operativo:

- **In ambiente Linux**: la JVM invoca la system call `clone()`. Nel kernel Linux non esiste una differenza strutturale profonda tra processo e thread: entrambi sono rappresentati dalla medesima struttura `task_struct`. Quando si crea un thread, `clone()` viene invocata con i flag `CLONE_VM` (condivisione dello spazio di memoria), `CLONE_FS` (condivisione del filesystem) e `CLONE_FILES` (condivisione della tabella dei descrittori di file);
- **In ambiente Windows**: la JVM invoca la funzione Win32 `CreateThread()`, che alloca le strutture native nel kernel NT;
- **La Riserva dello Stack**: per ciascun thread nativo creato, il sistema operativo riserva in anticipo uno spazio di indirizzamento virtuale per lo stack (tipicamente **1 MB** di default su sistemi a 64 bit).

Il Limite di Scala: Perché "Un Thread per Ogni Cosa" Fallisce

Poiché ogni Platform Thread richiede $\approx$ 1 MB di stack riservato e una struttura dati nel kernel, un processo non può creare thread a piacimento:

- Con 10.000 thread nativi contemporanei, solo per allocare gli stack servirebbero oltre 10 GB di

memoria virtuale;
- Lo scheduler del sistema operativo andrebbe incontro a saturazione (*thrashing*), trascorrendo più tempo ad effettuare context switch che ad eseguire il codice utile dell'applicazione;
- Attorno a 3.000–5.000 thread nativi contemporanei, il runtime Java termina con `java.lang.OutOfMemoryError: unable to create native thread`.

Questo limite strutturale ha imposto l'invenzione dei **Thread Pool** (Sezione 5.4) e, più recentemente, dei **Virtual Threads** (Sezione 5.8).

5.2.2 Ispezione Forense: `jstack` e i Thread della JVM

Quando un programma Java è in esecuzione, digitando da terminale il comando `jstack <PID>` (oppure `jcmd <PID> Thread.dump_to_file`), la JVM restituisce la fotografia istantanea (*thread dump*) di tutti i thread attivi:

```
"main" #1 prio=5 os_prio=0 cpu=14200ms tid=0x0000... in Object.wait()
"AWT-EventQueue-0" #15 prio=6 os_prio=0 cpu=120ms tid=0x0000... waiting on condition
"Common-Cleaner" #12 daemon prio=8 tid=0x0000... waiting on condition
```

Si nota immediatamente che anche un banale programma monothread possiede internamente decine di thread avviati dalla JVM: il thread `main`, i thread del Garbage Collector, il compilatore JIT e, non appena si istanzia una finestra grafica, `AWT-EventQueue-0`.

5.3 Il Primo Thread: Compito, Lavoratore e Ciclo di Vita

5.3.1 Compito vs Lavoratore: `Runnable` e `Thread`

Java separa nettamente **che cosa** va eseguito da **chi** lo esegue:

- Il **Compito** (*Task*): è definito dall'interfaccia funzionale `java.lang.Runnable`, contenente il solo metodo `void run()`;
- Il **Lavoratore** (*Worker*): è la classe `java.lang.Thread`, che chiede al sistema operativo un thread nativo per eseguire il compito.

Creazione e Avvio del Primo Thread

```java
package it.unibs.pajc.thread;

public class PrimoThreadDemo {
    public static void main(String[] args) {
        // 1. Definizione del compito tramite Runnable (lambda)
        Runnable compito = () -> {
            String nome = Thread.currentThread().getName();
            System.out.println("Esecuzione sul thread worker: " + nome);
        };

        // 2. Assegnazione del compito al lavoratore
        Thread t = new Thread(compito, "Lavoratore-1");

        // 3. Avvio del lavoratore
        t.start();

        System.out.println("Il main ha completato l'avvio e prosegue in parallelo.");
    }
}
```

Principio Architetturale: L'Errore Tipico d'Esame: `start()` vs `run()`

La differenza tra `start()` e `run()` è netta e fondamentale:
- `t.start()`: chiede alla JVM di allocare le risorse native e creare un **nuovo thread di sistema operativo**. Il metodo ritorna istantaneamente e il compito viene eseguito in concorrenza reale;
- `t.run()`: **NON crea alcun nuovo thread!** È una banale chiamata di metodo sincrona eseguita **sul thread chiamante corrente** (`main`), bloccando l'esecuzione sequenziale.

Inoltre, invocare due volte `start()` sulla medesima istanza di `Thread` (anche se ha già terminato il proprio compito) solleva tassativamente un'eccezione a runtime: `java.lang.IllegalThreadStateException`.

5.3.2 Gli Stati del Ciclo di Vita di un Thread (`Thread.State`)

Un thread Java attraversa un ciclo vitale a 6 stati formali, interrogabili tramite `getState()`:

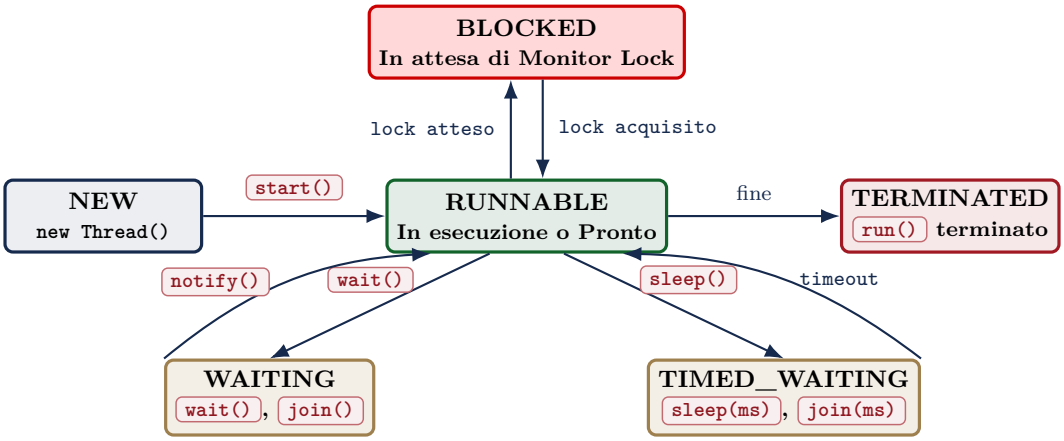


Figura 5.3: Diagramma formale degli stati del ciclo di vita di un Thread Java (`Thread.State`).

1. **NEW**: il thread è stato istanziato con `new`, ma `start()` non è ancora stato chiamato;
2. **RUNNABLE**: il thread è attualmente in esecuzione su un core CPU oppure è pronto (*ready*) nella coda dello scheduler in attesa di un quanto di tempo;
3. **BLOCKED**: il thread è sospeso in attesa di acquisire un lock intrinseco (`synchronized`) occupato da un altro thread;
4. **WAITING**: il thread è sospeso a tempo indefinito in attesa che un altro thread compia un'azione specifica (`Object.wait()`, `Thread.join()`);
5. **TIMED_WAITING**: il thread è sospeso con un tempo limite prefissato (`Thread.sleep(millis)`, `join(millis)`);
6. **TERMINATED**: il metodo `run()` ha concluso la sua esecuzione (normalmente o a causa di un'eccezione non catturata).

5.3.3 Coordinamento: `sleep()` e `join()`

- `Thread.sleep(millis)`: sospende l'esecuzione del thread corrente per la durata indicata ponendolo in `TIMED_WAITING`. Cede la CPU allo scheduler, ma **NON rilascia alcun lock acquisito**!
- `t.join()`: blocca il thread chiamante ponendolo nello stato `WAITING` fino a quando il thread bersaglio t non entra nello stato `TERMINATED`. Se t è già terminato, `join()` ritorna istantaneamente.

5.3.4 Fermare un Thread: L'Interruzione Cooperativa (`interrupt()`)

In Java i metodi di terminazione forzata come `stop()` o `destroy()` sono stati formalmente rimossi o disabilitati: interrompere bruscamente un thread a metà calcolo lascia le strutture dati parzialmente mutate e i lock aperti, corrompendo la memoria della JVM.

Java adotta una **politica di cancellazione rigorosamente cooperativa**:

- Invocare `t.interrupt()` non uccide il thread, ma alza una bandiera booleana interna (*interrupt status*);
- Se il thread si trova in uno stato di attesa bloccante (`sleep()`, `join()`, `wait()`), l'attesa viene interrotta istantaneamente, viene scatenata un'`InterruptedException` e **la bandiera viene resettata automaticamente a false**;
- Se il thread è in computazione pura, deve controllare periodicamente il flag invocando `Thread.currentThread().`

Principio Architetturale: Il Peccato Capitale del Catch Vuoto

Un errore gravissimo, purtroppo generato in automatico da molti ambienti IDE, è ingoiare l'interruzione:

```
// GRAVE ERRORE ARCHITETTURALE: il catch vuoto ingoia la richiesta di stop!
```

```
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    // Il flag e' stato azzerato e il programma continua come se nulla fosse!
}
```

La regola aurea ingegneristica impone di **ripristinare sempre la bandiera** o rilanciare l'eccezione:

```
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt(); // Ripristina lo stato di interrupt!
    return; // Esce ordinatamente dal task
}
```

5.3.5 Lambda e Stato: Cattura ed *Effectively Final* (§3.5 D04)

Un'istanza di `Runnable` può essere definita sinteticamente tramite una lambda expression passata direttamente al costruttore del thread:

```
int limite = 1000;
new Thread(() -> System.out.println("Limite: " + limite)).start();
```

Tuttavia, quando una lambda fa riferimento a variabili locali del metodo circostante, il compilatore Java impone una restrizione fondamentale: le variabili catturate devono essere **final** oppure **effectively final** (ossia variabili alle quali non viene mai assegnato un nuovo valore dopo l'inizializzazione).

La ragione architetturale risiede nel modello di memoria della JVM:

- Le variabili primitive locali risiedono nello **stack frame** del thread che ha eseguito il metodo (es. il thread `main`);
- Il nuovo thread eseguirà la lambda in un istante temporale asincrono imprevedibile, all'interno del proprio **stack privato**, potenzialmente quando il metodo del thread chiamante è già terminato e il suo stack frame è stato deallocato;
- La JVM risolve questa discrepanza effettuando una **cattura per valore** (*capture by value*): copia il valore della variabile locale all'interno dei campi sintetici generati per la lambda;
- Se fosse consentito riassegnare la variabile locale dopo la creazione della lambda, i due thread opererebbero su valori divergenti creando anomalie insanabili.

L'Illusione dello Stato Condiviso tramite Array Per aggirare il vincolo di *effectively final*, alcuni programmatori utilizzano un array di un solo elemento come contenitore mutabile:

```
long[] contatore = new long[1]; // Il riferimento all'array e' final
new Thread(() -> contatore[0]++).start(); // Mutazione del contenuto nello Heap!
```

Sebbene il compilatore accetti questa sintassi (perché il riferimento all'array non viene riassegnato), questa pratica è pericolosa: espone la cella di memoria nello Heap a corse critiche non sincronizzate tra thread concorrenti. Come enunciato dal Prof. Redolfi, per condividere o accumulare risultati tra thread non si manipolano array mutabili catturati, ma si adottano classi dedicate con incapsulamento (come `CompitoPrimi` analizzato di seguito) oppure interfacce asincrone con valore di ritorno (`Callable<V>`).

5.3.6 L'Esempio Guida: I Primi su Due Thread

Possiamo ora dividere il conteggio dei primi su due thread manuali per sfruttare due core:

Conteggio dei Primi su Due Thread Manuali (Capitolo 3 D04)

```
public class PrimiDueThread {
    // Classe per incapsulare il compito e raccogliere il risultato parziale
    static class CompitoPrimi implements Runnable {
        private final long da, a;
        private long conteggio = 0;

        public CompitoPrimi(long da, long a) {
            this.da = da;
            this.a = a;
```

```
        }

        @Override
        public void run() {
            conteggio = PrimeFinderApp.contaPrimi(da, a);
        }

        public long getConteggio() { return conteggio; }
    }

    public static void main(String[] args) throws InterruptedException {
        long max = 100_000_000L;
        long meta = max / 2;

        CompitoPrimi c1 = new CompitoPrimi(1, meta);
        CompitoPrimi c2 = new CompitoPrimi(meta + 1, max);

        Thread t1 = new Thread(c1, "Core-1");
        Thread t2 = new Thread(c2, "Core-2");

        long tInizio = System.currentTimeMillis();
        t1.start();
        t2.start();

        t1.join(); // Attende la prima meta'
        t2.join(); // Attende la seconda meta'
        long tFine = System.currentTimeMillis();

        long totale = c1.getConteggio() + c2.getConteggio();
        System.out.printf("Trovati %d primi in %d ms (2 thread manuali).\n", totale,
    (tFine - tInizio));
    }
}
```

Questo programma funziona, dimezza quasi il tempo di calcolo, ma **non scala**: se il processore ha 16 core, dovremmo scrivere 16 classi a mano, 16 chiamate a `start()`, 16 a `join()`, e il tempo totale sarebbe comunque imposto dal compito più lento. Serve un'astrazione industriale: il framework Executor.

5.4 Il Framework Executor: Disaccoppiamento tra Compito ed Esecuzione

5.4.1 Perché Non un Thread per Ogni Compito?

La creazione manuale di thread (`new Thread(task).start()`) presenta tre problemi insuperabili:

1. **Costo di allocazione**: richiede una chiamata di sistema al kernel e la riserva di 1 MB di stack;
2. **Pericolo di Thread Explosion**: in un server che riceve migliaia di richieste, il sistema collassa per esaurimento memoria;
3. **Mancanza di riutilizzo**: una volta completato `run()`, il thread muore e non può più essere riutilizzato per un altro compito.

Java 5 ha introdotto l'interfaccia `java.util.concurrent.ExecutorService`, che separa nettamente la sottomissione del compito dalla gestione fisica del pool di thread.

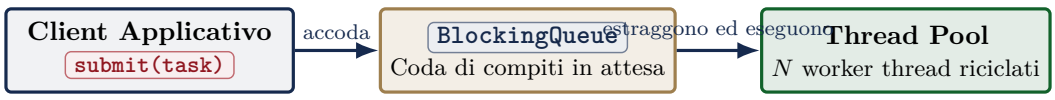


Figura 5.4: Architettura del Thread Pool: disaccoppiamento tra la coda dei compiti e i thread operai riutilizzabili.

5.4.2 Le Factory Standard di `Executors`

La classe di utilità `java.util.concurrent.Executors` offre factory preconfezionate:

- `Executors.newFixedThreadPool(int nThreads)`: alloca un numero fisso di thread. I compiti in eccesso attendono in una coda illimitata (`LinkedBlockingQueue`). Ideale per compiti CPU-bound prevedibili;
- `Executors.newCachedThreadPool()`: crea nuovi thread al bisogno e riutilizza quelli inattivi. I thread inattivi per oltre 60 secondi vengono distrutti. Ideale per molti task brevi e leggeri I/O-bound;
- `Executors.newSingleThreadExecutor()`: impiega un solo thread operaio, garantendo l'esecuzione rigorosamente sequenziale nell'ordine di accodamento;
- `Executors.newScheduledThreadPool(int corePoolSize)`: consente di pianificare compiti a intervalli temporali periodici o con ritardo prefissato.

5.4.3 Differenza Cruciale: `execute()` vs `submit()`

- `void execute(Runnable r)`: accoda il compito. Se il compito lancia un'eccezione a runtime (`RuntimeExceptio` non catturata, il thread operaio muore, la JVM stampa lo stacktrace su `System.err` e il pool genera un nuovo thread per rimpiazzarlo;
- `Future<?> submit(Runnable/Callable task)`: accoda il compito e restituisce un'istanza di `Future`. Se il compito lancia un'eccezione, il thread operaio **non muore**: l'eccezione viene catturata dal pool e registrata all'interno del `Future`. Verrà scatenata solo quando il chiamante invocherà `future.get()` (incapsulata in una `ExecutionException`).

5.4.4 Spegnere un Executor e il Supporto ad AutoCloseable

I thread all'interno di un pool non sono thread demoni (*non-daemon*): essi rimangono perennemente in attesa di nuovi compiti nella coda. Se il programmatore non chiude esplicitamente il pool, **la JVM non terminerà mai**!

Il protocollo canonico di chiusura controllata (*Two-Phase Termination*):

```
ExecutorService pool = Executors.newFixedThreadPool(4);
// Sottomissione dei compiti...
pool.shutdown(); // 1. Rifiuta nuovi compiti ma lascia completare quelli in coda
try {
    if (!pool.awaitTermination(60, TimeUnit.SECONDS)) { // 2. Attende fino a 60 secondi
        pool.shutdownNow(); // 3. Forza l'interruzione dei compiti ancora attivi
    }
} catch (InterruptedException ex) {
    pool.shutdownNow();
    Thread.currentThread().interrupt();
}
```

Principio Architetturale: Novità Java 19+: ExecutorService è AutoCloseable!

Nelle versioni moderne di Java (Java 19+ e Java 21 LTS), l'interfaccia `ExecutorService` estende `AutoCloseable`. Ciò consente di utilizzare la sintassi pulita del **try-with-resources**:

```
try (ExecutorService pool = Executors.newFixedThreadPool(numCore)) {
    pool.submit(compito1);
    pool.submit(compito2);
} // Alla chiusura del blocco, close() chiama automaticamente shutdown() e attende la
    fine!
```

5.4.5 Dimensionamento Ottimale del Pool

Quanti thread assegnare a un pool? Dipende dalla natura dei compiti:

- **Compiti CPU-Bound** (calcoli matematici puri, compressione video): il limite fisico è il numero di processori logici disponibili. Assegnare più thread non velocizza il programma, ma aumenta l'overhead del context switch:

$$N_{\text{threads}} = N_{\text{core}} (+1 \text{ per assorbire eventuali page fault})$$

dove $N_{\text{core}} =$ `Runtime.getRuntime().availableProcessors()`.

- **Compiti I/O-Bound** (lettura file, query SQL, chiamate REST su socket): i thread trascorrono gran parte del tempo sospesi in attesa dei dati. Si applica la **Formula di Brian Goetz**:

$$N_{\text{threads}} = N_{\text{core}} \times \left(1 + \frac{W}{C}\right)$$

dove W è il tempo di attesa (*Wait Time*) e C è il tempo di calcolo effettivo (*Compute Time*). Se un'interrogazione impiega $90ms$ di attesa di rete e $10ms$ di calcolo ($W/C = 9$), su una macchina a 4 core la dimensione ottimale del pool è $4 \times (1 + 9) = 40$ thread.

5.5 Calcolo Asincrono con Valore: `Callable<V>` e `Future<V>`

5.5.1 Da Runnable a Callable

L'interfaccia storica `Runnable` presenta due gravi limiti architetturali: non restituisce alcun risultato computazionale (`void run()`) e non può lanciare eccezioni controllate (*checked exception*). Per superare questi vincoli, Java introduce l'interfaccia funzionale tipizzata `Callable<V>`:

Interfaccia Callable<V> (java.util.concurrent)

```
@FunctionalInterface
public interface Callable<V> {
    V call() throws Exception; // Restituisce un tipo V generico e ammette eccezioni
}
```

Sottomettendo un `Callable<V>` a un `ExecutorService`, il metodo restituisce immediatamente un token asincrono: l'interfaccia `Future<V>`.

- `V get()`: blocca il thread chiamante in attesa del completamento e restituisce il risultato V. Se il compito è fallito con un'eccezione, `get()` solleva una `ExecutionException` che incapsula la causa reale (`getCause()`);
- `V get(long timeout, TimeUnit unit)`: attende con un tempo limite massimo, sollevando `TimeoutException` se il calcolo ritarda eccessivamente;
- `boolean isDone()`: verifica se il compito ha concluso l'esecuzione senza bloccare;
- `boolean cancel(boolean mayInterruptIfRunning)`: richiede la cancellazione del compito; se il flag è `true`, invia un segnale di interrupt al thread esecutore.

5.5.2 L'Esempio Guida: Contare i Primi fino a 10^8 su Tutti i Core

Conteggio Parallelo con FixedThreadPool e Future (Capitolo 4 D04)

```
package it.unibs.pajc.thread;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.*;

public class PrimeCounterBenchmark {
    public static void main(String[] args) throws Exception {
        long max = 100_000_000L;
        int numCore = Runtime.getRuntime().availableProcessors();
        int numCompiti = 64; // Decomposizione fine dell'intervallo

        System.out.printf("Avvio conteggio fino a %d su %d core con %d compiti...\n",
                max, numCore, numCompiti);

        long t0 = System.currentTimeMillis();

        long totalePrimi = 0;
        try (ExecutorService pool = Executors.newFixedThreadPool(numCore)) {
            List<Future<Long>> futures = new ArrayList<>();
            long passo = max / numCompiti;

            for (int k = 0; k < numCompiti; k++) {
```

```
                final long da = k * passo + 1;
                final long a = (k == numCompiti - 1) ? max : (k + 1) * passo;

                Callable<Long> compito = () -> PrimeFinderApp.contaPrimi(da, a);
                futures.add(pool.submit(compito));
            }

            // Aggregazione finale bloccante sui risultati
            for (Future<Long> f : futures) {
                totalePrimi += f.get();
            }
        } // Chiude automaticamente l'executor service

        long t1 = System.currentTimeMillis();
        System.out.printf("Trovati %d numeri primi in %d ms.\n", totalePrimi, (t1 -
    t0));
    }
}
```

5.5.3 Le Misure Empiriche e la Legge di Amdahl

Nelle lezioni del corso (Slide L04a e Dispensa D04), il professore ha presentato i risultati del benchmark eseguiti su un laptop dotato di 16 core logici (12 ad alte prestazioni P-core e 4 ad alta efficienza E-core):

Numero Thread Worker	Tempo di Calcolo (ms)	Speedup Relativo (S_N)
1 thread (sequenziale)	$14.150ms$	$1.00\times$
2 thread	$7.320ms$	$1.93\times$
4 thread	$3.910ms$	$3.61\times$
8 thread	$2.310ms$	$6.13\times$
16 thread	$1.710ms$	$8.27\times$
32 thread	$1.650ms$	$8.57\times$

Tabella 5.1: Misure empiriche d'aula: tempo e speedup al variare dei thread su 16 core hardware.

Definizione: La Legge di Amdahl (1967)

Se una frazione p del carico di lavoro totale è parallelizzabile e la frazione complementare $(1-p)$ è intrinsecamente sequenziale (es. sottomissione dei task, aggregazione dei risultati, sincronizzazioni), lo **Speedup teorico massimo** ottenibile impiegando N processori paralleli è rigidamente limitato da:

$$S(N) = \frac{1}{(1-p) + \frac{p}{N}}$$

Anche ipotizzando un numero infinito di core fisici ($N \to \infty$), il guadagno prestazionale converge asintoticamente a un tetto invalicabile:

$$S_{\max} = \lim_{N \to \infty} S(N) = \frac{1}{1-p}$$

Se anche solo il 5% del programma è sequenziale ($1 - p = 0.05$), non si potrà mai superare uno speedup di $20\times$, indipendentemente dalla potenza hardware acquistata.

5.5.4 Perché 64 Compiti e Non 16? Lo Sbilanciamento del Carico

Un quesito fondamentale posto a lezione è il seguente: se disponiamo di 16 core, perché suddividere l'intervallo in 64 compiti anziché in 16?

La risposta risiede nell'**eterogeneità computazionale dell'algoritmo**:

- Il test di primalità per un intero n esegue divisioni per tentativi fino a $\sqrt{n}$, con una complessità computazionale asintotica pari a $\mathcal{O}(\sqrt{n})$;
- Verificare la primalità dei numeri compresi tra 1 e $25 \cdot 10^6$ richiede un numero drasticamente inferiore di operazioni rispetto alla verifica dei numeri compresi tra $75 \cdot 10^6$ e $100 \cdot 10^6$;

- Se dividessimo l'intervallo in soli 16 blocchi rigidi, i thread assegnati ai primi blocchi finirebbero in pochi decimi di secondo rimanendo completamente inattivi, mentre i thread assegnati agli ultimi blocchi lavorerebbero a lungo: il tempo totale sarebbe penalizzato dallo sbilanciamento del carico (*load imbalance*);
- Suddividendo invece l'intervallo in **64 compiti fini**, i thread operai che completano rapidamente i blocchi iniziali estraggono immediatamente nuovi compiti dalla coda del pool (*dynamic work distribution*), saturando costantemente tutti i core della CPU fino al termine dell'elaborazione.

5.6 Quando i Thread Condividono: Race Condition, Bytecode e Visibilità

Nei capitoli precedenti i compiti erano completamente indipendenti: ciascun thread operava su dati propri e restituiva un valore tramite `Future`. Il momento in cui due o più thread leggono e scrivono concorrentemente sul medesimo oggetto nello Heap segna la nascita dei bug più insidiosi dell'informatica.

5.6.1 L'Anatomia di una Race Condition a Livello di Bytecode

Si consideri il contatore apparentemente banale: `count++`. Un programmatore inesperto potrebbe ritenere che l'operatore `++` sia un'istruzione indivisibile. Analizzando il codice disassemblato generato dal comando `javap -c`, scopriamo che `count++` si traduce in tre istruzioni distinte della JVM:

```
1 aload_0            // 1. Carica il riferimento a 'this' nello stack locale
2 getfield #count    // 2. Legge il valore attuale di 'count' dallo Heap
3 iconst_1           // 3. Carica la costante 1 nello stack di operandi
4 iadd               // 4. Somma 1
5 putfield #count    // 5. Scrive il nuovo valore nel campo 'count' nello Heap
```

L'Interlacciamento Distruttivo delle Istruzioni

Se due thread T_1 e T_2 eseguono contemporaneamente `count++` partendo dal valore iniziale 10:

1. T_1 esegue `getfield` e legge 10;
2. Lo scheduler del SO sospende T_1 e cede il tempo a T_2;
3. T_2 esegue `getfield` (legge 10), calcola $10 + 1 = 11$ e scrive 11 con `putfield`;
4. T_1 si risveglia: il suo stack privato possedeva già il valore 10, calcola $10 + 1 = 11$ e sovrascrive lo heap con 11!

Due incrementi distinti hanno prodotto un aumento di 1 sola unità: l'incremento di T_2 è andato perso per sempre (**Lost Update**). Se due thread eseguono ciascuno 500.000 incrementi, il risultato finale sarà un valore casuale e corrotto compreso tra 500.000 e 1.000.000.

5.6.2 Visibilità della Memoria e la Parola Chiave `volatile`

Oltre all'atomicità, esiste il problema della **Visibilità**:

- Le CPU moderne integrano cache hardware velocissime multilivello (L1, L2, L3) e write-buffers locali a ciascun core;
- Se il thread T_1 sul Core 1 scrive `running = false`, tale scrittura può rimanere confinata nel write-buffer locale e non essere riversata immediatamente nella RAM;
- Il thread T_2 sul Core 2 continua a leggere `running = true` dalla propria cache L1, rimanendo intrappolato in un loop infinito!
- Inoltre, sia il compilatore JIT sia l'hardware della CPU possono riordinare liberamente le istruzioni indipendenti (*Instruction Reordering*) per massimizzare la pipeline.

Definizione: Parola Chiave `volatile`

La dichiarazione `volatile boolean running = true;` impone al compilatore e alla CPU l'inserimento di barriere di memoria hardware (*Memory Barriers*):

- Ogni operazione di **scrittura** su una variabile `volatile` viene forzata immediatamente nella memoria RAM e invalida le cache locali degli altri core (stabilendo una relazione formale di **happens-before**);
- Ogni operazione di **lettura** legge direttamente il valore fresco dalla memoria;
- Impedisce il riordinamento delle istruzioni a monte e a valle della barriera.

ATTENZIONE CRITICA: `volatile` garantisce unicamente la **visibilità immediata**, ma **NON

garantisce l'atomicità** delle operazioni composte (`count++` su variabile volatile continua a perdere incrementi!).

5.6.3 Sincronizzazione Intrinseca: Il Monitor Lock (`synchronized`)

Per garantire contemporaneamente sia la **mutua esclusione** (atomicità) sia la **visibilità della memoria**, Java associa a ogni singolo oggetto residente nello Heap un **Monitor Lock** (*Intrinsic Lock*).

Modalità di Sincronizzazione in Java

```
public class ContatoreSincronizzato {
    private int count = 0;
    private final Object lockDedicato = new Object();

    // 1. Metodo di istanza: lock acquisito sull'istanza corrente ('this')
    public synchronized void incrementaThis() {
        this.count++;
    }

    // 2. Blocco sincronizzato fine-grained: lock su un oggetto privato dedicato
    public void incrementaBlocco() {
        // Operazioni preliminari non concorrenti...
        synchronized (this.lockDedicato) {
            this.count++;
        }
    }

    // 3. Metodo statico: lock acquisito sull'oggetto Class
    (ContatoreSincronizzato.class)
    public static synchronized void resetGlobale() {
        // Lock condiviso tra tutte le istanze della classe
    }
}
```

Principio Architetturale: Rientranza del Lock (Reentrant Monitor)

I monitor di Java sono **rientranti** (*Reentrant*): se un thread che già detiene il lock di un oggetto incontra un secondo blocco sincronizzato sul medesimo oggetto (ad esempio invocando un metodo sincronizzato all'interno di un altro metodo sincronizzato della stessa classe), la JVM riconosce che il possessore è lo stesso e lo lascia passare senza auto-bloccarsi in stallo.

5.6.4 Coordinamento con `wait()`, `notify()` e la Regola del Loop While

Quando un thread ha acquisito il monitor ma una condizione logica non è verificata (ad esempio, un buffer è vuoto), può rilasciare il lock e attendere:

- `lock.wait()`: **rilascia atomicamente il monitor lock** e pone il thread nello stato `WAITING` all'interno del *wait-set* del monitor;
- `lock.notifyAll()`: risveglia tutti i thread attualmente sospesi sul monitor. Essi competono per riacquisire il lock e riprendere l'esecuzione;
- `lock.notify()`: risveglia un **unico** thread scelto arbitrariamente dalla JVM.

Perché Preferire Sempre `notifyAll()` a `notify()` (§5.4 D04) L'impiego di `notify()` è fonte di bug insidiosi noti come **segnali persi** (*missed signals*):

1. Se molteplici thread sono in attesa per condizioni logiche differenti (es. produttori in attesa di spazio libero e consumatori in attesa di nuovi elementi);
2. Invocando `notify()`, la JVM potrebbe risvegliare casualmente un produttore quando invece si è liberato un dato per un consumatore;
3. Il produttore si risveglia, verifica che il buffer è ancora pieno, e si rimette immediatamente in `wait()`;
4. Poiché nessun altro thread è stato svegliato, il segnale è svanito nel nulla e tutti i thread rimangono bloccati a tempo indeterminato.

Per questa ragione, la prassi ingegneristica raccomanda di utilizzare sempre `notifyAll()`: ogni thread rivaluterà la propria condizione nel ciclo `while` e solo chi ha il predicato soddisfatto proseguirà.

Principio Architetturale: La Regola d'Oro: Il Ciclo While contro gli Spurious Wakeups

Un thread in attesa su `wait()` può risvegliarsi in modo anomalo a causa di risvegli spuri della CPU (*spurious wakeups*) o perché un altro thread risvegliato per primo ha già consumato la risorsa. È una **regola tassativa di programmazione** verificare sempre la condizione all'interno di un ciclo `while`, MAI con un semplice `if`:

```
synchronized (lock) {
    while (!condizionePronta) { // MAI usare 'if' qui!
        lock.wait();
    }
    // Esegui azione protetta
}
```

5.6.5 Deadlock e le Quattro Condizioni di Coffman

L'impiego improprio di lock annidati può condurre al **Deadlock** (*Stallo Cieco*): due o più thread rimangono bloccati a tempo indefinito, ciascuno in attesa del lock detenuto dall'altro.

Definizione: Le Quattro Condizioni di Coffman (1971)

Affinché si verifichi un Deadlock, devono coesistere contemporaneamente quattro condizioni:

1. **Mutua Esclusione**: risorsa detenuta in modo esclusivo;
2. **Possesso e Attesa (*Hold and Wait*)**: un thread detiene un lock e ne attende un altro;
3. **Assenza di Prelazione (*No Preemption*)**: un lock non può essere sottratto con la forza;
4. **Attesa Circolare (*Circular Wait*)**: esiste un ciclo chiuso di dipendenze ($T_1 \to L_2 \to T_2 \to L_1$).

Principio Architetturale: Tecnica Architetturale di Prevenzione: Global Lock Ordering

Per distruggere alla radice la quarta condizione (l'attesa circolare), tutti i thread dell'applicazione devono acquisire i lock **sempre e solo nel medesimo ordine globale prefissato**. Se occorre trasferire denaro tra due conti A e B, si confrontano i rispettivi identificativi univoci: il lock del conto con ID minore viene acquisito sempre per primo, eliminando a priori la possibilità di stalli ciclici.

5.6.6 Thread Safety per Costruzione

Prima ancora di ricorrere ai lock, l'ingegnere del software adotta tre strategie di design per prevenire i conflitti alla fonte:

1. **Confinamento nello Stack (*Thread Confinement*)**: le variabili locali allocate all'interno dei metodi risiedono nello stack privato del thread. Se un oggetto viene creato, utilizzato e distrutto all'interno di un metodo senza mai pubblicarne il riferimento nello Heap, esso è intrinsecamente immune da race condition;
2. **Immutabilità (*Immutability*)**: un oggetto immutabile (come i `record` di Java 16+ o classi con campi `final` e privi di metodi setter) non può essere mutato dopo la costruzione. Più thread possono leggere contemporaneamente un oggetto immutabile senza alcun lock di sincronizzazione;
3. **Confinamento dei Dati (*Data Confinement*)**: incapsulare lo stato mutabile all'interno di una classe e accedervi unicamente tramite metodi sincronizzati ben definiti.

5.7 Gli Strumenti Avanzati di `java.util.concurrent`

5.7.1 Lock Espliciti: `ReentrantLock`

La classe `ReentrantLock` sostituisce `synchronized` fornendo funzionalità avanzate:

- `tryLock(long timeout, TimeUnit unit)`: tenta di acquisire il lock per un tempo massimo, consentendo al programma di rinunciare ed evitare stalli prolungati;
- Politica di equità (*fairness*): garantisce l'acquisizione FIFO eliminando la starvation dei thread;
- Protocollo tassativo `try-finally`:

```java
ReentrantLock lock = new ReentrantLock();
lock.lock();
try {
    // Sezione critica protetta
} finally {
    lock.unlock(); // RILASCIO TASSATIVO NEL FINALLY!
}
```

5.7.2 Lettori e Scrittori: `ReadWriteLock`

La classe `ReentrantReadWriteLock` dissocia il lock in due componenti coordinate:

- **Lock di Lettura (`readLock()`)**: molteplici thread lettori possono accedere contemporaneamente in parallelo senza bloccarsi a vicenda;
- **Lock di Scrittura (`writeLock()`)**: l'accesso in scrittura è rigorosamente esclusivo (nessun altro lettore o scrittore ammesso).

È la struttura ideale per collezioni a forte prevalenza di letture (es. 99% letture e 1% aggiornamenti), anche se un flusso ininterrotto di lettori può causare la starvation dello scrittore.

5.7.3 Risorse a Posti Limitati: `Semaphore`

La classe `Semaphore` modella un insieme di permessi numerici (*permits*). Un thread invoca `acquire()` per ottenere un permesso (bloccandosi se sono esauriti) e `release()` per restituirlo al termine:

```java
Semaphore corsia = new Semaphore(10); // Massimo 10 veicoli simultanei
corsia.acquire();
try {
    transitaCorsia();
} finally {
    corsia.release();
}
```

5.7.4 Variabili Atomiche Lock-Free (`java.util.concurrent.atomic`)

Le classi `AtomicInteger`, `AtomicLong` e `AtomicReference` implementano operazioni aritmetiche atomiche (`incrementAndGet()`, `compareAndSet()`) senza utilizzare alcun blocco sincronizzato.

Sotto il cofano, sfruttano l'istruzione hardware **CAS** (*Compare-And-Swap*) implementata nativamente dal silicio delle CPU moderne:

$$\text{CAS}(\text{indirizzo}, \text{valoreAtteso}, \text{nuovoValore})$$

La CPU confronta il valore in memoria e lo aggiorna in un unico ciclo di clock hardware solo se corrisponde a quello atteso, ritentando automaticamente in un loop ottimistico qualora un altro core abbia modificato il valore nel frattempo.

5.7.5 La Soluzione alla Contesa Estrema: `LongAdder`

Quando decine di thread concorrenti incrementano costantemente una singola variabile `AtomicLong`, si verifica il fenomeno della **contesa sul bus di memoria**: tutti i thread competono sul medesimo indirizzo di memoria, i tentativi CAS falliscono continuamente a raffica e le prestazioni crollano.

Per risolvere questo problema, Java 8 ha introdotto **`LongAdder`**:

- Anziché memorizzare un unico valore centrale, `LongAdder` alloca dinamicamente una **tabella interna di celle a 64 bit distribuite** (*striped cells*);
- Ciascun thread opera e incrementa la propria cella locale senza alcuna contesa e senza fallimenti CAS;
- La somma complessiva viene calcolata solo quando il programma richiede il totale invocando `sum()`.

Meccanismo Concorrente	Tempo Esecuzione Contesa Estrema	Caratteristiche Chiave
Contatore Monothread	$18ms$	Non thread-safe, corrompe i dati
`synchronized`	$74ms$	Serializzazione forzata dei thread
`AtomicLong` (CAS singolo)	$391ms$	Degradato dai continui tentativi CAS falliti
`LongAdder` (Celle distribuite)	$66ms$	**Nessuna contesa, throughput massimo**

Tabella 5.2: Benchmark d'aula: tempo impiegato per eseguire 10.000.000 incrementi distribuiti su 16 thread.

5.7.6 Collezioni Concorrenti Thread-Safe

- `ConcurrentHashMap`: tabella hash ad altissimo throughput. Non sincronizza l'intera mappa, ma opera tramite primitive CAS e locking a livello di singolo bucket; espone metodi atomici composti come `putIfAbsent()` e `computeIfAbsent()`;
- `CopyOnWriteArrayList`: ogni mutazione (`add()`, `remove()`) alloca e riscrive una nuova copia immutabile dell'array sottostante. Le letture sono lock-free e prive di sincronizzazione: è la struttura ottimale per liste di listener grafici (moltissime iterazioni, rarissime modifiche);
- `BlockingQueue` (`ArrayBlockingQueue`, `LinkedBlockingQueue`): realizza il pattern Produttore-Consumatore fornendo metodi bloccanti thread-safe `put()` (attende se la coda è piena) e `take()` (attende se la coda è vuota).

5.7.7 Quale Strumento Scegliere: La Guida Decisionale (§6.6 D04)

Di fronte alla vasta gamma di primitive fornite dalla JVM e dal package `java.util.concurrent`, lo sviluppatore deve saper selezionare lo strumento con il minimo costo prestazionale e il massimo rigore formale. Il Prof. Redolfi sintetizza questa tassonomia nella seguente matrice decisionale canonica:

Problema Ingegneristico	**Strumento Raccomandato**
Una bandiera scritta da un thread e letta da altri	`volatile boolean` oppure `AtomicBoolean`
Un contatore o un riferimento scalare singolo	`AtomicInteger` / `AtomicLong` / `AtomicReference`; `LongAdder` se la contesa è elevata
Più campi correlati da aggiornare atomicamente	Blocco `synchronized` su lock privato; `ReentrantLock` se occorrono timeout o equità (*fairness*)
Accessi con moltissime letture e rarissime scritture	`ReadWriteLock` (`ReentrantReadWriteLock`); `CopyOnWriteArrayList` per collezioni di listener
Regolazione di una risorsa a capienza limitata (N permessi)	`Semaphore`
Passaggio asincrono e disaccoppiato di dati fra thread	`BlockingQueue` (`ArrayBlockingQueue` o `LinkedBlockingQueue`)
Dizionario o cache condivisa ad alto throughput	`ConcurrentHashMap` con mutazioni atomiche (`compute()`, `merge()`)
Attesa coordinata che una condizione logica si verifichi	`wait()`/`notifyAll()` su monitor, `Condition` esplicita o `CountDownLatch`
Assenza di stato condiviso	**La scelta migliore in assoluto: Immutabilità e Confinamento**

Tabella 5.3: Matrice di selezione degli strumenti concorrenti (Capitolo 6 D04).

5.8 Il Tempo, l'Attesa e i Thread Virtuali (Java 21)

5.8.1 Compiti Differiti e Periodici: `ScheduledExecutorService`

Per eseguire compiti ritardati o periodici si impiega `ScheduledExecutorService`:

- `schedule(task, delay, unit)`: esegue il compito una sola volta dopo il ritardo prefissato;
- `scheduleAtFixedRate(task, init, period, unit)`: esegue il compito a intervalli regolari calcolati dall'istante di avvio (*fixed rate*);
- `scheduleWithFixedDelay(task, init, delay, unit)`: attende che il compito precedente sia terminato, e solo allora avvia il timer di attesa prima della successiva esecuzione (*fixed delay*).

5.8.2 Stream Paralleli Visti da Sotto: `parallelStream()` (§7.2 D04)

Con quanto appreso sugli executor, è possibile comprendere esattamente cosa accade a basso livello quando si invoca `.parallel()` su uno stream Java:

```
long n = LongStream.rangeClosed(1, 100_000_000)
    .parallel()
    .filter(PrimeTester::isPrime)
    .count();
```

Il funzionamento interno orchestrato dalla JVM prevede tre passaggi:

1. **Partizionamento Ricorsivo**: la libreria interroga lo `Spliterator` della sorgente invocando `trySplit()` ricorsivamente, spezzando i dati in sotto-intervalli finché non raggiungono una grana fine;

2. **Sottomissione a ForkJoinPool**: ogni blocco viene consegnato come compito al pool comune della JVM, il **`ForkJoinPool.commonPool()`**. Si tratta di un pool con algoritmo di *work-stealing* dotato di $N_{\text{CPU}} - 1$ thread operai, a cui si aggiunge il thread chiamante stesso che partecipa all'esecuzione;
3. **Fusione dei Risultati**: al termine dell'albero di sottomissione, i risultati parziali vengono ridotti e combinati. In pratica, la libreria riproduce esattamente l'architettura a 64 compiti analizzata nel Capitolo 4.

Il Pericolo degli Stream Paralleli con Operazioni Bloccanti Poiché il `commonPool()` è un'istanza singola e condivisa dall'intera JVM per tutti i flussi paralleli dell'applicazione:

È SEVERAMENTE VIETATO eseguire operazioni I/O o socket bloccanti dentro uno stream parallelo!

Se un thread del `commonPool()` si blocca in attesa di un server HTTP lento, quel thread non partecipa al calcolo: pochi compiti bloccanti saturano l'intero pool comune, paralizzando qualsiasi altro stream o elaborazione asincrona in esecuzione sull'intera applicazione.

5.8.3 La Rivoluzione dei Virtual Threads (Java 21 LTS, JEP 444)

Con il rilascio di **Java 21**, il modello di concorrenza ha subito la mutazione più radicale della sua storia con l'introduzione dei **Virtual Threads** (Project Loom).

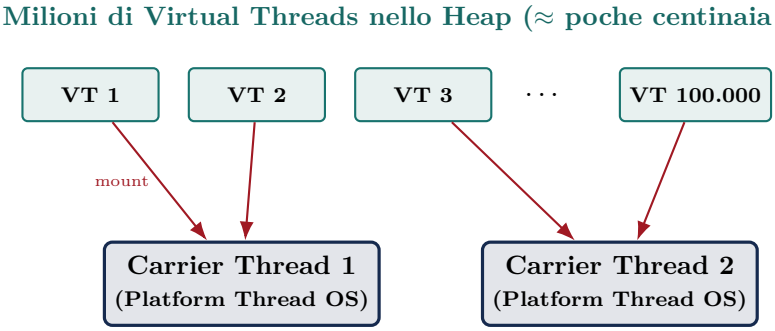


Figura 5.5: Architettura M:N dei Virtual Threads: milioni di flussi leggeri montati su pochi Carrier Threads fisici.

Il funzionamento dei Virtual Threads è straordinario:

1. Un Virtual Thread occupa inizialmente solo **poche centinaia di byte** nello Heap della JVM (e non 1 MB di memoria nativa!);
2. La JVM esegue i Virtual Threads montandoli (*mount*) su un piccolo pool di platform thread del sistema operativo detti **Carrier Threads** (pari al numero di core CPU);
3. Quando il Virtual Thread compie un'operazione bloccante (lettura da socket HTTP, accesso a database JDBC, lettura file su disco o `Thread.sleep()`):
   La JVM effettua l'unmount istantaneo del Virtual Thread dal Carrier Thread!
   Lo stack del Virtual Thread viene congelato nello Heap e il Carrier Thread fisico del sistema operativo viene immediatamente riutilizzato per eseguire un altro Virtual Thread!
4. Quando l'operazione di I/O si sblocca, la JVM rimonta il Virtual Thread su un Carrier Thread disponibile e il codice riprende linearmente da dove era stato sospeso.

Creazione Massiva di 100.000 Virtual Threads in Java 21

```
1  package it.unibs.pajc.thread;
2
3  import java.time.Duration;
4  import java.util.concurrent.Executors;
5
6  public class VirtualThreadDemo {
7      public static void main(String[] args) throws Exception {
8          // NON si usano piu' pool fissi limitati: ogni compito riceve un Virtual
       Thread dedicato!
9          try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
```

```
            for (int i = 0; i < 100_000; i++) {
                final int id = i;
                executor.submit(() -> {
                    Thread.sleep(Duration.ofSeconds(1)); // Attesa bloccante non costosa
                    if (id % 25_000 == 0) {
                        System.out.printf("Virtual Thread #%d completato con successo.\n", id);
                    }
                    return id;
                });
            }
        } // Attende automaticamente la conclusione di tutti i 100.000 thread!

        System.out.println("Tutti i 100.000 compiti asincroni sono terminati.");
    }
}
```

Principio Architetturale: Quando Usare i Virtual Threads e Quando NO

- **SI (Ideali per carichi I/O-Bound)**: server web, microservizi HTTP, interrogazioni a database, architetture a micro-agenti. Permette di abbandonare la complessità del reactive programming e tornare a scrivere codice imperativo sincrono lineare;
- **NO (Inutili per carichi CPU-Bound)**: nel calcolo intensivo (come il conteggio dei numeri primi o la cifratura dati), il collo di bottiglia è la capacità di calcolo dei core fisici. Creare 100.000 virtual thread per contare i primi non porta alcun beneficio rispetto a un pool di 16 thread nativi.

5.8.4 L'Esempio Guida in Tre Modi: Calcolo vs Attesa (§7.4 D04)

A coronamento della trattazione sulla concorrenza, il Prof. Redolfi mette a confronto i diversi modelli di esecuzione sottoponendoli a due carichi di lavoro diametralmente opposti:

1. **Carico di Calcolo (CPU-Bound)**: conteggio dei numeri primi fino a 10^8 suddiviso in 64 compiti fini;
2. **Carico di Attesa (I/O-Bound)**: 10.000 compiti indipendenti, ciascuno dei quali simula una chiamata di rete o una lettura su disco bloccandosi per $1s$ con `Thread.sleep(1000)`.

I risultati sperimentali misurati su una macchina di sviluppo a 16 core logici evidenziano in modo inequivocabile i limiti e i punti di forza di ciascuna architettura:

Modello di Esecuzione	**Calcolo (10^8 Primi)**	**Attesa ($10.000 \times 1s$ I/O)**
Pool Fisso (16 Platform Threads)	$1{,}4s$	$625s$ ($\approx 10{,}4min$, 625 turni da 16)
Pool Fisso (200 Platform Threads)	$1{,}5s$	$50s$ (50 turni sequenziali)
Cached Pool (`Executors.newCachedThreadPool`)	$1{,}5s$	$1{,}9s$ (10.000 thread OS nativi generati)
Stream Parallelo (`.parallel()`)	$1{,}4s$	**Non Adatto** (satura il `commonPool`)
Virtual Threads (Java 21, 1 per compito)	$1{,}5s$	$1{,}0s$ **(Trionfo: ≈ 20 thread OS, zero overhea**

Tabella 5.4: La prova comparativa d'aula: CPU-bound vs I/O-bound a confronto su 16 core (D04).

Analisi Comparativa dei Risultati

- **Sul calcolo puro**, le prestazioni sono identiche ($\approx 1{,}4 - 1{,}5s$): il collo di bottiglia fisico sono i 16 core della CPU. Creare 10.000 thread virtuali non apporta alcun beneficio rispetto a 16 thread nativi;
- **Sull'attesa bloccante**, il pool fisso crolla miseramente: i 16 thread del kernel rimangono congelati nel sonno, costringendo i 10.000 compiti a formare una coda di 625 turni sequenziali per oltre 10 minuti di stallo;
- Il Cached Pool riesce a completare in $1{,}9s$, ma a un prezzo proibitivo: istanzia 10.000 thread nativi del sistema operativo, consumando oltre $10GB$ di memoria virtuale e rischiando il crash della macchina (*Thread Explosion*);
- I **Virtual Threads** completano tutti i 10.000 compiti nell'istante esatto dell'attesa ($1{,}0s$), impiegando soltanto una ventina di Carrier Threads fisici e poche decine di megabyte nello Heap.

5.9 Thread e Swing: Risolvere il Congelamento con SwingWorker

Torniamo ora all'applicazione iniziale 'PrimeFinderApp' e all'immagine guida introdotta nella Sezione 5.1.

5.9.1 La Regola del Single-Threaded Subsystem e `EventQueue.invokeLater()`

Tutti i componenti grafici di Swing **non sono thread-safe**. Per azzerare i lock nel rendering a 60 FPS, Swing impone che qualsiasi modifica ai componenti visibili debba avvenire **esclusivamente all'interno dell'Event Dispatch Thread (EDT)**.

Se un thread worker di background desidera aggiornare una label o una tabella, deve inserire un compito nella coda `EventQueue`:

```
EventQueue.invokeLater(() -> {
    etichettaRisultato.setText("Calcolo completato: " + totale);
});
```

5.9.2 L'Astrazione Dedicata: `javax.swing.SwingWorker<T, V>`

Per gestire in modo strutturato compiti di lunga durata con risultati intermedi, avanzamento percentuale e annullamento, Java Swing fornisce la classe astratta tipizzata `SwingWorker<T, V>`:

- T: tipo del risultato finale restituito al termine del calcolo;
- V: tipo dei dati parziali trasmessi progressivamente all'interfaccia.

I quattro metodi cardinali di `SwingWorker`:

1. `T doInBackground() throws Exception`: eseguito su un **thread worker di background** del pool interno di Swing. Contiene il calcolo pesante e non deve MAI toccare componenti grafici;
2. `publish(V... chunks)`: invocato dall'interno di `doInBackground()` per inviare blocchi di dati parziali;
3. `process(List<V> chunks)`: eseguito **automaticamente sull'EDT**. Riceve i dati parziali e aggiorna in sicurezza barre di avanzamento (`JProgressBar`) o tabelle;
4. `done()`: eseguito **automaticamente sull'EDT** al termine di `doInBackground()`. Consente di recuperare il risultato con `get()`, riabilitare i pulsanti e gestire eventuali errori.

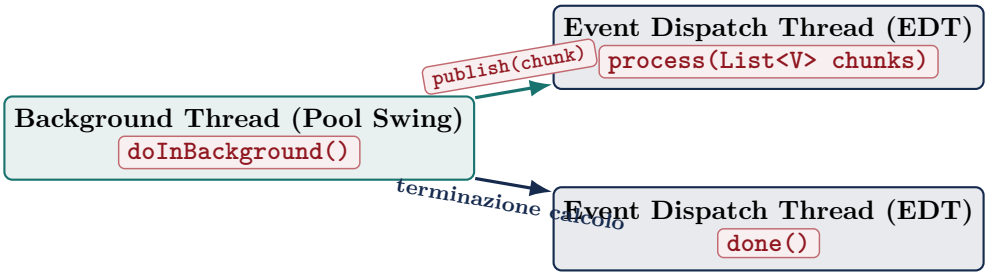


Figura 5.6: Il flusso di esecuzione di `SwingWorker`: disaccoppiamento tra thread di background ed EDT.

5.9.3 La Soluzione Completa: PrimeFinderApp Reattiva con SwingWorker

PrimeFinderApp v3: GUI Reattiva con SwingWorker e Annullamento (Capitolo 8 D04)

```
package it.unibs.pajc.thread;

import javax.swing.*;
import java.awt.*;
import java.util.List;

public class PrimeFinderAppReattiva extends JFrame {
    private final JTextField campoLimite = new JTextField("100000000", 10);
    private final JButton bottoneConta = new JButton("Conta");
    private final JButton bottoneAnnulla = new JButton("Annulla");
    private final JProgressBar barraAvanzamento = new JProgressBar(0, 100);
    private final JLabel etichettaRisultato = new JLabel("Pronto.");

    private CalcoloWorker workerCorrente = null;

    public PrimeFinderAppReattiva() {
        super("Contatore Primi Reattivo");
```

```
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new FlowLayout());

        bottoneAnnulla.setEnabled(false);
        barraAvanzamento.setStringPainted(true);

        add(new JLabel("Fino a:"));
        add(campoLimite);
        add(bottoneConta);
        add(bottoneAnnulla);
        add(barraAvanzamento);
        add(etichettaRisultato);

        bottoneConta.addActionListener(e -> {
            long max = Long.parseLong(campoLimite.getText().trim());
            bottoneConta.setEnabled(false);
            bottoneAnnulla.setEnabled(true);
            barraAvanzamento.setValue(0);
            etichettaRisultato.setText("Calcolo in corso...");

            workerCorrente = new CalcoloWorker(max);
            workerCorrente.execute(); // Avvia lo SwingWorker in background
        });

        bottoneAnnulla.addActionListener(e -> {
            if (workerCorrente != null) {
                workerCorrente.cancel(true); // Richiede l'interruzione cooperativa
            }
        });

        pack();
        setLocationRelativeTo(null);
    }

    private class CalcoloWorker extends SwingWorker<Long, Integer> {
        private final long max;

        public CalcoloWorker(long max) {
            this.max = max;
        }

        @Override
        protected Long doInBackground() throws Exception {
            long contatore = 0;
            long step = max / 100;
            for (long i = 1; i <= max; i++) {
                if (Thread.currentThread().isInterrupted()) {
                    return contatore; // Uscita cooperativa se annullato
                }
                if (PrimeFinderApp.isPrime(i)) contatore++;

                if (step > 0 && i % step == 0) {
                    publish((int) ((i * 100) / max)); // Invia percentuale parziale
                }
            }
            return contatore;
        }

        @Override
        protected void process(List<Integer> chunks) {
            // ESEGUITO SULL'EDT: sicuro per aggiornare la barra grafica!
            int ultimaPercentuale = chunks.get(chunks.size() - 1);
            barraAvanzamento.setValue(ultimaPercentuale);
        }

        @Override
```

```
84          protected void done() {
85              // ESEGUITO SULL'EDT: ripristina i controlli dell'interfaccia
86              bottoneConta.setEnabled(true);
87              bottoneAnnulla.setEnabled(false);
88              if (isCancelled()) {
89                  etichettaRisultato.setText("Calcolo annullato dall'utente.");
90              } else {
91                  try {
92                      long totale = get();
93                      etichettaRisultato.setText(String.format("Trovati %d numeri
    primi.", totale));
94                  } catch (Exception ex) {
95                      etichettaRisultato.setText("Errore durante il calcolo: " +
    ex.getMessage());
96                  }
97              }
98          }
99      }
100
101     public static void main(String[] args) {
102         EventQueue.invokeLater(() -> new PrimeFinderAppReattiva().setVisible(true));
103     }
104 }
```

5.9.4 Rientro nell'EDT e la Trappola di `invokeAndWait` (§8.5 D04)

Mentre `EventQueue.invokeLater(r)` accoda il compito e ritorna immediatamente in modo asincrono, il toolkit AWT mette a disposizione anche `EventQueue.invokeAndWait(r)`, che sospende il thread chiamante finché l'EDT non ha terminato l'esecuzione di `r`. Questo metodo è concepito per scenari in cui un worker thread ha bisogno di un valore di risposta dall'interfaccia utente (ad esempio il testo digitato in un campo o la conferma da una finestra di dialogo modale).

La Trappola Mortale: Mai Chiamare `invokeAndWait` dall'EDT! Se `EventQueue.invokeAndWait(r)` viene invocato accidentalmente da un metodo in esecuzione sull'EDT (come un listener di un pulsante):

La JVM solleva immediatamente un `Error` bloccante!

Se la JVM non intercettasse questa chiamata con un errore, si verificherebbe un **deadlock istantaneo e irreversibile**: l'EDT si metterebbe in attesa del completamento del compito `r`, ma nessun altro thread può eseguire compiti sull'EDT tranne l'EDT stesso! L'EDT attenderebbe se stesso all'infinito.

> **Principio Architetturale: La Ricetta Canonica: Esecuzione Sicura su EDT da Qualsiasi Thread**
>
> Per aggiornare un componente grafico da una libreria o da un metodo che potrebbe essere invocato sia da thread di background sia direttamente dall'EDT, il Prof. Redolfi raccomanda di utilizzare il predicato `EventQueue.isDispatchThread()`:
>
> ```
> public void aggiornaTesto(String nuovoTesto) {
>     Runnable azione = () -> etichetta.setText(nuovoTesto);
>     if (EventQueue.isDispatchThread()) {
>         azione.run(); // Gia' sull'EDT: esecuzione immediata sincrona
>     } else {
>         EventQueue.invokeLater(azione); // Fuori dall'EDT: accodamento asincrono sicuro
>     }
> }
> ```
>
> Questo pattern elimina ogni rischio di deadlock e garantisce il rispetto formale della regola del singolo thread grafico.

5.9.5 Compiti Periodici sull'Interfaccia: `javax.swing.Timer` (§8.7 D04)

Quando un'applicazione grafica necessita di un compito periodico che modifica i componenti dello schermo (un orologio digitale, un'animazione a frame, un lampeggio di stato o il polling periodico di una barra):

- L'impiego di `ScheduledExecutorService` (Sezione 5.8) eseguirebbe il compito sul thread sbagliato (un worker thread del pool), imponendo a ogni iterazione di re-inoltrare l'aggiornamento con `invokeLater()`;

- Il framework Swing fornisce la classe specializzata **`javax.swing.Timer`**:

```
// Esegue l'azione ogni 1000 ms DIRETTAMENTE sull'EDT:
Timer timer = new Timer(1000, evento -> {
    etichettaOra.setText(LocalTime.now().format(DateTimeFormatter.ofPattern("HH:mm:ss")));
});
timer.start();
...
timer.stop(); // Arresta il timer
```

Principio Architetturale: Criterio di Scelta: `javax.swing.Timer` vs `ScheduledExecutorService`

- **`javax.swing.Timer`**: va utilizzato per compiti **brevi e leggeri** che interagiscono con i pixel e i componenti grafici, poiché il listener gira direttamente sull'EDT senza richiedere dispatching;
- **`ScheduledExecutorService`**: va utilizzato per compiti **pesanti o bloccanti** (letture su disco, interrogazioni HTTP di rete periodiche, calcolo matematico) che devono tassativamente risiedere fuori dall'EDT per non congelare la finestra.

5.10 Orizzonti Avanzati della Concorrenza Moderna

A conclusione della trattazione, la moderna piattaforma Java mette a disposizione strumenti avanzati per scenari computazionali complessi:

- `CompletableFuture<T>`: astrazione per la programmazione asincrona dichiarativa. Permette di concatenare trasformazioni e aggregazioni di compiti concorrenti (`thenApply()`, `thenCompose()`, `thenCombine()`, `allOf()`) senza mai bloccare con `get()`;
- `ForkJoinPool` e `RecursiveTask<V>`: architettura basata sul paradigma *Divide-et-Impera* ricorsivo e su algoritmi di **Work-Stealing**: quando un worker thread termina il proprio sotto-albero computazionale, preleva ("ruba") automaticamente compiti dalla coda degli altri worker thread ancora occupati;
- **Structured Concurrency** (*JEP 453*, `StructuredTaskScope`): introdotta come funzionalità in anteprima in Java 21+, tratta insiemi di compiti concorrenti correlati come una singola unità logica di lavoro, garantendo che se un sotto-compito fallisce, tutti gli altri compiti correlati vengano cancellati automaticamente prima di uscire dal blocco di codice;
- **Scoped Values** (*JEP 446*, `ScopedValue`): sostituiscono in modo efficiente e immutabile le storiche `ThreadLocal`, condividendo dati di contesto in sola lettura tra milioni di Virtual Threads a costo di allocazione zero.

Consultazione Eserciziario e Formulario su Thread e Concorrenza

Per le prove pratiche d'esame, i benchmark empirici e gli esercizi completi tratti dalla Palestra d'aula del Prof. Redolfi, si consulti l'**Appendice A** (Esercizi A.5–A.5). Per le tabelle sinottiche di consultazione rapida delle API e dei costrutti concorrenti, si consulti l'**Appendice B**.

Appendice A

Eserciziario e Tracce di Laboratorio

Il presente capitolo raccoglie organicamente tutte le tracce, le specifiche di collaudo e le consegne degli esercizi del corso, integrando le attività svolte durante le lezioni frontali, i compiti assegnati per casa (*homework*) e le simulazioni di progetti complessi.

Conformemente all'impostazione metodologica e al rigore architetturale del corso, ciascun esercizio è catalogato formalmente secondo **tre livelli progressivi di difficoltà**:

- **Livello 1 (Base – Sintassi, API e Strutture Dati)**: focalizzato sulla padronanza delle firme dei metodi, dei tipi primitivi e wrapper, e delle classi core del runtime Java.
- **Livello 2 (Intermedio – Algoritmico e Problem Solving)**: richiede la combinazione di più principi (incapsulamento, polimorfismo, programmazione funzionale, gestione dell'I/O ed eccezioni).
- **Livello 3 (Avanzato – Architettura ad Oggetti Multi-file)**: progetti d'ingegneria del software su larga scala articolati su 4–8+ classi e interfacce interconnesse, con contratti formali, thread-safety, asserzioni TDD e assenza totale di codice simulato.

A.1 Capitolo 1: Warm-up e Fondamenti della Programmazione a Oggetti

Fascia 1: Base (Sintassi, API e Strutture Dati)

Esercizio 1.1: StringMagic – Manipolazione Testuale, Incapsulamento e TDD

Livello: *Livello 1 – Base.*
Argomento del Task: Creazione di classi con stato immutabile/incapsulato, costruttori, scansione algoritmica di sequenze di caratteri e collaudo guidato dai test (*Test-Driven Development*).
Riferimento Didattico: Slide 41 (*PAJC_L01_Warm-up.pdf* – Sezione *"HOMEWORK"*).

Consegna Operativa: Si richiede di realizzare una classe denominata `StringMagic` per la rappresentazione incapsulata di un testo, esponendo:

1. Costruttore esplicito che riceve la stringa base;
2. Metodo `reverse()` che restituisce una nuova stringa con i caratteri invertiti;
3. Metodo `maskVowel()` che sostituisce tutte le vocali (minuscole e maiuscole) con il carattere asterisco `'*'`;
4. Metodo `freq(char c)` che restituisce il conteggio esatto delle occorrenze del carattere `c`.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: StringMagicTest.java

```
StringMagic s = new StringMagic("un seme magico");
assert "ocigam emes nu".equals(s.reverse()) : "Errore reverse!";
assert "*n s*m* m*g*c*".equals(s.maskVowel()) : "Errore maskVowel!";
assert s.freq('m') == 2 : "Errore frequenza di 'm'!";
assert s.freq('z') == 0 : "Errore frequenza carattere assente!";
```

Esercizio 1.2: FileUtil – Ispezione del Filesystem, Visualizzazione Numerata e Hex Dump

Livello: *Livello 1 – Base.*
Argomento del Task: Gestione degli stream di I/O a caratteri e a byte (`BufferedReader`, `FileInputStream`), gestione deterministica delle risorse (`try-with-resources`) ed esame forense esadecimale.
Riferimento Didattico: Slide 56 (*PAJC_L01_Warm-up.pdf* – Sezione *"COMPITO"*).

Consegna Operativa: Progettare una classe di utilità `FileUtil` dotata dei seguenti metodi statici:

1. `print(String filename)`: legge un file di testo e ne visualizza le righe precedute dal numero progressivo allineato a destra (`"%3d | %s\n"`);
2. `dump(String filename)`: apre un flusso a byte e visualizza l'esadecimale a blocchi di 16 byte per riga, stampando offset a 8 cifre (`00000000:`), byte esadecimali separati da spazi e caratteri ASCII stampabili (sostituendo con `'.'` i caratteri di controllo non stampabili).

Esercizio 1.3: CollectionStats – Statistiche descrittive su Liste e Insiemi

Livello: *Livello 1 – Base.*
Argomento del Task: Utilizzo di collezioni generiche (`List`, `Set`, `Map`), calcolo di frequenze, moda statistica e rilevazione duplicati.
Riferimento Didattico: Esercitazione Warm-up sui generics e collections.

Consegna Operativa: Realizzare una classe generica `CollectionStats<T>` che incapsula una collezione di elementi omogenei e fornisce i seguenti metodi analitici:

1. `distinctCount()`: restituisce il numero di elementi distinti (sfruttando un `HashSet`);
2. `frequencyMap()`: genera una `Map<T, Integer>` contenente il conteggio delle occorrenze di ciascun elemento;
3. `mode()`: restituisce l'elemento con la frequenza più elevata (in caso di parità, il primo incontrato).

Fascia 2: Intermedio (Algoritmico e Problem Solving)

Esercizio 1.4: Gerarchia Geometrica vs Protocollo Visivo – Disaccoppiamento Ortogonale

Livello: *Livello 2 – Intermedio.*
Argomento del Task: Polimorfismo, principio di singola responsabilità, separazione tra tassonomia (`extends`) e contratti comportamentali (`implements`).
Riferimento Didattico: Slide 14–39 (*PAJC_L01_Warm-up.pdf*).

Consegna Operativa:

1. Dichiarare l'interfaccia `Shape2d` con i metodi `double perimeter()` e `double area()`;
2. Implementare le classi concrete `Rectangle`, `Square`, `Circle` ed `Ellipse`;
3. Introdurre l'interfaccia ortogonale `Drawable` con il metodo `void draw()`;
4. Creare una classe `Spaceship` che implementa `Drawable` pur non essendo una forma geometrica, evidenziando il polimorfismo orizzontale;
5. Popolare una lista polimorfa `List<Drawable>` e iterare invocando `draw()`.

Esercizio 1.5: Macchina a Stati Finiti (FSM) tramite Enum con Metodi Polimorfi

Livello: *Livello 2 – Intermedio.*
Argomento del Task: Pattern State implementato mediante enumerativi Java avanzati, metodi astratti in `enum` e transizioni di stato deterministiche.
Riferimento Didattico: Slide 60–61 (*PAJC_L01_Warm-up.pdf*).

Consegna Operativa: Modellare una FSM per un distributore automatico o processo di validazione con 4 stati: `IDLE` → `AUTHENTICATING` → `PROCESSING` → `COMPLETED`. Ciascun valore dell'`enum StatoWorkflow` deve:

1. Possedere un campo `String descrizione` e il rispettivo getter;
2. Implementare il metodo astratto `public abstract StatoWorkflow nextStep(boolean esito);`
3. Gestire l'eventuale transizione verso lo stato di errore `FAILED` in caso di `esito == false`.

Esercizio 1.6: Polinomio Incapsulato con Algebra Simbolica e Derivata

Livello: *Livello 2 – Intermedio.*
Argomento del Task: Incapsulamento difensivo, aritmetica su mappe sparse (`Map<Integer, Double>` grado-coefficiente), uguaglianza formale (`equals` e `hashCode`).

Consegna Operativa: Progettare la classe `Polinomio` che rappresenta un polinomio a coefficienti reali a una variabile:

1. Metodo `aggiungiTermine(int grado, double coefficiente)`;
2. Metodo `valuta(double x)`: calcola il valore del polinomio in x con il metodo di Horner o esponenziazione diretta;
3. Metodo `derivata()`: restituisce un nuovo `Polinomio` rappresentante la derivata prima;
4. Metodo `somma(Polinomio altro)`: restituisce un nuovo polinomio somma.

Fascia 3: Avanzato (Architettura ad Oggetti Multi-file)

Esercizio 1.7: MiniFileSystemSimulator – Architettura Multi-file Composita

Livello: *Livello 3 – Avanzato (Architettura a 5+ Classi).*
Argomento del Task: Design Pattern Composite, gestione della gerarchia ad albero, percorsi assoluti e relativi, permessi Unix-like e navigazione ricorsiva.

Consegna Operativa: Progettare una suite di classi interconnesse per la simulazione di un filesystem in memoria:

1. **Classe astratta** `FSNode`: radice comune con nome, nodo genitore (`DirectoryNode`) e timestamp di creazione;
2. **Classe** `FileNode`: estende `FSNode`, contiene un payload di byte e il calcolo della dimensione fisica;
3. **Classe** `DirectoryNode`: estende `FSNode`, contiene una collezione di figli (`Map<String, FSNode>`), metodi per aggiungere, rimuovere, cercare per nome e calcolare ricorsivamente la dimensione totale;
4. **Classe** `FileSystemEngine`: gestisce il nodo root (`"/"`), il nodo corrente (*working directory*) e simula i comandi tipici: `cd(String path)`, `ls()`, `mkdir(String name)`, `touch(String name, byte[] data)` e `pwd()`.

Esercizio 1.8: VehicleFleetManager – Telemetria, Contratti di Manutenzione ed Emissioni

Livello: *Livello 3 – Avanzato (Architettura a 6+ Classi).*
Argomento del Task: Gerarchia polimorfa profonda, interfacce di certificazione, storicizzazione cronologica degli eventi e calcolo energetico.

Consegna Operativa: Realizzare un sistema per la gestione di una flotta aziendale di veicoli:

1. Interfaccia `Veicolo`: metodi `targa()`, `chilometraggio()`, `consumoMedioPerKm()`;
2. Sottoclassi concrete: `AutoElettrica` (capacità batteria, degrado celle), `CamionDiesel` (portata merci, classe Euro), `FurgoneIbrido`;
3. Interfaccia `SottoponibileARevisione` con contratto formale per il controllo periodico;
4. Classe `RegistroManutenzioni` che registra interventi, costi e notifiche di scadenza chilometrica;
5. Classe `FlottaAziendale` per il calcolo dell'impronta carbonica totale e ordinamento per efficienza energetica.

Progetti Evolutivi Modulari Interconnessi

Esercizio 1.9: [Evolutivo Filone A – Parte 1] InventoryStore: Modello Prodotti, Scorte e Persistenza su File

Livello: *Livello 2 – Intermedio (Progetto Evolutivo – Filone E-Commerce, Parte 1).*
Argomento del Task: Incapsulamento OOP, gerarchia di prodotti, gestione eccezioni checked/unchecked, I/O binario/testuale e validazione integrità con checksum.
Riferimento Didattico: Slide 44–58 (*PAJC_L01_Warm-up.pdf*) e §1.9.

Consegna Operativa: Si richiede di progettare il nucleo di un sistema di gestione magazzino ed e-commerce che verrà progressivamente esteso nei capitoli successivi:

1. Progettare la classe astratta `Articolo` (codice SKU alfanumerico univoco, nome, prezzo base `double`, quantità a magazzino `int`) e le sottoclassi concrete `ProdottoFisico` (con peso in kg e costo di spedizione) e `ProdottoDigitale` (con URL di download e dimensione in MB);
2. Applicare l'incapsulamento difensivo: l'inventario non può accettare prezzi negativi o scorte inferiori a zero (sollevare `IllegalArgumentException`);
3. Realizzare la classe `Magazzino` che mantiene una collezione indicizzata `Map<String, Articolo>` ed espone metodi per carico merce, scarico merce (sollevando una checked exception custom `ScorteInsufficientiException`) e ricerca per codice;
4. Implementare la persistenza dello stato su file mediante due metodi:
   - `salvaSuFile(String filename)`: serializza l'inventario in un file di testo formattato CSV, calcolando e accodando all'ultima riga un checksum (es. somma hash o SHA256 dei record);
   - `caricaDaFile(String filename)`: legge e ricostruisce il catalogo verificando tassativamente il checksum (sollevare `DatiCorrottiException` se il file è manomesso).

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: InventoryStoreTest.java

```
Magazzino m = new Magazzino();
m.aggiungi(new ProdottoFisico("SKU-01", "Tastiera Meccanica", 89.90, 15, 1.2));
m.aggiungi(new ProdottoDigitale("SKU-02", "E-Book Java Pro", 29.99, 100,
    "https://store/ebook.pdf", 14.5));

assert m.cerca("SKU-01").getPrezzoBase() == 89.90 : "Errore ricerca!";
m.preleva("SKU-01", 5);
assert m.cerca("SKU-01").getQuantita() == 10 : "Errore scarico scorte!";

m.salvaSuFile("inventario.csv");
Magazzino ripristinato = Magazzino.caricaDaFile("inventario.csv");
assert ripristinato.cerca("SKU-02").getNome().equals("E-Book Java Pro") : "Errore
    ripristino!";
```

Esercizio 1.10: [Evolutivo Filone B – Parte 1] UrbanRoadNetwork: Grafo Stradale, Veicoli e Topologia da File

Livello: *Livello 2 – Intermedio (Progetto Evolutivo – Filone Smart City, Parte 1).*
Argomento del Task: Strutture dati a grafo (liste di adiacenza), manipolazione stringhe con tokenizzatore, lettura file di configurazione e modellazione OOP di entità semantiche.
Riferimento Didattico: §1.4 e §1.9.

Consegna Operativa: Costruire il modello geometrico e topologico di una rete stradale cittadina:

1. Definire la classe `Incrocio` (id univoco, coordinate x, y, lista di strade incidenti) e la classe `TrattaStradale` (nodo sorgente, nodo destinazione, lunghezza in metri, limite di velocità e capienza massima veicoli);
2. Definire la gerarchia di veicoli con classe astratta `Veicolo` (targa, lunghezza fisica in metri, velocità

attuale) e sottoclassi concrete `Automobile`, `Autobus` e `Ambulanza` (con priorità di emergenza);

3. Progettare la classe `ReteStradale` che consente di caricare la topologia della città da un file di configurazione strutturato a sezioni (`[INCROCI]` e `[STRADE]`);
4. Implementare il metodo `inserisciVeicolo(Veicolo v, String idStrada)` che verifica la capienza fisica residua della tratta, sollevando un'eccezione checked `StradaSaturaException` qualora lo spazio occupato ecceda la lunghezza totale della carreggiata.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: UrbanRoadNetworkTest.java

```
ReteStradale rete = new ReteStradale();
rete.caricaConfigurazione("mappa_citta.txt");

assert rete.getIncroci().size() >= 2 : "Incroci non caricati!";
TrattaStradale viaRoma = rete.getTratta("TR-01");
rete.inserisciVeicolo(new Automobile("AB123CD", 4.2, 50.0), "TR-01");
assert viaRoma.getVeicoliPresenti().size() == 1 : "Veicolo non inserito!";
```

Esercizio 1.11: Riferimenti, Aliasing e Copia Difensiva in Memoria

Livello: *Livello 2 – Intermedio (Modello di Memoria JVM).*
Argomento del Task: Gestione dei riferimenti a oggetti nello Stack e nello Heap, effetto di aliasing nell'assegnamento tra riferimenti, mutabilità condivisa e implementazione di copie difensive con costruttore di copia.
Riferimento Didattico: Dispensa ufficiale *PAJC-26-D01* (Esercizio sui Riferimenti e Modello di Memoria) e §1.1.3.

Consegna Operativa: Si richiede di analizzare e verificare sperimentalmente il comportamento della JVM nella manipolazione di riferimenti a oggetti mutabili:

1. Progettare la classe `Punto2D` con coordinate intere incapsulate x, y, costruttore parametrico, metodi getter/setter e metodo `toString()`;
2. Implementare un costruttore di copia `public Punto2D(Punto2D altro)` per duplicare lo stato creando un nuovo oggetto indipendente sullo Heap;
3. Dimostrare in un metodo di test l'effetto dell'aliasing: assegnando `Punto2D p2 = p1`, qualsiasi modifica apportata tramite `p2.setX(99)` si riflette istantaneamente su `p1.getX()`, poiché entrambe le variabili puntano al medesimo indirizzo di memoria;
4. Dimostrare l'efficacia della copia difensiva: istanziando `Punto2D p3 = new Punto2D(p1)`, la mutazione di `p3` lascia inalterato lo stato di `p1`;
5. Scrivere la suite di collaudo con asserzioni che certifichi l'uguaglianza dei valori e la disuguaglianza dei riferimenti (`p1 != p3 && p1.equals(p3)`).

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: RiferimentiMemoryTest.java

```
Punto2D p1 = new Punto2D(10, 20);

// 1. Dimostrazione Aliasing (stesso oggetto nello Heap)
Punto2D p2 = p1;
p2.setX(50);
assert p1.getX() == 50 : "Fallimento aliasing: p1 e p2 devono condividere lo
    stesso stato!";
assert p1 == p2 : "I riferimenti p1 e p2 devono puntare alla stessa istanza!";

// 2. Dimostrazione Copia Difensiva (nuovo oggetto indipendente)
Punto2D p3 = new Punto2D(p1);
p3.setX(100);
assert p1.getX() == 50 : "La mutazione di p3 non deve alterare p1!";
assert p1 != p3 : "p1 e p3 devono essere istanze distinte nello Heap!";
assert p3.getX() == 100 && p3.getY() == 20 : "Stato di p3 non corretto!";
```

A.2 Capitolo 2: Classi Anonime, Lambda, Interfacce Moderne e Sealed

Fascia 1: Base (Sintassi e Refactoring)

Esercizio 2.1: Refactoring Progressivo: Da Classe Anonima a Lambda e Method Reference

Livello: *Livello 1 – Base.*
Argomento del Task: Contrazione sintattica, eliminazione del boilerplate, target typing e mapping delle 4 categorie di method reference.
Riferimento Didattico: Dispensa D02 (*Esercizio 2.1*) e Slide 10–13 (*PAJC-26-L02.pdf*).

Consegna Operativa: Riscrivere tre classi anonime tradizionali (`Runnable`, `Comparator<Integer>`, `Function<String, String>`) prima in forma di lambda expression concisa e successivamente (ove lecito) in forma di method reference, giustificando la non-equivalenza semantica di `System.out::println` per `Runnable`.

Esercizio 2.2: Audit di Conformità e Certificazione SAM (@FunctionalInterface)

Livello: *Livello 1 – Base.*
Argomento del Task: Definizione formale di interfaccia funzionale, regole di conteggio dei metodi astratti, esclusione di default/statici e metodi di `Object`.
Riferimento Didattico: Dispensa D02 (*Esercizio 2.2*) e Slide 18–19 (*PAJC-26-L02.pdf*).

Consegna Operativa: Analizzare cinque interfacce con diverse combinazioni di metodi astratti, metodi default e firme di `Object` (`equals`), determinando quali sono conformi a `@FunctionalInterface` e dimostrando il rigetto delle non conformi da parte del compilatore.

Esercizio 2.3: Analisi Forense della Cattura di Stato: Il Contatore non Effectively Final

Livello: *Livello 1 – Base.*
Argomento del Task: Modello di memoria Stack vs Heap, ciclo di vita dei record di attivazione e vincolo di immutabilità `effectively final`.
Riferimento Didattico: Dispensa D02 (*Esercizio 2.3*) e Slide 8 (*PAJC-26-L02.pdf*).

Consegna Operativa: Dato un ciclo che tenta di incrementare una variabile locale all'interno di una lambda, spiegare l'errore del compilatore e implementare le due soluzioni: (A) contenitore Heap invariante (`int[]` o `AtomicInteger`) e (B) redesign funzionale puro senza effetti collaterali.

Fascia 2: Intermedio (Contratti e Funzionale)

Esercizio 2.4: Architettura a Pipeline con Interfaccia Funzionale Personalizzata

Livello: *Livello 2 – Intermedio.*
Argomento del Task: Progettazione di interfacce SAM custom, metodi di ordine superiore e composizione di trasformazioni in cascata.
Riferimento Didattico: Dispensa D02 (*Esercizio 2.4*).

Consegna Operativa: Dichiarare `@FunctionalInterface interface Trasformazione { String applica(String s); }` e realizzare il metodo statico `applicaTutte(String, List<Trasformazione>)` collaudandolo con trim, toUpperCase e aggiunta di punteggiatura.

Esercizio 2.5: Estensione del Motore della Calcolatrice ed Esportazione Immutabile

Livello: *Livello 2 – Intermedio.*
Argomento del Task: Operatori matematici standard (`Math`), method references statici e principio di incapsulamento difensivo delle collezioni (*Defensive Copying*).
Riferimento Didattico: Dispensa D02 (*Esercizio 2.5*) e Slide 29–30 (*PAJC-26-L02.pdf*).

Consegna Operativa: Estendere `Motore` registrando operatori `"%"`, `"max"` e `"min"`, ed esporre `operatoriDisponibili()` proteggendo la mappa interna da modifiche client tramite `Set.copyOf()`.

Esercizio 2.6: FileUtil con Iniezione di Comportamento e Callback di Colorazione

Livello: *Livello 2 – Intermedio.*
Argomento del Task: Inversione del controllo su I/O, pattern callback mediante `BiConsumer<Integer, String>` e formattazione dinamica delle righe.
Riferimento Didattico: Slide 30 (*PAJC-26-L02.pdf*).

Consegna Operativa: Aggiungere a `FileUtil` il metodo `print(filename, lineDecorator)` con parametro `BiConsumer<Integer, String>` che delega la formattazione di ogni riga a una funzione esterna (es. righe alternate colorate o filtraggio righe vuote).

Esercizio 2.7: Matrix<T> Generica e MatrixOperator Funzionale ad Alto Ordine

Livello: *Livello 2 – Intermedio.*
Argomento del Task: Generics, interfacce funzionali ad alto ordine e trasformazione di matrici bidimensionali cella per cella.
Riferimento Didattico: Slide 30 (*PAJC-26-L02.pdf*).

Consegna Operativa: Realizzare `Matrix<T>` con il metodo `map(MatrixOperator<T> op)` che restituisce una nuova matrice applicando la trasformazione ad ogni cella (r, c).

Fascia 3: Avanzato (Sealed Hierarchies, Modern Interfaces e Pattern Matching)

Esercizio 2.8: Catalogo Prodotti con Comparatori Combinabili Multipli e Null-Safe

Livello: *Livello 3 – Avanzato.*
Argomento del Task: Integrazione tra `Comparable` e factory moderne di `Comparator` (`comparing`, `thenComparing`, `reversed`, `nullsLast`), coerenza con `equals`.
Riferimento Didattico: Dispensa D02b (*PAJC-26-D02b.pdf*).

Consegna Operativa: Progettare una classe immutabile `Prodotto` (con codice SKU, categoria, prezzo `double`, recensione media `Double` opzionale):

1. Implementare l'ordinamento naturale con `Comparable<Prodotto>` basato unicamente su SKU, garantendo perfetta coerenza con `equals()` e `hashCode()`;
2. Progettare comparatori statici combinabili complessi:

- Ordinamento per categoria (alfabetico), poi per prezzo decrescente, e infine per recensione con valori nulli posizionati in fondo (`Comparator.nullsLast`);

3. Dimostrare in un test suite che l'ordinamento rispetta le proprietà matematiche di antisimmetria e transitività.

Esercizio 2.9: Albero delle Espressioni Matematiche con Gerarchia Sealed e Pattern Matching

Livello: *Livello 3 – Avanzato (Java 17–21 Standard).*
Argomento del Task: Gerarchie chiuse con `sealed` e `permits`, pattern matching per `switch` sui tipi, valutazione ricorsiva e derivazione simbolica.
Riferimento Didattico: Dispensa D02b (*PAJC-26-D02b.pdf* – Sezione Sealed Hierarchies).

Consegna Operativa: Costruire un motore di calcolo simbolico per espressioni algebriche:

1. Dichiarare l'interfaccia sigillata:
   `public sealed interface Expr permits Costante, Variabile, Somma, Prodotto {}`
2. Definire i quattro tipi concreti come `record` immutabili (`final`);
3. Implementare il metodo puro di valutazione:
   `public static double eval(Expr e, Map<String, Double> ambiente)`
   utilizzando il pattern matching per lo `switch` sui tipi con verifica di esaustività del compilatore (senza clausola `default`);
4. Implementare il metodo di derivazione analitica `public static Expr derivata(Expr e, String var)` applicando simbolicamente la regola della somma e del prodotto di Leibniz ($D(f \cdot g) = f' \cdot g + f \cdot g'$).

Progetti Evolutivi Modulari Interconnessi

Esercizio 2.10: [Evolutivo Filone A – Parte 2] FunctionalDiscountEngine & ChainedComparatorCart

Livello: *Livello 2 – Intermedio (Progetto Evolutivo – Filone E-Commerce, Parte 2).*
Argomento del Task: Espansione del modulo `Magazzino` (Es. A.1) mediante programmazione funzionale: comportamento iniettabile con `Function`, regole di sconto con `Predicate`, composizione di comparatori (`Comparator.comparing`) e copie difensive immutabili.
Riferimento Didattico: §2.8 e §2.13.2.

Consegna Operativa: Riprendere ed estendere l'architettura dell'esercizio A.1:

1. Progettare la classe `Carrello` contenente una collezione ordinata di articoli selezionati dall'utente e la rispettiva quantità;
2. Dichiarare l'interfaccia funzionale `RegolaSconto`:
   `@FunctionalInterface public interface RegolaSconto { double calcolaSconto(Carrello c); }`
3. Implementare factory statiche di sconti mediante lambda expression:
   - Sconto percentuale sul totale se l'importo supera una determinata soglia;
   - Coupon a codice promozionale fisso (sottrazione diretta);
   - Sconto categoria (es. 20% sui soli `ProdottoDigitale`);
4. Progettare la classe `MotorePromozioni` che riceve una lista di `RegolaSconto` e le applica in cascata combinandole con composizione funzionale, assicurando che lo sconto totale non superi il valore del carrello;
5. Implementare nel `Carrello` metodi di ordinamento che accettano comparatori combinati: ordinare gli articoli per prezzo decrescente, e a parità di prezzo per nome alfabetico (`Comparator.comparing(Articolo::getPrezzo).reversed().thenComparing(Articolo::getNome)`).

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: FunctionalDiscountEngineTest.java

```
Carrello cart = new Carrello();
cart.aggiungi(new ProdottoFisico("SKU-01", "Monitor 4K", 300.0, 5, 4.0), 1);
cart.aggiungi(new ProdottoDigitale("SKU-02", "Corso Java", 100.0, 50, "url",
    1.0), 2); // 2 x 100 = 200

MotorePromozioni promo = new MotorePromozioni();
// Regola: 10% se totale > 400
promo.registraRegola(c -> c.totaleLordo() > 400 ? c.totaleLordo() * 0.10 : 0.0);

assert cart.totaleLordo() == 500.0 : "Totale errato!";
assert promo.calcolaScontoTotale(cart) == 50.0 : "Sconto errato!";
assert promo.totaleNetto(cart) == 450.0 : "Netto errato!";
```

Esercizio 2.11: [Evolutivo Filone B – Parte 2] TrafficLightFSM & FunctionalDriverBehavior

Livello: *Livello 2 – Intermedio (Progetto Evolutivo – Filone Smart City, Parte 2).*
Argomento del Task: Espansione del simulatore di traffico (Es. A.1) con macchine a stati a semaforo (`enum`), strategie di guida funzionali iniettate tramite `Predicate<Veicolo>` e `Function<Veicolo, Velocita>`.
Riferimento Didattico: §1.10 e §2.9.

Consegna Operativa: Evolvere il modello della rete stradale dell'esercizio A.1 trasformando il sistema da statico a dinamico:

1. Modellare il semaforo cittadino mediante un `enum` evoluto `StatoSemaforo`:
   - Valori: `ROSSO`, `GIALLO`, `VERDE`;
   - Ciascun valore definisce un metodo astratto `public abstract StatoSemaforo transizione();`
   - Ciascun valore possiede un predicato associato `public abstract boolean consenteTransito(Veicolo v);` l'ambulanza in emergenza deve poter transitare anche con il rosso!
2. Associare a ciascun incrocio una collezione di semafori regolati temporalmente;
3. Definire la politica di accelerazione/decelerazione del veicolo tramite un'interfaccia funzionale `ComportamentoGuida`:
   `@FunctionalInterface public interface ComportamentoGuida { double calcolaVelocita(Veicolo v, double distanzaOstacolo); }`
4. Creare tre stili di guida predefiniti con lambda: *Prudente* (decelera gradualmente a partire da 50 metri), *Aggressivo* (mantiene la velocità massima finché non è a 15 metri dall'ostacolo) ed *Eco-Friendly*;
5. Implementare il metodo `simulaPassoTemporale(double deltaSecondi)` che aggiorna la posizione di tutti i veicoli in base allo stato dei semafori e alle rispettive funzioni di comportamento.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: TrafficSimulationStepTest.java

```
Automobile auto = new Automobile("MI999ZZ", 4.0, 50.0);
auto.setComportamento((v, dist) -> dist < 20.0 ? 0.0 : v.getVelocitaMassima());

Ambulanza ambulanza = new Ambulanza("EMERG-1", 5.5, 90.0, true);

StatoSemaforo rosso = StatoSemaforo.ROSSO;
assert !rosso.consenteTransito(auto) : "L'auto non puo' passare con il rosso!";
assert rosso.consenteTransito(ambulanza) : "L'ambulanza deve poter transitare con
    il rosso!";
```

Esercizio 2.12: Cronometro per Task Generici e Confronto Prestazionale tra Stringhe

Livello: *Livello 2 – Intermedio (Metodi di Ordine Superiore e Lambda).*
Argomento del Task: Realizzazione di utility di profilazione cronometrica tramite interfaccia funzionale `Runnable`, invocazione polimorfa di lambda expression e confronto sperimentale dell'efficienza temporale tra concatenazione con operatore `+=`, `StringBuilder` e `StringBuffer`.
Riferimento Didattico: Laboratorio ufficiale *LabLambda* (`LambdaApp.java`) e §2.5.

Consegna Operativa: Nel progetto di laboratorio `LabLambda`, si richiede di costruire uno strumento di misurazione temporale flessibile per quantificare il costo dell'accumulo di stringhe:

1. Realizzare il metodo statico `long cronometra(Runnable task)`:
   - Rileva il timestamp iniziale `long t0 = System.nanoTime();`
   - Esegue il compito passato come parametro invocando `task.run();`
   - Rileva il timestamp finale `long t1 = System.nanoTime();`
   - Calcola e restituisce il tempo trascorso convertito in millisecondi: `(t1 - t0) / 1_000_000;`
2. Realizzare il metodo statico ausiliario `void misura(String desc, Runnable task)` che stampa a console la stringa formattata con la descrizione e il tempo impiegato in millisecondi;
3. Nel metodo `main()`, utilizzare le lambda per misurare e confrontare sperimentalmente tre strategie di costruzione di una sequenza di 100.000 o 1.000.000 di caratteri:
   - Concatenazione iterativa con `s += "x"` (dimostrandone l'inefficienza dovuta all'immutabilità della classe `String`);
   - Accumulo sequenziale tramite `StringBuilder` (buffer mutabile ad alte prestazioni per contesti mono-thread);
   - Accumulo sequenziale tramite `StringBuffer` (buffer mutabile sincronizzato thread-safe).

Specifiche di Collaudo e Test Case (TDD):

Codice del Laboratorio: LambdaApp.java

```
int ntest = 1_000_000;

misura("Concatenazione con +=", () -> {
    String s = "";
    for (int i = 0; i < 50_000; i++) s += "x"; // Limitato per evitare attese
});

misura("StringBuilder", () -> {
    var sb = new StringBuilder();
    for (int i = 0; i < ntest; i++) sb.append("x");
});

misura("StringBuffer", () -> {
    var sb = new StringBuffer();
    for (int i = 0; i < ntest; i++) sb.append("x");
});
```

A.3 Capitolo 3: Stream Computazionali, Pipeline e Collettori

Fascia 1: Base (Pipeline e Metodi Core)

Esercizio 3.1: Filtro, Mappatura e Statistiche su Flussi di Stringhe

Livello: *Livello 1 – Base.*
Argomento del Task: Pipeline lineari con `filter`, `map`, `distinct`, `sorted` e raccolta con `toList()`.
Riferimento Didattico: Dispensa D03 (*PAJC-26-D03.pdf* – §1–2) e Slide L03.

Consegna Operativa: Data una lista di stringhe rappresentanti titoli di libri (contenenti spazi e lettere

miste):

1. Rimuovere gli spazi superflui (`trim()`) e scartare i titoli vuoti o più corti di 3 caratteri;
2. Convertire tutti i titoli in formato maiuscolo;
3. Eliminare i duplicati (`distinct()`);
4. Ordinare alfabeticamente e raccogliere i primi 5 risultati in una lista immutabile.

Esercizio 3.2: IntStream e Analisi di Primalità Funzionale

Livello: *Livello 1 – Base.*
Argomento del Task: Flussi primitivi specializzati, azzeramento del boxing, `IntStream.rangeClosed()` e `summaryStatistics()`.
Riferimento Didattico: Dispensa D03 (*PAJC-26-D03.pdf* – §4).

Consegna Operativa:

1. Realizzare il predicato `public static boolean isPrimo(int n)` usando un'unica pipeline funzionale: `n > 1 && IntStream.rangeClosed(2, (int)Math.sqrt(n)).noneMatch(d -> n % d == 0);`
2. Generare tutti i numeri primi compresi tra 1 e 10.000;
3. Calcolarne in un unico passaggio la somma totale, il valore massimo e la media sfruttando `IntSummaryStatistics`.

Esercizio 3.3: Generatore di Fibonacci e Sequenze Geometriche con Stream.iterate

Livello: *Livello 1 – Base.*
Argomento del Task: Flussi infiniti, iterazione algoritmica mediante funzioni unarie e troncamento con `limit()`.
Riferimento Didattico: Slide L03 e Appunti del 30/09.

Consegna Operativa:

1. Generare i primi N termini della sequenza di Fibonacci operando su array di coppie `long[] {a, b}`;
2. Generare la sequenza infinita delle potenze di due $(1, 2, 4, 8, \dots)$, scartare le prime 5 potenze con `skip(5)` ed estrarre le successive 10 convertendole in stringa formattata.

Fascia 2: Intermedio (Laziness, Factory e Collettori)

Esercizio 3.4: Rilevazione Sperimentale della Laziness e dell'Ordine di Esecuzione

Livello: *Livello 2 – Intermedio.*
Argomento del Task: Studio forense dell'elaborazione verticale vs orizzontale, intercettazione con `peek()` e short-circuit.
Riferimento Didattico: Lezione 30/09 (trascrizione e note d'aula).

Consegna Operativa: Scrivere un programma di benchmarking didattico che dimostra:

1. Che l'assenza di un'operazione terminale produce zero computazione;
2. Che scambiando l'ordine di `filter()` e `map()` su un insieme di 100.000 stringhe, il numero totale di invocazioni della funzione di mappatura crolla proporzionalmente alla selettività del filtro;
3. Misurare e stampare il numero esatto di passaggi eseguiti nei due casi mediante un contatore atomico `AtomicInteger`.

Esercizio 3.5: Risoluzione del Single-Use Stream tramite Supplier Factory

Livello: *Livello 2 – Intermedio.*
Argomento del Task: Gestione dell'`IllegalStateException`, pattern Factory per flussi ripetibili e concorrenza.

Riferimento Didattico: Dispensa D03 (§3).

Consegna Operativa: Progettare una classe `StreamQuery<T>` che incapsula una sorgente dati e una catena di filtri racchiusa in un `Supplier<Stream<T>>`:

1. Esporre metodi riutilizzabili: `count()`, `first()`, `toList()` e `anyMatch()`;
2. Dimostrare che il client può invocare ripetutamente questi metodi in qualsiasi ordine senza mai incorrere in `IllegalStateException`;
3. Validare il comportamento in un contesto multi-thread con 4 worker concorrenti.

Esercizio 3.6: Censimento e Partizionamento del Bimondo Geometrico

Livello: *Livello 2 – Intermedio.*
Argomento del Task: `Collectors.groupingBy`, aggregazioni downstream con `counting()` e `partitioningBy`.
Riferimento Didattico: Lezione 30/09 (Esercizio Bimondo in una riga).

Consegna Operativa: A partire da una lista eterogenea di figure `Shape2d` del Capitolo 1:

1. Raggruppare le forme per classe concreta calcolando il conteggio istanze in un'unica espressione (`groupingBy(Object::getClass, counting())`);
2. Partizionare le forme in base all'area (superiore o inferiore a 50.0);
3. Calcolare per ciascun gruppo la somma totale dei perimetri tramite downstream collector (`Collectors.summingDouble(Shape2d::perimeter)`).

Esercizio 3.7: LogAnalyzer – Parsing e Aggregazione di Log Server con Files.lines

Livello: *Livello 2 – Intermedio.*
Argomento del Task: Stream I/O ad alte prestazioni con `Files.lines()`, espressioni regolari e raggruppamenti statistici.

Consegna Operativa: Dato un file di log web server nel formato standard `"DATA_ORA [LIVELLO] IP CODICE_HTTP MESSAGGIO"`:

1. Utilizzare `Files.lines(Path)` in un blocco `try-with-resources` per garantire il rilascio immediato del file descriptor del sistema operativo;
2. Estrarre la lista degli indirizzi IP unici che hanno generato errori HTTP ≥ 400;
3. Generare una mappa `Map<Integer, Long>` con il conteggio degli accessi per ciascun codice di stato HTTP;
4. Identificare i primi 3 IP con il maggior volume di richieste complessive.

Fascia 3: Avanzato (Architettura Multi-file Funzionale)

Esercizio 3.8: FinancialPortfolioAnalyzer – Analisi Portafoglio con FlatMap e Custom Collectors

Livello: *Livello 3 – Avanzato (Architettura a 5 Classi).*
Argomento del Task: Trasformazioni complesse 1-a-molti con `flatMap()`, collettori personalizzati, raggruppamenti multidimensionali e calcolo del rischio.

Consegna Operativa: Realizzare un motore finanziario articolato sulle seguenti classi:

1. `TitoloAzionario`: ticker, settore industriale, prezzo corrente;
2. `Transazione`: data, tipo (ACQUISTO/VENDITA), quantità, prezzo unitario;
3. `PosizionePortafoglio`: titolo, lista cronologica di transazioni;
4. `Portafoglio`: intestatario, lista di posizioni;
5. Progettare una suite di metodi analitici basati esclusivamente su Stream:
   - Calcolo del valore di mercato totale del portafoglio;

- Ripartizione percentuale del capitale investito per settore industriale (`Map<String, Double>`);
- Estrazione delle transazioni storiche con volume superiore a soglia mediante `flatMap()`.

Esercizio 3.9: FullTextSearchEngine – Motore di Ricerca ad Indice Inverso Funzionale

Livello: *Livello 3 – Avanzato (Architettura a 6 Classi).*
Argomento del Task: Elaborazione del linguaggio naturale (NLP) funzionale, rimozione stop-words, indicizzazione inversa con Stream paralleli e ranking TF-IDF.

Consegna Operativa: Costruire un motore di ricerca per documenti testuali:

1. Classe immutabile `Documento` (id, titolo, testo integrale);
2. Pipeline di tokenizzazione: divide il testo in token alfanumerici, converte in minuscolo, elimina caratteri non ASCII e filtra un set predefinito di stop-words;
3. Costruzione dell'**Indice Inverso** (`Map<String, Set<Integer>>`) che mappa ciascun termine all'insieme di ID dei documenti in cui compare, generata con un singolo `collect(groupingBy(...))`;
4. Algoritmo di interrogazione booleana (`AND`, `OR`) su query multi-parola con calcolo di rilevanza TF-IDF e ordinamento dei documenti più pertinenti.

Progetti Evolutivi Modulari Interconnessi

Esercizio 3.10: [Evolutivo Filone A – Parte 3] SalesAnalyticsEngine: Reportistica Vendite e Collettori Downstream

Livello: *Livello 2 – Intermedio (Progetto Evolutivo – Filone E-Commerce, Parte 3).*
Argomento del Task: Espansione del modulo vendite (Es. A.2) con pipeline Stream avanzate: trasformazione 1-a-molti con `flatMap`, aggregazioni complesse con `Collectors.groupingBy`, downstream collectors multipli e partizionamento.
Riferimento Didattico: §3.8 e §B.2.

Consegna Operativa: Integrare nel sistema e-commerce un motore analitico basato interamente su Stream per supportare il Business Intelligence aziendale:

1. Progettare la classe `Ordine` (id, data, cliente, lista di articoli acquistati con quantità, stato: `IN_ATTESA`, `EVASO`, `ANNULLATO`);
2. Implementare la classe `RegistroVendite` contenente lo storico di tutti gli ordini;
3. Realizzare una suite di metodi analitici puri tramite un'unica pipeline di Stream per ciascun requisito:
   - `fatturatoPerCategoria()`: calcola il ricavo totale generato raggruppato per categoria merceologica (`Map<String, Double>`) combinando `flatMap`, `groupingBy` e `summingDouble()`;
   - `clientiTopSpender(int limit)`: estrae i primi N clienti con il maggior volume di spesa complessivo negli ordini evasi;
   - `partizionaPerFasciaDiSpesa(double soglia)`: partiziona gli ordini in due liste (`Map<Boolean, List<Ordine>>`) con valore totale superiore o inferiore alla soglia specificata;
   - `articoloPiuVenduto()`: individua l'articolo con la massima somma di quantità acquistate in assoluto, restituito in forma di `Optional<Articolo>`.
4. Dimostrare che tutte le interrogazioni sono thread-safe ed operano su copie immutabili o flussi paralleli (`parallelStream()`) senza corruzione dei dati.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: SalesAnalyticsEngineTest.java

```
RegistroVendite registro = new RegistroVendite();
// Popolamento con ordini di prova...
Map<String, Double> fatturato = registro.fatturatoPerCategoria();
assert fatturato.containsKey("Hardware") : "Categoria assente!";
assert fatturato.get("Hardware") > 0.0 : "Fatturato non positivo!";

List<Cliente> top3 = registro.clientiTopSpender(3);
assert top3.size() == 3 : "Top 3 non corretta!";
```

Esercizio 3.11: [Evolutivo Filone B – Parte 3] CongestionAnalyzer & FlowMetricsSlidingWindow

Livello: *Livello 2 – Intermedio (Progetto Evolutivo – Filone Smart City, Parte 3).*
Argomento del Task: Espansione del simulatore di traffico (Es. A.2) tramite analisi real-time dei sensori di flusso con Stream: calcolo velocità medie, partizionamento tratte congestionate e sliding window temporale.
Riferimento Didattico: §3.6 e §3.8.

Consegna Operativa: Aggiungere al simulatore stradale cittadino una centralina analitica che elabora i passaggi dei veicoli rilevati dalle telecamere:

1. Modellare l'evento di transito `TransitoRecord` (idSensore, targaVeicolo, timestampMillis, velocitaRilevata);
2. Implementare la classe `AnalizzatoreTraffico` alimentata da un flusso continuo o da una lista di eventi;
3. Realizzare le seguenti metriche analitiche tramite Stream:
   - `velocitaMediaPerTratta()`: mappa ciascuna strada alla velocità media registrata nell'ultimo intervallo (`Map<String, Double>`);
   - `tratteCongestionate(double velocitaCritica)`: estrae l'insieme delle tratte stradali la cui velocità media è inferiore alla soglia critica, ordinandole per severità del blocco;
   - `rilevazioneInfrazioniVelocita(double limite)`: filtra i veicoli che hanno ecceduto il limite consentito ed emette una stringa CSV formattata con `Collectors.joining("\n")`;
4. Implementare un meccanismo a **Finestra Scorrevole (Sliding Window)** che conserva solo gli eventi degli ultimi 60 secondi ed elimina automaticamente i transiti obsoleti mediante `removeIf()` prima di ogni calcolo statistico.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: TrafficAnalyticsTest.java

```
AnalizzatoreTraffico analyzer = new AnalizzatoreTraffico();
analyzer.registraTransito(new TransitoRecord("TR-01", "MI111AA",
    System.currentTimeMillis(), 25.0));
analyzer.registraTransito(new TransitoRecord("TR-01", "MI222BB",
    System.currentTimeMillis(), 15.0));

Map<String, Double> medie = analyzer.velocitaMediaPerTratta();
assert medie.get("TR-01") == 20.0 : "Media calcolata errata!";
assert analyzer.tratteCongestionate(30.0).contains("TR-01") : "Tratta non
    rilevata come congestionata!";
```

A.4 Capitolo 4: Interfacce Grafiche Swing ed Architettura ad Eventi

Fascia 1: Base (Finestre e Componenti Atomici)

Esercizio 4.1: Prima Finestra con Swing: Avvio Sicuro su EDT e Interazione Base

Livello: *Livello 1 – Base.*
Argomento del Task: Inizializzazione sicura della GUI su Event Dispatch Thread (`EventQueue.invokeLater`), `JFrame`, `JButton`, `JLabel`.
Riferimento Didattico: Slide L04 (*PAJC_L04_Eventi&GUI.pdf*) e Lezione 30/09.

Consegna Operativa: Realizzare una finestra Swing con le seguenti specifiche:

1. Avvio tassativamente incapsulato in `EventQueue.invokeLater()`;
2. Titolo della finestra: `"Benvenuto in Swing"`;
3. Chiusura su crocetta impostata con `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)`;
4. Contenuto: una `JLabel` centrale con testo `"Stato: In Attesa"` e un `JButton` con testo `"Attiva"`;
5. Alla pressione del pulsante, il testo della label deve aggiornarsi in `"Stato: Operazione Completata con Successo!"`.

Esercizio 4.2: Studio Comparativo dei Layout Manager al Ridimensionamento

Livello: *Livello 1 – Base.*
Argomento del Task: Verifica empirica della fragilità di `AbsoluteLayout` rispetto alla resilienza di `BorderLayout` e `FlowLayout`.
Riferimento Didattico: Slide L04 e §4.5.

Consegna Operativa: Creare un'applicazione di test contenente tre finestre distinte con 4 pulsanti ciascuna:

1. **Finestra A**: utilizza `setLayout(null)` con coordinate `setBounds()` fisse;
2. **Finestra B**: utilizza `FlowLayout`;
3. **Finestra C**: utilizza `BorderLayout`;
4. Documentare formalmente il comportamento visivo di ciascuna finestra quando l'utente la raddoppia in larghezza o la riduce al 50%.

Fascia 2: Intermedio (Interattività e Listener)

Esercizio 4.3: Contatore Interattivo a Due Vie con Limiti di Soglia

Livello: *Livello 2 – Intermedio.*
Argomento del Task: Sincronizzazione bidirezionale modello-vista, pulsanti di incremento/decremento, validazione input su `JTextField`.

Consegna Operativa: Costruire una GUI per la gestione di un contatore intero:

1. Display centrale costituito da un `JTextField` che visualizza il valore corrente (modificabile anche manualmente dall'utente);
2. Due pulsanti laterali: `"-"` (decrementa) e `"+"` (incrementa);
3. Un pulsante inferiore `"Reset"` che azzera il contatore;
4. Vincolo di sicurezza: il valore non può scendere sotto zero né superare 100. Se l'utente digita caratteri non numerici, la pressione di Invio deve ripristinare il valore precedente ed emettere un segnale acustico (`Toolkit.getDefaultToolkit().beep()`).

Esercizio 4.4: Convertitore di Temperatura e Valuta con Dialoghi di Validazione

Livello: *Livello 2 – Intermedio.*
Argomento del Task: Gestione eventi semantici, menu a scelta singola (`JComboBox` o radio buttons), gestione dialoghi modali (`JOptionPane`).

Consegna Operativa: Progettare una calcolatrice di conversione unita':

1. Consente di convertire temperature da Celsius a Fahrenheit e Kelvin;
2. Disposizione grafica con `BorderLayout` per la struttura e un `GridLayout(3, 2, 5, 5)` per i campi input/output;
3. Gestione di `NumberFormatException`: se il campo è vuoto o contiene testo alfabetico, intercettare l'errore e mostrare un messaggio di allerta con `JOptionPane.showMessageDialog()`.

Esercizio 4.5: Calcolatrice Grafica Desktop con Integrazione del Motore Funzionale

Livello: *Livello 2 – Intermedio.*
Argomento del Task: Integrazione architetturale MVC, collegamento del `Motore` (Capitolo 2) con un'interfaccia a griglia `GridLayout`.

Consegna Operativa: Assemblare una calcolatrice grafica completa:

1. **Modello**: istanza della classe `Motore` per l'esecuzione dei calcoli aritmetici;
2. **Vista**: `JTextField` superiore per la visualizzazione dell'espressione e griglia 4×4 contenente cifre numeriche (`0`–`9`) e simboli (`+`, `-`, `*`, `/`, `=`, `C`);
3. **Controller**: gestione degli stati (primo operando, operatore, secondo operando) evitando azzeramenti imprevisti.

Fascia 3: Avanzato (Componenti Custom e Architettura Event-Driven Multi-file)

Esercizio 4.6: Il Componente Custom PnlKeypad con Incapsulamento ed EventListenerList

Livello: *Livello 3 – Avanzato (Pattern Componente Riutilizzabile).*
Argomento del Task: Componenti compositi che estendono `JPanel`, incapsulamento totale dei widget interni, pattern proxy di eventi, gestione multicast con `EventListenerList`.
Riferimento Didattico: Lezione 30/09 e §4.7.

Consegna Operativa: Reimplementare e raffinare il componente `PnlDigits` presentato a lezione:

1. La classe `PnlKeypad` deve estendere `JPanel` e configurare una griglia 4×3;
2. **Incapsulamento rigido**: i 12 pulsanti interni (`0`–`9`, `"*"`, `"#"`) devono essere tassativamente privati e inaccessibili all'esterno;
3. Implementare la registrazione multicast esponendo `addActionListener(ActionListener)` e `removeActionListener(ActionListener)`;
4. Utilizzare `EventListenerList` per memorizzare gli ascoltatori ed emettere un nuovo `ActionEvent` avente `PnlKeypad` come sorgente e il tasto premuto come `actionCommand`;
5. Integrare il componente in una finestra che simula l'inserimento di un codice PIN segreto con mascheramento a video.

Esercizio 4.7: SensorTelemetryDashboard – Monitoraggio Concorrente Multi-file con EDT

Livello: *Livello 3 – Avanzato (Architettura a 6 Classi).*
Argomento del Task: Concorrenza tra thread di simulazione in background ed Event Dispatch Thread, disaccoppiamento MVC, listener custom e rendering dinamico.

Consegna Operativa: Costruire una dashboard di controllo per una sala server con monitoraggio di temperatura e carico CPU:

1. `Sensore`: classe eseguibile su thread autonomo (`Runnable`) che genera ciclicamente valori di telemetria con jitter gaussiano;
2. `SensorListener`: interfaccia custom con callback `onTelemetryReceived(SensorEvent e)`;
3. `PnlMetricCard`: componente grafico personalizzato che mostra valore corrente, indicatore colorato (verde per normale, giallo per attenzione, rosso per allarme);

4. **Vincolo di Sicurezza sui Thread**: le notifiche provenienti dal thread del sensore devono essere iniettate sull'EDT esclusivamente tramite `SwingUtilities.invokeLater()` prima di toccare i componenti grafici;
5. Pulsanti di start/stop per ciascun sensore e pannello di riepilogo con log scorrevole (`JScrollPane`).

Esercizio 4.8: VectorPaintStudio – Canvas Grafico Interattivo con PaintComponent e Undo/Redo

Livello: *Livello 3 – Avanzato (Architettura a 7 Classi).*
Argomento del Task: Rendering custom 2D sovrascrivendo `paintComponent(Graphics g)`, gestione eventi a basso livello (`MouseListener`, `MouseMotionListener`), pattern Command con stack di Undo/Redo.

Consegna Operativa: Realizzare un editor vettoriale bidimensionale:

1. `CanvasPanel`: estende `JPanel` e sovrascrive `paintComponent(Graphics g)`, attivando l'antialiasing tramite `Graphics2D`;
2. Toolbar superiore con pulsanti per selezionare la forma da disegnare (Rettangolo, Cerchio, Linea) e il colore corrente (`JColorChooser`);
3. Intercettazione del trascinamento del mouse (*drag*): preview dinamica durante il movimento e memorizzazione della figura al rilascio;
4. Gestione dello storico con due `Deque` di comandi per supportare le operazioni di **Annulla (Undo)** e **Ripristina (Redo)** da tastiera (combinazioni `Ctrl+Z` e `Ctrl+Y`).

Progetti Evolutivi Modulari Interconnessi

Esercizio 4.9: [Evolutivo Filone A – Parte 4] StoreManagerDesktopApp: GUI Swing con JTable, Filtri e Carrello Asincrono

Livello: *Livello 2 – Intermedio (Progetto Evolutivo – Filone E-Commerce, Parte 4).*
Argomento del Task: Integrazione completa del sistema e-commerce (Es. A.1, Es. A.2, Es. A.3) in un'applicazione grafica desktop Swing con tabella dati (`JTable`), modello personalizzato (`AbstractTableModel`), ordinamento/filtraggio dinamico con Stream/Regex, gestione del carrello acquisti e checkout non bloccante su EDT.
Riferimento Didattico: Slide L04, §4.5, §B.2 e §B.2.

Consegna Operativa: Si richiede di dotare il sistema di vendita realizzato nei capitoli precedenti di una postazione desktop completa per l'addetto alle vendite:

1. **Modello della Tabella (`ArticoloTableModel`)**: Estendere `AbstractTableModel` per collegare la lista di articoli dell'`InventoryStore` a una `JTable`. Le colonne devono essere: `Codice`, `Descrizione`, `Categoria`, `Prezzo Base (€)`, `Giacenza`, `Stato`. Rendere non modificabili le celle e tipizzare correttamente le classi delle colonne tramite `getColumnClass()`;
2. **Pannello Superiore di Ricerca e Filtraggio**: Inserire un `JTextField` e un selettore di categoria `JComboBox<String>`. Collegare un `TableRowSorter<ArticoloTableModel>` alla tabella per filtrare istantaneamente i record digitati dall'operatore senza ricaricare la vista;
3. **Pannello Laterale del Carrello (`PnlCartView`)**: Costruire un componente grafico composito per la gestione del carrello dell'utente in tempo reale:
   - `JList<RigaCarrello>` o mini-tabella con le voci selezionate e relative quantità;
   - Pulsanti `"Aggiungi"`, `"Rimuovi"` e `"Svuota"`;
   - Selettore profilo cliente (Standard, VIP, Studente) per l'attivazione delle strategie di sconto definite in Es. A.2;
   - Riepilogo contabile dinamico: visualizzazione di sub-totale, sconti applicati dal `FunctionalDiscountEngine` e prezzo finale calcolato;
4. **Checkout Asincrono e Aggiornamento Scorte**: Alla pressione del pulsante `"Completa Ordine"`:

- Verificare la giacenza effettiva nel catalogo;
- Decrementare le quantità in magazzino;
- Registrare la transazione nel `RegistroVendite` (Es. A.3);
- Notificare la tabella con `fireTableDataChanged()`;
- Mostrare una ricevuta riassuntiva in un `JOptionPane.showMessageDialog()` modale. Garantire che l'aggiornamento grafico avvenga interamente nel rispetto del thread EDT (`SwingUtilities.invokeLater()`).

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: StoreManagerAppTest.java

```
InventoryStore store = new InventoryStore();
store.aggiungi(new Articolo("ART-01", "Monitor 4K", "Hardware", 350.0, 10));
ArticoloTableModel tableModel = new ArticoloTableModel(store);

assert tableModel.getRowCount() == 1 : "Conteggio righe errato!";
assert tableModel.getValueAt(0, 0).equals("ART-01") : "ID articolo non
    corrispondente!";

Carrello carrello = new Carrello();
carrello.aggiungi(store.trova("ART-01").orElseThrow(), 2);
double totale = carrello.calcolaTotale(new DiscountEngineV2());
assert totale > 0.0 : "Calcolo carrello nullo o negativo!";
```

Esercizio 4.10: [Evolutivo Filone B – Parte 4] TrafficMonitorSimulatorCanvas: Mappa Grafica 2D Animata su Canvas e Swing Timer

Livello: *Livello 2 – Intermedio (Progetto Evolutivo – Filone Smart City, Parte 4).*
Argomento del Task: Visualizzazione grafica dinamica e simulazione visuale del grafo stradale (Es. A.1), dei semafori a stati finiti (Es. A.2) e del monitoraggio code (Es. A.3) su canvas custom Swing con rendering `Graphics2D`, antialiasing ed animazione a 25+ FPS pilotata da `javax.swing.Timer`.
Riferimento Didattico: §B.2, §B.2 e §B.2.

Consegna Operativa: Integrare il simulatore di mobilità urbana con una dashboard geografica visuale bidimensionale interattiva:

1. **Canvas Grafico della Città (`PnlTrafficCanvas`)**: Estendere `JPanel` e sovrascrivere `paintComponent(Graphics g)`:
   - Effettuare il cast a `Graphics2D` ed abilitare il rendering con antialiasing (`RenderingHints.KEY_ANTIALIASING`);
   - Disegnare gli incroci (nodi del grafo) come cerchi con identificativo e semaforo associato;
   - Il semaforo deve essere renderizzato con il colore dello stato corrente restituito dalla macchina a stati (`Rosso`, `Giallo`, `Verde`);
   - Disegnare le tratte stradali (archi del grafo) come linee o corsie collegate tra gli incroci, evidenziando in rosso le tratte congestionate rilevate da `AnalizzatoreTraffico`;
2. **Animazione dei Veicoli con Traslazione Continua**: Ciascun veicolo in transito su una tratta deve essere visualizzato come un rettangolo o cerchietto colorato che si muove dall'incrocio di partenza a quello di arrivo interpolando linearmente le coordinate ad ogni frame:

$$x(t) = x_A + t \cdot (x_B - x_A), \quad y(t) = y_A + t \cdot (y_B - y_A), \quad t \in [0.0, 1.0]$$

3. **Loop di Animazione con `javax.swing.Timer`**: Configurare un timer Swing periodico con frequenza di aggiornamento a 40 ms ($\approx$ 25 frame al secondo) che:
   - Esegue l'avanzamento logico di un quanto temporale nel motore di simulazione;
   - Aggiorna le posizioni dei veicoli e gli stati temporizzati dei semafori;

- Richiama `repaint()` sul pannello canvas;

4. **Pannello Telemetria Laterale**: Collocare a destra (`BorderLayout.EAST`) un pannello informativo con contatori real-time dei veicoli attivi, velocità media cittadina ed un pulsante `"Semaforo d'Emergenza"` che forza tutti i semafori su `GIALLO_LAMPEGGIANTE`.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: TrafficCanvasTest.java

```
ReteStradale rete = new ReteStradale();
Incrocio n1 = new Incrocio("INC-1", 50, 50);
Incrocio n2 = new Incrocio("INC-2", 250, 150);
rete.aggiungiTratta(new Tratta(n1, n2, 50.0));

PnlTrafficCanvas canvas = new PnlTrafficCanvas(rete);
canvas.setSize(800, 600);

// Verifica dell'interpolazione fisica di un veicolo:
VeicoloSimulato auto = new VeicoloSimulato("MI000XX", n1, n2, 100); // 100 frame totali
auto.avanzaFrame();
Point pos = auto.getPosizioneCorrente();
assert pos.x > 50 : "Avanzamento orizzontale fallito!";
assert pos.y > 50 : "Avanzamento verticale fallito!";
```

Esercizio 4.11: Interfaccia Grafica con JComboBox, Pannelli Annidati e Gestione Multi-Listener

Livello: *Livello 2 – Intermedio (Swing, Layout e Gestione Eventi).*
Argomento del Task: Composizione di interfacce grafiche mediante `BorderLayout` e `JPanel`, utilizzo del componente a selezione `JComboBox`, associazione di `ActionListener` multipli e disaccoppiamento tra azioni operative e diagnostica (*debug listener*).
Riferimento Didattico: Laboratori ufficiali *LabGUI01* (`LabGUI01App.java`) e *LabGUI02* (`LabGUI02App.java`) e §B.2.

Consegna Operativa: Integrando le architetture dei progetti di laboratorio `LabGUI01` e `LabGUI02`, si richiede di realizzare una finestra interattiva completa:

1. **Struttura e Layout**:
   - Impostare il contenitore principale (`frame.getContentPane()`) con `BorderLayout`;
   - Inserire nella regione `BorderLayout.NORTH` un pannello `JPanel` contenente un menu a discesa `JComboBox<String>` inizializzato con le voci `{"ALFA", "BETA", "GAMMA"}` e una `JLabel` ausiliaria per la visualizzazione della selezione corrente;
   - Inserire nella regione `BorderLayout.SOUTH` un'etichetta di stato principale `lblInfo` con font in evidenza (24pt);
2. **Gestione Selezione ComboBox**: Agganciare al `JComboBox` un `ActionListener` tramite lambda expression che, alla selezione di un elemento, recupera l'opzione con `getSelectedItem().toString()` e aggiorna tempestivamente l'etichetta del pannello;
3. **Ascoltatore Condiviso e Debug Dinamico**:
   - Configurare due pulsanti di comando (`btnUno` e `btnDue`) che condividono il medesimo `ActionListener`: la pressione imposta su `lblInfo` il comando azionato (`e.getActionCommand()`) e ne colora il testo di rosso;
   - Introdurre un pulsante di controllo `btnDebug` che, se premuto, aggiunge dinamicamente ai pulsanti un secondo ascoltatore per stampare su console standard la sorgente fisica dell'evento (`e.getSource()`);
4. **Esecuzione Sicura**: Garantire l'avvio della finestra confinato nell'Event Dispatch Thread tramite `EventQueue.invokeLater()`.

Specifiche di Collaudo e Codice del Laboratorio:

Codice del Laboratorio: LabGUIApp.java

```
EventQueue.invokeLater(() -> {
    JFrame frame = new JFrame("Esercitazione LabGUI");
    frame.setBounds(100, 100, 450, 300);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    JLabel lblInfo = new JLabel("--", JLabel.CENTER);
    lblInfo.setFont(new Font("SansSerif", Font.PLAIN, 24));
    frame.getContentPane().add(lblInfo, BorderLayout.SOUTH);

    JPanel panel = new JPanel();
    frame.getContentPane().add(panel, BorderLayout.NORTH);

    JComboBox<String> comboBox = new JComboBox<>(new String[] {"ALFA", "BETA",
    "GAMMA"});
    JLabel lblCombo = new JLabel("--");
    panel.add(comboBox);
    panel.add(lblCombo);

    comboBox.addActionListener(e ->
    lblCombo.setText(comboBox.getSelectedItem().toString()));

    JButton btnUno = new JButton("Btn Uno");
    JButton btnDue = new JButton("Btn Due");
    JButton btnDebug = new JButton("Attiva Debug");
    panel.add(btnUno); panel.add(btnDue); panel.add(btnDebug);

    ActionListener btnListener = e -> {
        lblInfo.setText(e.getActionCommand());
        lblInfo.setForeground(Color.RED);
    };
    btnUno.addActionListener(btnListener);
    btnDue.addActionListener(btnListener);

    ActionListener debugInfo = e -> System.out.println("Sorgente evento: " +
    e.getSource());
    btnDebug.addActionListener(e -> {
        btnUno.addActionListener(debugInfo);
        btnDue.addActionListener(debugInfo);
    });

    frame.setVisible(true);
});
```

A.5 Capitolo 5: Thread, Concorrenza, Executor e Virtual Threads

Fascia 1: Base (Ciclo di Vita e Sincronizzazione Elementare)

Esercizio 5.1: Calcolo Concorrente di Numeri Primi con Thread, Runnable e Partizionamento Manuale

Livello: *Livello 1 – Base.*
Argomento del Task: Creazione esplicita di thread nativi, interfaccia `Runnable`, partizionamento matematico di intervalli discreti e attesa sincronizzata con `join()`.
Riferimento Didattico: Dispensa D04 (*Esercizio 4.1*) e Slide L04a (Slide 10–18).

Consegna Operativa: Si richiede di realizzare un'applicazione Java per il conteggio dei numeri primi compresi nell'intervallo $[1, N]$ con $N = 10.000.000$:

1. Implementare il predicato matematico `public static boolean isPrime(long n)`;
2. Suddividere l'intervallo totale in M partizioni disgiunte di ampiezza uniforme, dove $M =$

`Runtime`.`getRuntime`().`availableProcessors`();

3. Istanziare M thread operai passando a ciascuno un'istanza di `Runnable` o estendendo una classe dedicata che memorizza il conteggio parziale;
4. Avviare concorrentemente tutti i thread mediante `start()` (non `run()`!);
5. Nel thread principale (`main`), attendere la conclusione di tutti i worker iterando su un array di thread ed invocando `join()`;
6. Sommare i parziali, stampare il tempo di calcolo complessivo e confrontarlo con l'esecuzione a singolo thread.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: PrimeThreadTest.java

```
long max = 1_000_000L;
int threads = 4;
long totale = ManualThreadPrimeCounter.contaPrimi(max, threads);
assert totale == 78498L : "Conteggio numeri primi fino a 1.000.000 errato!";
```

Esercizio 5.2: Migrazione ad ExecutorService con Callable<Long> e Future<Long>

Livello: *Livello 1 – Base.*
Argomento del Task: Utilizzo del Thread Pool industriale (`FixedThreadPool`), interfaccia tipizzata `Callable<V>`, gestione token `Future<V>` e chiusura controllata del pool.
Riferimento Didattico: Dispensa D04 (*Esercizio 4.2*) e Slide L04a (Slide 20–26).

Consegna Operativa: Rifattorizzare il calcolo dell'Esercizio A.5 eliminando la gestione manuale di `Thread`:

1. Istanziare un `ExecutorService` tramite `Executors.newFixedThreadPool(core)`;
2. Modellare ciascuna partizione come una lambda expression o classe anonima conforme a `Callable<Long>`;
3. Sottomettere i task al pool accumulando le istanze di `Future<Long>` restituite in una `List<Future<Long»`;
4. Calcolare la somma totale iterando sulla lista ed invocando `f.get()`;
5. Chiudere ordinatamente il pool con `pool.shutdown()` e `pool.awaitTermination(60, TimeUnit.SECONDS)`.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: ExecutorPrimeTest.java

```
ExecutorService pool = Executors.newFixedThreadPool(4);
long totale = ExecutorPrimeCounter.calcola(1_000_000L, pool);
pool.shutdown();
assert totale == 78498L : "Conteggio con Executor errato!";
```

Esercizio 5.3: Dimostrazione Empirica di Race Condition e Disamina Bytecode di count++

Livello: *Livello 1 – Base.*
Argomento del Task: Analisi forense delle corse critiche su variabili condivise, disassemblaggio bytecode (`javap`), confronto prestazionale e di correttezza tra non-sincronizzato, `synchronized`, `AtomicLong` e `LongAdder`.
Riferimento Didattico: Dispensa D04 (*Esercizio 4.3*) e Slide L04a (Slide 27–35).

Consegna Operativa:

1. Progettare quattro contatori concorrenti:
   - `UnsafeCounter`: campo `long count` modificato con `count++`;

- `SynchronizedCounter`: metodo `public synchronized void increment();`
- `AtomicCounter`: basato su `AtomicLong.incrementAndGet();`
- `AdderCounter`: basato sulla classe ad alto throughput `LongAdder.increment();`

2. Avviare 4 thread concorrenti che eseguono ciascuno 250.000 incrementi (totale atteso: 1.000.000);
3. Dimostrare sperimentalmente che `UnsafeCounter` fallisce sistematicamente producendo un valore inferiore ad 1.000.000, e spiegare tramite le istruzioni di bytecode `getfield`, `iadd`, `putfield` il motivo esatto della perdita;
4. Misurare e confrontare il tempo di esecuzione in millisecondi dei quattro approcci.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: RaceConditionBytecodeTest.java

```
int threads = 4;
int incPerThread = 250_000;
long atteso = (long) threads * incPerThread; // 1.000.000

UnsafeCounter c1 = new UnsafeCounter();
SynchronizedCounter c2 = new SynchronizedCounter();
AtomicCounter c3 = new AtomicCounter();
AdderCounter c4 = new AdderCounter();

ExecutorService pool = Executors.newFixedThreadPool(threads);
CountDownLatch latch = new CountDownLatch(threads);

for (int i = 0; i < threads; i++) {
    pool.submit(() -> {
        for (int j = 0; j < incPerThread; j++) {
            c1.increment();
            c2.increment();
            c3.increment();
            c4.increment();
        }
        latch.countDown();
    });
}

latch.await();
pool.shutdown();

System.out.printf("Esiti -> Unsafe: %d, Sync: %d, Atomic: %d, Adder: %d\n",
        c1.get(), c2.get(), c3.get(), c4.get());

assert c1.get() < atteso : "Anomalia: UnsafeCounter non ha prodotto race
    condition!";
assert c2.get() == atteso : "Errore: SynchronizedCounter ha perso incrementi!";
assert c3.get() == atteso : "Errore: AtomicCounter ha perso incrementi!";
assert c4.get() == atteso : "Errore: AdderCounter ha perso incrementi!";
```

Fascia 2: Intermedio (Pattern di Coordinamento e Utilità Concorrenti)

Esercizio 5.4: Pattern Produttore-Consumatore Canonico con wait() e notifyAll()

Livello: *Livello 2 – Intermedio.*
Argomento del Task: Coordinamento primitivo tra thread, monitor intrinseci, gestione della coda circolare a dimensione fissa, prevenzione dei risvegli spuri (*spurious wakeups*) con `while` e notifica a diffusione.
Riferimento Didattico: Dispensa D04 (*Esercizio 4.4*) e Slide L04a.

Consegna Operativa: Implementare una coda bloccante generica `BoundedBuffer<T>` di capacità fissa K:

1. Utilizzare un array interno `Object[]` gestito con puntatori circolari di testa e coda;
2. Implementare il metodo `public synchronized void put(T item) throws InterruptedException`: se il buffer è pieno, deve attendere in un loop `while (count == capacity) wait();`
3. Implementare il metodo `public synchronized T take() throws InterruptedException`: se il buffer è vuoto, deve attendere in un loop `while (count == 0) wait();`
4. Invocare tassativamente `notifyAll()` dopo ogni inserimento e prelievo per risvegliare i thread concorrenti;
5. Collaudare la struttura con 3 thread produttori e 2 thread consumatori, assicurando che nessun elemento venga perso o duplicato.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: BoundedBufferTest.java

```
BoundedBuffer<Integer> buffer = new BoundedBuffer<>(5);
int totaleElementi = 100;
AtomicInteger consumati = new AtomicInteger(0);

Thread p1 = new Thread(() -> {
    for (int i = 0; i < 50; i++) {
        try { buffer.put(i); } catch (InterruptedException e) {
    Thread.currentThread().interrupt(); }
    }
});
Thread p2 = new Thread(() -> {
    for (int i = 50; i < 100; i++) {
        try { buffer.put(i); } catch (InterruptedException e) {
    Thread.currentThread().interrupt(); }
    }
});

Runnable consumerTask = () -> {
    for (int i = 0; i < 50; i++) {
        try { buffer.take(); consumati.incrementAndGet(); }
        catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    }
};
Thread c1 = new Thread(consumerTask);
Thread c2 = new Thread(consumerTask);

p1.start(); p2.start(); c1.start(); c2.start();
p1.join(); p2.join(); c1.join(); c2.join();

assert consumati.get() == totaleElementi : "Elementi consumati non corrispondenti
    al totale!";
assert buffer.isEmpty() : "Il buffer dovrebbe essere vuoto al termine!";
```

Esercizio 5.5: Produttore-Consumatore Industriale con ArrayBlockingQueue e ReentrantLock

Livello: *Livello 2 – Intermedio.*
Argomento del Task: Utilizzo avanzato di `ArrayBlockingQueue` ed implementazione con `ReentrantLock` e due code di attesa separate `Condition` (`notFull`, `notEmpty`).
Riferimento Didattico: Dispensa D04 (*Esercizio 4.5*).

Consegna Operativa: Riprogettare il buffer bloccante dell'Esercizio A.5 impiegando i lock espliciti di `java.util.concurrent.locks`:

1. Utilizzare un'istanza di `ReentrantLock`;

2. Creare due condizioni indipendenti: `Condition notFull = lock.newCondition()` e `Condition notEmpty = lock.newCondition();`
3. Nel metodo `put()`, se la coda è piena invocare `notFull.await()`, e all'inserimento segnalare selettivamente con `notEmpty.signal()`;
4. Nel metodo `take()`, se la coda è vuota invocare `notEmpty.await()`, e al prelievo segnalare con `notFull.signal()`;
5. Spiegare perché le due condition distinte offrono prestazioni nettamente superiori a `notifyAll()`, eliminando il risveglio inutile di thread non interessati.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: ExplicitLockQueueTest.java

```
LockConditionBuffer<String> queue = new LockConditionBuffer<>(10);
ExecutorService pool = Executors.newFixedThreadPool(6);
CountDownLatch latch = new CountDownLatch(1000);

for (int i = 0; i < 500; i++) {
    final String msg = "TICKET-" + i;
    pool.submit(() -> {
        try { queue.put(msg); } catch (InterruptedException e) {
    Thread.currentThread().interrupt(); }
    });
    pool.submit(() -> {
        try { String res = queue.take(); assert res != null; latch.countDown(); }
        catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    });
}

boolean completato = latch.await(5, TimeUnit.SECONDS);
pool.shutdown();

assert completato : "Timeout: potenziale stallo o deadlock tra le condition
    notFull e notEmpty!";
assert queue.size() == 0 : "Coda non vuota al termine del collaudo!";
```

Esercizio 5.6: Riproduzione Deterministica di Deadlock e Risoluzione con Global Lock Ordering

Livello: *Livello 2 – Intermedio.*
Argomento del Task: Analisi formale dello stallo ciclico tra due lock concorrenti, diagnosi delle 4 condizioni di Coffman e risoluzione architetturale mediante ordinamento globale dei lock.
Riferimento Didattico: Dispensa D04 (*Esercizio 4.6*).

Consegna Operativa:

1. Modellare la classe `ContoCorrente` (id numerico univoco, saldo in euro);
2. Scrivere un metodo ingenuo `trasferisci(ContoCorrente da, ContoCorrente a, double importo)` che acquisisce prima `synchronized(da)` e poi annidato `synchronized(a)`;
3. Avviare due thread che eseguono in contemporanea: Thread 1 trasferisce da Conto 1 a Conto 2, mentre Thread 2 trasferisce da Conto 2 a Conto 1. Verificare che l'applicazione collassa in Deadlock;
4. Riscrivere il metodo applicando il **Global Lock Ordering**: confrontare gli ID numerici dei due conti ed acquisire sempre per primo il lock del conto con ID minore, garantendo che nessun ciclo possa mai generarsi.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: GlobalLockOrderingTest.java

```
ContoCorrente cA = new ContoCorrente(101, 1000.0);
ContoCorrente cB = new ContoCorrente(102, 1000.0);

Thread t1 = new Thread(() -> {
    for (int i = 0; i < 50_000; i++) cA.trasferisciOrdinato(cB, 1.0);
});
Thread t2 = new Thread(() -> {
    for (int i = 0; i < 50_000; i++) cB.trasferisciOrdinato(cA, 1.0);
});

t1.start(); t2.start();
t1.join(3000); t2.join(3000);

assert !t1.isAlive() && !t2.isAlive() : "DEADLOCK: i thread risultano ancora
    bloccati in mutua attesa!";
assert (cA.getSaldo() + cB.getSaldo()) == 2000.0 : "Violazione dell'invariante di
    conservazione monetaria!";
```

Esercizio 5.7: GUI Reattiva con SwingWorker ed Event Dispatch Thread

Livello: *Livello 2 – Intermedio.*
Argomento del Task: Integrazione tra concorrenza di calcolo e grafica Swing, disaccoppiamento dei thread di background dall'EDT e aggiornamento progressivo di una `JProgressBar`.
Riferimento Didattico: Dispensa D04 (*Esercizio 4.7*) e §5.9.

Consegna Operativa: Costruire una finestra grafica con un pulsante `"Avvia Calcolo"`, una barra di avanzamento e una label informativa:

1. Alla pressione del pulsante, disabilitarlo ed avviare un task `SwingWorker<Long, Integer>`;
2. Il metodo `doInBackground()` scandisce 10.000.000 di numeri calcolando la primalità;
3. Ogni 100.000 iterazioni, invoca `publish(percentuale)`;
4. Il metodo `process()` aggiorna il valore della `JProgressBar` in modo sicuro sull'EDT;
5. Il metodo `done()` riabilita il pulsante e visualizza il risultato finale;
6. Dimostrare che durante l'elaborazione l'utente può ridimensionare e trascinare la finestra senza alcun blocco visivo.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: PrimeSwingWorkerTest.java

```
JProgressBar bar = new JProgressBar(0, 100);
JLabel status = new JLabel("In attesa");
CountDownLatch doneLatch = new CountDownLatch(1);

PrimeComputationWorker worker = new PrimeComputationWorker(1_000_000L, bar,
    status, doneLatch);
worker.execute();

boolean completato = doneLatch.await(5, TimeUnit.SECONDS);
assert completato : "SwingWorker non completato entro il timeout di 5s!";
assert bar.getValue() == 100 : "La progress bar non ha raggiunto il 100%!";
assert status.getText().contains("78498") : "Risultato finale del conteggio primi
    non aggiornato su label!";
```

Fascia 3: Avanzato (High-Scale Systems & Next-Gen Java)

Esercizio 5.8: Benchmark di Scalabilità Massiva con 100.000 Virtual Threads in Java 21

Livello: *Livello 3 – Avanzato (Java 21 LTS Standard).*
Argomento del Task: Architettura M:N dei Virtual Threads (Project Loom), verifica empirica dell'unmounting su chiamate bloccanti e confronto prestazionale con i Platform Threads del kernel.
Riferimento Didattico: Dispensa D04 (*Esercizio 4.8*) e §5.8.

Consegna Operativa:

1. Scrivere un'applicazione di benchmarking che utilizza `Executors.newVirtualThreadPerTaskExecutor()`;
2. Sottomettere 100.000 task indipendenti, ciascuno dei quali simula una latenza di rete bloccante invocando `Thread.sleep(Duration.ofSeconds(1))`;
3. Utilizzare il costrutto `try-with-resources` sull'executor per attendere automaticamente la conclusione di tutti i task;
4. Misurare il tempo di esecuzione totale e la memoria fisica occupata dalla JVM;
5. Dimostrare teoricamente e sperimentalmente che il medesimo test eseguito su un pool di Platform Threads convenzionali provocherebbe il collasso immediato del sistema per esaurimento della RAM o errore di allocazione nativa del sistema operativo.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: VirtualThreadScaleTest.java

```
long t0 = System.currentTimeMillis();
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 100_000; i++) {
        executor.submit(() -> {
            Thread.sleep(1000);
            return 1;
        });
    }
}
long t1 = System.currentTimeMillis();
System.out.printf("100.000 Virtual Thread completati in: %d ms (Atteso: ~1.5 -
    2.5 secondi)\n", (t1 - t0));
assert (t1 - t0) < 5000 : "Esecuzione virtual thread eccessivamente lenta!";
```

Esercizio 5.9: [Evolutivo Filone A – Parte 5] ConcurrentOrderProcessor & MultiThreadInventorySync

Livello: *Livello 3 – Avanzato (Progetto Evolutivo – Filone Magazzino & E-Commerce, Parte 5).*
Argomento del Task: Elaborazione asincrona concorrente degli ordini di acquisto multithread, prevenzione di over-selling e corse critiche sulle giacenze di magazzino (`InventoryStore`), lock granulari per articolo (`ReentrantLock`), ordinamento globale dei lock contro deadlock multi-articolo, coda di spedizione `BlockingQueue` e tracciamento contabile con `LongAdder`.
Riferimento Didattico: §5.6, §5.7 ed evoluzione di Es. A.1, Es. A.2, Es. A.3, Es. A.4.

Consegna Operativa: Completare l'infrastruttura di gestione del negozio digitale introducendo l'elaborazione concorrente degli acquisti:

1. **Magazzino Thread-Safe con Lock Granulari**: Estendere `InventoryStore` per consentire a molteplici thread di verificare e scalare le scorte in parallelo. Associare a ciascun codice articolo un `ReentrantLock` dedicato (oppure impiegare `ConcurrentHashMap` con mutazioni atomiche su `compute()`), garantendo che ordini su articoli disgiunti non subiscano contesa né serializzazione;
2. **Elaborazione Asincrona tramite Worker Pool**: Configurare un thread pool (`ExecutorService`) con K worker che consumano ordini da una `BlockingQueue<Ordine>`: i client di checkout (Es. A.4) sottomettano ordini in modo asincrono senza bloccare l'interfaccia utente;

3. **Prenotazione Atomica Multi-Articolo e Anti-Deadlock**: Qualora un carrello contenga più articoli distinti (es. 2 unità di SKU-A e 1 unità di SKU-B), l'acquisizione dei lock di inventario deve avvenire rigorosamente secondo il **Global Lock Ordering** (ordinamento lessicografico crescente degli SKU) per prevenire deadlock tra ordini incrociati. Se un articolo risulta esaurito, l'intera transazione deve essere respinta (politica *All-or-Nothing*) rilasciando immediatamente tutti i lock già acquisiti;
4. **Metriche di Cassa e Notifiche di Esaurimento**: Aggiornare il fatturato globale concorrente mediante `LongAdder` (azzerando la contesa tra core) e notificare gli eventi di esaurimento scorte ai listener registrati tramite `CopyOnWriteArrayList`.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: ConcurrentOrderProcessorTest.java

```
InventoryStore store = new InventoryStore();
store.caricaArticolo("ART-01", 100); // 100 unita' disponibili
store.caricaArticolo("ART-02", 50);  // 50 unita' disponibili

ConcurrentOrderProcessor processor = new ConcurrentOrderProcessor(store, 4);
int numClienti = 80;
CountDownLatch latch = new CountDownLatch(numClienti);

// 80 clienti concorrenti tentano di acquistare 2 unita' di ART-01 e 1 unita' di ART-02:
for (int i = 0; i < numClienti; i++) {
    Map<String, Integer> carrello = Map.of("ART-01", 2, "ART-02", 1);
    processor.sottomettiOrdine(new Ordine("ORD-" + i, carrello), latch);
}

boolean completato = latch.await(10, TimeUnit.SECONDS);
processor.shutdown();

assert completato : "Timeout nell'elaborazione concorrente degli ordini!";
// Essendo disponibili solo 50 unita' di ART-02, esattamente 50 ordini devono andare a buon fine e 30 devono fallire:
assert processor.getOrdiniCompletati() == 50 : "Over-selling rilevato su ART-02!";
assert processor.getOrdiniFalliti() == 30 : "Conteggio ordini falliti errato!";
assert store.getGiacenza("ART-01") == 0 : "Giacenza residua di ART-01 errata (atteso 0: 100 - 50*2)!";
assert store.getGiacenza("ART-02") == 0 : "Giacenza residua di ART-02 errata!";
```

Esercizio 5.10: [Evolutivo Filone B – Parte 5] ConcurrentUrbanTrafficSimulator: Simulatore Multi-Agente con Virtual Threads & Semaphore per Corsie

Livello: *Livello 3 – Avanzato (Progetto Evolutivo – Filone Smart City & Mobilità, Parte 5).*
Argomento del Task: Simulazione massiva multi-agente di mobilità urbana in Java 21 LTS con 10.000 veicoli concorrenti modellati come Virtual Threads leggeri, gestione della capacità fisica delle corsie tramite `Semaphore`, regolazione degli incroci a precedenza/semaforo e telemetria non bloccante disaccoppiata verso il canvas grafico Swing.
Riferimento Didattico: §5.8, §5.7 ed evoluzione di Es. A.1, Es. A.2, Es. A.3, Es. A.4.

Consegna Operativa: Portare il simulatore di mobilità urbana alla massima scala computazionale mediante le funzionalità di concorrenza moderna di Java 21:

1. **Veicoli come Virtual Threads Autonomi**: Ogni veicolo immesso nella rete stradale è eseguito come un task indipendente all'interno di un `newVirtualThreadPerTaskExecutor()`; il ciclo vitale del veicolo modella la navigazione passo-passo lungo il grafo stradale, calcolando tempi di transito con attese bloccanti `Thread.sleep()` a zero consumo di thread OS del kernel;
2. **Capacità di Corsia con `Semaphore`**: Ciascuna tratta stradale dispone di una capienza fisica massima di veicoli simultanei K (es. 15 auto). L'ingresso sulla corsia impone l'acquisizione di un permesso con `corsiaSemaphore.acquire()`; il rilascio avviene solo quando il veicolo raggiunge

l'incrocio successivo;

3. **Attraversamento Incroci con Priorità Dinamica**: Negli incroci semaforizzati guidati dalla FSM (Es. A.2), i veicoli transitano solo durante la fase verde. Negli incroci a precedenza, la confluenza tra corsie è protetta da un `ReentrantLock` equo (`fair = true`) per garantire l'assenza di starvation dei flussi secondari;
4. **Telemetria Atomica verso il Canvas Grafico**: Per consentire al canvas Swing (Es. A.4) di visualizzare le posizioni delle migliaia di auto a 25 FPS senza bloccare i thread di simulazione né saturare l'EDT, i veicoli pubblicano le loro coordinate correnti su un buffer ad array atomico (`AtomicReference`), permettendo al timer Swing di estrarre lo snapshot globale in $O(1)$ senza lock di attesa.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: ConcurrentTrafficSimulatorTest.java

```
ReteStradale rete = new ReteStradale();
Incrocio inA = new Incrocio("INC-A", 0, 0);
Incrocio inB = new Incrocio("INC-B", 100, 0);
TrattaAutostradale tratta = new TrattaAutostradale(inA, inB, 10); // Capienza max: 10 veicoli
rete.aggiungiTratta(tratta);

ConcurrentTrafficSimulator sim = new ConcurrentTrafficSimulator(rete);
int veicoliTotali = 5000;
CountDownLatch traguardoLatch = new CountDownLatch(veicoliTotali);

long start = System.currentTimeMillis();
sim.avviaSimulazioneMassiva(veicoliTotali, traguardoLatch);

boolean finiti = traguardoLatch.await(15, TimeUnit.SECONDS);
long elapsed = System.currentTimeMillis() - start;
sim.stop();

assert finiti : "Timeout: la simulazione con 5.000 veicoli non ha terminato entro 15s!";
assert tratta.getMaxVeicoliRilevati() <= 10 : "Violazione di capienza della tratta: superato il limite del semaforo!";
System.out.printf("5.000 veicoli Virtual Thread simulati con successo in %d ms\n", elapsed);
```

Esercizio 5.11: Gestione del Ciclo di Vita del ThreadPool: CachedThreadPool, Shutdown e AwaitTermination

Livello: *Livello 2 – Intermedio (Concorrenza e Executor Framework).*
Argomento del Task: Gestione del ciclo di vita dei thread worker mediante `ExecutorService`, sottomissione di task multipli asincroni con `submit()`, protocollo canonico di arresto controllato (*Two-Phase Termination* con `shutdown()`, `awaitTermination()` e `shutdownNow()`) e reazione cooperativa all'interruzione (`Thread.currentThread().interrupt()`).
Riferimento Didattico: Laboratorio ufficiale *LezThread01* (`MyTask.java` e `ThreadApp03.java`) e §5.4.

Consegna Operativa: Nel progetto di laboratorio `LezThread01`, si richiede di costruire un sistema di orchestrazione di task asincroni conforme al paradigma moderno dei pool di thread:

1. **Definizione del Task Lavoratore (`MyTask`)**: Realizzare la classe `MyTask` che implementa `Runnable`:
   - Campi incapsulati: nome del task (`String id`), numero di iterazioni da compiere (`int steps`) e tempo di sospensione in millisecondi (`int delay`);
   - Nel metodo `run()`, eseguire un ciclo che stampa il progresso del task e sospende il thread corrente con `Thread.sleep(delay)`;

- In caso di `InterruptedException`, interrompere immediatamente il ciclo e ripristinare il flag di interruzione invocando `Thread.currentThread().interrupt();`

2. **Istanziazione del Pool Dinamico**: Nel `main()`, istanziare un thread pool elastico con `Executors.newCachedThreadPool()` e sottomettere 5 task concorrenti con parametri di carico differenziati;
3. **Protocollo di Arresto Controllato (Two-Phase Termination)**: Implementare la sequenza di terminazione corretta:
   - Invocare `shutdown()`: il pool rifiuta nuovi compiti ma lascia completare quelli già sottomessi;
   - Invocare `awaitTermination(5, TimeUnit.SECONDS)`: il thread principale attende fino a 5 secondi il termine naturale dei lavoratori;
   - Se il timeout scade senza che tutti i task siano terminati, forzare l'arresto immediato con `shutdownNow()` (che invia un segnale di interrupt ai task attivi);
   - Gestire l'eventuale `InterruptedException` forzando `shutdownNow()` e ripristinando l'interrupt del thread chiamante.

Specifiche di Collaudo e Codice del Laboratorio:

Codice del Laboratorio: ThreadApp03.java

```
ExecutorService executor = Executors.newCachedThreadPool();

executor.submit(new MyTask("task 1", 10, 300));
executor.submit(new MyTask("task 2", 10, 200));
executor.submit(new MyTask("task 3", 10, 100));
executor.submit(new MyTask("task 4", 10, 250));
executor.submit(new MyTask("task 5", 1000, 100)); // Task lungo

executor.shutdown(); // Non accetta nuovi compiti
try {
    if (!executor.awaitTermination(5, TimeUnit.SECONDS)) {
        System.out.println("Timeout: forzatura arresto immediato...");
        executor.shutdownNow(); // Invia interrupt() ai task attivi
    }
} catch (InterruptedException ex) {
    executor.shutdownNow();
    Thread.currentThread().interrupt(); // Ripristina flag interrupt
}
```

Palestra Ufficiale del Corso (Dalla Dispensa D04, Cap. 9 – Prof. Massimiliano Redolfi)

Esercizio 5.12: Contare i Thread: start() vs run() e il Ciclo di Vita (D04 Es. 9.1)

Livello: *Livello 1 – Base (Stile Prova Scritta).*
Argomento del Task: Differenza cardinale tra `start()` e `run()`, gestione del ciclo di vita dei thread, sollevamento di `IllegalThreadStateException` su thread terminati e monitoraggio con `Thread.activeCount()`.
Riferimento Didattico: Dispensa D04 (*Esercizio 9.1*) e Slide L04a.

Consegna Operativa: Si consideri il frammento base:

```
Runnable r = () -> System.out.println("ciao da " + Thread.currentThread().getName());
```

Per ciascuno dei seguenti quattro frammenti, determinare senza eseguire quanti thread vengono creati oltre a `main`, cosa viene stampato e se si verificano eccezioni:

1. `new Thread(r).start(); new Thread(r).start(); r.run();`
2. `Thread t = new Thread(r); t.run(); t.start();`
3. `Thread u = new Thread(r); u.start(); u.join(); u.start();`
4. `for (int i = 0; i < 3; i++) new Thread(r).run();`

Scrivere un programma che esegue ciascun blocco all'interno di un `try-catch`, stampando il numero di thread vivi con `Thread.activeCount()` e il nome del thread corrente.

Attenzione del Docente:

- In (b) e (d) si legge la parola `Thread` e si tende erroneamente a contare thread: **ciò che conta è solo start()**! Il metodo `run()` è una normale invocazione sequenziale su `main`;
- In (c), il secondo `start()` su un thread già terminato non lo riavvia affatto: solleva tassativamente `IllegalThreadStateException`, poiché un thread in stato `TERMINATED` non può tornare in `RUNNABLE`.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: ThreadCountLifecycleTest.java

```
Runnable r = () -> System.out.println("Saluto da: " +
    Thread.currentThread().getName());

// Caso (b): run() non crea thread, start() ne crea esattamente 1
Thread t = new Thread(r);
t.run(); // Eseguito su main!
t.start(); // Eseguito su Thread-x concorrente
t.join();

// Caso (c): verifica del crash su doppio start()
Thread u = new Thread(r);
u.start();
u.join();
boolean eccezioneCatturata = false;
try {
    u.start(); // Illegale!
} catch (IllegalThreadStateException ex) {
    eccezioneCatturata = true;
}
assert eccezioneCatturata : "Errore: atteso IllegalThreadStateException su
    riavvio!";
```

Esercizio 5.13: Numeri Felici Multithread con Callable e Future (D04 Es. 9.2)

Livello: *Livello 1 – Base (Stile Prova Scritta).*
Argomento del Task: Classe con metodo multithread, pool di thread, sottomissione di `Callable<Integer>`, aggregazione con `Future<Integer>` e gestione rigorosa degli estremi di partizione.
Riferimento Didattico: Dispensa D04 (*Esercizio 9.2*).

Consegna Operativa: Un numero intero positivo è detto **felice** se, sostituendolo ripetutamente con la somma dei quadrati delle sue cifre decimali, la sequenza converge a 1 (es. $19 \to 1^2 + 9^2 = 82 \to 68 \to 100 \to 1$); altrimenti entra in un ciclo periodico chiuso che contiene il numero 4.

1. Realizzare la classe `Felici` con i metodi `public static boolean isFelice(int n)` e `public static int conta(int da, int a)`;
2. Implementare il metodo concorrente `public static int contaMT(int da, int a, int nThread)`, che partiziona l'intervallo $[da, a]$ in $nThread$ blocchi, sottomette i compiti a un `ExecutorService` e somma i risultati parziali tramite `Future`;
3. Verificare nel `main()` che `conta()` e `contaMT()` restituiscano l'identico risultato fino a 10.000.000 e misurare lo speedup al variare dei thread $(1, 2, 4, 8)$.

Attenzione del Docente: L'errore più frequente nelle prove scritte risiede nella gestione degli estremi dell'intervallo: un numero contato due volte (estremo condiviso tra compiti) o mai contato (quando

l'ampiezza non divide esattamente l'intervallo). L'ultimo compito deve estendersi esplicitamente fino ad a.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: NumeriFeliciTest.java

```
assert Felici.isFelice(19) : "19 deve essere felice!";
assert !Felici.isFelice(4) : "4 non e' felice (ciclo chiuso)!";
assert Felici.isFelice(7) : "7 deve essere felice!";

int da = 1, a = 1_000_000;
int seq = Felici.conta(da, a);
int par = Felici.contaMT(da, a, Runtime.getRuntime().availableProcessors());

assert seq == par : String.format("Incongruenza: sequenziale=%d, parallelo=%d",
    seq, par);
System.out.printf("Numeri felici fino a %d: %d (Verifica superata).\n", a, par);
```

Esercizio 5.14: Il Crivello di Eratostene a Blocchi e LongAdder (D04 Es. 9.3)

Livello: *Livello 2 – Intermedio.*
Argomento del Task: Crivello segmentato per numeri primi fino a 10^9, confinamento nello stack (*Thread Confinement*) di array locali, aggregazione massiva ad altissimo throughput tramite `LongAdder`.
Riferimento Didattico: Dispensa D04 (*Esercizio 9.3*).

Consegna Operativa: Contare i numeri primi fino a 10^9 con test di divisione è troppo oneroso. Si richiede di progettare un **crivello segmentato concorrente**:

1. Calcolare una tantum i numeri primi fino a $\sqrt{10^9} \approx 31.623$ mediante un crivello classico sequenziale in memoria;
2. Suddividere l'intervallo $[1, 10^9]$ in blocchi omogenei da 1.000.000 di elementi ciascuno;
3. Ciascun compito alloca un proprio array locale `boolean[] composto` di ampiezza pari al blocco (confinamento nel task!), lo crivella con i primi base precalcolati e incrementa il conteggio dei sopravvissuti su un'istanza condivisa di `LongAdder`;
4. Eseguire su un pool con tanti thread quanti i core fisici della macchina e confrontare il tempo con l'approccio sequenziale.

Attenzione del Docente: L'array dei booleani deve nascere tassativamente all'interno del compito: è il **confinamento dei dati**! Un unico array condiviso di un miliardo di elementi occuperebbe 1 GB di Heap e genererebbe conflitti catastrofici; un array per blocco occupa appena 1 MB nello stack locale ed è immune da contesa.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: CrivelloSegmentatoTest.java

```
long max = 1_000_000_000L;
LongAdder totalePrimi = new LongAdder();

long t0 = System.currentTimeMillis();
CrivelloSegmentato.esegui(max, totalePrimi);
long t1 = System.currentTimeMillis();

long risultato = totalePrimi.sum();
System.out.printf("Crivello 10^9 completato in %d ms. Primi trovati: %d\n", (t1 -
    t0), risultato);
// Risultato analitico atteso dalla teoria dei numeri:
assert risultato == 50_847_534L : "Conteggio errato per i primi fino a 10^9!";
```

Esercizio 5.15: Sorgente di Eventi Thread-Safe con CopyOnWriteArrayList (D04 Es. 9.4)

Livello: *Livello 2 – Intermedio.*
Argomento del Task: Modifica di collezioni durante l'iterazione, fallimento di `ArrayList` per `ConcurrentModificationException`, inefficacia di `synchronized` per rientranza e soluzione definitiva con `CopyOnWriteArrayList`.
Riferimento Didattico: Dispensa D04 (*Esercizio 9.4*).

Consegna Operativa: Progettare una classe `SorgenteEventi` dotata dei metodi `addListener(Listener l)`, `removeListener(Listener l)` e `notifica(String evento)` che invoca tutti i listener registrati:

1. Avviare molteplici thread che notificano eventi a ripetizione mentre un thread concorrente aggiunge e rimuove listener;
2. Uno dei listener, al terzo evento ricevuto, deve **rimuovere se stesso** all'interno del metodo di callback;
3. Dimostrare che impiegando una comune `ArrayList`, l'applicazione lancia `ConcurrentModificationException`;
4. Spiegare perché proteggere il ciclo `for` con `synchronized(this)` **non risolve il problema** (a causa della rientranza del monitor sul medesimo thread);
5. Risolvere il problema adottando `CopyOnWriteArrayList` e documentare nel commento perché il listener che si auto-rimuove riceve comunque l'evento in corso in modo del tutto coerente.

Attenzione del Docente: Chi mette `synchronized` attorno al `for` di notifica evita l'interferenza tra thread diversi, ma non evita l'eccezione scatenata dal listener che si rimuove all'interno del callback sullo stesso thread! Il monitor è rientrante e lo lascia passare. La copia per scrittura è l'unica risposta corretta.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: CopyOnWriteListenerTest.java

```
SorgenteEventi sorgente = new SorgenteEventi(); // Internamente
    CopyOnWriteArrayList
AtomicInteger eventiRicevuti = new AtomicInteger(0);

Listener autoRimuovente = new Listener() {
    private int count = 0;
    @Override
    public void onEvent(String ev) {
        eventiRicevuti.incrementAndGet();
        if (++count == 3) {
            sorgente.removeListener(this); // Auto-rimozione durante l'iterazione!
        }
    }
};

sorgente.addListener(autoRimuovente);
for (int i = 0; i < 10; i++) {
    sorgente.notifica("Evento-" + i);
}

assert eventiRicevuti.get() == 3 : "Il listener avrebbe dovuto ricevere
    esattamente 3 eventi!";
```

Esercizio 5.16: Code al Supermercato: BlockingQueue, Sentinelle Multiple e Statistiche (D04 Es. 9.5)

Livello: *Livello 2 – Intermedio.*
Argomento del Task: Pattern Produttore-Consumatore con `BlockingQueue`, protocollo di ter-

minazione a sentinella multipla (*Poison Pill*) e statistiche aggregate concorrenti con `LongAdder` e `LongAccumulator`.
Riferimento Didattico: Dispensa D04 (*Esercizio 9.5*).

Consegna Operativa: Modellare la gestione di una barriera casse di un supermercato:

1. Definire il record immutabile `record Cliente(int id, long arrivoMillis, int numArticoli)`;
2. Un thread produttore genera 100 clienti a intervalli casuali e li deposita in una `ArrayBlockingQueue<Cliente>` di capacità limitata;
3. Tre casse (ciascuna eseguita su un proprio thread operaio) estraggono il prossimo cliente e ne simulano il servizio con una pausa proporzionale al numero di articoli;
4. Alla conclusione dei 100 clienti, il generatore deve arrestare tutte e tre le casse in modo ordinato senza perdere alcun cliente;
5. Calcolare e stampare a fine giornata: clienti serviti per ciascuna cassa, tempo medio di attesa in coda e tempo massimo di attesa registrato (usando `LongAdder` e `LongAccumulator`).

Attenzione del Docente: Con tre casse consumatrici, **una sola sentinella di arresto ferma una sola cassa**! Le altre due rimarrebbero bloccate per sempre su `take()`. La soluzione corretta consiste nell'inserire in coda esattamente **tre sentinelle distinte** (una per cassa), oppure fare in modo che la prima cassa che incontra la sentinella la reinserisca in coda prima di terminare.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: SupermercatoQueueTest.java

```
BlockingQueue<Cliente> coda = new ArrayBlockingQueue<>(20);
Cliente SENTINELLA = new Cliente(-1, 0, 0);

AtomicInteger servitiTotali = new AtomicInteger(0);
LongAdder tempoAttesaTotale = new LongAdder();
LongAccumulator attesaMassima = new LongAccumulator(Long::max, 0);

// Avvio 3 casse operaie...
// Generazione 100 clienti reali + 3 sentinelle...
// Attesa terminazione thread casse...

assert servitiTotali.get() == 100 : "Devono essere serviti esattamente 100
    clienti!";
assert coda.isEmpty() : "La coda deve risultare vuota a fine giornata!";
System.out.printf("Attesa media: %d ms, Attesa max: %d ms.\n",
        tempoAttesaTotale.sum() / 100, attesaMassima.get());
```

Esercizio 5.17: Scaricare in Parallelo con 50 Virtual Threads e Barra ASCII (D04 Es. 9.6)

Livello: *Livello 2 – Intermedio (Java 21 LTS).*
Argomento del Task: Utilizzo dei Virtual Threads per carichi I/O-bound ad alta concorrenza, contatore atomico `AtomicLong` e thread osservatore di telemetria.
Riferimento Didattico: Dispensa D04 (*Esercizio 9.6*).

Consegna Operativa: Simulare lo scaricamento concorrente di 50 file di dimensione casuale compresa tra 1 e $20MB$:

1. Ogni download modella la latenza sospendendosi con `Thread.sleep(1)` per ogni $10KB$ e aggiorna un'istanza condivisa di `AtomicLong` a blocchi di $100KB$;
2. Avviare tutti i 50 download concorrenti tramite un Virtual Thread per ciascun file (`newVirtualThreadPerTaskExecutor()`);
3. Nel frattempo, un thread convenzionale di piattaforma legge l'`AtomicLong` ogni $200ms$ e stampa a console sulla medesima riga una barra di avanzamento visuale (`[========== ] 52%`);

4. Misurare il tempo totale di esecuzione e confrontarlo con un'esecuzione su un pool a 4 thread nativi.

Attenzione del Docente: La barra di avanzamento non necessita di valori matematicamente esatti al microsecondo: legge un valore atomico consistente, ancorché vecchio di pochi istanti. Bloccare tutti i 50 thread per sincronizzare la lettura della barra sarebbe un gravissimo errore di design!

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: VirtualDownloadTest.java

```
AtomicLong byteScaricati = new AtomicLong(0);
long byteTotali = 50L * 10 * 1024 * 1024; // Stima media

long t0 = System.currentTimeMillis();
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 50; i++) {
        executor.submit(() -> {
            for (int k = 0; k < 100; k++) {
                Thread.sleep(10);
                byteScaricati.addAndGet(100 * 1024);
            }
            return true;
        });
    }
}
long t1 = System.currentTimeMillis();
System.out.printf("50 download completati in %d ms con Virtual Threads.\n", (t1 -
    t0));
assert byteScaricati.get() == 50L * 100 * 100 * 1024 : "Byte scaricati non
    corrispondenti!";
```

Esercizio 5.18: PrimeFinderApp Completa: SwingWorker a 64 Compiti e Cancellazione a Tre Livelli (D04 Es. 9.7)

Livello: *Livello 3 – Avanzato (Progetto Completo Swing + Concorrenza).*
Argomento del Task: Integrazione tra GUI reattiva, `SwingWorker`, pool interno di calcolo a 64 task e propagazione cooperativa della cancellazione a tre livelli.
Riferimento Didattico: Dispensa D04 (*Esercizio 9.7*) e Slide L04a.

Consegna Operativa: Realizzare la versione definitiva e industriale di `PrimeFinderApp`:

1. Interfaccia dotata di campo per il limite N, campo per il numero di thread worker (default pari ai core della macchina), pulsanti "Conta" e "Annulla", barra di avanzamento e label informativa;
2. Al clic su "Conta", uno `SwingWorker` istanzia un pool dedicato con il numero di thread scelto, suddivide l'intervallo $[1, N]$ in 64 compiti, li sottomette e aggiorna l'avanzamento man mano che i `Future` completano;
3. Il pulsante "Annulla" interrompe tempestivamente l'elaborazione (i compiti in corso devono accorgersene entro $100ms$);
4. Alla chiusura della finestra, nessun thread operaio deve rimanere orfano nella JVM.

Attenzione del Docente: Invocare `cancel(true)` su `SwingWorker` invia l'interrupt unicamente al thread del worker, **non ai 16 thread del pool interno**! Il worker ha la responsabilità etica di propagare il segnale: deve invocare `shutdownNow()` sul pool nel blocco `finally`, e i singoli compiti computazionali devono verificare periodicamente `Thread.currentThread().isInterrupted()`. Sono tre livelli di interruzione cooperativa a cascata.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: PrimeFinderAppFullTest.java

```
PrimeFinderAppReattiva app = new PrimeFinderAppReattiva();
assert app != null : "Creazione app fallita!";
// Verifica che l'architettura supporti l'annullamento cooperativo immediato
```

Esercizio 5.19: Un Mandelbrot in Piccolo: Rendering Progressivo e Annullamento Obsoleti (D04 Es. 9.8)

Livello: *Livello 3 – Avanzato.*
Argomento del Task: Rendering multithread progressivo, consegna asincrona di immagini tramite `EventQueue.invokeLater()`, zoom interattivo, cancellazione compiti obsoleti e differenza vitale tra `isInterrupted()` e `interrupted()`.
Riferimento Didattico: Dispensa D04 (*Esercizio 9.8*).

Consegna Operativa: Costruire un pannello grafico Swing (600 × 400 pixel) per la visualizzazione dell'insieme frattale di Mandelbrot ($z \leftarrow z^2 + c$):

1. Suddividere il calcolo in tre passate successive a risoluzione crescente (un pixel ogni 8, ogni 2, tutti i pixel);
2. Ciascuna passata genera una `BufferedImage` e la consegna alla GUI tramite `EventQueue.invokeLater()`, così che l'immagine grezza compaia all'istante e si perfezioni progressivamente;
3. Al clic del mouse, l'immagine si ricentra sulle coordinate cliccate e raddoppia lo zoom: i compiti della passata precedente non ancora eseguiti vanno cancellati con `Future.cancel()`, e quelli in corso devono interrompersi subito per non sovrascrivere la nuova inquadratura con frame obsoleti.

Attenzione del Docente: Un compito che controlla lo stato di interruzione invocando il metodo statico `Thread.interrupted()` **azzera la bandiera al primo controllo positivo**! Se prima di consegnare l'immagine verifica di nuovo la bandiera, la troverà abbassata e consegnerà l'immagine obsoleta coprendo quella nuova. È un errore reale visto sul campo: si deve usare il metodo di istanza `isInterrupted()`, che legge lo stato senza modificarlo.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: MandelbrotRendererTest.java

```
MandelbrotCalculator calc = new MandelbrotCalculator(600, 400);
Future<?> f = calc.avviaPassata(1); // Passata grezza
calc.richiediZoom(300, 200); // Trigger annullamento immediato
assert f.isCancelled() || f.isDone() : "Il compito obsoleto doveva essere
    annullato!";
```

Esercizio 5.20: La Cache che Calcola una Volta Sola: Memoizer con FutureTask (D04 Es. 9.9)

Livello: *Livello 3 – Avanzato (Architettura Concorrente Zero-Contesa).*
Argomento del Task: Realizzazione di una cache thread-safe ad altissime prestazioni (`Memo<K, V>`), token asincrono `FutureTask<V>` come segnaposto, eliminazione dei blocchi lunghi all'interno di `computeIfAbsent()` e prevenzione di deadlock.
Riferimento Didattico: Dispensa D04 (*Esercizio 9.9*).

Consegna Operativa: Progettare una classe generica `Memo<K, V>` che avvolge una funzione costosa `Function<K, V>` e garantisce che per ciascuna chiave K il calcolo venga eseguito **una sola volta**, anche se 16 thread richiedono la medesima chiave contemporaneamente:

1. Utilizzare internamente una `ConcurrentHashMap<K, Future<V>`;
2. All'invocazione di `get(K key)`, impiegare `computeIfAbsent()` per inserire un'istanza di

`FutureTask<V>` come puro segnaposto atomico istantaneo;

3. Il thread che ha inserito con successo il segnaposto esegue il calcolo con `task.run()` **fuori dal lock della mappa**; tutti gli altri thread che trovano il segnaposto già presente attendono il risultato invocando `task.get()`;
4. Se la funzione genera un'eccezione, rimuovere la chiave per non lasciare la cache avvelenata;
5. Collaudare con 16 thread che richiedono le stesse 10 chiavi: verificare che la funzione costosa venga invocata esattamente 10 volte e non 160.

Attenzione del Docente: Eseguire il calcolo costoso all'interno della lambda di `computeIfAbsent()` è un errore disastroso per la concorrenza! La mappa trattiene il lock esclusivo sul bucket per tutta la durata del calcolo: gli altri thread che richiedono chiavi diverse residenti nello stesso bucket rimarrebbero bloccati, e se la funzione costosa tentasse a sua volta di accedere alla mappa si verificherebbe un deadlock immediato. L'impiego di `FutureTask` come segnaposto separa l'inserimento atomico istantaneo dal calcolo vero e proprio.

Specifiche di Collaudo e Test Case (TDD):

Test Case TDD: MemoizerCacheTest.java

```
AtomicInteger chiamateEffettive = new AtomicInteger(0);
Function<Integer, String> funzioneLenta = k -> {
    chiamateEffettive.incrementAndGet();
    try { Thread.sleep(500); } catch (InterruptedException e) {
    Thread.currentThread().interrupt(); }
    return "RISULTATO-" + k;
};

Memo<Integer, String> cache = new Memo<>(funzioneLenta);
ExecutorService pool = Executors.newFixedThreadPool(16);
CountDownLatch latch = new CountDownLatch(160);

// 16 thread richiedono in parallelo 10 chiavi (160 richieste totali)
for (int i = 0; i < 160; i++) {
    final int chiave = i % 10;
    pool.submit(() -> {
        String res = cache.get(chiave);
        assert res.equals("RISULTATO-" + chiave);
        latch.countDown();
    });
}

latch.await();
pool.shutdown();

// Asserzione fondamentale: la funzione deve essere stata chiamata ESATTAMENTE 10
    volte!
assert chiamateEffettive.get() == 10 :
        String.format("Violazione cache: attese 10 chiamate, eseguite %d!",
    chiamateEffettive.get());
System.out.printf("Memoizer collaudato: 160 richieste concorrenti gestite con
    sole %d computazioni.\n",
        chiamateEffettive.get());
```

Appendice B

Compendio e Formulario Tecnico di Java

Guida alla Consultazione Rapida

Questo compendio costituisce il riferimento enciclopedico e operativo permanente per lo studio, lo svolgimento degli esercizi di laboratorio e la preparazione dell'esame. È strutturato in due parti complementari:

1. **Parte I – Schemi Concettuali e Principi Architetturali**: compendio teorico sui pilastri della programmazione a oggetti (UML, ereditarietà, polimorfismo, eccezioni, generics, complessità computazionale e algoritmi di ricerca/grafi).
2. **Parte II – Formulario Operativo delle API**: catalogo analitico e tabellare di tutte le classi, interfacce e metodi fondamentali di Java standard, con firme, tipi di ritorno, parametri e descrizioni operative.

B.1 PARTE I: SCHEMI CONCETTUALI E PATTERN ARCHITETTURALI

Questa prima parte raccoglie e formalizza i modelli teorici, le convenzioni grafiche di modellazione a oggetti e i criteri architetturali che governano la progettazione del software in ambiente Java. Vengono approfonditi la notazione UML, i principi cardine di ingegneria del software (incapsulamento, coesione, accoppiamento, aliasing e copie difensive), la tassonomia gerarchica delle eccezioni e l'analisi asintotica comparativa delle collezioni dati.

B.1.1 Notazione UML e Relazioni tra Classi

Nel paradigma a oggetti, il diagramma delle classi UML descrive la struttura statica del sistema specificando entità, attributi, operazioni e relazioni reciproche:

Definizione: Specificatori di Visibilità e Modificatori UML

- `+public`: visibilità totale, accessibile da qualsiasi classe e package.
- `#protected`: accessibile dalle classi dello stesso package e da tutte le sottoclassi.
- `-private`: incapsulamento stretto, accessibile esclusivamente all'interno della classe stessa.
- `~package-private`: visibilità di default (senza modificatore esplicito), riservata al medesimo package.
- **Membri Static**: sottolineati nel diagramma UML, legati alla classe e non all'istanza.
- **Costanti FINAL**: indicate in maiuscolo o con vincolo `{readOnly}`, immutabili dopo l'assegnazione.
- **Classi/Metodi Astratti**: indicati in *corsivo*, contengono firme senza implementazione diretta.

Dipendenza (Dependency)	**Relazione "knows-about" (debolissima)**: una classe usa temporaneamente un'altra (es. parametro di un metodo o variabile locale) senza memorizzarne il riferimento nel proprio stato di istanza. **Simbolo Grafico UML**: - - - - - - - - - - ▸ *(linea tratteggiata con freccia aperta a V rivolta al fornitore)*
Associazione (Association)	**Relazione "has-a" strutturale**: legame duraturo tra due oggetti con cicli di vita indipendenti. Si traduce in un campo di istanza che mantiene il riferimento. **Simbolo Grafico UML**: ——→ *(navigabile con freccia)* oppure —— *(bidirezionale)*
Aggregazione (Aggregation)	**Relazione "has-a" Intero-Parte (debole)**: il contenitore aggrega elementi che esistono indipendentemente da esso (es. Squadra e Calciatori). **Simbolo Grafico UML**: ◇—— *(linea continua con diamante vuoto sul lato dell'aggregato/contenitore)*
Composizione (Composition)	**Relazione "contains-a" Intero-Parte (forte/esclusiva)**: le parti sono generate internamente e il loro ciclo di vita coincide rigidamente con quello del contenitore (es. Libro e Pagine). **Simbolo Grafico UML**: ◆—— *(linea continua con diamante pieno sul lato del contenitore)*
Ereditarietà (Generalization)	**Relazione "is-a"**: gerarchia di specializzazione in cui la sottoclasse assorbe stato e comportamento della classe base per estenderli (`extends`). **Simbolo Grafico UML**: ——▷ *(linea continua con triangolo chiuso vuoto rivolto alla classe base/padre)*
Realizzazione (Realization)	**Relazione "can-do"**: firma e adempimento di un contratto di interfaccia comportamentale da parte di una classe concreta (`implements`). **Simbolo Grafico UML**: - - - - - - - - - - ▷ *(linea tratteggiata con triangolo chiuso vuoto rivolto all'interfaccia)*

Figura B.1: Quadro sinottico delle relazioni fondamentali nel diagramma delle classi UML.

B.1.2 Principi Guida di Ingegneria del Software

- **Incapsulamento e Information Hiding**: l'accesso ai dettagli interni e alla struttura dei dati di un oggetto è precluso all'esterno; la manipolazione avviene esclusivamente attraverso l'interfaccia pubblica dei metodi, proteggendo gli invarianti di classe.
- **Coesione Elevata (High Cohesion)**: ogni classe o modulo deve perseguire una singola responsabilità logica ben delimitata (*Single Responsibility Principle*). Più una classe è coesa, più è comprensibile, manutenibile e testabile.
- **Accoppiamento Debole (Loose Coupling)**: le classi devono interagire riducendo al minimo le dipendenze reciproche. Un basso accoppiamento isola le modifiche, evitando reazioni a catena nell'intero sistema.
- **Variabili vs Oggetti**: le variabili allocate nello Stack contengono **riferimenti** (puntatori sicuri) ad aree di memoria allocate nello Heap. Un oggetto nello Heap può essere contemporaneamente referenziato da più variabili (fenomeno dell'*Aliasing*).

- **Uguaglianza: `==` vs `equals()`**: l'operatore `==` valuta unicamente la coincidenza degli indirizzi di memoria (stessa istanza nello heap). Il metodo `equals()` valuta l'uguaglianza logica dello stato interno. Qualsiasi classe che sovrascriva `equals()` **deve obbligatoriamente sovrascrivere anche** `hashCode()` per preservare il corretto funzionamento nelle tabelle hash.
- **Shallow Copy vs Deep Copy (Copia Difensiva)**:
  - *Shallow Copy*: assegna direttamente il riferimento della lista o dell'oggetto (`this.dati = dati;`); modifiche esterne si ripercuotono sull'oggetto interno.
  - *Deep Copy (Copia Difensiva)*: crea una struttura dati nuova e indipendente clonando gli elementi (`this.dati = new ArrayList<>(dati);`), isolando lo stato da interferenze esterne.

B.1.3 Ereditarietà, Polimorfismo e Classi Astratte

Caratteristica	Overloading (Sovraccarico)	Overriding (Sovrascrittura)
Ambito	All'interno della stessa classe (o sottoclasse).	Tra superclasse e sottoclasse.
Firma del metodo	**Stesso nome**, ma lista parametri differente per tipo o numero.	**Firma identica** (stesso nome, stessi parametri, tipo ritorno covariante).
Risoluzione	A tempo di compilazione (**Static Binding** / Early Binding).	A tempo di esecuzione (**Dynamic Binding** / Late Binding).
Annotazione	Nessuna.	Obbligatoria/Consigliata: `@Override`.
Scopo	Fornire varianti ergonomiche della stessa operazione.	Specializzare o ridefinire il comportamento di una superclasse.

Proprietà	Classe Astratta (`abstract class`)	Interfaccia (`interface`)
Semantica ontologica	Definisce un'identità (**Cos'è**: *IS-A*).	Definisce un contratto/protocollo (**Cosa sa fare**: *CAN-DO*).
Metodi con corpo	Ammessi sia concreti che astratti.	Ammessi solo se qualificati come `default` o `static` (da Java 8).
Variabili di istanza	Ammette qualsiasi campo di stato (`private`, `protected`, mutabile).	Ammette esclusivamente costanti pubbliche: implicitamente `public static final`.
Costruttori	Consentiti (invocati tramite `super()` dalle figlie).	Vietati (non possiede costruttori).
Ereditarietà	Singola: una classe può estendere (`extends`) una sola classe base.	Multipla: una classe può implementare (`implements`) un numero arbitrario di interfacce.
Risoluzione Conflitti	Regola "Le classi vincono" su metodi di default.	Se due interfacce forniscono lo stesso metodo di default, l'override è obbligatorio.

B.1.4 Gestione delle Eccezioni

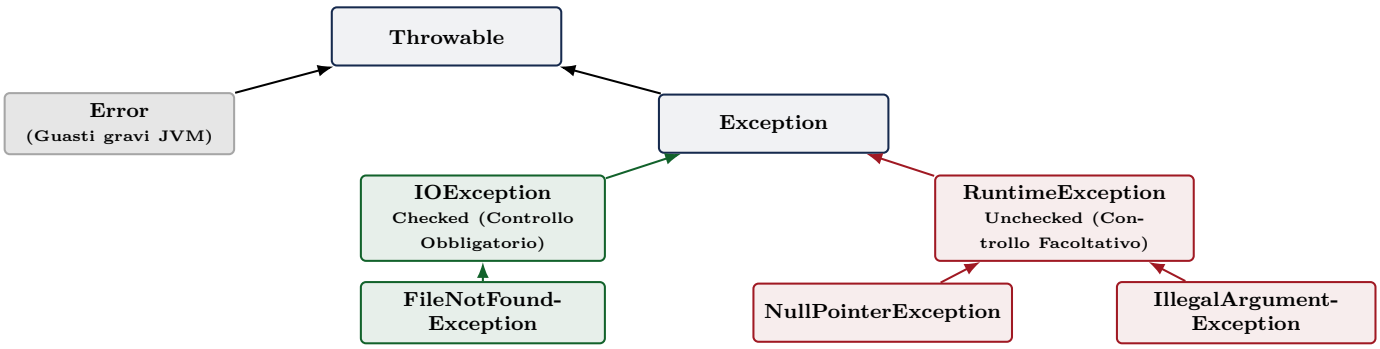


Figura B.2: Gerarchia delle eccezioni in Java: distinzione tra Checked (obbligo di gestione) e Unchecked.

- **Checked Exceptions** (derivano da `Exception` escludendo `RuntimeException`): rappresentano condizioni anomale esterne prevedibili (es. file mancante, socket interrotto). Il compilatore impone la clausola `throws` nella firma o l'incapsulamento in un blocco `try-catch`.
- **Unchecked Exceptions** (derivano da `RuntimeException`): rappresentano bug di programmazione (es. puntatore nullo, indice array errato). Non richiedono dichiarazione formale.
- **Clausola `finally`**: blocco eseguito sempre, sia in caso di successo sia in presenza di eccezione non catturata, indispensabile per il rilascio di risorse.

- **Costrutto `try-with-resources`**: introdotto per dichiarare risorse che implementano l'interfaccia `AutoCloseabl` garantisce l'invocazione automatica di `close()` al termine del blocco.

B.1.5 Complessità Computazionale delle Strutture Dati

Il confronto asintotico che segue è il riferimento operativo per la scelta della struttura dati più adatta a un dato problema: si legge per righe, incrociando il costo delle operazioni fondamentali con il criterio di selezione riportato nella seconda tabella.

Struttura Dati	Accesso Indice	Ricerca Valore	Inserimento	Rimozione
Array primitivo (`T[]`)	$O(1)$	$O(n)$	*N/A (dim. fissa)*	*N/A (dim. fissa)*
`ArrayList`	$O(1)$	$O(n)$	$O(1)^*$ coda / $O(n)$	$O(n)$
`LinkedList`	$O(n)$	$O(n)$	$O(1)$ capi / $O(n)$	$O(1)$ capi / $O(n)$
`ArrayDeque`	$O(1)$ capi	$O(n)$	$O(1)^*$ capi	$O(1)$ capi
`HashSet` / `HashMap`	*N/A (chiave)*	$O(1)$ medio	$O(1)$ medio	$O(1)$ medio
`TreeSet` / `TreeMap`	*N/A (chiave)*	$O(\log n)$	$O(\log n)$	$O(\log n)$

Nota: $O(1)^*$ indica costo costante *ammortizzato* (incrementale su raddoppio buffer o capacità dinamica).

Criteri di Selezione Architetturale

Struttura	Quando preferirla	Attenzione (trade-off critico)
Array (`T[]`)	Dimensione fissa nota a priori; calcolo intensivo su tipi primitivi, senza overhead di boxing/unboxing.	Nessun ridimensionamento: la dimensione è congelata all'istanziazione.
`ArrayList`	Scelta di default per le collezioni sequenziali; ottima per accessi casuali e append in coda.	Riallocazione a +50% con copia; inserimento/rimozione centrale $O(n)$ per lo *shift* degli elementi.
`LinkedList`	Inserimenti/rimozioni ripetuti solo agli estremi, o iterazione bidirezionale pesante con `ListIterator`.	Overhead ~24–32 B/nodo, scarsa cache locality; `get(i)` resta $O(n)$.
`ArrayDeque`	Code FIFO, pile LIFO, buffer ad anello: da preferire sempre a `LinkedList` e alla classe obsoleta `Stack`.	Nessun accesso efficiente a un indice arbitrario interno.
`HashSet` / `HashMap`	Lookup a costo pressoché costante per chiave, quando l'ordine è irrilevante.	`equals()`/`hashCode()` devono essere coerenti; *rehashing* $O(n)$ oltre il load factor 0.75.
`TreeSet` / `TreeMap`	Serve un ordinamento garantito o operazioni di range (`subMap()`, `headSet()`).	Costo $O(\log n)$ su ogni operazione; unicità basata su `compareTo()==0`, non su `equals()`.

B.1.6 Modelli Matematici della Scalabilità Concorrente: Amdahl e Goetz

La progettazione di architetture concorrenti richiede di quantificare a priori il beneficio prestazionale ottenibile dall'aggiunta di core hardware e di dimensionare correttamente il numero di thread nei pool.

Definizione: Legge di Amdahl (1967) e Limite Asintotico dello Speedup

Dato un programma computazionale, sia $P \in [0, 1]$ la frazione di tempo di esecuzione dell'algoritmo che può essere eseguita in perfetto parallelismo su N processori, e sia $(1 - P)$ la frazione intrinsecamente sequenziale (non parallelizzabile, come I/O, sincronizzazioni o aggregazioni finali).
Lo **Speedup** teorico $S(N)$, definito come il rapporto tra il tempo di esecuzione su un singolo processore T_1 e il tempo su N processori T_N, è espresso dalla formula:

$$S(N) = \frac{T_1}{T_N} = \frac{1}{(1 - P) + \frac{P}{N}} \tag{B.1}$$

All'aumentare indefinito del numero di processori fisici ($N \to \infty$), il termine $\frac{P}{N}$ decade a zero: lo speedup non cresce indefinitamente, ma converge asintoticamente al **Limite Superiore di Amdahl**:

$$S_{\max} = \lim_{N \to \infty} S(N) = \frac{1}{1 - P} \tag{B.2}$$

Frazione Parallela (P)	**Frazione Seriale ($1 - P$)**	**Speedup Massimo Teorico ($S_{\max}$)**
50%	50%	**2**× (nessun hardware potrà mai velocizzare più del doppio)
75%	25%	**4**×
90%	10%	**10**×
95%	5%	**20**× (anche con 100.000 core, limite massimo 20×)
99%	1%	**100**×

Definizione: Formula di Brian Goetz per il Dimensionamento dei Thread Pool

Per determinare il numero ottimale di worker thread N_{threads} all'interno di un `ThreadPoolExecutor` per prevenire sia la sotto-utilizzazione della CPU sia la saturazione della memoria con degrado da context switch, si applica la formula di Brian Goetz:

$$N_{\text{threads}} = N_{\text{CPU}} \times U_{\text{CPU}} \times \left(1 + \frac{W}{C}\right) \tag{B.3}$$

dove:

- N_{CPU} = `Runtime.getRuntime().availableProcessors()` è il numero di processori logici disponibili;
- $U_{\text{CPU}} \in [0, 1]$ è il coefficiente di utilizzazione target della CPU (in contesti di calcolo dedicato, $U_{\text{CPU}} = 1$);
- W/C è il rapporto tra il tempo medio trascorso in attesa (*Wait time*: blocco su disco, query SQL a database, chiamate socket HTTP) e il tempo effettivo di elaborazione della CPU (*Compute time*).

Casi d'uso pratici:

- *Carico CPU-Bound Puro* ($W \approx 0$): $N_{\text{threads}} = N_{\text{CPU}}(+1)$ per saturare la pipeline senza contesa;
- *Carico I/O-Bound Intenso* (es. 90% attesa di rete, $W/C = 9$): $N_{\text{threads}} = N_{\text{CPU}} \times (1+9) = 10 \times N_{\text{CPU}}$. Con Java 21, per questi scenari la prassi moderna prescrive l'adozione di un Virtual Thread per compito anziché l'espansione dei platform thread nativi.

B.2 PARTE II: FORMULARIO OPERATIVO DELLE API DI JAVA

Classe Fondamentale `java.lang.Object`

- `boolean equals(Object obj)` → `boolean`
  Verifica l'uguaglianza logica tra l'istanza corrente e l'oggetto passato.
  Implementazione di default: confronta unicamente i riferimenti di memoria (`this == obj`).
  Contratto formale dell'override: la relazione deve essere *riflessiva* (`x.equals(x)`), *simmetrica* (`x.equals(y) == y.equals(x)`), *transitiva*, *coerente* e *non-nulla* (`x.equals(null) == false`).
  Parametri: `obj` l'oggetto di riferimento con cui confrontare l'istanza.
  Esempio canonico:

```
if (this == obj) return true;
if (obj == null || getClass() != obj.getClass()) return false;
Persona other = (Persona) obj;
return eta == other.eta && Objects.equals(nome, other.nome);
```

- `int hashCode()` → `int`
  Restituisce il codice hash intero associato all'oggetto, utilizzato dalle strutture dati hash (`HashMap`, `HashSet`, `Hashtable`).

Contratto cardinale: se `a.equals(b) == true`, allora **tassativamente** `a.hashCode() == b.hashCode()`. Due oggetti diversi possono avere lo stesso hash (collisione), ma una buona funzione deve distribuire i valori uniformemente.
Esempio: `return Objects.hash(nome, eta);`

- `String toString()` → `String`
  Restituisce la rappresentazione testuale dell'oggetto. Di default: `NomeClasse@hexHashCode`. Da sovrascrivere sempre per agevolare il debug e la diagnostica nei log.
- `final Class<?> getClass()` → `Class<?>`
  Restituisce l'oggetto riflettivo che descrive il tipo dinamico effettivo dell'istanza a run-time. Essendo `final`, non può essere sovrascritto.
- `final void wait() throws InterruptedException` → `void`
  Sospende il thread corrente e rilascia temporaneamente il lock intrinseco del monitor dell'oggetto, fino a quando un altro thread non invoca `notify()` o `notifyAll()` sullo stesso monitor.
  Variante con timeout: `wait(long timeoutMillis)` attende al massimo il tempo specificato. Deve essere sempre invocato all'interno di un blocco o metodo `synchronized`.
- `final void notify()` / `final void notifyAll()` → `void`
  Risveglia rispettivamente un singolo thread arbitrario (`notify()`) oppure tutti i thread (`notifyAll()`) in attesa sul monitor dell'oggetto. Deve essere invocato con il lock acquisito.

Manipolazione di Stringhe: `String`, `StringBuilder` e `CharSequence`

Classe Immutabile `java.lang.String`

- `char charAt(int index)` → `char`
  Restituisce il carattere presente all'indice specificato (0-based). Lancia `IndexOutOfBoundsException` se l'indice è negativo o $\geq$ length().
- `int length()` → `int`
  Restituisce il numero totale di caratteri (unità di codice UTF-16) presenti nella stringa.
- `String substring(int beginIndex, int endIndex)` → `String`
  Estrae la porzione di stringa nell'intervallo semichiuso [beginIndex, endIndex).
  Parametri: `beginIndex` indice iniziale (*incluso*); `endIndex` indice finale (*escluso*).
  Esempio: `"Universita".substring(0, 3)` restituisce `"Uni"`.
  Variante a 1 parametro: `substring(int beginIndex)` estrae fino al termine della stringa.
- `boolean equals(Object anObject)` / `boolean equalsIgnoreCase(String another)` → `boolean`
  Confronta il contenuto testuale carattere per carattere. `equalsIgnoreCase` ignora le differenze tra maiuscole e minuscole.
- `int compareTo(String anotherString)` → `int`
  Confronto lessicografico: restituisce un valore < 0 se l'istanza precede, 0 se identiche, > 0 se segue.
  Variante: `compareToIgnoreCase(String str)`.
- `int indexOf(String str)` / `int lastIndexOf(String str)` → `int`
  Restituisce l'indice della prima (o ultima) occorrenza della sottostringa, oppure -1 se assente.
  Varianti: ammettono un secondo parametro `int fromIndex` da cui avviare la ricerca.
- `boolean contains(CharSequence s)` → `boolean`
  Verifica se la sequenza di caratteri è contenuta nella stringa.
- `boolean startsWith(String prefix)` / `boolean endsWith(String suffix)` → `boolean`
  Verifica se la stringa inizia o termina con la sequenza specificata.
- `boolean isEmpty()` / `boolean isBlank()` → `boolean`
  `isEmpty()` ritorna `true` se length() == 0; `isBlank()` (Java 11) ritorna `true` anche se la stringa contiene solo caratteri di spaziatura (*whitespaces*).

- `String trim()` / `String strip()` → `String`
  Rimuovono gli spazi iniziali e finali. `strip()` (Java 11) è conforme a Unicode (rimuove tutti i caratteri di spaziatura internazionali). Disponibili anche `stripLeading()` e `stripTrailing()`.
- `String toLowerCase()` / `String toUpperCase()` → `String`
  Restituisce una nuova stringa con tutti i caratteri convertiti in minuscolo o maiuscolo.
- `String replace(CharSequence target, CharSequence replacement)` → `String`
  Sostituisce tutte le occorrenze letterali della sequenza `target` con `replacement`.
- `String replaceAll(String regex, String replacement)` → `String`
  Sostituisce tutte le corrispondenze dell'**espressione regolare** `regex` con la stringa di rimpiazzo.
- `String[] split(String regex)` / `String[] split(String regex, int limit)` → `String[]`
  Suddivide la stringa in un array di token delimitati dalla regex.
- `static String valueOf(Object obj)` / `static String valueOf(int i)` → `String`
  Converte qualsiasi tipo primitivo o oggetto nella sua rappresentazione stringa (gestisce in sicurezza `null` restituendo `"null"`).
- `static String join(CharSequence delimiter, CharSequence... elements)` → `String`
  Concatena gli elementi interponendo il delimitatore specificato.
  Esempio: `String.join("-", "2026", "10", "04")` produce `"2026-10-04"`.
- `static String format(String format, Object... args)` → `String`
  Genera una stringa formattata secondo gli specificatori standard (`%d` intero, `%s` stringa, `%.2f` decimale).
- `String repeat(int count)` → `String`
  (Java 11) Restituisce una nuova stringa composta dalla ripetizione dell'istanza per `count` volte.
- `Stream<String> lines()` → `Stream<String>`
  (Java 11) Restituisce un flusso lazy di righe estratte dalla stringa, separate da `\n`, `\r` o `\r\n`.

Classe Mutabile `java.lang.StringBuilder`

- `StringBuilder append(T val)` → `StringBuilder`
  Accoda la rappresentazione testuale del valore (primitivo, oggetto o stringa) al buffer corrente. Restituisce `this` per consentire l'*inlining* fluente.
- `StringBuilder insert(int offset, T val)` → `StringBuilder`
  Inserisce la sequenza alla posizione `offset`, traslando a destra i caratteri successivi.
- `StringBuilder delete(int start, int end)` → `StringBuilder`
  Rimuove i caratteri compresi nell'intervallo $[\text{start}, \text{end})$.
  Variante: `deleteCharAt(int index)` elimina il singolo carattere alla posizione indicata.
- `StringBuilder replace(int start, int end, String str)` → `StringBuilder`
  Sostituisce i caratteri dal range $[\text{start}, \text{end})$ con la nuova stringa fornita.
- `StringBuilder reverse()` → `StringBuilder`
  Inverte specularmente l'ordine di tutti i caratteri memorizzati nel buffer.
- `void setCharAt(int index, char ch)` → `void`
  Sovrascrive il carattere alla posizione `index` con `ch` senza allocare una nuova stringa.
- `int length()` / `int capacity()` / `void setLength(int newLength)` → `int/void`
  Rispettivamente il numero di caratteri effettivi, la capienza del buffer interno prima del raddoppio e la modifica forzata della lunghezza (tronca o riempie con null-byte).

Classi Wrapper, Parsing e Ispezione Caratteri (`java.lang`)

Parsing e Utilità Numeriche: `java.lang.Integer` e `java.lang.Double`

- `static int parseInt(String s)` → `int` (throws `NumberFormatException`)

Effettua il parsing della stringa decimale restituendo il primitivo `int`.
Variante con base: `parseInt(String s, int radix)` effettua il parsing in base 2 (binario), 16 (esadecimale), etc.
Esempio: `int val = Integer.parseInt("2026"); int hex = Integer.parseInt("FF", 16); // 255`

- `static Integer valueOf(String s)` / `static Integer valueOf(int i)` → `Integer`
Restituisce l'oggetto wrapper riutilizzando la cache dei valori $[-128, 127]$.
- `static int compare(int x, int y)` → `int`
Confronta due interi primitivi: < 0 se $x < y$, 0 se uguali, > 0 se $x > y$. Indispensabile nei comparatori.
- `static int max(int a, int b)` / `static int min(int a, int b)` → `int`
Restituisce il massimo o il minimo tra due interi (utilizzabili come method reference: `Integer::max`).
- **Costanti di Limite**: `Integer.MAX_VALUE` ($2^{31} - 1 = 2\,147\,483\,647$), `Integer.MIN_VALUE` ($-2^{31}$).
- `static double parseDouble(String s)` → `double` (throws `NumberFormatException`)
Converte la stringa in virgola mobile nel primitivo `double`.
- `static int compare(double d1, double d2)` → `int`
Confronta due double gestendo correttamente valori speciali come `Double.NaN` e zero con segno (-0.0 vs $+0.0$).
- `static boolean isNaN(double v)` / `static boolean isInfinite(double v)` → `boolean`
Verifica se il valore è rispettivamente un "Not-a-Number" o un infinito positivo/negativo.
- `static boolean parseBoolean(String s)` → `boolean`
Restituisce `true` se la stringa non è nulla ed è uguale a `"true"` (case-insensitive), altrimenti `false`.

Ispezione e Trasformazione Caratteri: `java.lang.Character`

- `static boolean isDigit(char ch)` → `boolean`
Verifica se il carattere è una cifra numerica ($'0' \ldots '9'$).
- `static boolean isLetter(char ch)` → `boolean`
Verifica se il carattere è una lettera alfabetica.
- `static boolean isLetterOrDigit(char ch)` → `boolean`
Verifica se il carattere è alfanumerico (lettera o cifra).
- `static boolean isWhitespace(char ch)` → `boolean`
Verifica se il carattere è uno spazio, tabulazione (`'\t'`) o a capo (`'\n'`).
- `static boolean isUpperCase(char ch)` / `static boolean isLowerCase(char ch)` → `boolean`
Verifica se il carattere è maiuscolo o minuscolo.
- `static char toUpperCase(char ch)` / `static char toLowerCase(char ch)` → `char`
Converte il singolo carattere in maiuscolo o minuscolo.

Java I/O Classico: File, Stream e Reader/Writer (`java.io`)

Astrazione File System: `java.io.File`

- `boolean exists()` → `boolean`
Verifica se il file o la directory esiste fisicamente sul supporto di memorizzazione.
- `boolean isFile()` / `boolean isDirectory()` → `boolean`
Verifica se il percorso punta rispettivamente a un file regolare oppure a una directory.
- `long length()` → `long`
Restituisce la dimensione del file espressa in byte (restituisce `0L` se il file non esiste).
- `String getName()` / `String getAbsolutePath()` → `String`
Restituisce rispettivamente il solo nome del file/cartella o il percorso assoluto canonico.

- `boolean createNewFile() throws IOException` → `boolean`
  Crea atomicamente un nuovo file vuoto; restituisce `false` se il file esiste già.
- `boolean delete()` → `boolean`
  Elimina fisicamente il file o la directory (le cartelle devono essere vuote per essere rimosse).
- `boolean mkdir()` / `boolean mkdirs()` → `boolean`
  Crea la cartella singola (`mkdir`) o l'intera gerarchia di directory padre mancanti (`mkdirs`).
- `String[] list()` / `File[] listFiles()` → `String[] / File[]`
  Restituisce i nomi o gli oggetti `File` relativi a tutti i figli contenuti nella cartella.
- `boolean renameTo(File dest)` → `boolean`
  Rinomina o sposta il file verso la destinazione specificata da `dest`.
- `boolean canRead()` / `boolean canWrite()` / `boolean canExecute()` → `boolean`
  Interroga i permessi del sistema operativo per l'accesso in lettura, scrittura o esecuzione.

Parsing Testuale: `java.util.Scanner`

- `String nextLine()` → `String`
  Legge e consuma l'intera riga fino al terminatore di riga (attenzione ai caratteri `\n` residui!).
- `String next()` → `String`
  Legge il prossimo token delimitato da spazi bianchi.
- `int nextInt()` / `double nextDouble()` → `int/double`
  Converte il prossimo token nel rispettivo tipo primitivo (solleva `InputMismatchException` se non compatibile).
- `boolean hasNext()` / `boolean hasNextInt()` → `boolean`
  Verifica preventivamente se è presente un token del tipo specificato prima di leggerlo.
- `Scanner useDelimiter(String pattern)` → `Scanner`
  Imposta l'espressione regolare utilizzata come delimitatore tra i token.

Stream Bufferizzati e Stampa Formattata

- `BufferedReader` `.readLine()` → `String` (throws `IOException`)
  Legge una riga di testo; restituisce `null` quando viene raggiunto l'End-Of-File (EOF).
- `BufferedWriter` `.write(String str)` → `void` (throws `IOException`)
  Scrive la stringa nel buffer di memoria.
- `BufferedWriter` `.newLine()` → `void` (throws `IOException`)
  Scrive il terminatore di riga nativo del sistema operativo ospitante.
- `PrintWriter` `.printf(String format, Object... args)` → `PrintWriter`
  Scrive testo formattato su stream (file o socket) con la medesima sintassi di `System.out.printf`.

Serializzazione di Oggetti (`java.io.Serializable`)

- `ObjectOutputStream` `.writeObject(Object obj)` → `void` (throws `IOException`)
  Serializza lo stato dell'oggetto e del grafo collegato su flusso binario (`Serializable`):
  `try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(file))) { out.writeObject(obj); }`
- `ObjectInputStream` `.readObject()` → `Object` (throws `IOException, ClassNotFoundException`)
  Deserializza e ricostruisce l'oggetto dallo stream binario; richiede il cast esplicito al tipo concreto.
  Modificatore `transient`: i campi contrassegnati con `transient` non vengono serializzati e ripartono dal loro valore di default.

Modern File I/O: `java.nio.file.Path` e `java.nio.file.Files` (Java 8–21)

Interfaccia di Percorso `java.nio.file.Path`

- `static Path of(String first, String... more)` → `Path`
  (Java 11) Factory method moderno raccomandato per istanziare un percorso (sostituisce `Paths.get()`).
  Esempio: `Path p = Path.of("dati", "2026", "report.csv");`
- `Path getFileName()` / `Path getParent()` / `Path toAbsolutePath()` → `Path`
  Estrae rispettivamente l'ultimo elemento del percorso, la directory genitrice o la risoluzione assoluta.
- `Path resolve(String other)` / `Path resolve(Path other)` → `Path`
  Risolve e concatena un percorso relativo rispetto al path corrente.

Operazioni Massive su File: Classe di Utilità `java.nio.file.Files`

- `lines(path)` → `Stream<String>` (throws `IOException`)
  Firma: `static Stream<String> lines(Path path)`
  Flusso Lazy di Righe: apre il file e ne legge le righe su richiesta in uno `Stream<String>`. Consumo di memoria $O(1)$, ideale per file giganteschi.
  Regola tassativa: lo stream implementa `AutoCloseable`; deve essere racchiuso in un blocco `try-with-resources` per garantire il rilascio immediato del file descriptor nel sistema operativo.
  Esempio:

```
try (Stream<String> stream = Files.lines(Path.of("server.log"))) {
    long errori = stream.filter(line -> line.contains("ERROR")).count();
}
```

- `readAllLines(path)` → `List<String>` (throws `IOException`)
  Firma: `static List<String> readAllLines(Path path)`
  Legge interamente il file in memoria caricando tutte le righe in una lista di stringhe ($O(n)$ in memoria RAM).
- `readString(path)` → `String` (throws `IOException`)
  Firma: `static String readString(Path path)`
  (Java 11) Legge l'intero contenuto del file direttamente in una singola stringa decodificata in UTF-8.
- `writeString(path, csq, options...)` → `Path` (throws `IOException`)
  Firma: `static Path writeString(Path path, CharSequence csq, OpenOption... options)`
  (Java 11) Scrive atomicamente la stringa specificata nel file. Con `StandardOpenOption.CREATE`, `StandardOpenOption.APPEND` esegue l'append in coda.
- `write(path, lines, options...)` → `Path` (throws `IOException`)
  Firma: `static Path write(Path path, Iterable<? extends CharSequence> lines, OpenOption... opts)`
  Scrive una sequenza iterabile di righe all'interno del file.
- `exists(path, options...)` → `boolean`
  Firma: `static boolean exists(Path path, LinkOption... options)`
  Verifica l'esistenza reale del file o cartella sul file system.
- `copy(source, target, options...)` → `Path` (throws `IOException`)
  Firma: `static Path copy(Path source, Path target, CopyOption... options)`
  Copia il file da sorgente a destinazione. Opzione chiave: `StandardCopyOption.REPLACE_EXISTING`.
- `move(source, target, options...)` → `Path` (throws `IOException`)
  Firma: `static Path move(Path source, Path target, CopyOption... options)`
  Sposta o rinomina atomicamente il file.
- `delete(path)` / `deleteIfExists(path)` → `void / boolean` (throws `IOException`)
  Firme: `static void delete(Path path)`, `static boolean deleteIfExists(Path path)`

Elimina il file indicato; la variante `deleteIfExists` non lancia eccezione se il file è assente.

- `createDirectories(dir, attrs...)` → `Path` (throws `IOException`)
  Firma: `static Path createDirectories(Path dir, FileAttribute<?>... attrs)`
  Crea l'intera gerarchia di directory mancanti senza sollevare errori se alcune cartelle esistono già.
- `walk(start, maxDepth, options...)` → `Stream<Path>` (throws `IOException`)
  Firma: `static Stream<Path> walk(Path start, int maxDepth, FileVisitOption... options)`
  Attraversa ricorsivamente l'albero delle directory a partire da `start` fino a `maxDepth` livelli, restituendo un flusso computazionale di percorsi.

Interfaccia Radice `Collection<E>` e Iteratori (`java.util`)

- `boolean add(E e)` → `boolean`
  Inserisce l'elemento; ritorna `true` se la collezione è stata modificata, `false` se l'elemento era già presente (negli insiemi `Set`).
  Parametri: `e` l'elemento conforme al vincolo generico da inserire.
- `boolean addAll(Collection<? extends E> c)` → `boolean`
  Inserisce tutti gli elementi contenuti nella collezione specificata (*unione*).
- `boolean remove(Object o)` → `boolean`
  Rimuove una singola occorrenza dell'oggetto basandosi sul confronto logico `equals()`.
- `boolean removeAll(Collection<?> c)` → `boolean`
  Rimuove tutti gli elementi presenti anche nella collezione passata (*differenza insiemistica*).
- `boolean retainAll(Collection<?> c)` → `boolean`
  Mantiene unicamente gli elementi presenti anche nella collezione passata (*intersezione*).
- `boolean removeIf(Predicate<? super E> filter)` → `boolean`
  (Java 8) Rimuove in un unico passaggio tutti gli elementi che soddisfano il predicato booleano.
  Parametri: `filter` predicato funzionale che restituisce `true` per gli elementi da eliminare.
  Esempio: `studenti.removeIf(s -> s.getMedia() < 18.0);`
- `boolean contains(Object o)` / `boolean containsAll(Collection<?> c)` → `boolean`
  Verifica la presenza del singolo elemento o di tutti gli elementi della collezione fornita.
- `void clear()` → `void`
  Svuota integralmente la collezione rimuovendo ogni riferimento.
- `boolean isEmpty()` / `int size()` → `boolean/int`
  Rispettivamente verifica se la dimensione è zero e restituisce il numero di elementi contenuti.
- `Iterator<E> iterator()` → `Iterator<E>`
  Restituisce l'iteratore standard per attraversare la sequenza di elementi. Metodi chiave di `Iterator<E>`:
  - `boolean hasNext()`: verifica se vi sono ulteriori elementi nella collezione.
  - `E next()`: restituisce il prossimo elemento e avanza il cursore ($O(1)$).
  - `void remove()`: rimuove l'ultimo elemento restituito da `next()`. È **l'unico metodo sicuro** per eliminare elementi durante un ciclo evitando `ConcurrentModificationException`.
- `Object[] toArray()` → `Object[]`
  Converte la collezione in un array nativo di tipo `Object[]`.
- `<T> T[] toArray(T[] a)` → `T[]`
  Converte la collezione in un array del tipo specificato; se `a` ha capienza sufficiente viene riutilizzato, altrimenti ne viene allocato uno nuovo della stessa dimensione.
  Esempio: `String[] arr = lista.toArray(new String[0]);`

- `default <T> T[] toArray(IntFunction<T[]> generator)` → `T[]`
  (Java 11) Converte la collezione in array tipizzato usando un costruttore di array come generator.
  Esempio: `String[] arr = lista.toArray(String[]::new);`
- `default Stream<E> stream()` / `default Stream<E> parallelStream()` → `Stream<E>`
  Crea rispettivamente uno stream sequenziale o parallelo supportato dal framework Fork/Join.

Interfaccia Posizionale `List<E>` e `ListIterator<E>`

- `E get(int index)` → `E`
  Restituisce l'elemento posizionato all'indice specificato ($O(1)$ su `ArrayList`, $O(n)$ su `LinkedList`).
  Parametri: `index` indice posizionale ($0 \leq \text{index} < \text{size}()$).
- `E set(int index, E element)` → `E`
  Sostituisce l'elemento all'indice e restituisce il valore precedentemente memorizzato.
- `void add(int index, E element)` → `void`
  Inserisce l'elemento all'indice indicato, traslando a destra di una posizione tutti gli elementi successivi.
- `E remove(int index)` → `E`
  Elimina l'elemento all'indice specificato, trasla a sinistra i successivi e restituisce il valore eliminato.
- `int indexOf(Object o)` / `int lastIndexOf(Object o)` → `int`
  Restituisce la posizione della prima (o ultima) occorrenza dell'oggetto, oppure -1 se assente.
- `List<E> subList(int fromIndex, int toIndex)` → `List<E>`
  Restituisce una **vista** (*view*) della lista compresa nell'intervallo [fromIndex, toIndex).
  Attenzione: le modifiche apportate alla vista si riflettono direttamente sulla lista originale e viceversa.
- `default void sort(Comparator<? super E> c)` → `void`
  (Java 8) Ordina la lista sul posto secondo il comparatore passato (passare `null` per ordine naturale).
  Esempio: `lista.sort(Comparator.naturalOrder());`
- `default void replaceAll(UnaryOperator<E> operator)` → `void`
  (Java 8) Sostituisce ciascun elemento della lista con il risultato dell'applicazione dell'operatore.
  Esempio: `parole.replaceAll(String::toUpperCase);`
- `static <E> List<E> of(E... elements)` → `List<E>`
  (Java 9) Factory method per creare una lista **immutabile e non modificabile**. Non accetta elementi `null` (solleva `NullPointerException`).
  Esempio: `List<String> immutabile = List.of("A", "B", "C");`
- `static <E> List<E> copyOf(Collection<? extends E> coll)` → `List<E>`
  (Java 10) Restituisce una copia immutabile indipendente della collezione fornita.

Iteratore Bidirezionale `ListIterator<E>`

- `boolean hasPrevious()` / `E previous()` → `boolean / E`
  Verifica l'esistenza dell'elemento precedente e lo estrae muovendo il cursore a ritroso.
- `int nextIndex()` / `int previousIndex()` → `int`
  Restituisce l'indice posizionale che verrebbe letto con la successiva chiamata a `next()` o `previous()`.
- `void set(E e)` / `void add(E e)` → `void`
  `set` sovrascrive l'ultimo elemento restituito; `add` inserisce un elemento nella posizione corrente.

Code e Pile: `Queue<E>`, `Deque<E>` e `ArrayDeque<E>`

Operazione	Solleva Eccezione	Valore Speciale	Descrizione Comportamentale
Inserimento	`boolean add(E e)`	`boolean offer(E e)`	Inserisce in coda; restituisce `false` se la coda è piena.
Rimozione	`E remove()`	`E poll()`	Estrae dalla testa; restituisce `null` se vuota.
Ispezione	`E element()`	`E peek()`	Legge la testa senza estrarla; restituisce `null` se vuota.
Coda a Due Estremi (`Deque<E>`) e Comandi Pila LIFO			
Inserimento Estremi	`addFirst(e)` / `addLast(e)`	`offerFirst(e)` / `offerLast(e)`	Inserisce rispettivamente in testa o in coda.
Rimozione Estremi	`removeFirst()` / `removeLast()`	`pollFirst()` / `pollLast()`	Estrae rispettivamente dalla testa o dalla coda.
Ispezione Estremi	`getFirst()` / `getLast()`	`peekFirst()` / `peekLast()`	Legge la testa o la coda senza estrazione.
Operazioni Pila	`push(E e)` (= addFirst)	`pop()` (= removeFirst)	Gestione a pila LIFO (*Last-In First-Out*).

Dizionari e Mappe: `Map<K, V>` e Metodi Funzionali Java 8+

- `V get(Object key)` → `V`
  Restituisce il valore associato alla chiave specificata, oppure `null` se la chiave è assente.
- `V put(K key, V value)` → `V`
  Associa il valore alla chiave. Se la chiave era già presente, sovrascrive il valore e restituisce il precedente valore memorizzato (oppure `null` se nuova).
- `V getOrDefault(Object key, V defaultValue)` → `V`
  Restituisce il valore mappato se presente; se la chiave non esiste restituisce il valore `defaultValue`.
  Esempio: `int conteggio = mappa.getOrDefault("parola", 0);`
- `V putIfAbsent(K key, V value)` → `V`
  Inserisce la coppia chiave-valore solo se la chiave non è presente o è attualmente associata a `null`.
- `V remove(Object key)` / `boolean remove(Object key, Object value)` → `V / boolean`
  La prima variante rimuove la chiave e rende il valore; la seconda variante rimuove l'entry solo se la chiave è attualmente associata esattamente a quell'oggetto `value`.
- `boolean containsKey(Object key)` / `boolean containsValue(Object value)` → `boolean`
  Verifica rapida della chiave ($O(1)$ su `HashMap`) o scansione lineare dei valori ($O(n)$).
- `Set<K> keySet()` / `Collection<V> values()` / `Set<Map.Entry<K, V» entrySet()`
  Restituiscono rispettivamente l'insieme delle chiavi, la raccolta dei valori e l'insieme delle coppie chiave-valore.
  Metodi dell'entry `Map.Entry<K, V>`: `K getKey()`, `V getValue()`, `V setValue(V value)`.
  Pattern di iterazione canonico:

```
for (Map.Entry<String, Integer> entry : mappa.entrySet()) {
    String k = entry.getKey();
    Integer v = entry.getValue();
}
```

Metodi di Calcolo Funzionale Moderni (Java 8+)

- `V computeIfAbsent(K key, Function<? super K, ? extends V> mappingFunction)` → `V`
  Se la chiave non è presente (o è associata a `null`), calcola il valore tramite la funzione passata e lo inserisce nella mappa. Restituisce il valore finale.
  Parametri: `key` la chiave da cercare; `mappingFunction` la funzione che genera il valore a partire dalla

chiave.
Esempio pratico (Creazione automatica di liste innestate):

```
Map<String, List<Studente>> perCorso = new HashMap<>();
perCorso.computeIfAbsent("Informatica", k -> new ArrayList<>()).add(studente);
```

- `V computeIfPresent(K key, BiFunction<? super K, ? super V, ? extends V> remappingFunction)` → `V`
  Se la chiave è presente e non-nulla, calcola un nuovo valore usando chiave e vecchio valore. Se la funzione restituisce `null`, **rimuove la chiave dalla mappa**.
- `V merge(K key, V value, BiFunction<? super V, ? super V, ? extends V> remappingFunction)` → `V`
  Algoritmo di Fusione Chiave-Valore: se la chiave non esiste o è associata a `null`, associa direttamente `value`. Altrimenti calcola il nuovo valore fondendo il vecchio e il nuovo tramite la funzione binaria (`remappingFunction`). Se la funzione restituisce `null`, elimina la chiave.
  Parametri: `key` chiave; `value` valore da associare se assente; `remappingFunction` funzione $(v_{\text{old}}, v_{\text{new}}) \to v_{\text{merged}}$.
  Esempio canonico (Frequenziario perfetto):

```
Map<String, Integer> frequenze = new HashMap<>();
for (String parola : testo) {
    frequenze.merge(parola, 1, Integer::sum);
}
```

- `void forEach(BiConsumer<? super K, ? super V> action)` → `void`
  Itera su ogni coppia della mappa invocando la lambda specificata.
  Esempio: `mappa.forEach((k, v) -> System.out.println(k + " = " + v));`
- `static <K, V> Map<K, V> of(K k1, V v1, ...)` / `static <K, V> Map<K, V> copyOf(Map<K, V> m)` → `Map<K, V>`
  (Java 9/10) Creano mappe non modificabili e immutabili. Sollevano eccezione in presenza di chiavi duplicate o valori `null`.

Confronto e Ordinamento: `Comparable<T>`, `Comparator<T>` e Costruzione Fluida

Ordinamento Naturale Intrinseco: `Comparable<T>`

- `int compareTo(T other)` → `int`
  Definisce l'ordine canonico dell'oggetto confrontato con l'istanza `other`.
  Convenzione numerica: restituisce un valore intero strettamente < 0 se `this < other`; restituisce 0 se lo stato è equivalente; restituisce > 0 se `this > other`.

Ordinamento Esterno e Combinatori Fluenti: `Comparator<T>`

- `int compare(T o1, T o2)` → `int`
  Metodo SAM: restituisce < 0 se $o_1 < o_2$, 0 se uguali, > 0 se $o_1 > o_2$.
- `static <T, U extends Comparable<? super U>> Comparator<T> comparing(Function<? super T, ? e` → `Comparator<T>`
  Costruisce un comparatore estraendo una chiave ordinabile secondo il suo ordine naturale.
  Esempio: `Comparator<Persona> comp = Comparator.comparing(Persona::getCognome);`
- `static <T> Comparator<T> comparingInt(ToIntFunction<? super T> keyExtractor)` → `Comparator<T>`
  Variante specializzata per chiavi intere primitive che azzera l'overhead di boxing/unboxing. Disponibili anche `comparingDouble` e `comparingLong`.
  Esempio: `Comparator.comparingInt(Persona::getEta);`

- `default Comparator<T> thenComparing(Comparator<? super T> other)` → `Comparator<T>`
  Criterio di spareggio (tie-breaker): se il comparatore corrente restituisce 0 (parità), valuta il secondo comparatore fornito.
  Esempio combinato:

```
Comparator<Persona> comp = Comparator.comparing(Persona::getCognome)
                                     .thenComparing(Persona::getNome)
                                     .thenComparingInt(Persona::getEta);
```

- `default Comparator<T> reversed()` → `Comparator<T>`
  Inverte la direzione dell'ordinamento applicato dal comparatore corrente.
- `static <T extends Comparable<? super T>» Comparator<T> naturalOrder()` / `reverseOrder()` → `Comparator<T>`
  Restituiscono comparatori basati sull'ordine naturale intrinseco o sul suo inverso.
- `static <T> Comparator<T> nullsFirst(Comparator<? super T> comparator)` / `nullsLast(...)` → `Comparator<T>`
  Adattatori sicuri per gestire valori `null` posizionandoli rispettivamente in testa o in coda.

Libreria Matematica: `java.lang.Math`

- `static double abs(double a)` / `static int abs(int a)` → `double / int`
  Restituisce il valore assoluto non negativo del numero specificato (disponibile anche per `long` e `float`).
- `static double max(double a, double b)` / `static int max(int a, int b)` → `double / int`
  Restituisce il valore maggiore tra i due argomenti passati (disponibili analoghi metodi `min`).
- `static double pow(double base, double exp)` → `double`
  Calcola l'elevamento a potenza base^{exp}.
- `static double sqrt(double a)` → `double`
  Calcola la radice quadrata positiva ($\sqrt{a}$).
- `static double ceil(double a)` / `static double floor(double a)` → `double`
  Arrotonda per eccesso (il più piccolo intero $\geq a$) o per difetto (il più grande intero $\leq a$).
- `static long round(double a)` / `static int round(float a)` → `long / int`
  Arrotonda il numero all'intero più vicino (sommando 0.5 e applicando il troncamento).
- `static double random()` → `double`
  Genera un valore pseudocasuale distribuito uniformemente nell'intervallo reale $[0.0, 1.0)$.
- `static int clamp(long value, int min, int max)` → `int`
  (Java 21) Delimita e costringe il valore all'interno dell'intervallo chiuso $[\text{min}, \text{max}]$.

Manipolazione di Array: `java.util.Arrays` e Utilità `java.util.Collections`

Classe di Utilità `java.util.Arrays`

- `sort(a)` / `sort(a, comparator)` → `void`
  Firme: `static void sort(T[] a)`, `static <T> void sort(T[] a, Comparator<? super T> c)`
  Ordina l'array sul posto secondo l'ordine naturale o il comparatore specificato.
- `parallelSort(a)` → `void`
  Firma: `static void parallelSort(T[] a)`
  Ordina l'array sfruttando il multithreading Fork/Join (consigliato per array con $> 10^5$ elementi).
- `binarySearch(a, key)` / `binarySearch(a, c, key)` → `int`
  Firme: `static <T> int binarySearch(T[] a, T key)`, `static <T> int binarySearch(T[] a, Comparator<? super T> c, T key)`

Ricerca dicotomica $O(\log n)$: l'array **deve essere preventivamente ordinato**. Restituisce l'indice se trovato; se assente restituisce un indice negativo $-(\text{insertion point}) - 1$.

- `equals(a1, a2)` / `deepEquals(a1, a2)` → `boolean`
  Firme: `static boolean equals(T[] a, T[] a2)`, `static boolean deepEquals(Object[] a1, Object[] a2)`
  Confronta due array monodimensionali (`equals`) o multidimensionali annidati (`deepEquals`).
- `copyOf(original, newLength)` → `T[]`
  Firma: `static <T> T[] copyOf(T[] original, int newLength)`
  Clona l'array estendendolo o troncandolo alla nuova dimensione indicata.
- `fill(a, val)` → `void`
  Firma: `static void fill(T[] a, T val)`
  Assegna il valore `val` a tutti gli elementi dell'array.
- `asList(a...)` → `List<T>`
  Firma: `static <T> List<T> asList(T... a)`
  Crea una lista a dimensione fissa basata sull'array sottostante (modifiche ai valori si riflettono sull'array).
- `stream(array)` → `Stream / IntStream`
  Firme: `static <T> Stream<T> stream(T[] a)`, `static IntStream stream(int[] a)`
  Genera uno stream sequenziale a partire da un array di oggetti o tipi primitivi.

Classe di Utilità `java.util.Collections`

- `static void shuffle(List<?> list)` / `static void reverse(List<?> list)` → `void`
  Rimescola casualmente gli elementi in ordine uniforme o inverte specularmente l'ordine della lista.
- `static <T extends Comparable<? super T>> T min(Collection<? extends T> coll)` → `T`
  Restituisce il valore minimo della collezione (anche con `Comparator<? super T>`).
- `static int frequency(Collection<?> c, Object o)` → `int`
  Conta quante volte l'oggetto `o` appare all'interno della collezione passata.
- `static boolean disjoint(Collection<?> c1, Collection<?> c2)` → `boolean`
  Restituisce `true` se le due collezioni non hanno elementi in comune (*disgiunte*).
- `static <T> List<T> unmodifiableList(List<? extends T> list)` → `List<T>`
  Restituisce una vista non modificabile della lista (disponibile anche per `unmodifiableSet`, `unmodifiableMap`).

Programmazione Funzionale: Catalogo `java.util.function`

Interfaccia SAM	Metodo Astratto	Firma Tipi	Descrizione e Ruolo Operativo
`Supplier<T>`	`T get()`	$() \to T$	Generatore/Fornitore di valori senza input.
`Consumer<T>`	`void accept(T t)`	$T \to$ void	Consumatore con effetto collaterale (es. print).
`BiConsumer<T, U>`	`void accept(T, U)`	$(T, U) \to$ void	Consumatore a due argomenti (es. entry mappa).
`Function<T, R>`	`R apply(T t)`	$T \to R$	Trasformatore da tipo T a tipo R.
`BiFunction<T, U, R>`	`R apply(T, U)`	$(T, U) \to R$	Funzione binaria tra tipi eterogenei.
`Predicate<T>`	`boolean test(T t)`	$T \to$ bool	Criterio di filtro/asserzione booleana.
`BiPredicate<T, U>`	`boolean test(T, U)`	$(T, U) \to$ bool	Relazione di test tra due elementi.
`UnaryOperator<T>`	`T apply(T t)`	$T \to T$	Operatore unario chiuso sullo stesso tipo.
`BinaryOperator<T>`	`T apply(T, T)`	$(T, T) \to T$	Operatore binario omogeneo (somme, max).

Combinatori e Metodi di Default Fondamentali

- **Predicati**: `p1.and(p2)`, `p1.or(p2)`, `p1.negate()`, `Predicate.not(metodoRef)` (Java 11: es. `Predicate.not(String::isBlank)`).
- **Funzioni**: `f1.andThen(f2)` ($f_2(f_1(x))$), `f1.compose(f2)` ($f_1(f_2(x))$), `Function.identity()` (restituisce il valore invariato).
- **Operatori Binari**: `BinaryOperator.maxBy(comparator)`, `BinaryOperator.minBy(comparator)`.

Specializzazioni Primitive Chiave (Zero Boxing)

- `ToIntFunction<T>` (`int applyAsInt(T value)`), `ToDoubleFunction<T>` (`double applyAsDouble(T value)`).
- `IntFunction<R>` (`R apply(int value)`), `IntPredicate` (`boolean test(int v)`), `IntConsumer` (`void accept(int v)`).
- `IntUnaryOperator` (`int applyAsInt(int v)`), `IntBinaryOperator` (`int applyAsInt(int a, int b)`).

Sintassi e Tassonomia Completa dei Method Reference

Le Quattro Categorie Canoniche (`qualificatore::nome`)

- **1. Metodo d'Istanza su Oggetto Specifico**: `istanzaEsistente::metodo`
  L'oggetto bersaglio è già vincolato e noto a priori; gli argomenti della lambda diventano argomenti del metodo.
  Sintassi: `System.out::println` ≡ `x -> System.out.println(x)`
- **2. Metodo Statico di Classe**: `NomeClasse::metodoStatico`
  Non vi è alcun oggetto ricevente; i parametri della lambda vengono passati direttamente al metodo statico.
  Sintassi: `Math::max` ≡ `(a, b) -> Math.max(a, b)`; `Integer::sum` ≡ `(a, b) -> Integer.sum(a, b)`
- **3. Metodo d'Istanza su Ricevente Arbitrario**: `NomeClasse::metodoDiIstanza`
  Regola chiave: il **primo parametro** della lambda funge da oggetto bersaglio (*receiver*) su cui viene invocato il metodo; gli eventuali parametri successivi diventano gli argomenti.
  Sintassi 1 param: `String::trim` ≡ `s -> s.trim()`
  Sintassi 2 param: `String::compareTo` ≡ `(s, t) -> s.compareTo(t)`
- **4. Costruttore**: `NomeClasse::new`
  Riferimento al costruttore per istanziazione differita del tipo specificato.
  Sintassi: `ArrayList::new` ≡ `() -> new ArrayList<>()`; `String[]::new` ≡ `size -> new String[size]`

Stream API: Pipeline, Flussi Primitivi e `Optional<T>` (`java.util.stream`)

Creazione di Stream

- `collezione.stream()` / `collezione.parallelStream()` → `Stream<E>`
  Crea uno stream sequenziale o parallelo (gestito dal framework concorrente Fork/Join).
- `Arrays.stream(T[] array)` → `Stream<T>`
  Crea uno stream da array (disponibile anche per primitivi: `IntStream`, `LongStream`, `DoubleStream`).
- `Stream.of(T... values)` / `Stream.empty()` → `Stream<T>`
- `Stream.iterate(T seed, UnaryOperator<T> f)` → `Stream<T>`
  Genera uno stream infinito ordinato: $s_0, f(s_0), f(f(s_0)), \ldots$.
- `Stream.iterate(T seed, Predicate<? super T> hasNext, UnaryOperator<T> next)` → `Stream<T>`

(Java 9) Versione con predicato di arresto simile al ciclo `for(seed; hasNext; next)`.

- `Stream.generate(Supplier<? extends T> s)` → `Stream<T>`
  Genera uno stream infinito non ordinato alimentato da un fornitore di valori.

Operazioni Intermedie (Lazy – Valutazione Pigra)

- `Stream<T> filter(Predicate<? super T> predicate)` → `Stream<T>`
  Mantiene unicamente gli elementi che soddisfano la condizione booleana del predicato.
- `<R> Stream<R> map(Function<? super T, ? extends R> mapper)` → `Stream<R>`
  Trasforma ciascun elemento da tipo T a tipo R tramite la funzione fornita (trasformazione 1-a-1).
- `IntStream mapToInt(ToIntFunction<? super T> mapper)` → `IntStream`
  Trasforma lo stream in un flusso numerico primitivo a zero boxing. Analoghi: `mapToLong`, `mapToDouble`.
- `<R> Stream<R> flatMap(Function<? super T, ? extends Stream<? extends R>> mapper)` → `Stream<R>`
  Appiattisce stream annidati: mappa ciascun elemento in uno stream e fonde tutti i flussi in un unico stream piano (trasformazione 1-a-molti).
- `Stream<T> distinct()` → `Stream<T>`
  Elimina i duplicati preservando l'ordine (*stateful*, si basa su `equals()` e `hashCode()`).
- `Stream<T> sorted()` / `Stream<T> sorted(Comparator<? super T> comparator)` → `Stream<T>`
  Ordina gli elementi dello stream (*stateful*, non applicabile su flussi infiniti senza limit).
- `Stream<T> peek(Consumer<? super T> action)` → `Stream<T>`
  Esegue un'azione non invasiva (tipicamente log o debug) su ciascun elemento senza alterare il flusso.
- `Stream<T> limit(long maxSize)` / `Stream<T> skip(long n)` → `Stream<T>`
  Tronca lo stream ai primi `maxSize` elementi o scarta i primi `n` elementi.
- `Stream<T> takeWhile(Predicate<? super T> predicate)` → `Stream<T>`
  (Java 9) Estrae gli elementi dall'inizio finché il predicato è `true`; al primo fallimento tronca lo stream.
- `Stream<T> dropWhile(Predicate<? super T> predicate)` → `Stream<T>`
  (Java 9) Scarta gli elementi finché il predicato è `true`, emettendo tutti i successivi non appena fallisce.

Operazioni Terminali (Eager – Consumano e Chiudono lo Stream)

- `void forEach(Consumer<? super T> action)` / `void forEachOrdered(...)` → `void`
  Esegue l'azione su ciascun elemento. `forEachOrdered` rispetta l'ordine anche in parallelo.
- `Object[] toArray()` / `<A> A[] toArray(IntFunction<A[]> generator)` → `A[]`
  Raccoglie gli elementi in un array tipizzato.
  Esempio: `String[] arr = stream.toArray(String[]::new);`

Focus Riduzioni: I Tre Overload Fondamentali di `reduce`

- `reduce(accumulator)` → `Optional<T>`
  Firma formale: `Optional<T> reduce(BinaryOperator<T> accumulator)`
  Esegue una riduzione associativa senza valore iniziale. Restituisce un `Optional` vuoto se lo stream è privo di elementi.
  Parametri: `accumulator` operatore binario associativo $(T, T) \to T$.
  Esempio: `Optional<Integer> max = numeri.stream().reduce(Integer::max);`
- `reduce(identity, accumulator)` → `T`
  Firma formale: `T reduce(T identity, BinaryOperator<T> accumulator)`
  Esegue la riduzione partendo da un valore di identità. Ritorna `identity` se lo stream è vuoto.
  Parametri: `identity` elemento neutro dell'operatore ($op(identity, x) = x$); `accumulator` funzione $(T, T) \to T$.
  Esempio: `int sum = numeri.stream().reduce(0, (a, b) -> a + b);`

- `reduce(identity, accumulator, combiner)` → `U`
  Firma formale: `<U> U reduce(U identity, BiFunction<U, ? super T, U> acc, BinaryOperator<U> comb)`
  Riduzione Generalizzata con Cambio di Tipo: consente di accumulare gli elementi dello stream (di tipo T) in un risultato finale di tipo completamente differente (U).
  Scomposizione Analitica dei Tre Componenti:
  - `identity`: valore iniziale di tipo U (restituito se lo stream è vuoto); deve fungere da elemento neutro per il combinatore: $combiner(u, identity) = u$.
  - `accumulator`: funzione $(U, T) \to U$ che incorpora un elemento dello stream (T) nel valore parziale accumulato (U).
  - `combiner`: funzione binaria associativa $(U, U) \to U$ invocata negli **stream paralleli** per fondere i risultati parziali prodotti concorrentemente dai vari worker thread. In esecuzione strettamente sequenziale non viene eseguita, ma è richiesta dal contratto formale della firma.

  Esempio Operativo (Calcolo della lunghezza complessiva di tutte le stringhe in parallelo):

```
int totaleCaratteri = listaStringhe.parallelStream()
    .reduce(0,
            (parziale, str) -> parziale + str.length(), // Accumulatore (U, T) -> U
            Integer::sum);                              // Combinatore (U, U) -> U
```

Focus Raccoglitori: I Due Overload di `collect`

- `collect(collector)` → `R`
  Firma formale: `<R, A> R collect(Collector<? super T, A, R> collector)`
  Riduzione mutabile terminale standard delegata a un'istanza di `Collector` (vedi scheda successiva).
- `collect(supplier, accumulator, combiner)` → `R`
  Firma formale: `<R> R collect(Supplier<R> sup, BiConsumer<R, ? super T> acc, BiConsumer<R, R> comb)`
  Riduzione mutabile diretta senza istanziare formalmente un `Collector`.
  Ruolo dei Tre Parametri:
  - `supplier`: factory che istanzia il contenitore di accumulo mutabile (es. `ArrayList::new`).
  - `accumulator`: azione che aggiunge un elemento T al contenitore mutabile R (es. `List::add`).
  - `combiner`: azione che unisce due contenitori mutabili R in caso di stream parallelo (es. `List::addAll`).

  Esempio: `List<String> out = stream.collect(ArrayList::new, List::add, List::addAll);`

Altre Operazioni Terminali

- `long count()` → `long`
  Restituisce il numero totale di elementi transitati nello stream.
- `Optional<T> min(Comparator<? super T> comp)` / `Optional<T> max(...)` → `Optional<T>`
  Estrae il valore minimo o massimo in base al comparatore fornito.
- `boolean anyMatch(Predicate)` / `allMatch(Predicate)` / `noneMatch(Predicate)` → `boolean`
  Valutazioni a cortocircuito (*short-circuit*): interrompono l'elaborazione non appena l'esito è certo.
- `Optional<T> findFirst()` / `Optional<T> findAny()` → `Optional<T>`
  Estrae il primo elemento (rispetta l'ordine di incontro) o un elemento qualsiasi (ottimizzato per parallelo).

Flussi Primitivi a Zero-Boxing (`IntStream`, `LongStream`, `DoubleStream`)

- `IntStream.range(int start, int end)` / `rangeClosed(start, end)` → `IntStream`
  Genera intervallo numerico consecutivo escluso o incluso l'estremo superiore `end`.
- `int sum()` / `OptionalDouble average()` / `OptionalInt min()` / `OptionalInt max()`
  Operazioni aritmetiche aggregate immediate senza creazione di oggetti wrapper.

- `IntSummaryStatistics summaryStatistics()` → `IntSummaryStatistics`
  Calcola contemporaneamente conteggio, somma, minimo, massimo e media in un solo passaggio (`getCount()`, `getSum()`, `getAverage()`, etc.).
- `Stream<Integer> boxed()` → `Stream<Integer>`
  Effettua il boxing degli elementi primitivi riconvertendoli nel rispettivo stream di wrapper.
- `<U> Stream<U> mapToObj(IntFunction<? extends U> mapper)` → `Stream<U>`
  Converte il flusso primitivo in un flusso di oggetti mappando ciascun primitivo.

La Classe Contenitore Antinull: `java.util.Optional<T>`

- `of(value)` → `Optional<T>` (solleva `NullPointerException` se nullo).
- `ofNullable(value)` → `Optional<T>` (restituisce `empty()` se nullo).
- `empty()` → `Optional<T>`
- `isPresent()` / `isEmpty()` → `boolean` (Java 11: verifica presenza o assenza).
- `get()` → `T` (estrae il valore; lancia `NoSuchElementException` se vuoto: usare con cautela!).
- `ifPresent(action)` → `void` (esegue l'azione se il valore è presente).
- `ifPresentOrElse(action, emptyAction)` → `void`
  (Java 9) Se presente esegue l'azione sul valore, altrimenti esegue l'azione di fallback `emptyAction`.
- `filter(predicate)` → `Optional<T>`
- `map(mapper)` → `Optional<U>`
- `flatMap(mapper)` → `Optional<U>`
  Firma: `<U> Optional<U> flatMap(Function<? super T, ? extends Optional<? extends U>> mapper)`
  Applica la funzione evitando l'annidamento `Optional<Optional<U>>`.
- `or(supplier)` → `Optional<T>`
  (Java 9) Ritorna lo stesso `Optional` se presente, altrimenti restituisce l'alternativa prodotta dal supplier.
- `stream()` → `Stream<T>`
  (Java 9) Converte l'optional in uno stream di 0 o 1 elemento (eccellente per `flatMap(Optional::stream)`).
- `orElse(other)` → `T` (restituisce il valore di default se vuoto).
- `orElseGet(supplier)` → `T` (valutazione lazy del valore di fallback).
- `orElseThrow()` / `orElseThrow(excSupplier)` → `T`
  Firme: `T orElseThrow()`, `<X extends Throwable> T orElseThrow(Supplier<? extends X> exc) throws X`

Collettori Avanzati: Catalogo della Classe `java.util.stream.Collectors`

Accumulo in Collezioni e Mappe

- `toList()` / `toUnmodifiableList()` → `Collector`
  Firme: `static <T> Collector<T, ?, List<T>> toList()`, `toUnmodifiableList()`
  Accumula gli elementi in una `List` mutabile o non modificabile (Java 10).
- `toSet()` / `toUnmodifiableSet()` → `Collector`
  Firme: `static <T> Collector<T, ?, Set<T>> toSet()`, `toUnmodifiableSet()`
  Accumula gli elementi in un `Set` eliminando automaticamente i duplicati.
- `toCollection(factory)` → `Collector`
  Firma: `static <T, C extends Collection<T>> Collector<T, ?, C> toCollection(Supplier<C>`

`factory)`
Accumula in un'implementazione specifica.
Esempio: `Collectors.toCollection(TreeSet::new)`.

- `toMap(keyMapper, valueMapper)` → `Collector`
Firma: `static <T, K, U> Collector<T, ?, Map<K, U» toMap(keyMapper, valueMapper)`
Raccoglie in `Map`. **Attenzione**: lancia `IllegalStateException` in caso di chiavi duplicate!
- `toMap(keyMapper, valueMapper, mergeFunction)` → `Collector`
Firma: `static <T, K, U> Collector<T, ?, Map<K, U» toMap(keyM, valM, BinaryOperator<U> mergeFn)`
Risolve le collisioni di chiave applicando la funzione di merge binaria.
Esempio: `toMap(Persona::getCitta, Persona::getNome, (n1, n2) -> n1 + ", " + n2)`.
- `toMap(keyM, valM, mergeFn, mapFactory)` → `Collector`
Firma: `static <T, K, U, M extends Map<K, U» Collector<T, ?, M> toMap(keyM, valM, mergeFn, Supplier<M> factory)`
Consente di specificare la classe concreta della mappa di destinazione (es. `TreeMap::new`).

Manipolazione Testuale

- `joining()` / `joining(del)` / `joining(del, prefix, suffix)` → `Collector`
Firme: `static Collector<CharSequence, ?, String> joining([del[, prefix, suffix]])`
Concatena le stringhe con eventuale delimitatore, prefisso e suffisso.
Esempio: `joining(", ", "[", "]")` produce `"[A, B, C]"`.

Raggruppamento e Partizionamento

- `groupingBy(classifier)` → `Collector`
Firma: `static <T, K> Collector<T, ?, Map<K, List<T»> groupingBy(Function<? super T, ? extends K> classifier)`
Raggruppa gli elementi per chiave producendo una mappa `Map<K, List<T»`.
- `groupingBy(classifier, downstream)` → `Collector`
Firma: `static <T, K, A, D> Collector<T, ?, Map<K, D» groupingBy(classifier, Collector downstream)`
Raggruppa gli elementi e applica una riduzione secondaria (*downstream*) sui valori del gruppo.
Esempio (Conteggio elementi per città):

```
Map<String, Long> perCitta = persone.stream()
    .collect(Collectors.groupingBy(Persona::getCitta, Collectors.counting()));
```

- `groupingBy(classifier, mapFactory, downstream)` → `Collector`
Firma: `static <T, K, D, M extends Map<K, D» Collector<T, ?, M> groupingBy(classifier, Supplier<M> factory, downstream)`
Specifica l'implementazione della mappa restituita (es. `TreeMap::new` per avere chiavi ordinate).
- `partitioningBy(predicate)` / `partitioningBy(predicate, downstream)` → `Collector`
Firme: `static <T> Collector<T, ?, Map<Boolean, List<T»> partitioningBy(Predicate<? super T> predicate)`, `static <T, D, A> Collector<T, ?, Map<Boolean, D» partitioningBy(predicate, Collector downstream)`
Partiziona gli elementi in due liste in base al valore booleano del predicato (`true` o `false`) o applica un collettore downstream.

Riduzioni Downstream e Collettori Composti

- `counting()` → `Collector<T, ?, Long>`
Conta gli elementi presenti nel gruppo.
- `summingInt(mapper)` / `summingDouble(mapper)` → `Collector`
Somma i valori numerici estratti da ciascun elemento.

- `averagingDouble(mapper)` → `Collector`
  Calcola la media aritmetica dei valori estratti.
- `mapping(mapper, downstream)` → `Collector`
  Firma: `static <T, U, A, R> Collector<T, ?, R> mapping(Function mapper, Collector downstream)`
  Applica una funzione di mapping agli elementi prima di passarli al collettore downstream.
  Esempio: `groupingBy(Persona::getCitta, mapping(Persona::getNome, toList()))`.
- `filtering(predicate, downstream)` → `Collector`
  Firma: `static <T, A, R> Collector<T, ?, R> filtering(Predicate predicate, Collector downstream)`
  (Java 9) Filtra gli elementi all'interno di ciascun gruppo prima dell'accumulo downstream.
- `flatMapping(mapper, downstream)` → `Collector`
  Firma: `static <T, U, A, R> Collector<T, ?, R> flatMapping(Function mapper, Collector downstream)`
  (Java 9) Appiattisce collezioni innestate durante il raggruppamento.
- `reducing(op)` / `reducing(identity, op)` / `reducing(identity, mapper, op)` → `Collector`
  Esegue una riduzione sugli elementi con operatore associativo.
- `collectingAndThen(downstream, finisher)` → `Collector`
  Firma: `static <T, A, R, RR> Collector<T, A, RR> collectingAndThen(Collector downstream, Function finisher)`
  Applica una funzione di finitura trasformativa al termine del processo di accumulo (es. `collectingAndThen(toList(), Collections::unmodifiableList)` per collezioni non modificabili).

Java Swing, AWT, Eventi ed EDT (`javax.swing` & `java.awt`)

Gerarchia dei Componenti Grafici

- `JFrame`: `setTitle(String)`, `setSize(int w, int h)`, `setLocationRelativeTo(null)` (centra lo schermo), `pack()` (ridimensiona sui componenti), `setVisible(boolean)`, `setDefaultCloseOperation(int)` (costanti: `JFrame.EXIT_ON_CLOSE`, `DISPOSE_ON_CLOSE`).
- `JButton`: `addActionListener(ActionListener l)`, `setText(String)`, `getText()` → `String`, `setEnabled(boolean)`.
- `JTextField`: `getText()` → `String`, `setText(String)`, `setColumns(int)`, `setEditable(boolean)`.
- `JTextArea`: `append(String)`, `setText(String)`, `getText()` → `String`, `setEditable(boolean)`, `setRows(int)`, `setColumns(int)`.
- `JScrollPane`: contenitore a barre di scorrimento (`new JScrollPane(componente)`); indispensabile per avvolgere `JTable` o `JTextArea` e renderle scorrevoli.
- `JLabel`: `setText(String)`, `getText()` → `String`, `setIcon(Icon)`.
- `JCheckBox` / `JRadioButton`: `isSelected()` → `boolean`, `setSelected(boolean)`, `addItemListener(ItemListener)`. I radio button si raggruppano con `new ButtonGroup().add(radioBtn);`.
- `JComboBox<E>`: `addItem(E item)`, `getSelectedItem()` → `Object`, `getSelectedIndex()` → `int`.
- `JTable` e `DefaultTableModel`: `table.setModel(model)`, `model.addRow(Object[] rowData)`, `model.getValueAt(int r, int c)`, `model.setValueAt(Object val, int r, int c)`, `model.fireTableDataChanged()`.

Finestre di Dialogo Modali: `javax.swing.JOptionPane`

- `showMessageDialog(parent, msg)`: mostra un avviso modale informativo:

`JOptionPane.showMessageDialog(this, "Operazione completata con successo!");`

- `showInputDialog(parent, prompt)` → `String`: finestra di input per acquisire una stringa dall'utente (restituisce `null` se annullata).
- `showConfirmDialog(parent, prompt)` → `int`: richiede una conferma (costanti: `JOptionPane.YES_OPTION`, `NO_OPTION`, `CANCEL_OPTION`).

Gestione del Layout (`LayoutManager`)

- `BorderLayout`: 5 regioni cardinali (`NORTH`, `SOUTH`, `EAST`, `WEST`, `CENTER`). North/South prendono altezza preferita e tutta la larghezza; East/West larghezza preferita; Center occupa tutto lo spazio residuo. Default di `JFrame`.
  Uso: `panel.add(componente, BorderLayout.CENTER);`
- `FlowLayout`: Dispone i componenti in riga da sinistra a destra con a capo automatico. Default di `JPanel`.
  Costruttore: `new FlowLayout(FlowLayout.LEFT, hgap, vgap);`
- `GridLayout`: Griglia a celle di dimensioni identiche (`new GridLayout(rows, cols, hgap, vgap);`).
- `BoxLayout`: Disposizione monodimensionale su un asse (`BoxLayout.X_AXIS` o `Y_AXIS`). Supporta colle elastiche (`Box.createHorizontalGlue()`) e distanziatori fissi (`Box.createRigidArea(new Dimension(w, h))`).
- `GridBagLayout`: Griglia complessa flessibile governata da `GridBagConstraints` (`gridx, gridy, gridwidth, weightx, fill, insets`).

Modello ad Eventi e Listener

- `ActionListener` → `void actionPerformed(ActionEvent e)`: Click pulsanti, invio su campi di testo.
- `ItemListener` → `void itemStateChanged(ItemEvent e)`: Variazione selezione su checkbox/radio button.
- `MouseListener` / `MouseAdapter` → `mouseClicked`, `mousePressed`, `mouseReleased`, `mouseEntered`, `mouseExited`.
- `WindowListener` / `WindowAdapter` → `windowClosing(WindowEvent e)`: Intercetta la chiusura della finestra.

Concorrenza Grafica ed Event Dispatch Thread (EDT)

- **Regola del Single-Thread**: tutti i componenti di Swing non sono thread-safe. Tutte le letture/scritture sui componenti grafici devono avvenire rigidamente sull'Event Dispatch Thread (EDT).
- `SwingUtilities.invokeLater(Runnable r)` / `EventQueue.invokeLater(Runnable r)`: accoda un compito asincrono da eseguire sull'EDT appena la coda si svuota. Non blocca il thread chiamante.
  Avvio GUI standard: `EventQueue.invokeLater(() -> new MioFrame().setVisible(true));`
- `EventQueue.isDispatchThread()` / `SwingUtilities.isEventDispatchThread()` → `boolean`: interroga il runtime per accertare se il thread corrente è l'EDT.
- **La Ricetta Universale di Aggiornamento Sicuro**:

```
void aggiornaGrafica(Runnable r) {
    if (EventQueue.isDispatchThread()) r.run();
    else EventQueue.invokeLater(r);
}
```

- `EventQueue.invokeAndWait(Runnable r)`: blocca il thread chiamante finché l'EDT non ha completato l'esecuzione di `r`. Solleva `InterruptedException` ed `InvocationTargetException`.
  Trappola di Deadlock: se invocato direttamente dall'EDT, lancia immediatamente un `Error` (l'EDT si auto-bloccherebbe attendendo un compito che solo lui può eseguire).

Timer Grafico: `javax.swing.Timer`

- Timer per animazioni, orologi e polling visivo: consegna i suoi `ActionEvent` **direttamente sull'EDT**.
- `Timer(int delayMillis, ActionListener listener)`: crea un timer con intervallo e listener.
- `start()`: avvia il conteggio; `stop()`: arresta il timer; `restart()`: riavvia azzerando il delay iniziale.
- `setRepeats(boolean flag)`: se `false`, esegue il compito una sola volta (*one-shot*); default `true`.
- `setInitialDelay(int delay)`: imposta un ritardo iniziale differente per il primo scatto.
- **Criterio di Scelta**: usare `javax.swing.Timer` per compiti brevi che aggiornano componenti GUI; usare `ScheduledExecutorService` per compiti pesanti o I/O bloccante periodico fuori dall'EDT.

Task Asincroni di Background: `javax.swing.SwingWorker<T, V>`

- Esegue calcoli intensivi o I/O prolungato in background senza congelare l'EDT, garantendo la pubblicazione di progressi intermedi e la finalizzazione sicura sull'interfaccia:
  - **Parametri Generics**: `T` è il tipo del risultato finale (restituito da `get()`); `V` è il tipo dei dati intermedi parziali inviati all'EDT.
  - `protected abstract T doInBackground() throws Exception` [**Worker Thread**]: obbligatorio. Esegue il calcolo pesante fuori dall'EDT.
  - `protected final void publish(V... chunks)` [**Worker Thread**]: invia uno o più risultati intermedi parziali verso l'EDT.
  - `protected void process(List<V> chunks)` [**EDT**]: riceve i blocchi aggregati inviati da `publish()` e aggiorna in sicurezza i componenti grafici (es. `JProgressBar`, `JTable`).
  - `protected void done()` [**EDT**]: invocato automaticamente al termine del calcolo o dopo annullamento. Ideale per riattivare bottoni, nascondere spinner e raccogliere il valore finale con `get()`.
  - `public final void execute()`: sottomette lo worker al thread pool interno gestito da Swing.
  - `public final T get()` / `get(long timeout, TimeUnit unit)`: bloccante; estrae il valore restituito da `doInBackground()`. Solleva `ExecutionException` in caso di fallimento.
  - `public final boolean cancel(boolean mayInterruptIfRunning)`: richiede la cancellazione cooperativa impostando lo stato di interruzione;
  - `public final boolean isCancelled()` / `isDone()` → `boolean`: interrogano lo stato del task.
  - `protected final void setProgress(int progress)`: notifica l'avanzamento percentuale (0..100); osservabile registrando un listener con `addPropertyChangeListener(PropertyChangeListener l)`.

Disegno Grafico 2D Personalizzato

- Sovrascrittura di `protected void paintComponent(Graphics g)` in una sottoclasse di `JPanel`.
- Invocazione obbligatoria come prima istruzione di `super.paintComponent(g);` per ripulire lo sfondo.
- Cast al contesto grafico avanzato: `Graphics2D g2 = (Graphics2D) g;`.
- Attivazione dell'antialiasing: `g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);`.
- Tracciamento e riempimento: `g2.setColor(Color)`, `g2.setStroke(Stroke)`, `g2.draw(Shape)`, `g2.fill(Shape)`.
- Forzatura del ridisegno da codice: `pannello.repaint()` accoda la richiesta di rendering all'EDT.

Thread, Concorrenza ed Executor Framework (`java.lang` & `java.util.concurrent`)

Primitive di Concorrenza: `java.lang.Thread` e `java.lang.Runnable`

- `Runnable`: Interfaccia funzionale SAM con firma `void run()`; definisce l'unità computazionale da eseguire.

- `Thread`: Incapsula un flusso di esecuzione concorrente:
  - `new Thread(Runnable task)`: Crea un thread associato al task.
  - `start()`: Avvia l'esecuzione parallela allocando lo stack ed invocando `run()` nel nuovo thread del sistema operativo. **Attenzione**: non invocare mai `run()` direttamente (eseguirebbe il codice nel thread chiamante).
  - `join()` / `join(long millis)`: Blocca il thread chiamante ponendolo in stato `WAITING` fino alla conclusione del thread bersaglio.
  - `Thread.sleep(long millis)`: Metodo statico; sospende il thread corrente per la durata indicata (`TIMED_WAITING`). Solleva `InterruptedException`.
  - `interrupt()`: Invia una richiesta cooperativa di terminazione impostando il flag di interruzione. Se il thread è sospeso su `sleep()`, `wait()` o `join()`, viene sollevata tempestivamente una `InterruptedException`.
  - `isInterrupted()` → `boolean`: Interroga lo stato del flag senza azzerarlo.
  - `Thread.interrupted()` → `boolean`: Metodo statico; legge il flag e lo **azzera** (usare con cautela).
  - `Thread.currentThread()` → `Thread`: Restituisce il riferimento all'istanza del thread in esecuzione.
  - `Thread.currentThread().isVirtual()` → `boolean` (Java 21): Rileva se il thread corrente è un Virtual Thread o un Platform Thread nativo del kernel.
  - **Fluent Builder Virtual Threads** (Java 21):
    ```
    Thread.ofVirtual().name("vt-worker-", 0).start(task);
    Thread unstarted = Thread.ofVirtual().unstarted(task); unstarted.start();
    ```
  - **Fluent Builder Platform Threads**:
    ```
    Thread.ofPlatform().name("os-worker").daemon().start(task);
    ```

Framework di Esecuzione: `ExecutorService` e Factory `Executors`

- `Executors.newFixedThreadPool(int n)`: Istanzia un pool a dimensione fissa con n worker thread. Dimensionamento ottimale CPU-bound: $N_{\text{threads}} = N_{\text{core}}(+1)$.
- `Executors.newCachedThreadPool()`: Istanzia un pool elastico che crea nuovi thread secondo necessità e distrugge quelli inattivi per oltre 60 secondi. Attenzione: rischio Thread Explosion sotto carichi I/O massivi!
- `Executors.newSingleThreadExecutor()`: Unico worker thread per esecuzione strettamente sequenziale FIFO.
- `Executors.newScheduledThreadPool(int core)`: Per compiti differiti (`schedule()`) o periodici (`scheduleAtFixedRate()`, `scheduleWithFixedDelay()`).
- `Executors.newVirtualThreadPerTaskExecutor()` (Java 21): Alloca un Virtual Thread leggero user-mode nello Heap per ciascun compito sottomesso. Prestazioni ottimali per carichi I/O-bound.
- `submit(Runnable/Callable task)` vs `execute(Runnable task)`: con `submit()`, le eccezioni non catturate vengono incapsulate in `ExecutionException` su `get()`; con `execute()`, l'eccezione termina il worker e stampa lo stacktrace su `System.err`.
- **Gestione Risorse Automatica (Java 19+)**: `ExecutorService` implementa `AutoCloseable`; all'uscita dal blocco `try-with-resources`, `close()` invoca `shutdown()` e attende il completamento.
- **Protocollo di Terminazione Controllata (Two-Phase Termination)**:
  - `shutdown()`: Rifiuta la sottomissione di nuovi compiti lasciando completare quelli già accodati.
  - `awaitTermination(long timeout, TimeUnit unit)` → `boolean`: Attende fino al tempo limite prefissato la conclusione naturale dei task.
  - `shutdownNow()` → `List<Runnable>`: Forza l'arresto immediato inviando `interrupt()` a tutti i thread attivi e restituisce la lista dei task non avviati.

Task Produttori di Risultato: `Callable<V>` e `Future<T>`

- `Callable<V>`: Interfaccia funzionale con firma `V call() throws Exception`; a differenza di `Runnable`, restituisce un valore tipizzato e ammette eccezioni checked.
- `Future<T>`: Token asincrono per monitorare e recuperare l'esito della computazione:
  - `get()` → T: Invocazione bloccante per estrarre il valore finale. Solleva `InterruptedException` o `ExecutionException`.
  - `get(long timeout, TimeUnit unit)` → T: Invocazione con attesa vincolata a un tempo limite (`TimeoutException`).
  - `isDone()` → `boolean`: Rileva se il task ha completato l'elaborazione.
  - `cancel(boolean mayInterruptIfRunning)` → `boolean`: Tenta l'annullamento dell'esecuzione del task propagando l'interrupt se richiesto.
- `invokeAll(Collection<Callable<T>>)` → `List<Future<T>>`: Sottomette tutti i task e ritorna solo quando tutti hanno concluso.
- `invokeAny(Collection<Callable<T>>)` → T: Restituisce il risultato del primo task che completa con successo, cancellando gli altri.

Sincronizzazione Intrinseca ed Esplicita

- `synchronized`: Monitor lock intrinseco dell'oggetto; mutua esclusione atomica e visibilità della memoria:
  - Metodo di istanza: lock su `this`; Metodo statico: lock sull'oggetto `Classe.class`.
  - Blocco fine-grained: `synchronized(lockPrivato) { /* sezione critica */ }`.
  - Rientranza: un thread può riacquisire più volte lo stesso monitor senza auto-bloccarsi.
  - Coordinamento con `wait()` e `notifyAll()`: rilascia atomicamente il lock; obbligo del ciclo `while (!cond) wait()` contro gli spurious wakeups e uso di `notifyAll()` per evitare segnali persi (*missed signals*).
- `java.util.concurrent.locks.ReentrantLock`: lock esplicito alternativo a `synchronized`; supporta `tryLock(timeout, unit)`, equità (*fairness*) e impone il blocco `try-finally { lock.unlock(); }`.
- `java.util.concurrent.locks.Condition`: generata da `reentrantLock.newCondition()`; consente condizioni di attesa multiple e distinte sullo stesso lock (`await()`, `signal()`, `signalAll()`).
- `java.util.concurrent.locks.ReentrantReadWriteLock`: separa `readLock()` (molti lettori in parallelo) e `writeLock()` (scrittura esclusiva). Ottimale per strutture a prevalenza di lettura.
- `java.util.concurrent.Semaphore`: gestisce un pool di K permessi; `acquire()` per prendere un permesso, `release()` nel blocco `finally` per restituirlo.

Sincronizzatori di Coordinamento Avanzati

- `java.util.concurrent.CountDownLatch`: barriera a conteggio alla rovescia non riutilizzabile:
  - `new CountDownLatch(int count)`: inizializza il contatore scalare a N.
  - `countDown()`: decrementa atomicamente il contatore di 1; quando raggiunge 0, tutti i thread in attesa vengono sbloccati.
  - `await()` / `await(long timeout, TimeUnit unit)` → `boolean`: blocca il thread chiamante finché il contatore non si azzera.
  - *Uso d'elezione*: sincronizzazione dello start simultaneo di N worker o attesa del completamento nei test unitari TDD.
- `java.util.concurrent.CyclicBarrier`: punto di incontro comune riutilizzabile ciclicamente:
  - `new CyclicBarrier(int parties, Runnable barrierAction)`: attende N thread; l'ultimo che arriva esegue l'azione opzionale di barriera.
  - `await()` → `int`: sospende il thread fino all'arrivo di tutti i *parties*; si resetta automaticamente al turno successivo.

Variabili Atomiche e LongAdder per Altissima Contesa

- Modificatore `volatile`: garantisce visibilità immediata in RAM (relazione *happens-before*) e vieta il riordinamento delle istruzioni, ma **non garantisce atomicità** su operazioni composte.
- `java.util.concurrent.atomic.AtomicLong` / `AtomicInteger`: operazioni lock-free basate sull'istruzione hardware CAS (*Compare-And-Swap*): `incrementAndGet()`, `addAndGet(delta)`, `compareAndSet(atteso, nuovo)`.
- `java.util.concurrent.atomic.LongAdder`: introdotto per superare il collo di bottiglia del CAS sotto contesa estrema; alloca una tabella di celle a 64 bit distribuite per thread: `increment()` aggiorna la cella locale a contesa zero, `sum()` calcola il totale aggregato.

Collezioni Concorrenti Thread-Safe

- `ConcurrentHashMap<K, V>`: hashing lock-free su bucket con CAS e metodi atomici: `putIfAbsent()`, `computeIfAbsent(k, f)`. Nota: non eseguire calcoli lenti dentro la lambda di `computeIfAbsent()` per non bloccare il bucket.
- `CopyOnWriteArrayList<T>`: ogni scrittura clona l'array sottostante; iterazioni lock-free immuni da `ConcurrentModificationException`. Ottimale per liste di listener grafici.
- `BlockingQueue<T>` (`ArrayBlockingQueue`, `LinkedBlockingQueue`): per pattern Produttore-Consumatore; metodi bloccanti `put(item)` e `take()`. Arresto pulito tramite sentinelle multiple (*Poison Pill*).

Matrice Decisionale: Quale Strumento Scegliere (§6.6 D04)

- Guida sintetica per la scelta dello strumento ottimale all'esame:

Problema Ingegneristico	Strumento Concorrente Ottimale
Bandiera scritta da uno e letta da altri	`volatile boolean` oppure `AtomicBoolean`
Contatore scalare singolo	`AtomicLong` / `AtomicInteger`; `LongAdder` se la contesa è elevata
Più campi correlati da aggiornare insieme	Blocco `synchronized` privato; `ReentrantLock` se servono timeout o fairness
Moltissime letture, rare scritture	`ReadWriteLock`; `CopyOnWriteArrayList` per collezioni listener
Risorsa a capienza limitata (N posti)	`Semaphore`
Passaggio dati tra produttori e consumatori	`BlockingQueue` (`ArrayBlockingQueue` o `LinkedBlockingQueue`)
Dizionario concorrente ad alto throughput	`ConcurrentHashMap` con mutazioni atomiche (`computeIfAbsent()`, `merge()`)
Attesa coordinata che una condizione sia vera	`wait()`/`notifyAll()` su monitor, `Condition` esplicita o `CountDownLatch`
Nessuno stato condiviso	**La scelta migliore in assoluto: Immutabilità e Confinamento**

Formattazione a Terminale e Codici di Escape ANSI

Struttura delle Sequenze di Controllo ANSI

- Una sequenza di comando colore ANSI ha la struttura standard: `"\u001B[" + codice + "m"`.
- `\u001B[0m` (`RESET`): Ripristina all'istante lo stile e il colore predefinito della console.
- **Modificatori di Stile**: `BOLD` (1), `DIM` (2), `UNDERLINE` (4), `REVERSE` (7).
- **Colori Base di Primo Piano (Foreground)**: `BLACK` (30), `RED` (31), `GREEN` (32), `YELLOW` (33), `BLUE` (34), `MAGENTA` (35), `CYAN` (36), `WHITE` (37).
- **Colori ad Alta Luminosità (Bright)**: `BRIGHT_BLACK` (90), `BRIGHT_RED` (91), `BRIGHT_GREEN` (92), `BRIGHT_YELLOW` (93), `BRIGHT_BLUE` (94), `BRIGHT_MAGENTA` (95), `BRIGHT_CYAN` (96), `BRIGHT_WHITE` (97).

Pattern Canonico: Enum Ansi (dai progetti FileUtil e LabAnsiColor)

- Implementazione elegante ed incapsulata mediante enumerativo Java:

```
public enum Ansi {
    RESET(0), BOLD(1), UNDERLINE(4), REVERSE(7),
    BLACK(30), RED(31), GREEN(32), YELLOW(33),
    BLUE(34), MAGENTA(35), CYAN(36), WHITE(37),
    BRIGHT_BLACK(90), BRIGHT_RED(91), BRIGHT_GREEN(92), BRIGHT_YELLOW(93),
    BRIGHT_BLUE(94), BRIGHT_MAGENTA(95), BRIGHT_CYAN(96), BRIGHT_WHITE(97);

    private final int code;
```

```
        Ansi(int code) { this.code = code; }

        public String code() { return "\u001B[" + code + "m"; }
        public String paint(String s) { return code() + s + RESET.code(); }
    }
```

- **Esempio di Utilizzo**: `System.out.println(Ansi.GREEN.paint("Esecuzione completata!"));`